

TYCampport3

3

Generated by Doxygen 1.8.17

1 Main Page	1
1.1 Note	1
2 Module Index	3
2.1 Modules	3
3 Class Index	5
3.1 Class List	5
4 File Index	9
4.1 File List	9
5 Module Documentation	53
5.1 TurboJPEG	53
5.1.1 Detailed Description	55
5.1.2 Macro Definition Documentation	56
5.1.2.1 TJ_NUMCS	56
5.1.2.2 TJ_NUMERR	56
5.1.2.3 TJ_NUMPF	56
5.1.2.4 TJ_NUMSAMP	57
5.1.2.5 TJ_NUMXOP	57
5.1.2.6 TJFLAG_ACCURATEDCT	57
5.1.2.7 TJFLAG_BOTTOMUP	57
5.1.2.8 TJFLAG_FASTDCT	57
5.1.2.9 TJFLAG_FASTUPSAMPLE	58
5.1.2.10 TJFLAG_NOREALLOC	58
5.1.2.11 TJFLAG_PROGRESSIVE	58
5.1.2.12 TJFLAG_STOPONWARNING	58
5.1.2.13 TJPAD	59
5.1.2.14 TJSCALED	59
5.1.2.15 TJXOPT_COPYNONE	59
5.1.2.16 TJXOPT_CROP	59
5.1.2.17 TJXOPT_GRAY	60
5.1.2.18 TJXOPT_NOOUTPUT	60
5.1.2.19 TJXOPT_PERFECT	60
5.1.2.20 TJXOPT_PROGRESSIVE	60
5.1.2.21 TJXOPT_TRIM	61
5.1.3 Typedef Documentation	61
5.1.3.1 tjhandle	61
5.1.3.2 tjtransform	61
5.1.4 Enumeration Type Documentation	61
5.1.4.1 TJCS	61
5.1.4.2 TJERR	62
5.1.4.3 TJPF	62

5.1.4.4 TJSAMP	63
5.1.4.5 TJXOP	64
5.1.5 Function Documentation	65
5.1.5.1 tjAlloc()	65
5.1.5.2 tjBufSize()	65
5.1.5.3 tjBufSizeYUV2()	66
5.1.5.4 tjCompress2()	66
5.1.5.5 tjCompressFromYUV()	68
5.1.5.6 tjCompressFromYUVPlanes()	69
5.1.5.7 tjDecodeYUV()	71
5.1.5.8 tjDecodeYUVPlanes()	72
5.1.5.9 tjDecompress2()	73
5.1.5.10 tjDecompressHeader3()	73
5.1.5.11 tjDecompressToYUV2()	74
5.1.5.12 tjDecompressToYUVPlanes()	75
5.1.5.13 tjDestroy()	76
5.1.5.14 tjEncodeYUV3()	76
5.1.5.15 tjEncodeYUVPlanes()	77
5.1.5.16 tjFree()	78
5.1.5.17 tjGetErrorCode()	79
5.1.5.18 tjGetErrorStr2()	79
5.1.5.19 tjGetScalingFactors()	80
5.1.5.20 tjInitCompress()	80
5.1.5.21 tjInitDecompress()	80
5.1.5.22 tjInitTransform()	81
5.1.5.23 tjLoadImage()	81
5.1.5.24 tjPlaneHeight()	82
5.1.5.25 tjPlaneSizeYUV()	82
5.1.5.26 tjPlaneWidth()	83
5.1.5.27 tjSaveImage()	83
5.1.5.28 tjTransform()	84
6 Class Documentation	87
6.1 AlgVersion Struct Reference	87
6.1.1 Detailed Description	87
6.2 DepthEnhanceParameters Struct Reference	87
6.2.1 Detailed Description	88
6.3 DepthInpainterParameters Struct Reference	88
6.3.1 Detailed Description	88
6.4 DepthSpeckleFilterParameters Struct Reference	88
6.4.1 Detailed Description	88
6.5 f16 Struct Reference	88

6.5.1 Detailed Description	89
6.6 gz_header_s Struct Reference	89
6.6.1 Detailed Description	89
6.7 gzFile_s Struct Reference	89
6.7.1 Detailed Description	89
6.8 JHUFF_TBL Struct Reference	90
6.8.1 Detailed Description	90
6.9 jpeg_common_struct Struct Reference	90
6.9.1 Detailed Description	90
6.10 jpeg_component_info Struct Reference	90
6.10.1 Detailed Description	91
6.11 jpeg_compress_struct Struct Reference	91
6.11.1 Detailed Description	93
6.12 jpeg_decompress_struct Struct Reference	93
6.12.1 Detailed Description	95
6.13 jpeg_destination_mgr Struct Reference	95
6.13.1 Detailed Description	95
6.14 jpeg_error_mgr Struct Reference	96
6.14.1 Detailed Description	96
6.15 jpeg_marker_struct Struct Reference	96
6.15.1 Detailed Description	97
6.16 jpeg_memory_mgr Struct Reference	97
6.16.1 Detailed Description	97
6.17 jpeg_progress_mgr Struct Reference	98
6.17.1 Detailed Description	98
6.18 jpeg_scan_info Struct Reference	98
6.18.1 Detailed Description	98
6.19 jpeg_source_mgr Struct Reference	98
6.19.1 Detailed Description	99
6.20 JQUANT_TBL Struct Reference	99
6.20.1 Detailed Description	99
6.21 nsimd_aarch64_vf16 Struct Reference	99
6.21.1 Detailed Description	99
6.22 nsimd_aarch64_vlf16 Struct Reference	99
6.22.1 Detailed Description	100
6.23 nsimd_avx2_vf16 Struct Reference	100
6.23.1 Detailed Description	100
6.24 nsimd_avx2_vlf16 Struct Reference	100
6.24.1 Detailed Description	100
6.25 nsimd_avx512_knl_vf16 Struct Reference	100
6.25.1 Detailed Description	101
6.26 nsimd_avx512_knl_vlf16 Struct Reference	101

6.26.1 Detailed Description	101
6.27 nsimd_avx512_skylake_vf16 Struct Reference	101
6.27.1 Detailed Description	101
6.28 nsimd_avx512_skylake_vlf16 Struct Reference	101
6.28.1 Detailed Description	102
6.29 nsimd_avx_vf16 Struct Reference	102
6.29.1 Detailed Description	102
6.30 nsimd_avx_vlf16 Struct Reference	102
6.30.1 Detailed Description	102
6.31 nsimd_cpu_vf16 Struct Reference	103
6.31.1 Detailed Description	103
6.32 nsimd_cpu_vf32 Struct Reference	103
6.32.1 Detailed Description	103
6.33 nsimd_cpu_vf64 Struct Reference	103
6.33.1 Detailed Description	104
6.34 nsimd_cpu_vi16 Struct Reference	104
6.34.1 Detailed Description	104
6.35 nsimd_cpu_vi32 Struct Reference	104
6.35.1 Detailed Description	104
6.36 nsimd_cpu_vi64 Struct Reference	105
6.36.1 Detailed Description	105
6.37 nsimd_cpu_vi8 Struct Reference	105
6.37.1 Detailed Description	105
6.38 nsimd_cpu_vlf16 Struct Reference	106
6.38.1 Detailed Description	106
6.39 nsimd_cpu_vlf32 Struct Reference	106
6.39.1 Detailed Description	106
6.40 nsimd_cpu_vlf64 Struct Reference	106
6.40.1 Detailed Description	107
6.41 nsimd_cpu_vli16 Struct Reference	107
6.41.1 Detailed Description	107
6.42 nsimd_cpu_vli32 Struct Reference	107
6.42.1 Detailed Description	107
6.43 nsimd_cpu_vli64 Struct Reference	108
6.43.1 Detailed Description	108
6.44 nsimd_cpu_vli8 Struct Reference	108
6.44.1 Detailed Description	108
6.45 nsimd_cpu_vlu16 Struct Reference	109
6.45.1 Detailed Description	109
6.46 nsimd_cpu_vlu32 Struct Reference	109
6.46.1 Detailed Description	109
6.47 nsimd_cpu_vlu64 Struct Reference	109

6.47.1 Detailed Description	110
6.48 nsimd_cpu_vlu8 Struct Reference	110
6.48.1 Detailed Description	110
6.49 nsimd_cpu_vu16 Struct Reference	110
6.49.1 Detailed Description	111
6.50 nsimd_cpu_vu32 Struct Reference	111
6.50.1 Detailed Description	111
6.51 nsimd_cpu_vu64 Struct Reference	111
6.51.1 Detailed Description	111
6.52 nsimd_cpu_vu8 Struct Reference	112
6.52.1 Detailed Description	112
6.53 nsimd_neon128_vf16 Struct Reference	112
6.53.1 Detailed Description	112
6.54 nsimd_neon128_vf64 Struct Reference	113
6.54.1 Detailed Description	113
6.55 nsimd_neon128_vlf16 Struct Reference	113
6.55.1 Detailed Description	113
6.56 nsimd_neon128_vlf64 Struct Reference	113
6.56.1 Detailed Description	113
6.57 nsimd_sse2_vf16 Struct Reference	114
6.57.1 Detailed Description	114
6.58 nsimd_sse2_vlf16 Struct Reference	114
6.58.1 Detailed Description	114
6.59 nsimd_sse42_vf16 Struct Reference	114
6.59.1 Detailed Description	114
6.60 nsimd_sse42_vlf16 Struct Reference	115
6.60.1 Detailed Description	115
6.61 nsimd_vmx_vf16 Struct Reference	115
6.61.1 Detailed Description	115
6.62 nsimd_vmx_vf64 Struct Reference	115
6.62.1 Detailed Description	115
6.63 nsimd_vmx_vf64 Struct Reference	116
6.63.1 Detailed Description	116
6.64 nsimd_vmx_vlf16 Struct Reference	116
6.64.1 Detailed Description	116
6.65 nsimd_vmx_vlf64 Struct Reference	116
6.65.1 Detailed Description	116
6.66 nsimd_vmx_vli64 Struct Reference	117
6.66.1 Detailed Description	117
6.67 nsimd_vmx_vlu64 Struct Reference	117
6.67.1 Detailed Description	117
6.68 nsimd_vmx_vu64 Struct Reference	117

6.68.1 Detailed Description	117
6.69 nsimd_vsx_vf16 Struct Reference	118
6.69.1 Detailed Description	118
6.70 nsimd_vsx_vlf16 Struct Reference	118
6.70.1 Detailed Description	118
6.71 OperatorInfo Struct Reference	118
6.71.1 Detailed Description	119
6.72 ParamDetail Struct Reference	119
6.72.1 Detailed Description	119
6.73 pattern_bin_param Struct Reference	120
6.73.1 Detailed Description	120
6.74 pattern_gray_param Struct Reference	120
6.74.1 Detailed Description	120
6.75 pattern_sine_param Struct Reference	120
6.75.1 Detailed Description	120
6.76 TY_PHC_GROUP_ATTR::phc_group_attr Struct Reference	121
6.76.1 Detailed Description	121
6.77 tjregion Struct Reference	121
6.77.1 Detailed Description	121
6.77.2 Member Data Documentation	121
6.77.2.1 h	121
6.77.2.2 w	122
6.77.2.3 x	122
6.77.2.4 y	122
6.78 tjscalingfactor Struct Reference	122
6.78.1 Detailed Description	122
6.78.2 Member Data Documentation	123
6.78.2.1 denom	123
6.78.2.2 num	123
6.79 tjtransform Struct Reference	123
6.79.1 Detailed Description	124
6.79.2 Member Data Documentation	124
6.79.2.1 customFilter	124
6.79.2.2 data	125
6.79.2.3 op	125
6.79.2.4 options	125
6.79.2.5 r	125
6.80 TY_ACC_BIAS Struct Reference	125
6.80.1 Detailed Description	125
6.81 TY_ACC_MISALIGNMENT Struct Reference	126
6.81.1 Detailed Description	126
6.82 TY_ACC_SCALE Struct Reference	126

6.82.1 Detailed Description	126
6.83 TY_AEC_ROI_PARAM Struct Reference	127
6.83.1 Detailed Description	127
6.84 TY_BYTEARRAY_ATTR Struct Reference	127
6.84.1 Detailed Description	128
6.84.2 Member Data Documentation	128
6.84.2.1 unit_size	128
6.84.2.2 valid_size	128
6.85 TY_CAMERA_CALIB_INFO Struct Reference	128
6.85.1 Detailed Description	129
6.86 TY_CAMERA_DISTORTION Struct Reference	129
6.86.1 Detailed Description	129
6.87 TY_CAMERA_EXTRINSIC Struct Reference	129
6.87.1 Detailed Description	129
6.88 TY_CAMERA_INTRINSIC Struct Reference	130
6.88.1 Detailed Description	130
6.89 TY_CAMERA_ROTATION Struct Reference	131
6.89.1 Detailed Description	131
6.90 TY_CAMERA_STATISTICS Struct Reference	131
6.90.1 Detailed Description	131
6.91 TY_CAMERA_TO_IMU Struct Reference	132
6.91.1 Detailed Description	132
6.92 TY_DEVICE_BASE_INFO Struct Reference	132
6.92.1 Detailed Description	133
6.93 TY_DEVICE_NET_INFO Struct Reference	133
6.93.1 Detailed Description	134
6.94 TY_DEVICE_USB_INFO Struct Reference	134
6.94.1 Detailed Description	134
6.95 TY_DI_WORKMODE Struct Reference	134
6.95.1 Detailed Description	134
6.96 TY_DO_WORKMODE Struct Reference	135
6.96.1 Detailed Description	135
6.97 TY_ENUM_ENTRY Struct Reference	135
6.97.1 Detailed Description	135
6.98 TY_EVENT_INFO Struct Reference	136
6.98.1 Detailed Description	136
6.99 TY_FEATURE_INFO Struct Reference	136
6.99.1 Detailed Description	136
6.100 TY_FLOAT_RANGE Struct Reference	137
6.100.1 Detailed Description	137
6.101 TY_FRAME_DATA Struct Reference	137
6.101.1 Detailed Description	138

6.102 TY_GYRO_BIAS Struct Reference	138
6.102.1 Detailed Description	138
6.103 TY_GYRO_MISALIGNMENT Struct Reference	138
6.103.1 Detailed Description	139
6.104 TY_GYRO_SCALE Struct Reference	139
6.104.1 Detailed Description	139
6.105 TY_IMAGE_DATA Struct Reference	140
6.105.1 Detailed Description	140
6.106 TY_IMU_DATA Struct Reference	140
6.106.1 Detailed Description	141
6.107 TY_INT_RANGE Struct Reference	141
6.107.1 Detailed Description	141
6.108 TY_INTERFACE_INFO Struct Reference	141
6.108.1 Detailed Description	142
6.109 TY_LASER_PARAM Struct Reference	142
6.109.1 Detailed Description	142
6.110 TY_LASER_PATTERN_PARAM Struct Reference	142
6.110.1 Detailed Description	143
6.111 TY_PHC_GROUP_ATTR Struct Reference	143
6.111.1 Detailed Description	144
6.112 TY_PIXEL_COLOR_DESC Struct Reference	144
6.112.1 Detailed Description	144
6.113 TY_PIXEL_DESC Struct Reference	144
6.113.1 Detailed Description	144
6.114 TY_TEMP_DATA Struct Reference	145
6.114.1 Detailed Description	145
6.115 TY_TOF_FREQ Struct Reference	145
6.115.1 Detailed Description	145
6.116 TY_TRIGGER_PARAM Struct Reference	145
6.116.1 Detailed Description	145
6.117 TY_TRIGGER_PARAM_EX Struct Reference	146
6.117.1 Detailed Description	146
6.118 TY_TRIGGER_TIMER_LIST Struct Reference	146
6.118.1 Detailed Description	146
6.119 TY_TRIGGER_TIMER_PERIOD Struct Reference	147
6.119.1 Detailed Description	147
6.120 TY_VECT_3F Struct Reference	147
6.120.1 Detailed Description	147
6.121 TY_VERSION_INFO Struct Reference	147
6.121.1 Detailed Description	148
6.122 TYDecodeResult Struct Reference	148
6.122.1 Detailed Description	148

6.123 TYEnumEntry Struct Reference	148
6.123.1 Detailed Description	148
6.124 TYImageInfo Struct Reference	149
6.124.1 Detailed Description	149
6.125 z_stream_s Struct Reference	149
6.125.1 Detailed Description	149
7 File Documentation	151
7.1 TYApi.h File Reference	151
7.1.1 Detailed Description	155
7.1.2 Function Documentation	155
7.1.2.1 TYCfgLogFile()	156
7.1.2.2 TYCfgLogServer()	156
7.1.2.3 TYClearBufferQueue()	157
7.1.2.4 TYCloseDevice()	157
7.1.2.5 TYCloseInterface()	158
7.1.2.6 TYDeinitLib()	159
7.1.2.7 TYDisableComponents()	159
7.1.2.8 TYDisableLog()	160
7.1.2.9 TYEnableComponents()	160
7.1.2.10 TYEnableLog()	161
7.1.2.11 TYEnqueueBuffer()	162
7.1.2.12 TYErrorString()	163
7.1.2.13 TYFetchFrame()	163
7.1.2.14 TYForceDeviceIP()	164
7.1.2.15 TYGetBool()	166
7.1.2.16 TYGetByteArray()	167
7.1.2.17 TYGetByteArrayAttr()	168
7.1.2.18 TYGetByteArraySize()	170
7.1.2.19 TYGetComponentIDs()	171
7.1.2.20 TYGetDeviceFeatureInfo()	172
7.1.2.21 TYGetDeviceFeatureNumber()	173
7.1.2.22 TYGetDeviceInfo()	174
7.1.2.23 TYGetDeviceInterface()	175
7.1.2.24 TYGetDeviceList()	176
7.1.2.25 TYGetDeviceNumber()	177
7.1.2.26 TYGetDeviceXML()	177
7.1.2.27 TYGetDeviceXMLSize()	178
7.1.2.28 TYGetEnabledComponents()	179
7.1.2.29 TYGetEnum()	180
7.1.2.30 TYGetEnumEntryCount()	181
7.1.2.31 TYGetEnumEntryInfo()	182

7.1.2.32 TYGetFeatureInfo()	184
7.1.2.33 TYGetFloat()	185
7.1.2.34 TYGetFloatRange()	186
7.1.2.35 TYGetFrameBufferSize()	187
7.1.2.36 TYGetInt()	188
7.1.2.37 TYGetInterfaceList()	190
7.1.2.38 TYGetInterfaceNumber()	190
7.1.2.39 TYGetIntRange()	191
7.1.2.40 TYGetString()	192
7.1.2.41 TYGetStringLength()	193
7.1.2.42 TYGetStruct()	196
7.1.2.43 TYHasDevice()	197
7.1.2.44 TYHasFeature()	198
7.1.2.45 TYHasInterface()	199
7.1.2.46 TYLibVersion()	200
7.1.2.47 TYOpenDevice()	200
7.1.2.48 TYOpenDeviceWithIP()	202
7.1.2.49 TYOpenInterface()	203
7.1.2.50 TYRegisterEventCallback()	204
7.1.2.51 TYRegisterImuCallback()	204
7.1.2.52 TYSendSoftTrigger()	205
7.1.2.53 TYSetBool()	206
7.1.2.54 TYSetByteArray()	207
7.1.2.55 TYSetEnum()	209
7.1.2.56 TYSetFloat()	210
7.1.2.57 TYSetInt()	212
7.1.2.58 TYSetLogLevel()	213
7.1.2.59 TYSetLogPrefix()	214
7.1.2.60 TYSetString()	214
7.1.2.61 TYSetStruct()	216
7.1.2.62 TYStartCapture()	217
7.1.2.63 TYStopCapture()	218
7.1.2.64 TYUpdateAllDeviceList()	219
7.1.2.65 TYUpdateDeviceList()	219
7.1.2.66 TYUpdateInterfaceList()	220
7.2 TYCoordinateMapper.h File Reference	220
7.2.1 Detailed Description	222
7.2.2 Function Documentation	222
7.2.2.1 TYInvertExtrinsic()	223
7.2.2.2 TYMapDepthImageToColorCoordinate()	223
7.2.2.3 TYMapDepthImageToPoint3d()	224
7.2.2.4 TYMapDepthToColorCoordinate()	224

7.2.2.5 TYMapDepthToPoint3d()	225
7.2.2.6 TYMapMono16ImageToDepthCoordinate()	225
7.2.2.7 TYMapMono8ImageToDepthCoordinate()	226
7.2.2.8 TYMapPoint3dToDepth()	227
7.2.2.9 TYMapPoint3dToDepthImage()	227
7.2.2.10 TYMapPoint3dToPoint3d()	228
7.2.2.11 TYMapRGB48ImageToDepthCoordinate()	228
7.2.2.12 TYMapRGBImageToDepthCoordinate()	229
7.2.2.13 TYMapRGBPixelsToDepthCoordinate()	230
7.3 TYDefs.h File Reference	231
7.3.1 Detailed Description	241
7.3.2 Macro Definition Documentation	241
7.3.2.1 TY_DECLARE_IMAGE_MODE1	241
7.3.3 Typedef Documentation	242
7.3.3.1 TY_ACC_BIAS	242
7.3.3.2 TY_ACC_MISALIGNMENT	242
7.3.3.3 TY_ACC_SCALE	242
7.3.3.4 TY_ACCESS_MODE_LIST	242
7.3.3.5 TY_BYTEARRAY_ATTR	243
7.3.3.6 TY_CAMERA_CALIB_INFO	243
7.3.3.7 TY_CAMERA_DISTORTION	243
7.3.3.8 TY_CAMERA_EXTRINSIC	243
7.3.3.9 TY_CAMERA_INTRINSIC	244
7.3.3.10 TY_CAMERA_ROTATION	244
7.3.3.11 TY_CAMERA_TO_IMU	245
7.3.3.12 TY_COMPONENT_ID	245
7.3.3.13 TY_DEVICE_BASE_INFO	245
7.3.3.14 TY_DEVICE_COMPONENT_LIST	245
7.3.3.15 TY_ENUM_ENTRY	246
7.3.3.16 TY_FEATURE_ID	246
7.3.3.17 TY_FLOAT_RANGE	246
7.3.3.18 TY_GYRO_BIAS	246
7.3.3.19 TY_GYRO_MISALIGNMENT	247
7.3.3.20 TY_GYRO_SCALE	247
7.3.3.21 TY_INTERFACE_INFO	247
7.3.3.22 TY_INTERFACE_TYPE_LIST	247
7.3.3.23 TY_PIXEL_BITS_LIST	248
7.3.3.24 TY_TRIGGER_MODE_LIST	248
7.3.4 Enumeration Type Documentation	248
7.3.4.1 TY_ACCESS_MODE_LIST	248
7.3.4.2 TY_DEVICE_COMPONENT_LIST	248
7.3.4.3 TY_FEATURE_ID_LIST	249

7.3.4.4 TY_INTERFACE_TYPE_LIST	252
7.3.4.5 TY_PIXEL_BITS_LIST	252
7.3.4.6 TY_PIXEL_FORMAT_LIST	252
7.3.4.7 TY_RESOLUTION_MODE_LIST	253
7.3.4.8 TY_TRIGGER_MODE_LIST	254
7.4 TYFeatureList.h File Reference	254
7.4.1 Detailed Description	263
7.4.2 Enumeration Type Documentation	263
7.4.2.1 TY_ACQ_MODE_LIST	263
7.4.2.2 TY_BIN_H_LIST	263
7.4.2.3 TY_BIN_V_LIST	264
7.4.2.4 TY_DEPTH_TOF_QUALITY_LIST	264
7.4.2.5 TY_DEV_CHARSET_LIST	264
7.4.2.6 TY_DEV_COMPRESS_LIST	265
7.4.2.7 TY_DEV_LINK_HB_MODE_LIST	265
7.4.2.8 TY_DEV_STREAM_ASYNC_LIST	265
7.4.2.9 TY_DEV_STREAM_CH_TYPE_LIST	266
7.4.2.10 TY_DEV_TEMP_SEL_LIST	266
7.4.2.11 TY_DEV_TIME_SYNC_LIST	266
7.4.2.12 TY_DEV_TYPE_LIST	267
7.4.2.13 TY_GAIN_SEL_LIST	267
7.4.2.14 TY_GEV_CCP_LIST	267
7.4.2.15 TY_LIGHT_SEL_LIST	267
7.4.2.16 TY_PIX_FMT_LIST	268
7.4.2.17 TY_SRC_SEL_LIST	268
7.4.2.18 TY_TRIG_ACT_LIST	269
7.4.2.19 TY_TRIG_MODE_LIST	269
7.4.2.20 TY_TRIG_SEL_LIST	269
7.4.2.21 TY_TRIG_SRC_LIST	270
7.4.2.22 TY_USER_SET_SEL_LIST	270
7.5 TYImageProc.h File Reference	271
7.5.1 Detailed Description	273
7.5.2 Function Documentation	273
7.5.2.1 TYDecodeImage()	273
7.5.2.2 TYDepthEnhanceFilter()	274
7.5.2.3 TYDepthImageInpainter()	274
7.5.2.4 TYDepthSpeckleFilter()	275
7.5.2.5 TYGetDecodeBufferSize()	275
7.5.2.6 TYGetDecodeTargetPixFmt()	276
7.5.2.7 TYGetImageAlgorithmVersion()	277
7.5.2.8 TYImageProcesAcceEnable()	277
7.5.2.9 TYUndistortImage()	277

7.5.2.10 TYUndistortImage2()	278
------------------------------	-----

Index	279
--------------	------------

Chapter 1

Main Page

Copyright(C)2016-2023 Percipio All Rights Reserved

1.1 Note

Depth camera, called "device", consists of several components. Each component is a hardware module or virtual module, such as RGB sensor, depth sensor. Each component has its own features, such as image width, exposure time, etc..

NOTE: The component TY_COMPONENT_DEVICE is a virtual component that contains all features related to the whole device, such as trigger mode, device IP.

Each frame consists of several images. Normally, all the images have identical timestamp, means they are captured at the same time.

Chapter 2

Module Index

2.1 Modules

Here is a list of all modules:

TurboJPEG	53
---------------------	----

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AlgVersion	87
DepthEnhenceParameters	87
DepthInpainterParameters	88
DepthSpeckleFilterParameters	88
f16	88
gz_header_s	89
gzFile_s	89
JHUFF_TBL	90
jpeg_common_struct	90
jpeg_component_info	90
jpeg_compress_struct	91
jpeg_decompress_struct	93
jpeg_destination_mgr	95
jpeg_error_mgr	96
jpeg_marker_struct	96
jpeg_memory_mgr	97
jpeg_progress_mgr	98
jpeg_scan_info	98
jpeg_source_mgr	98
JQUANT_TBL	99
nsimd_aarch64_vf16	99
nsimd_aarch64_vlf16	99
nsimd_avx2_vf16	100
nsimd_avx2_vlf16	100
nsimd_avx512_knl_vf16	100
nsimd_avx512_knl_vlf16	101
nsimd_avx512_skylake_vf16	101
nsimd_avx512_skylake_vlf16	101
nsimd_avx_vf16	102
nsimd_avx_vlf16	102
nsimd_cpu_vf16	103
nsimd_cpu_vf32	103
nsimd_cpu_vf64	103
nsimd_cpu_vf16	104
nsimd_cpu_vf32	104

nsimd_cpu_vf64	105
nsimd_cpu_vf8	105
nsimd_cpu_vlf16	106
nsimd_cpu_vlf32	106
nsimd_cpu_vlf64	106
nsimd_cpu_vli16	107
nsimd_cpu_vli32	107
nsimd_cpu_vli64	108
nsimd_cpu_vli8	108
nsimd_cpu_vlu16	109
nsimd_cpu_vlu32	109
nsimd_cpu_vlu64	109
nsimd_cpu_vlu8	110
nsimd_cpu_vu16	110
nsimd_cpu_vu32	111
nsimd_cpu_vu64	111
nsimd_cpu_vu8	112
nsimd_neon128_vf16	112
nsimd_neon128_vf64	113
nsimd_neon128_vlf16	113
nsimd_neon128_vlf64	113
nsimd_sse2_vf16	114
nsimd_sse2_vlf16	114
nsimd_sse42_vf16	114
nsimd_sse42_vlf16	115
nsimd_vmx_vf16	115
nsimd_vmx_vf64	115
nsimd_vmx_vf64	116
nsimd_vmx_vlf16	116
nsimd_vmx_vlf64	116
nsimd_vmx_vli64	117
nsimd_vmx_vlu64	117
nsimd_vmx_vu64	117
nsimd_vsx_vf16	118
nsimd_vsx_vlf16	118
OperatorInfo	118
ParamDetail	119
pattern_bin_param	120
pattern_gray_param	120
pattern_sine_param	120
TY_PHC_GROUP_ATTR::phc_group_attr	121
tjregion	121
tjscalingfactor	122
tjtransform	123
TY_ACC_BIAS	125
TY_ACC_MISALIGNMENT	126
TY_ACC_SCALE	126
TY_AEC_ROI_PARAM	127
TY_BYTEARRAY_ATTR	
Byte array data structure	127
TY_CAMERA_CALIB_INFO	128
TY_CAMERA_DISTORTION	129
TY_CAMERA_EXTRINSIC	129
TY_CAMERA_INTRINSIC	130
TY_CAMERA_ROTATION	131
TY_CAMERA_STATISTICS	131
TY_CAMERA_TO_IMU	132
TY_DEVICE_BASE_INFO	132

TY_DEVICE_NET_INFO	
Device network information	133
TY_DEVICE_USB_INFO	134
TY_DI_WORKMODE	134
TY_DO_WORKMODE	135
TY_ENUM_ENTRY	135
TY_EVENT_INFO	136
TY_FEATURE_INFO	136
TY_FLOAT_RANGE	
Float range data structure	137
TY_FRAME_DATA	137
TY_GYRO_BIAS	138
TY_GYRO_MISALIGNMENT	138
TY_GYRO_SCALE	139
TY_IMAGE_DATA	140
TY_IMU_DATA	140
TY_INT_RANGE	141
TY_INTERFACE_INFO	141
TY_LASER_PARAM	142
TY_LASER_PATTERN_PARAM	142
TY_PHC_GROUP_ATTR	143
TY_PIXEL_COLOR_DESC	144
TY_PIXEL_DESC	144
TY_TEMP_DATA	145
TY_TOF_FREQ	145
TY_TRIGGER_PARAM	145
TY_TRIGGER_PARAM_EX	146
TY_TRIGGER_TIMER_LIST	146
TY_TRIGGER_TIMER_PERIOD	147
TY_VECT_3F	147
TY_VERSION_INFO	147
TYDecodeResult	148
TYEnumEntry	148
TYImageInfo	149
z_stream_s	149

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

arm/aarch64/abs.h	??
arm/neon128/abs.h	??
arm/sve/abs.h	??
arm/sve1024/abs.h	??
arm/sve128/abs.h	??
arm/sve2048/abs.h	??
arm/sve256/abs.h	??
arm/sve512/abs.h	??
cpu/cpu/abs.h	??
ppc/vmx/abs.h	??
ppc/vsx/abs.h	??
x86/avx/abs.h	??
x86/avx2/abs.h	??
x86/avx512_knl/abs.h	??
x86/avx512_skylake/abs.h	??
x86/sse2/abs.h	??
x86/sse42/abs.h	??
arm/aarch64/acos_u10.h	??
arm/neon128/acos_u10.h	??
arm/sve/acos_u10.h	??
arm/sve1024/acos_u10.h	??
arm/sve128/acos_u10.h	??
arm/sve2048/acos_u10.h	??
arm/sve256/acos_u10.h	??
arm/sve512/acos_u10.h	??
cpu/cpu/acos_u10.h	??
ppc/vmx/acos_u10.h	??
ppc/vsx/acos_u10.h	??
x86/avx/acos_u10.h	??
x86/avx2/acos_u10.h	??
x86/avx512_knl/acos_u10.h	??
x86/avx512_skylake/acos_u10.h	??
x86/sse2/acos_u10.h	??
x86/sse42/acos_u10.h	??
arm/aarch64/acos_u35.h	??

arm/neon128/acos_u35.h	??
arm/sve/acos_u35.h	??
arm/sve1024/acos_u35.h	??
arm/sve128/acos_u35.h	??
arm/sve2048/acos_u35.h	??
arm/sve256/acos_u35.h	??
arm/sve512/acos_u35.h	??
cpu/cpu/acos_u35.h	??
ppc/vmx/acos_u35.h	??
ppc/vsx/acos_u35.h	??
x86/avx/acos_u35.h	??
x86/avx2/acos_u35.h	??
x86/avx512_knl/acos_u35.h	??
x86/avx512_skylake/acos_u35.h	??
x86/sse2/acos_u35.h	??
x86/sse42/acos_u35.h	??
arm/aarch64/acosh_u10.h	??
arm/neon128/acosh_u10.h	??
arm/sve/acosh_u10.h	??
arm/sve1024/acosh_u10.h	??
arm/sve128/acosh_u10.h	??
arm/sve2048/acosh_u10.h	??
arm/sve256/acosh_u10.h	??
arm/sve512/acosh_u10.h	??
cpu/cpu/acosh_u10.h	??
ppc/vmx/acosh_u10.h	??
ppc/vsx/acosh_u10.h	??
x86/avx/acosh_u10.h	??
x86/avx2/acosh_u10.h	??
x86/avx512_knl/acosh_u10.h	??
x86/avx512_skylake/acosh_u10.h	??
x86/sse2/acosh_u10.h	??
x86/sse42/acosh_u10.h	??
arm/aarch64/add.h	??
arm/neon128/add.h	??
arm/sve/add.h	??
arm/sve1024/add.h	??
arm/sve128/add.h	??
arm/sve2048/add.h	??
arm/sve256/add.h	??
arm/sve512/add.h	??
cpu/cpu/add.h	??
ppc/vmx/add.h	??
ppc/vsx/add.h	??
x86/avx/add.h	??
x86/avx2/add.h	??
x86/avx512_knl/add.h	??
x86/avx512_skylake/add.h	??
x86/sse2/add.h	??
x86/sse42/add.h	??
arm/aarch64/adds.h	??
arm/neon128/adds.h	??
arm/sve/adds.h	??
arm/sve1024/adds.h	??
arm/sve128/adds.h	??
arm/sve2048/adds.h	??
arm/sve256/adds.h	??
arm/sve512/adds.h	??

cpu/cpu/adds.h	??
ppc/vmx/adds.h	??
ppc/vsx/adds.h	??
x86/avx/adds.h	??
x86/avx2/adds.h	??
x86/avx512_knl/adds.h	??
x86/avx512_skylake/adds.h	??
x86/sse2/adds.h	??
x86/sse42/adds.h	??
arm/aarch64/addv.h	??
arm/neon128/addv.h	??
arm/sve/addv.h	??
arm/sve1024/addv.h	??
arm/sve128/addv.h	??
arm/sve2048/addv.h	??
arm/sve256/addv.h	??
arm/sve512/addv.h	??
cpu/cpu/addv.h	??
ppc/vmx/addv.h	??
ppc/vsx/addv.h	??
x86/avx/addv.h	??
x86/avx2/addv.h	??
x86/avx512_knl/addv.h	??
x86/avx512_skylake/addv.h	??
x86/sse2/addv.h	??
x86/sse42/addv.h	??
arm/aarch64/all.h	??
arm/neon128/all.h	??
arm/sve/all.h	??
arm/sve1024/all.h	??
arm/sve128/all.h	??
arm/sve2048/all.h	??
arm/sve256/all.h	??
arm/sve512/all.h	??
cpu/cpu/all.h	??
ppc/vmx/all.h	??
ppc/vsx/all.h	??
x86/avx/all.h	??
x86/avx2/all.h	??
x86/avx512_knl/all.h	??
x86/avx512_skylake/all.h	??
x86/sse2/all.h	??
x86/sse42/all.h	??
arm/aarch64/andb.h	??
arm/neon128/andb.h	??
arm/sve/andb.h	??
arm/sve1024/andb.h	??
arm/sve128/andb.h	??
arm/sve2048/andb.h	??
arm/sve256/andb.h	??
arm/sve512/andb.h	??
cpu/cpu/andb.h	??
ppc/vmx/andb.h	??
ppc/vsx/andb.h	??
x86/avx/andb.h	??
x86/avx2/andb.h	??
x86/avx512_knl/andb.h	??
x86/avx512_skylake/andb.h	??

x86/sse2/andb.h	??
x86/sse42/andb.h	??
arm/aarch64/andl.h	??
arm/neon128/andl.h	??
arm/sve/andl.h	??
arm/sve1024/andl.h	??
arm/sve128/andl.h	??
arm/sve2048/andl.h	??
arm/sve256/andl.h	??
arm/sve512/andl.h	??
cpu/cpu/andl.h	??
ppc/vmx/andl.h	??
ppc/vsx/andl.h	??
x86/avx/andl.h	??
x86/avx2/andl.h	??
x86/avx512_knl/andl.h	??
x86/avx512_skylake/andl.h	??
x86/sse2/andl.h	??
x86/sse42/andl.h	??
arm/aarch64/andnotb.h	??
arm/neon128/andnotb.h	??
arm/sve/andnotb.h	??
arm/sve1024/andnotb.h	??
arm/sve128/andnotb.h	??
arm/sve2048/andnotb.h	??
arm/sve256/andnotb.h	??
arm/sve512/andnotb.h	??
cpu/cpu/andnotb.h	??
ppc/vmx/andnotb.h	??
ppc/vsx/andnotb.h	??
x86/avx/andnotb.h	??
x86/avx2/andnotb.h	??
x86/avx512_knl/andnotb.h	??
x86/avx512_skylake/andnotb.h	??
x86/sse2/andnotb.h	??
x86/sse42/andnotb.h	??
arm/aarch64/andnotl.h	??
arm/neon128/andnotl.h	??
arm/sve/andnotl.h	??
arm/sve1024/andnotl.h	??
arm/sve128/andnotl.h	??
arm/sve2048/andnotl.h	??
arm/sve256/andnotl.h	??
arm/sve512/andnotl.h	??
cpu/cpu/andnotl.h	??
ppc/vmx/andnotl.h	??
ppc/vsx/andnotl.h	??
x86/avx/andnotl.h	??
x86/avx2/andnotl.h	??
x86/avx512_knl/andnotl.h	??
x86/avx512_skylake/andnotl.h	??
x86/sse2/andnotl.h	??
x86/sse42/andnotl.h	??
arm/aarch64/any.h	??
arm/neon128/any.h	??
arm/sve/any.h	??
arm/sve1024/any.h	??
arm/sve128/any.h	??

arm/sve2048/any.h	??
arm/sve256/any.h	??
arm/sve512/any.h	??
cpu/cpu/any.h	??
ppc/vmx/any.h	??
ppc/vsx/any.h	??
x86/avx/any.h	??
x86/avx2/any.h	??
x86/avx512_knl/any.h	??
x86/avx512_skylake/any.h	??
x86/sse2/any.h	??
x86/sse42/any.h	??
arm/aarch64/asin_u10.h	??
arm/neon128/asin_u10.h	??
arm/sve/asin_u10.h	??
arm/sve1024/asin_u10.h	??
arm/sve128/asin_u10.h	??
arm/sve2048/asin_u10.h	??
arm/sve256/asin_u10.h	??
arm/sve512/asin_u10.h	??
cpu/cpu/asin_u10.h	??
ppc/vmx/asin_u10.h	??
ppc/vsx/asin_u10.h	??
x86/avx/asin_u10.h	??
x86/avx2/asin_u10.h	??
x86/avx512_knl/asin_u10.h	??
x86/avx512_skylake/asin_u10.h	??
x86/sse2/asin_u10.h	??
x86/sse42/asin_u10.h	??
arm/aarch64/asin_u35.h	??
arm/neon128/asin_u35.h	??
arm/sve/asin_u35.h	??
arm/sve1024/asin_u35.h	??
arm/sve128/asin_u35.h	??
arm/sve2048/asin_u35.h	??
arm/sve256/asin_u35.h	??
arm/sve512/asin_u35.h	??
cpu/cpu/asin_u35.h	??
ppc/vmx/asin_u35.h	??
ppc/vsx/asin_u35.h	??
x86/avx/asin_u35.h	??
x86/avx2/asin_u35.h	??
x86/avx512_knl/asin_u35.h	??
x86/avx512_skylake/asin_u35.h	??
x86/sse2/asin_u35.h	??
x86/sse42/asin_u35.h	??
arm/aarch64/asinh_u10.h	??
arm/neon128/asinh_u10.h	??
arm/sve/asinh_u10.h	??
arm/sve1024/asinh_u10.h	??
arm/sve128/asinh_u10.h	??
arm/sve2048/asinh_u10.h	??
arm/sve256/asinh_u10.h	??
arm/sve512/asinh_u10.h	??
cpu/cpu/asinh_u10.h	??
ppc/vmx/asinh_u10.h	??
ppc/vsx/asinh_u10.h	??
x86/avx/asinh_u10.h	??

x86/avx2/asinh_u10.h	??
x86/avx512_knl/asinh_u10.h	??
x86/avx512_skylake/asinh_u10.h	??
x86/sse2/asinh_u10.h	??
x86/sse42/asinh_u10.h	??
arm/aarch64/atan2_u10.h	??
arm/neon128/atan2_u10.h	??
arm/sve/atan2_u10.h	??
arm/sve1024/atan2_u10.h	??
arm/sve128/atan2_u10.h	??
arm/sve2048/atan2_u10.h	??
arm/sve256/atan2_u10.h	??
arm/sve512/atan2_u10.h	??
cpu/cpu/atan2_u10.h	??
ppc/vmx/atan2_u10.h	??
ppc/vsx/atan2_u10.h	??
x86/avx/atan2_u10.h	??
x86/avx2/atan2_u10.h	??
x86/avx512_knl/atan2_u10.h	??
x86/avx512_skylake/atan2_u10.h	??
x86/sse2/atan2_u10.h	??
x86/sse42/atan2_u10.h	??
arm/aarch64/atan2_u35.h	??
arm/neon128/atan2_u35.h	??
arm/sve/atan2_u35.h	??
arm/sve1024/atan2_u35.h	??
arm/sve128/atan2_u35.h	??
arm/sve2048/atan2_u35.h	??
arm/sve256/atan2_u35.h	??
arm/sve512/atan2_u35.h	??
cpu/cpu/atan2_u35.h	??
ppc/vmx/atan2_u35.h	??
ppc/vsx/atan2_u35.h	??
x86/avx/atan2_u35.h	??
x86/avx2/atan2_u35.h	??
x86/avx512_knl/atan2_u35.h	??
x86/avx512_skylake/atan2_u35.h	??
x86/sse2/atan2_u35.h	??
x86/sse42/atan2_u35.h	??
arm/aarch64/atan_u10.h	??
arm/neon128/atan_u10.h	??
arm/sve/atan_u10.h	??
arm/sve1024/atan_u10.h	??
arm/sve128/atan_u10.h	??
arm/sve2048/atan_u10.h	??
arm/sve256/atan_u10.h	??
arm/sve512/atan_u10.h	??
cpu/cpu/atan_u10.h	??
ppc/vmx/atan_u10.h	??
ppc/vsx/atan_u10.h	??
x86/avx/atan_u10.h	??
x86/avx2/atan_u10.h	??
x86/avx512_knl/atan_u10.h	??
x86/avx512_skylake/atan_u10.h	??
x86/sse2/atan_u10.h	??
x86/sse42/atan_u10.h	??
arm/aarch64/atan_u35.h	??
arm/neon128/atan_u35.h	??

arm/sve/atan_u35.h	??
arm/sve1024/atan_u35.h	??
arm/sve128/atan_u35.h	??
arm/sve2048/atan_u35.h	??
arm/sve256/atan_u35.h	??
arm/sve512/atan_u35.h	??
cpu/cpu/atan_u35.h	??
ppc/vmx/atan_u35.h	??
ppc/vsx/atan_u35.h	??
x86/avx/atan_u35.h	??
x86/avx2/atan_u35.h	??
x86/avx512_knl/atan_u35.h	??
x86/avx512_skylake/atan_u35.h	??
x86/sse2/atan_u35.h	??
x86/sse42/atan_u35.h	??
arm/aarch64/atanh_u10.h	??
arm/neon128/atanh_u10.h	??
arm/sve/atanh_u10.h	??
arm/sve1024/atanh_u10.h	??
arm/sve128/atanh_u10.h	??
arm/sve2048/atanh_u10.h	??
arm/sve256/atanh_u10.h	??
arm/sve512/atanh_u10.h	??
cpu/cpu/atanh_u10.h	??
ppc/vmx/atanh_u10.h	??
ppc/vsx/atanh_u10.h	??
x86/avx/atanh_u10.h	??
x86/avx2/atanh_u10.h	??
x86/avx512_knl/atanh_u10.h	??
x86/avx512_skylake/atanh_u10.h	??
x86/sse2/atanh_u10.h	??
x86/sse42/atanh_u10.h	??
c_adv_api.h	??
c_adv_api_functions.h	??
arm/aarch64/cbrt_u10.h	??
arm/neon128/cbrt_u10.h	??
arm/sve/cbrt_u10.h	??
arm/sve1024/cbrt_u10.h	??
arm/sve128/cbrt_u10.h	??
arm/sve2048/cbrt_u10.h	??
arm/sve256/cbrt_u10.h	??
arm/sve512/cbrt_u10.h	??
cpu/cpu/cbrt_u10.h	??
ppc/vmx/cbrt_u10.h	??
ppc/vsx/cbrt_u10.h	??
x86/avx/cbrt_u10.h	??
x86/avx2/cbrt_u10.h	??
x86/avx512_knl/cbrt_u10.h	??
x86/avx512_skylake/cbrt_u10.h	??
x86/sse2/cbrt_u10.h	??
x86/sse42/cbrt_u10.h	??
arm/aarch64/cbrt_u35.h	??
arm/neon128/cbrt_u35.h	??
arm/sve/cbrt_u35.h	??
arm/sve1024/cbrt_u35.h	??
arm/sve128/cbrt_u35.h	??
arm/sve2048/cbrt_u35.h	??
arm/sve256/cbrt_u35.h	??

arm/sve512/cbrt_u35.h	??
cpu/cpu/cbrt_u35.h	??
ppc/vmx/cbrt_u35.h	??
ppc/vsx/cbrt_u35.h	??
x86/avx/cbrt_u35.h	??
x86/avx2/cbrt_u35.h	??
x86/avx512_knl/cbrt_u35.h	??
x86/avx512_skylake/cbrt_u35.h	??
x86/sse2/cbrt_u35.h	??
x86/sse42/cbrt_u35.h	??
arm/aarch64/ceil.h	??
arm/neon128/ceil.h	??
arm/sve/ceil.h	??
arm/sve1024/ceil.h	??
arm/sve128/ceil.h	??
arm/sve2048/ceil.h	??
arm/sve256/ceil.h	??
arm/sve512/ceil.h	??
cpu/cpu/ceil.h	??
ppc/vmx/ceil.h	??
ppc/vsx/ceil.h	??
x86/avx/ceil.h	??
x86/avx2/ceil.h	??
x86/avx512_knl/ceil.h	??
x86/avx512_skylake/ceil.h	??
x86/sse2/ceil.h	??
x86/sse42/ceil.h	??
arm/aarch64/cos_u10.h	??
arm/neon128/cos_u10.h	??
arm/sve/cos_u10.h	??
arm/sve1024/cos_u10.h	??
arm/sve128/cos_u10.h	??
arm/sve2048/cos_u10.h	??
arm/sve256/cos_u10.h	??
arm/sve512/cos_u10.h	??
cpu/cpu/cos_u10.h	??
ppc/vmx/cos_u10.h	??
ppc/vsx/cos_u10.h	??
x86/avx/cos_u10.h	??
x86/avx2/cos_u10.h	??
x86/avx512_knl/cos_u10.h	??
x86/avx512_skylake/cos_u10.h	??
x86/sse2/cos_u10.h	??
x86/sse42/cos_u10.h	??
arm/aarch64/cos_u35.h	??
arm/neon128/cos_u35.h	??
arm/sve/cos_u35.h	??
arm/sve1024/cos_u35.h	??
arm/sve128/cos_u35.h	??
arm/sve2048/cos_u35.h	??
arm/sve256/cos_u35.h	??
arm/sve512/cos_u35.h	??
cpu/cpu/cos_u35.h	??
ppc/vmx/cos_u35.h	??
ppc/vsx/cos_u35.h	??
x86/avx/cos_u35.h	??
x86/avx2/cos_u35.h	??
x86/avx512_knl/cos_u35.h	??

x86/avx512_skylake/cos_u35.h	??
x86/sse2/cos_u35.h	??
x86/sse42/cos_u35.h	??
arm/aarch64/cosh_u10.h	??
arm/neon128/cosh_u10.h	??
arm/sve/cosh_u10.h	??
arm/sve1024/cosh_u10.h	??
arm/sve128/cosh_u10.h	??
arm/sve2048/cosh_u10.h	??
arm/sve256/cosh_u10.h	??
arm/sve512/cosh_u10.h	??
cpu/cpu/cosh_u10.h	??
ppc/vmx/cosh_u10.h	??
ppc/vsx/cosh_u10.h	??
x86/avx/cosh_u10.h	??
x86/avx2/cosh_u10.h	??
x86/avx512_knl/cosh_u10.h	??
x86/avx512_skylake/cosh_u10.h	??
x86/sse2/cosh_u10.h	??
x86/sse42/cosh_u10.h	??
arm/aarch64/cosh_u35.h	??
arm/neon128/cosh_u35.h	??
arm/sve/cosh_u35.h	??
arm/sve1024/cosh_u35.h	??
arm/sve128/cosh_u35.h	??
arm/sve2048/cosh_u35.h	??
arm/sve256/cosh_u35.h	??
arm/sve512/cosh_u35.h	??
cpu/cpu/cosh_u35.h	??
ppc/vmx/cosh_u35.h	??
ppc/vsx/cosh_u35.h	??
x86/avx/cosh_u35.h	??
x86/avx2/cosh_u35.h	??
x86/avx512_knl/cosh_u35.h	??
x86/avx512_skylake/cosh_u35.h	??
x86/sse2/cosh_u35.h	??
x86/sse42/cosh_u35.h	??
arm/aarch64/cospi_u05.h	??
arm/neon128/cospi_u05.h	??
arm/sve/cospi_u05.h	??
arm/sve1024/cospi_u05.h	??
arm/sve128/cospi_u05.h	??
arm/sve2048/cospi_u05.h	??
arm/sve256/cospi_u05.h	??
arm/sve512/cospi_u05.h	??
cpu/cpu/cospi_u05.h	??
ppc/vmx/cospi_u05.h	??
ppc/vsx/cospi_u05.h	??
x86/avx/cospi_u05.h	??
x86/avx2/cospi_u05.h	??
x86/avx512_knl/cospi_u05.h	??
x86/avx512_skylake/cospi_u05.h	??
x86/sse2/cospi_u05.h	??
x86/sse42/cospi_u05.h	??
arm/aarch64/cvt.h	??
arm/neon128/cvt.h	??
arm/sve/cvt.h	??
arm/sve1024/cvt.h	??

arm/sve128/cvt.h	??
arm/sve2048/cvt.h	??
arm/sve256/cvt.h	??
arm/sve512/cvt.h	??
cpu/cpu/cvt.h	??
ppc/vmx/cvt.h	??
ppc/vsx/cvt.h	??
x86/avx/cvt.h	??
x86/avx2/cvt.h	??
x86/avx512_knl/cvt.h	??
x86/avx512_skylake/cvt.h	??
x86/sse2/cvt.h	??
x86/sse42/cvt.h	??
arm/aarch64/div.h	??
arm/neon128/div.h	??
arm/sve/div.h	??
arm/sve1024/div.h	??
arm/sve128/div.h	??
arm/sve2048/div.h	??
arm/sve256/div.h	??
arm/sve512/div.h	??
cpu/cpu/div.h	??
ppc/vmx/div.h	??
ppc/vsx/div.h	??
x86/avx/div.h	??
x86/avx2/div.h	??
x86/avx512_knl/div.h	??
x86/avx512_skylake/div.h	??
x86/sse2/div.h	??
x86/sse42/div.h	??
arm/aarch64/downcvt.h	??
arm/neon128/downcvt.h	??
arm/sve/downcvt.h	??
arm/sve1024/downcvt.h	??
arm/sve128/downcvt.h	??
arm/sve2048/downcvt.h	??
arm/sve256/downcvt.h	??
arm/sve512/downcvt.h	??
cpu/cpu/downcvt.h	??
ppc/vmx/downcvt.h	??
ppc/vsx/downcvt.h	??
x86/avx/downcvt.h	??
x86/avx2/downcvt.h	??
x86/avx512_knl/downcvt.h	??
x86/avx512_skylake/downcvt.h	??
x86/sse2/downcvt.h	??
x86/sse42/downcvt.h	??
arm/aarch64/eq.h	??
arm/neon128/eq.h	??
arm/sve/eq.h	??
arm/sve1024/eq.h	??
arm/sve128/eq.h	??
arm/sve2048/eq.h	??
arm/sve256/eq.h	??
arm/sve512/eq.h	??
cpu/cpu/eq.h	??
ppc/vmx/eq.h	??
ppc/vsx/eq.h	??

x86/avx/eq.h	??
x86/avx2/eq.h	??
x86/avx512_knl/eq.h	??
x86/avx512_skylake/eq.h	??
x86/sse2/eq.h	??
x86/sse42/eq.h	??
arm/aarch64/erf_u10.h	??
arm/neon128/erf_u10.h	??
arm/sve/erf_u10.h	??
arm/sve1024/erf_u10.h	??
arm/sve128/erf_u10.h	??
arm/sve2048/erf_u10.h	??
arm/sve256/erf_u10.h	??
arm/sve512/erf_u10.h	??
cpu/cpu/erf_u10.h	??
ppc/vmx/erf_u10.h	??
ppc/vsx/erf_u10.h	??
x86/avx/erf_u10.h	??
x86/avx2/erf_u10.h	??
x86/avx512_knl/erf_u10.h	??
x86/avx512_skylake/erf_u10.h	??
x86/sse2/erf_u10.h	??
x86/sse42/erf_u10.h	??
arm/aarch64/erfc_u15.h	??
arm/neon128/erfc_u15.h	??
arm/sve/erfc_u15.h	??
arm/sve1024/erfc_u15.h	??
arm/sve128/erfc_u15.h	??
arm/sve2048/erfc_u15.h	??
arm/sve256/erfc_u15.h	??
arm/sve512/erfc_u15.h	??
cpu/cpu/erfc_u15.h	??
ppc/vmx/erfc_u15.h	??
ppc/vsx/erfc_u15.h	??
x86/avx/erfc_u15.h	??
x86/avx2/erfc_u15.h	??
x86/avx512_knl/erfc_u15.h	??
x86/avx512_skylake/erfc_u15.h	??
x86/sse2/erfc_u15.h	??
x86/sse42/erfc_u15.h	??
arm/aarch64/exp10_u10.h	??
arm/neon128/exp10_u10.h	??
arm/sve/exp10_u10.h	??
arm/sve1024/exp10_u10.h	??
arm/sve128/exp10_u10.h	??
arm/sve2048/exp10_u10.h	??
arm/sve256/exp10_u10.h	??
arm/sve512/exp10_u10.h	??
cpu/cpu/exp10_u10.h	??
ppc/vmx/exp10_u10.h	??
ppc/vsx/exp10_u10.h	??
x86/avx/exp10_u10.h	??
x86/avx2/exp10_u10.h	??
x86/avx512_knl/exp10_u10.h	??
x86/avx512_skylake/exp10_u10.h	??
x86/sse2/exp10_u10.h	??
x86/sse42/exp10_u10.h	??
arm/aarch64/exp10_u35.h	??

arm/neon128/exp10_u35.h	??
arm/sve/exp10_u35.h	??
arm/sve1024/exp10_u35.h	??
arm/sve128/exp10_u35.h	??
arm/sve2048/exp10_u35.h	??
arm/sve256/exp10_u35.h	??
arm/sve512/exp10_u35.h	??
cpu/cpu/exp10_u35.h	??
ppc/vmx/exp10_u35.h	??
ppc/vsx/exp10_u35.h	??
x86/avx/exp10_u35.h	??
x86/avx2/exp10_u35.h	??
x86/avx512_knl/exp10_u35.h	??
x86/avx512_skylake/exp10_u35.h	??
x86/sse2/exp10_u35.h	??
x86/sse42/exp10_u35.h	??
arm/aarch64/exp2_u10.h	??
arm/neon128/exp2_u10.h	??
arm/sve/exp2_u10.h	??
arm/sve1024/exp2_u10.h	??
arm/sve128/exp2_u10.h	??
arm/sve2048/exp2_u10.h	??
arm/sve256/exp2_u10.h	??
arm/sve512/exp2_u10.h	??
cpu/cpu/exp2_u10.h	??
ppc/vmx/exp2_u10.h	??
ppc/vsx/exp2_u10.h	??
x86/avx/exp2_u10.h	??
x86/avx2/exp2_u10.h	??
x86/avx512_knl/exp2_u10.h	??
x86/avx512_skylake/exp2_u10.h	??
x86/sse2/exp2_u10.h	??
x86/sse42/exp2_u10.h	??
arm/aarch64/exp2_u35.h	??
arm/neon128/exp2_u35.h	??
arm/sve/exp2_u35.h	??
arm/sve1024/exp2_u35.h	??
arm/sve128/exp2_u35.h	??
arm/sve2048/exp2_u35.h	??
arm/sve256/exp2_u35.h	??
arm/sve512/exp2_u35.h	??
cpu/cpu/exp2_u35.h	??
ppc/vmx/exp2_u35.h	??
ppc/vsx/exp2_u35.h	??
x86/avx/exp2_u35.h	??
x86/avx2/exp2_u35.h	??
x86/avx512_knl/exp2_u35.h	??
x86/avx512_skylake/exp2_u35.h	??
x86/sse2/exp2_u35.h	??
x86/sse42/exp2_u35.h	??
arm/aarch64/exp_u10.h	??
arm/neon128/exp_u10.h	??
arm/sve/exp_u10.h	??
arm/sve1024/exp_u10.h	??
arm/sve128/exp_u10.h	??
arm/sve2048/exp_u10.h	??
arm/sve256/exp_u10.h	??
arm/sve512/exp_u10.h	??

cpu/cpu/exp_u10.h	??
ppc/vmx/exp_u10.h	??
ppc/vsx/exp_u10.h	??
x86/avx/exp_u10.h	??
x86/avx2/exp_u10.h	??
x86/avx512_knl/exp_u10.h	??
x86/avx512_skylake/exp_u10.h	??
x86/sse2/exp_u10.h	??
x86/sse42/exp_u10.h	??
arm/aarch64/expm1_u10.h	??
arm/neon128/expm1_u10.h	??
arm/sve/expm1_u10.h	??
arm/sve1024/expm1_u10.h	??
arm/sve128/expm1_u10.h	??
arm/sve2048/expm1_u10.h	??
arm/sve256/expm1_u10.h	??
arm/sve512/expm1_u10.h	??
cpu/cpu/expm1_u10.h	??
ppc/vmx/expm1_u10.h	??
ppc/vsx/expm1_u10.h	??
x86/avx/expm1_u10.h	??
x86/avx2/expm1_u10.h	??
x86/avx512_knl/expm1_u10.h	??
x86/avx512_skylake/expm1_u10.h	??
x86/sse2/expm1_u10.h	??
x86/sse42/expm1_u10.h	??
arm/aarch64/floor.h	??
arm/neon128/floor.h	??
arm/sve/floor.h	??
arm/sve1024/floor.h	??
arm/sve128/floor.h	??
arm/sve2048/floor.h	??
arm/sve256/floor.h	??
arm/sve512/floor.h	??
cpu/cpu/floor.h	??
ppc/vmx/floor.h	??
ppc/vsx/floor.h	??
x86/avx/floor.h	??
x86/avx2/floor.h	??
x86/avx512_knl/floor.h	??
x86/avx512_skylake/floor.h	??
x86/sse2/floor.h	??
x86/sse42/floor.h	??
arm/aarch64/fma.h	??
arm/neon128/fma.h	??
arm/sve/fma.h	??
arm/sve1024/fma.h	??
arm/sve128/fma.h	??
arm/sve2048/fma.h	??
arm/sve256/fma.h	??
arm/sve512/fma.h	??
cpu/cpu/fma.h	??
ppc/vmx/fma.h	??
ppc/vsx/fma.h	??
x86/avx/fma.h	??
x86/avx2/fma.h	??
x86/avx512_knl/fma.h	??
x86/avx512_skylake/fma.h	??

x86/sse2/fma.h	??
x86/sse42/fma.h	??
arm/aarch64/fmod.h	??
arm/neon128/fmod.h	??
arm/sve/fmod.h	??
arm/sve1024/fmod.h	??
arm/sve128/fmod.h	??
arm/sve2048/fmod.h	??
arm/sve256/fmod.h	??
arm/sve512/fmod.h	??
cpu/cpu/fmod.h	??
ppc/vmx/fmod.h	??
ppc/vsx/fmod.h	??
x86/avx/fmod.h	??
x86/avx2/fmod.h	??
x86/avx512_knl/fmod.h	??
x86/avx512_skylake/fmod.h	??
x86/sse2/fmod.h	??
x86/sse42/fmod.h	??
arm/aarch64/fms.h	??
arm/neon128/fms.h	??
arm/sve/fms.h	??
arm/sve1024/fms.h	??
arm/sve128/fms.h	??
arm/sve2048/fms.h	??
arm/sve256/fms.h	??
arm/sve512/fms.h	??
cpu/cpu/fms.h	??
ppc/vmx/fms.h	??
ppc/vsx/fms.h	??
x86/avx/fms.h	??
x86/avx2/fms.h	??
x86/avx512_knl/fms.h	??
x86/avx512_skylake/fms.h	??
x86/sse2/fms.h	??
x86/sse42/fms.h	??
arm/aarch64/fnma.h	??
arm/neon128/fnma.h	??
arm/sve/fnma.h	??
arm/sve1024/fnma.h	??
arm/sve128/fnma.h	??
arm/sve2048/fnma.h	??
arm/sve256/fnma.h	??
arm/sve512/fnma.h	??
cpu/cpu/fnma.h	??
ppc/vmx/fnma.h	??
ppc/vsx/fnma.h	??
x86/avx/fnma.h	??
x86/avx2/fnma.h	??
x86/avx512_knl/fnma.h	??
x86/avx512_skylake/fnma.h	??
x86/sse2/fnma.h	??
x86/sse42/fnma.h	??
arm/aarch64/fnms.h	??
arm/neon128/fnms.h	??
arm/sve/fnms.h	??
arm/sve1024/fnms.h	??
arm/sve128/fnms.h	??

arm/sve2048/fnms.h	??
arm/sve256/fnms.h	??
arm/sve512/fnms.h	??
cpu/cpu/fnms.h	??
ppc/vmx/fnms.h	??
ppc/vsx/fnms.h	??
x86/avx/fnms.h	??
x86/avx2/fnms.h	??
x86/avx512_knl/fnms.h	??
x86/avx512_skylake/fnms.h	??
x86/sse2/fnms.h	??
x86/sse42/fnms.h	??
functions.h	??
arm/aarch64/gather.h	??
arm/neon128/gather.h	??
arm/sve/gather.h	??
arm/sve1024/gather.h	??
arm/sve128/gather.h	??
arm/sve2048/gather.h	??
arm/sve256/gather.h	??
arm/sve512/gather.h	??
cpu/cpu/gather.h	??
ppc/vmx/gather.h	??
ppc/vsx/gather.h	??
x86/avx/gather.h	??
x86/avx2/gather.h	??
x86/avx512_knl/gather.h	??
x86/avx512_skylake/gather.h	??
x86/sse2/gather.h	??
x86/sse42/gather.h	??
arm/aarch64/gather_linear.h	??
arm/neon128/gather_linear.h	??
arm/sve/gather_linear.h	??
arm/sve1024/gather_linear.h	??
arm/sve128/gather_linear.h	??
arm/sve2048/gather_linear.h	??
arm/sve256/gather_linear.h	??
arm/sve512/gather_linear.h	??
cpu/cpu/gather_linear.h	??
ppc/vmx/gather_linear.h	??
ppc/vsx/gather_linear.h	??
x86/avx/gather_linear.h	??
x86/avx2/gather_linear.h	??
x86/avx512_knl/gather_linear.h	??
x86/avx512_skylake/gather_linear.h	??
x86/sse2/gather_linear.h	??
x86/sse42/gather_linear.h	??
arm/aarch64/ge.h	??
arm/neon128/ge.h	??
arm/sve/ge.h	??
arm/sve1024/ge.h	??
arm/sve128/ge.h	??
arm/sve2048/ge.h	??
arm/sve256/ge.h	??
arm/sve512/ge.h	??
cpu/cpu/ge.h	??
ppc/vmx/ge.h	??
ppc/vsx/ge.h	??

x86/avx/ge.h	??
x86/avx2/ge.h	??
x86/avx512_knl/ge.h	??
x86/avx512_skylake/ge.h	??
x86/sse2/ge.h	??
x86/sse42/ge.h	??
arm/aarch64/gt.h	??
arm/neon128/gt.h	??
arm/sve/gt.h	??
arm/sve1024/gt.h	??
arm/sve128/gt.h	??
arm/sve2048/gt.h	??
arm/sve256/gt.h	??
arm/sve512/gt.h	??
cpu/cpu/gt.h	??
ppc/vmx/gt.h	??
ppc/vsx/gt.h	??
x86/avx/gt.h	??
x86/avx2/gt.h	??
x86/avx512_knl/gt.h	??
x86/avx512_skylake/gt.h	??
x86/sse2/gt.h	??
x86/sse42/gt.h	??
arm/aarch64/hypot_u05.h	??
arm/neon128/hypot_u05.h	??
arm/sve/hypot_u05.h	??
arm/sve1024/hypot_u05.h	??
arm/sve128/hypot_u05.h	??
arm/sve2048/hypot_u05.h	??
arm/sve256/hypot_u05.h	??
arm/sve512/hypot_u05.h	??
cpu/cpu/hypot_u05.h	??
ppc/vmx/hypot_u05.h	??
ppc/vsx/hypot_u05.h	??
x86/avx/hypot_u05.h	??
x86/avx2/hypot_u05.h	??
x86/avx512_knl/hypot_u05.h	??
x86/avx512_skylake/hypot_u05.h	??
x86/sse2/hypot_u05.h	??
x86/sse42/hypot_u05.h	??
arm/aarch64/hypot_u35.h	??
arm/neon128/hypot_u35.h	??
arm/sve/hypot_u35.h	??
arm/sve1024/hypot_u35.h	??
arm/sve128/hypot_u35.h	??
arm/sve2048/hypot_u35.h	??
arm/sve256/hypot_u35.h	??
arm/sve512/hypot_u35.h	??
cpu/cpu/hypot_u35.h	??
ppc/vmx/hypot_u35.h	??
ppc/vsx/hypot_u35.h	??
x86/avx/hypot_u35.h	??
x86/avx2/hypot_u35.h	??
x86/avx512_knl/hypot_u35.h	??
x86/avx512_skylake/hypot_u35.h	??
x86/sse2/hypot_u35.h	??
x86/sse42/hypot_u35.h	??
arm/aarch64/if_else1.h	??

arm/neon128/if_else1.h	??
arm/sve/if_else1.h	??
arm/sve1024/if_else1.h	??
arm/sve128/if_else1.h	??
arm/sve2048/if_else1.h	??
arm/sve256/if_else1.h	??
arm/sve512/if_else1.h	??
cpu/cpu/if_else1.h	??
ppc/vmx/if_else1.h	??
ppc/vsx/if_else1.h	??
x86/avx/if_else1.h	??
x86/avx2/if_else1.h	??
x86/avx512_knl/if_else1.h	??
x86/avx512_skylake/if_else1.h	??
x86/sse2/if_else1.h	??
x86/sse42/if_else1.h	??
arm/aarch64/iota.h	??
arm/neon128/iota.h	??
arm/sve/iota.h	??
arm/sve1024/iota.h	??
arm/sve128/iota.h	??
arm/sve2048/iota.h	??
arm/sve256/iota.h	??
arm/sve512/iota.h	??
cpu/cpu/iota.h	??
ppc/vmx/iota.h	??
ppc/vsx/iota.h	??
x86/avx/iota.h	??
x86/avx2/iota.h	??
x86/avx512_knl/iota.h	??
x86/avx512_skylake/iota.h	??
x86/sse2/iota.h	??
x86/sse42/iota.h	??
jconfig.h	??
jerror.h	??
jmorecfg.h	??
jpeglib.h	??
arm/aarch64/le.h	??
arm/neon128/le.h	??
arm/sve/le.h	??
arm/sve1024/le.h	??
arm/sve128/le.h	??
arm/sve2048/le.h	??
arm/sve256/le.h	??
arm/sve512/le.h	??
cpu/cpu/le.h	??
ppc/vmx/le.h	??
ppc/vsx/le.h	??
x86/avx/le.h	??
x86/avx2/le.h	??
x86/avx512_knl/le.h	??
x86/avx512_skylake/le.h	??
x86/sse2/le.h	??
x86/sse42/le.h	??
arm/aarch64/len.h	??
arm/neon128/len.h	??
arm/sve/len.h	??
arm/sve1024/len.h	??

arm/sve128/len.h	??
arm/sve2048/len.h	??
arm/sve256/len.h	??
arm/sve512/len.h	??
cpu/cpu/len.h	??
ppc/vmx/len.h	??
ppc/vsx/len.h	??
x86/avx/len.h	??
x86/avx2/len.h	??
x86/avx512_knl/len.h	??
x86/avx512_skylake/len.h	??
x86/sse2/len.h	??
x86/sse42/len.h	??
arm/aarch64/lgamma_u10.h	??
arm/neon128/lgamma_u10.h	??
arm/sve/lgamma_u10.h	??
arm/sve1024/lgamma_u10.h	??
arm/sve128/lgamma_u10.h	??
arm/sve2048/lgamma_u10.h	??
arm/sve256/lgamma_u10.h	??
arm/sve512/lgamma_u10.h	??
cpu/cpu/lgamma_u10.h	??
ppc/vmx/lgamma_u10.h	??
ppc/vsx/lgamma_u10.h	??
x86/avx/lgamma_u10.h	??
x86/avx2/lgamma_u10.h	??
x86/avx512_knl/lgamma_u10.h	??
x86/avx512_skylake/lgamma_u10.h	??
x86/sse2/lgamma_u10.h	??
x86/sse42/lgamma_u10.h	??
arm/aarch64/load2a.h	??
arm/neon128/load2a.h	??
arm/sve/load2a.h	??
arm/sve1024/load2a.h	??
arm/sve128/load2a.h	??
arm/sve2048/load2a.h	??
arm/sve256/load2a.h	??
arm/sve512/load2a.h	??
cpu/cpu/load2a.h	??
ppc/vmx/load2a.h	??
ppc/vsx/load2a.h	??
x86/avx/load2a.h	??
x86/avx2/load2a.h	??
x86/avx512_knl/load2a.h	??
x86/avx512_skylake/load2a.h	??
x86/sse2/load2a.h	??
x86/sse42/load2a.h	??
arm/aarch64/load2u.h	??
arm/neon128/load2u.h	??
arm/sve/load2u.h	??
arm/sve1024/load2u.h	??
arm/sve128/load2u.h	??
arm/sve2048/load2u.h	??
arm/sve256/load2u.h	??
arm/sve512/load2u.h	??
cpu/cpu/load2u.h	??
ppc/vmx/load2u.h	??
ppc/vsx/load2u.h	??

x86/avx/load2u.h	??
x86/avx2/load2u.h	??
x86/avx512_knl/load2u.h	??
x86/avx512_skylake/load2u.h	??
x86/sse2/load2u.h	??
x86/sse42/load2u.h	??
arm/aarch64/load3a.h	??
arm/neon128/load3a.h	??
arm/sve/load3a.h	??
arm/sve1024/load3a.h	??
arm/sve128/load3a.h	??
arm/sve2048/load3a.h	??
arm/sve256/load3a.h	??
arm/sve512/load3a.h	??
cpu/cpu/load3a.h	??
ppc/vmx/load3a.h	??
ppc/vsx/load3a.h	??
x86/avx/load3a.h	??
x86/avx2/load3a.h	??
x86/avx512_knl/load3a.h	??
x86/avx512_skylake/load3a.h	??
x86/sse2/load3a.h	??
x86/sse42/load3a.h	??
arm/aarch64/load3u.h	??
arm/neon128/load3u.h	??
arm/sve/load3u.h	??
arm/sve1024/load3u.h	??
arm/sve128/load3u.h	??
arm/sve2048/load3u.h	??
arm/sve256/load3u.h	??
arm/sve512/load3u.h	??
cpu/cpu/load3u.h	??
ppc/vmx/load3u.h	??
ppc/vsx/load3u.h	??
x86/avx/load3u.h	??
x86/avx2/load3u.h	??
x86/avx512_knl/load3u.h	??
x86/avx512_skylake/load3u.h	??
x86/sse2/load3u.h	??
x86/sse42/load3u.h	??
arm/aarch64/load4a.h	??
arm/neon128/load4a.h	??
arm/sve/load4a.h	??
arm/sve1024/load4a.h	??
arm/sve128/load4a.h	??
arm/sve2048/load4a.h	??
arm/sve256/load4a.h	??
arm/sve512/load4a.h	??
cpu/cpu/load4a.h	??
ppc/vmx/load4a.h	??
ppc/vsx/load4a.h	??
x86/avx/load4a.h	??
x86/avx2/load4a.h	??
x86/avx512_knl/load4a.h	??
x86/avx512_skylake/load4a.h	??
x86/sse2/load4a.h	??
x86/sse42/load4a.h	??
arm/aarch64/load4u.h	??

arm/neon128/load4u.h	??
arm/sve/load4u.h	??
arm/sve1024/load4u.h	??
arm/sve128/load4u.h	??
arm/sve2048/load4u.h	??
arm/sve256/load4u.h	??
arm/sve512/load4u.h	??
cpu/cpu/load4u.h	??
ppc/vmx/load4u.h	??
ppc/vsx/load4u.h	??
x86/avx/load4u.h	??
x86/avx2/load4u.h	??
x86/avx512_knl/load4u.h	??
x86/avx512_skylake/load4u.h	??
x86/sse2/load4u.h	??
x86/sse42/load4u.h	??
arm/aarch64/loada.h	??
arm/neon128/loada.h	??
arm/sve/loada.h	??
arm/sve1024/loada.h	??
arm/sve128/loada.h	??
arm/sve2048/loada.h	??
arm/sve256/loada.h	??
arm/sve512/loada.h	??
cpu/cpu/loada.h	??
ppc/vmx/loada.h	??
ppc/vsx/loada.h	??
x86/avx/loada.h	??
x86/avx2/loada.h	??
x86/avx512_knl/loada.h	??
x86/avx512_skylake/loada.h	??
x86/sse2/loada.h	??
x86/sse42/loada.h	??
arm/aarch64/loadla.h	??
arm/neon128/loadla.h	??
arm/sve/loadla.h	??
arm/sve1024/loadla.h	??
arm/sve128/loadla.h	??
arm/sve2048/loadla.h	??
arm/sve256/loadla.h	??
arm/sve512/loadla.h	??
cpu/cpu/loadla.h	??
ppc/vmx/loadla.h	??
ppc/vsx/loadla.h	??
x86/avx/loadla.h	??
x86/avx2/loadla.h	??
x86/avx512_knl/loadla.h	??
x86/avx512_skylake/loadla.h	??
x86/sse2/loadla.h	??
x86/sse42/loadla.h	??
arm/aarch64/loadlu.h	??
arm/neon128/loadlu.h	??
arm/sve/loadlu.h	??
arm/sve1024/loadlu.h	??
arm/sve128/loadlu.h	??
arm/sve2048/loadlu.h	??
arm/sve256/loadlu.h	??
arm/sve512/loadlu.h	??

cpu/cpu/loadlu.h	??
ppc/vmx/loadlu.h	??
ppc/vsx/loadlu.h	??
x86/avx/loadlu.h	??
x86/avx2/loadlu.h	??
x86/avx512_knl/loadlu.h	??
x86/avx512_skylake/loadlu.h	??
x86/sse2/loadlu.h	??
x86/sse42/loadlu.h	??
arm/aarch64/loadu.h	??
arm/neon128/loadu.h	??
arm/sve/loadu.h	??
arm/sve1024/loadu.h	??
arm/sve128/loadu.h	??
arm/sve2048/loadu.h	??
arm/sve256/loadu.h	??
arm/sve512/loadu.h	??
cpu/cpu/loadu.h	??
ppc/vmx/loadu.h	??
ppc/vsx/loadu.h	??
x86/avx/loadu.h	??
x86/avx2/loadu.h	??
x86/avx512_knl/loadu.h	??
x86/avx512_skylake/loadu.h	??
x86/sse2/loadu.h	??
x86/sse42/loadu.h	??
arm/aarch64/log10_u10.h	??
arm/neon128/log10_u10.h	??
arm/sve/log10_u10.h	??
arm/sve1024/log10_u10.h	??
arm/sve128/log10_u10.h	??
arm/sve2048/log10_u10.h	??
arm/sve256/log10_u10.h	??
arm/sve512/log10_u10.h	??
cpu/cpu/log10_u10.h	??
ppc/vmx/log10_u10.h	??
ppc/vsx/log10_u10.h	??
x86/avx/log10_u10.h	??
x86/avx2/log10_u10.h	??
x86/avx512_knl/log10_u10.h	??
x86/avx512_skylake/log10_u10.h	??
x86/sse2/log10_u10.h	??
x86/sse42/log10_u10.h	??
arm/aarch64/log1p_u10.h	??
arm/neon128/log1p_u10.h	??
arm/sve/log1p_u10.h	??
arm/sve1024/log1p_u10.h	??
arm/sve128/log1p_u10.h	??
arm/sve2048/log1p_u10.h	??
arm/sve256/log1p_u10.h	??
arm/sve512/log1p_u10.h	??
cpu/cpu/log1p_u10.h	??
ppc/vmx/log1p_u10.h	??
ppc/vsx/log1p_u10.h	??
x86/avx/log1p_u10.h	??
x86/avx2/log1p_u10.h	??
x86/avx512_knl/log1p_u10.h	??
x86/avx512_skylake/log1p_u10.h	??

x86/sse2/log1p_u10.h	??
x86/sse42/log1p_u10.h	??
arm/aarch64/log2_u10.h	??
arm/neon128/log2_u10.h	??
arm/sve/log2_u10.h	??
arm/sve1024/log2_u10.h	??
arm/sve128/log2_u10.h	??
arm/sve2048/log2_u10.h	??
arm/sve256/log2_u10.h	??
arm/sve512/log2_u10.h	??
cpu/cpu/log2_u10.h	??
ppc/vmx/log2_u10.h	??
ppc/vsx/log2_u10.h	??
x86/avx/log2_u10.h	??
x86/avx2/log2_u10.h	??
x86/avx512_knl/log2_u10.h	??
x86/avx512_skylake/log2_u10.h	??
x86/sse2/log2_u10.h	??
x86/sse42/log2_u10.h	??
arm/aarch64/log2_u35.h	??
arm/neon128/log2_u35.h	??
arm/sve/log2_u35.h	??
arm/sve1024/log2_u35.h	??
arm/sve128/log2_u35.h	??
arm/sve2048/log2_u35.h	??
arm/sve256/log2_u35.h	??
arm/sve512/log2_u35.h	??
cpu/cpu/log2_u35.h	??
ppc/vmx/log2_u35.h	??
ppc/vsx/log2_u35.h	??
x86/avx/log2_u35.h	??
x86/avx2/log2_u35.h	??
x86/avx512_knl/log2_u35.h	??
x86/avx512_skylake/log2_u35.h	??
x86/sse2/log2_u35.h	??
x86/sse42/log2_u35.h	??
arm/aarch64/log_u10.h	??
arm/neon128/log_u10.h	??
arm/sve/log_u10.h	??
arm/sve1024/log_u10.h	??
arm/sve128/log_u10.h	??
arm/sve2048/log_u10.h	??
arm/sve256/log_u10.h	??
arm/sve512/log_u10.h	??
cpu/cpu/log_u10.h	??
ppc/vmx/log_u10.h	??
ppc/vsx/log_u10.h	??
x86/avx/log_u10.h	??
x86/avx2/log_u10.h	??
x86/avx512_knl/log_u10.h	??
x86/avx512_skylake/log_u10.h	??
x86/sse2/log_u10.h	??
x86/sse42/log_u10.h	??
arm/aarch64/log_u35.h	??
arm/neon128/log_u35.h	??
arm/sve/log_u35.h	??
arm/sve1024/log_u35.h	??
arm/sve128/log_u35.h	??

arm/sve2048/log_u35.h	??
arm/sve256/log_u35.h	??
arm/sve512/log_u35.h	??
cpu/cpu/log_u35.h	??
ppc/vmx/log_u35.h	??
ppc/vsx/log_u35.h	??
x86/avx/log_u35.h	??
x86/avx2/log_u35.h	??
x86/avx512_knl/log_u35.h	??
x86/avx512_skylake/log_u35.h	??
x86/sse2/log_u35.h	??
x86/sse42/log_u35.h	??
arm/aarch64/lt.h	??
arm/neon128/lt.h	??
arm/sve/lt.h	??
arm/sve1024/lt.h	??
arm/sve128/lt.h	??
arm/sve2048/lt.h	??
arm/sve256/lt.h	??
arm/sve512/lt.h	??
cpu/cpu/lt.h	??
ppc/vmx/lt.h	??
ppc/vsx/lt.h	??
x86/avx/lt.h	??
x86/avx2/lt.h	??
x86/avx512_knl/lt.h	??
x86/avx512_skylake/lt.h	??
x86/sse2/lt.h	??
x86/sse42/lt.h	??
arm/aarch64/mask_for_loop_tail.h	??
arm/neon128/mask_for_loop_tail.h	??
arm/sve/mask_for_loop_tail.h	??
arm/sve1024/mask_for_loop_tail.h	??
arm/sve128/mask_for_loop_tail.h	??
arm/sve2048/mask_for_loop_tail.h	??
arm/sve256/mask_for_loop_tail.h	??
arm/sve512/mask_for_loop_tail.h	??
cpu/cpu/mask_for_loop_tail.h	??
ppc/vmx/mask_for_loop_tail.h	??
ppc/vsx/mask_for_loop_tail.h	??
x86/avx/mask_for_loop_tail.h	??
x86/avx2/mask_for_loop_tail.h	??
x86/avx512_knl/mask_for_loop_tail.h	??
x86/avx512_skylake/mask_for_loop_tail.h	??
x86/sse2/mask_for_loop_tail.h	??
x86/sse42/mask_for_loop_tail.h	??
arm/aarch64/mask_storea1.h	??
arm/neon128/mask_storea1.h	??
arm/sve/mask_storea1.h	??
arm/sve1024/mask_storea1.h	??
arm/sve128/mask_storea1.h	??
arm/sve2048/mask_storea1.h	??
arm/sve256/mask_storea1.h	??
arm/sve512/mask_storea1.h	??
cpu/cpu/mask_storea1.h	??
ppc/vmx/mask_storea1.h	??
ppc/vsx/mask_storea1.h	??
x86/avx/mask_storea1.h	??

x86/avx2/mask_storea1.h	??
x86/avx512_knl/mask_storea1.h	??
x86/avx512_skylake/mask_storea1.h	??
x86/sse2/mask_storea1.h	??
x86/sse42/mask_storea1.h	??
arm/aarch64/mask_storeu1.h	??
arm/neon128/mask_storeu1.h	??
arm/sve/mask_storeu1.h	??
arm/sve1024/mask_storeu1.h	??
arm/sve128/mask_storeu1.h	??
arm/sve2048/mask_storeu1.h	??
arm/sve256/mask_storeu1.h	??
arm/sve512/mask_storeu1.h	??
cpu/cpu/mask_storeu1.h	??
ppc/vmx/mask_storeu1.h	??
ppc/vsx/mask_storeu1.h	??
x86/avx/mask_storeu1.h	??
x86/avx2/mask_storeu1.h	??
x86/avx512_knl/mask_storeu1.h	??
x86/avx512_skylake/mask_storeu1.h	??
x86/sse2/mask_storeu1.h	??
x86/sse42/mask_storeu1.h	??
arm/aarch64/masko_loada1.h	??
arm/neon128/masko_loada1.h	??
arm/sve/masko_loada1.h	??
arm/sve1024/masko_loada1.h	??
arm/sve128/masko_loada1.h	??
arm/sve2048/masko_loada1.h	??
arm/sve256/masko_loada1.h	??
arm/sve512/masko_loada1.h	??
cpu/cpu/masko_loada1.h	??
ppc/vmx/masko_loada1.h	??
ppc/vsx/masko_loada1.h	??
x86/avx/masko_loada1.h	??
x86/avx2/masko_loada1.h	??
x86/avx512_knl/masko_loada1.h	??
x86/avx512_skylake/masko_loada1.h	??
x86/sse2/masko_loada1.h	??
x86/sse42/masko_loada1.h	??
arm/aarch64/masko_loadu1.h	??
arm/neon128/masko_loadu1.h	??
arm/sve/masko_loadu1.h	??
arm/sve1024/masko_loadu1.h	??
arm/sve128/masko_loadu1.h	??
arm/sve2048/masko_loadu1.h	??
arm/sve256/masko_loadu1.h	??
arm/sve512/masko_loadu1.h	??
cpu/cpu/masko_loadu1.h	??
ppc/vmx/masko_loadu1.h	??
ppc/vsx/masko_loadu1.h	??
x86/avx/masko_loadu1.h	??
x86/avx2/masko_loadu1.h	??
x86/avx512_knl/masko_loadu1.h	??
x86/avx512_skylake/masko_loadu1.h	??
x86/sse2/masko_loadu1.h	??
x86/sse42/masko_loadu1.h	??
arm/aarch64/maskz_loada1.h	??
arm/neon128/maskz_loada1.h	??

arm/sve/maskz_loada1.h	??
arm/sve1024/maskz_loada1.h	??
arm/sve128/maskz_loada1.h	??
arm/sve2048/maskz_loada1.h	??
arm/sve256/maskz_loada1.h	??
arm/sve512/maskz_loada1.h	??
cpu/cpu/maskz_loada1.h	??
ppc/vmx/maskz_loada1.h	??
ppc/vsx/maskz_loada1.h	??
x86/avx/maskz_loada1.h	??
x86/avx2/maskz_loada1.h	??
x86/avx512_knl/maskz_loada1.h	??
x86/avx512_skylake/maskz_loada1.h	??
x86/sse2/maskz_loada1.h	??
x86/sse42/maskz_loada1.h	??
arm/aarch64/maskz_loadu1.h	??
arm/neon128/maskz_loadu1.h	??
arm/sve/maskz_loadu1.h	??
arm/sve1024/maskz_loadu1.h	??
arm/sve128/maskz_loadu1.h	??
arm/sve2048/maskz_loadu1.h	??
arm/sve256/maskz_loadu1.h	??
arm/sve512/maskz_loadu1.h	??
cpu/cpu/maskz_loadu1.h	??
ppc/vmx/maskz_loadu1.h	??
ppc/vsx/maskz_loadu1.h	??
x86/avx/maskz_loadu1.h	??
x86/avx2/maskz_loadu1.h	??
x86/avx512_knl/maskz_loadu1.h	??
x86/avx512_skylake/maskz_loadu1.h	??
x86/sse2/maskz_loadu1.h	??
x86/sse42/maskz_loadu1.h	??
arm/aarch64/max.h	??
arm/neon128/max.h	??
arm/sve/max.h	??
arm/sve1024/max.h	??
arm/sve128/max.h	??
arm/sve2048/max.h	??
arm/sve256/max.h	??
arm/sve512/max.h	??
cpu/cpu/max.h	??
ppc/vmx/max.h	??
ppc/vsx/max.h	??
x86/avx/max.h	??
x86/avx2/max.h	??
x86/avx512_knl/max.h	??
x86/avx512_skylake/max.h	??
x86/sse2/max.h	??
x86/sse42/max.h	??
arm/aarch64/min.h	??
arm/neon128/min.h	??
arm/sve/min.h	??
arm/sve1024/min.h	??
arm/sve128/min.h	??
arm/sve2048/min.h	??
arm/sve256/min.h	??
arm/sve512/min.h	??
cpu/cpu/min.h	??

ppc/vmx/min.h	??
ppc/vsx/min.h	??
x86/avx/min.h	??
x86/avx2/min.h	??
x86/avx512_knl/min.h	??
x86/avx512_skylake/min.h	??
x86/sse2/min.h	??
x86/sse42/min.h	??
arm/aarch64/mul.h	??
arm/neon128/mul.h	??
arm/sve/mul.h	??
arm/sve1024/mul.h	??
arm/sve128/mul.h	??
arm/sve2048/mul.h	??
arm/sve256/mul.h	??
arm/sve512/mul.h	??
cpu/cpu/mul.h	??
ppc/vmx/mul.h	??
ppc/vsx/mul.h	??
x86/avx/mul.h	??
x86/avx2/mul.h	??
x86/avx512_knl/mul.h	??
x86/avx512_skylake/mul.h	??
x86/sse2/mul.h	??
x86/sse42/mul.h	??
arm/aarch64/nbtrue.h	??
arm/neon128/nbtrue.h	??
arm/sve/nbtrue.h	??
arm/sve1024/nbtrue.h	??
arm/sve128/nbtrue.h	??
arm/sve2048/nbtrue.h	??
arm/sve256/nbtrue.h	??
arm/sve512/nbtrue.h	??
cpu/cpu/nbtrue.h	??
ppc/vmx/nbtrue.h	??
ppc/vsx/nbtrue.h	??
x86/avx/nbtrue.h	??
x86/avx2/nbtrue.h	??
x86/avx512_knl/nbtrue.h	??
x86/avx512_skylake/nbtrue.h	??
x86/sse2/nbtrue.h	??
x86/sse42/nbtrue.h	??
arm/aarch64/ne.h	??
arm/neon128/ne.h	??
arm/sve/ne.h	??
arm/sve1024/ne.h	??
arm/sve128/ne.h	??
arm/sve2048/ne.h	??
arm/sve256/ne.h	??
arm/sve512/ne.h	??
cpu/cpu/ne.h	??
ppc/vmx/ne.h	??
ppc/vsx/ne.h	??
x86/avx/ne.h	??
x86/avx2/ne.h	??
x86/avx512_knl/ne.h	??
x86/avx512_skylake/ne.h	??
x86/sse2/ne.h	??

x86/sse42/ne.h	??
arm/aarch64/neg.h	??
arm/neon128/neg.h	??
arm/sve/neg.h	??
arm/sve1024/neg.h	??
arm/sve128/neg.h	??
arm/sve2048/neg.h	??
arm/sve256/neg.h	??
arm/sve512/neg.h	??
cpu/cpu/neg.h	??
ppc/vmx/neg.h	??
ppc/vsx/neg.h	??
x86/avx/neg.h	??
x86/avx2/neg.h	??
x86/avx512_knl/neg.h	??
x86/avx512_skylake/neg.h	??
x86/sse2/neg.h	??
x86/sse42/neg.h	??
arm/aarch64/notb.h	??
arm/neon128/notb.h	??
arm/sve/notb.h	??
arm/sve1024/notb.h	??
arm/sve128/notb.h	??
arm/sve2048/notb.h	??
arm/sve256/notb.h	??
arm/sve512/notb.h	??
cpu/cpu/notb.h	??
ppc/vmx/notb.h	??
ppc/vsx/notb.h	??
x86/avx/notb.h	??
x86/avx2/notb.h	??
x86/avx512_knl/notb.h	??
x86/avx512_skylake/notb.h	??
x86/sse2/notb.h	??
x86/sse42/notb.h	??
arm/aarch64/notl.h	??
arm/neon128/notl.h	??
arm/sve/notl.h	??
arm/sve1024/notl.h	??
arm/sve128/notl.h	??
arm/sve2048/notl.h	??
arm/sve256/notl.h	??
arm/sve512/notl.h	??
cpu/cpu/notl.h	??
ppc/vmx/notl.h	??
ppc/vsx/notl.h	??
x86/avx/notl.h	??
x86/avx2/notl.h	??
x86/avx512_knl/notl.h	??
x86/avx512_skylake/notl.h	??
x86/sse2/notl.h	??
x86/sse42/notl.h	??
nsimd-all.h	??
nsimd.h	??
arm/aarch64/orb.h	??
arm/neon128/orb.h	??
arm/sve/orb.h	??
arm/sve1024/orb.h	??

arm/sve128/orb.h	??
arm/sve2048/orb.h	??
arm/sve256/orb.h	??
arm/sve512/orb.h	??
cpu/cpu/orb.h	??
ppc/vmx/orb.h	??
ppc/vsx/orb.h	??
x86/avx/orb.h	??
x86/avx2/orb.h	??
x86/avx512_knl/orb.h	??
x86/avx512_skylake/orb.h	??
x86/sse2/orb.h	??
x86/sse42/orb.h	??
arm/aarch64/orl.h	??
arm/neon128/orl.h	??
arm/sve/orl.h	??
arm/sve1024/orl.h	??
arm/sve128/orl.h	??
arm/sve2048/orl.h	??
arm/sve256/orl.h	??
arm/sve512/orl.h	??
cpu/cpu/orl.h	??
ppc/vmx/orl.h	??
ppc/vsx/orl.h	??
x86/avx/orl.h	??
x86/avx2/orl.h	??
x86/avx512_knl/orl.h	??
x86/avx512_skylake/orl.h	??
x86/sse2/orl.h	??
x86/sse42/orl.h	??
PFNC.h	??
arm/aarch64/pow_u10.h	??
arm/neon128/pow_u10.h	??
arm/sve/pow_u10.h	??
arm/sve1024/pow_u10.h	??
arm/sve128/pow_u10.h	??
arm/sve2048/pow_u10.h	??
arm/sve256/pow_u10.h	??
arm/sve512/pow_u10.h	??
cpu/cpu/pow_u10.h	??
ppc/vmx/pow_u10.h	??
ppc/vsx/pow_u10.h	??
x86/avx/pow_u10.h	??
x86/avx2/pow_u10.h	??
x86/avx512_knl/pow_u10.h	??
x86/avx512_skylake/pow_u10.h	??
x86/sse2/pow_u10.h	??
x86/sse42/pow_u10.h	??
arm/aarch64/put.h	??
arm/neon128/put.h	??
arm/sve/put.h	??
arm/sve1024/put.h	??
arm/sve128/put.h	??
arm/sve2048/put.h	??
arm/sve256/put.h	??
arm/sve512/put.h	??
cpu/cpu/put.h	??
ppc/vmx/put.h	??

ppc/vsx/put.h	??
x86/avx/put.h	??
x86/avx2/put.h	??
x86/avx512_knl/put.h	??
x86/avx512_skylake/put.h	??
x86/sse2/put.h	??
x86/sse42/put.h	??
arm/aarch64/rec.h	??
arm/neon128/rec.h	??
arm/sve/rec.h	??
arm/sve1024/rec.h	??
arm/sve128/rec.h	??
arm/sve2048/rec.h	??
arm/sve256/rec.h	??
arm/sve512/rec.h	??
cpu/cpu/rec.h	??
ppc/vmx/rec.h	??
ppc/vsx/rec.h	??
x86/avx/rec.h	??
x86/avx2/rec.h	??
x86/avx512_knl/rec.h	??
x86/avx512_skylake/rec.h	??
x86/sse2/rec.h	??
x86/sse42/rec.h	??
arm/aarch64/rec11.h	??
arm/neon128/rec11.h	??
arm/sve/rec11.h	??
arm/sve1024/rec11.h	??
arm/sve128/rec11.h	??
arm/sve2048/rec11.h	??
arm/sve256/rec11.h	??
arm/sve512/rec11.h	??
cpu/cpu/rec11.h	??
ppc/vmx/rec11.h	??
ppc/vsx/rec11.h	??
x86/avx/rec11.h	??
x86/avx2/rec11.h	??
x86/avx512_knl/rec11.h	??
x86/avx512_skylake/rec11.h	??
x86/sse2/rec11.h	??
x86/sse42/rec11.h	??
arm/aarch64/rec8.h	??
arm/neon128/rec8.h	??
arm/sve/rec8.h	??
arm/sve1024/rec8.h	??
arm/sve128/rec8.h	??
arm/sve2048/rec8.h	??
arm/sve256/rec8.h	??
arm/sve512/rec8.h	??
cpu/cpu/rec8.h	??
ppc/vmx/rec8.h	??
ppc/vsx/rec8.h	??
x86/avx/rec8.h	??
x86/avx2/rec8.h	??
x86/avx512_knl/rec8.h	??
x86/avx512_skylake/rec8.h	??
x86/sse2/rec8.h	??
x86/sse42/rec8.h	??

arm/aarch64/reinterpret.h	??
arm/neon128/reinterpret.h	??
arm/sve/reinterpret.h	??
arm/sve1024/reinterpret.h	??
arm/sve128/reinterpret.h	??
arm/sve2048/reinterpret.h	??
arm/sve256/reinterpret.h	??
arm/sve512/reinterpret.h	??
cpu/cpu/reinterpret.h	??
ppc/vmx/reinterpret.h	??
ppc/vsx/reinterpret.h	??
x86/avx/reinterpret.h	??
x86/avx2/reinterpret.h	??
x86/avx512_knl/reinterpret.h	??
x86/avx512_skylake/reinterpret.h	??
x86/sse2/reinterpret.h	??
x86/sse42/reinterpret.h	??
arm/aarch64/reinterpretl.h	??
arm/neon128/reinterpretl.h	??
arm/sve/reinterpretl.h	??
arm/sve1024/reinterpretl.h	??
arm/sve128/reinterpretl.h	??
arm/sve2048/reinterpretl.h	??
arm/sve256/reinterpretl.h	??
arm/sve512/reinterpretl.h	??
cpu/cpu/reinterpretl.h	??
ppc/vmx/reinterpretl.h	??
ppc/vsx/reinterpretl.h	??
x86/avx/reinterpretl.h	??
x86/avx2/reinterpretl.h	??
x86/avx512_knl/reinterpretl.h	??
x86/avx512_skylake/reinterpretl.h	??
x86/sse2/reinterpretl.h	??
x86/sse42/reinterpretl.h	??
arm/aarch64/remainder.h	??
arm/neon128/remainder.h	??
arm/sve/remainder.h	??
arm/sve1024/remainder.h	??
arm/sve128/remainder.h	??
arm/sve2048/remainder.h	??
arm/sve256/remainder.h	??
arm/sve512/remainder.h	??
cpu/cpu/remainder.h	??
ppc/vmx/remainder.h	??
ppc/vsx/remainder.h	??
x86/avx/remainder.h	??
x86/avx2/remainder.h	??
x86/avx512_knl/remainder.h	??
x86/avx512_skylake/remainder.h	??
x86/sse2/remainder.h	??
x86/sse42/remainder.h	??
arm/aarch64/round_to_even.h	??
arm/neon128/round_to_even.h	??
arm/sve/round_to_even.h	??
arm/sve1024/round_to_even.h	??
arm/sve128/round_to_even.h	??
arm/sve2048/round_to_even.h	??
arm/sve256/round_to_even.h	??

arm/sve512/round_to_even.h	??
cpu/cpu/round_to_even.h	??
ppc/vmx/round_to_even.h	??
ppc/vsx/round_to_even.h	??
x86/avx/round_to_even.h	??
x86/avx2/round_to_even.h	??
x86/avx512_knl/round_to_even.h	??
x86/avx512_skylake/round_to_even.h	??
x86/sse2/round_to_even.h	??
x86/sse42/round_to_even.h	??
arm/aarch64/rsqrt11.h	??
arm/neon128/rsqrt11.h	??
arm/sve/rsqrt11.h	??
arm/sve1024/rsqrt11.h	??
arm/sve128/rsqrt11.h	??
arm/sve2048/rsqrt11.h	??
arm/sve256/rsqrt11.h	??
arm/sve512/rsqrt11.h	??
cpu/cpu/rsqrt11.h	??
ppc/vmx/rsqrt11.h	??
ppc/vsx/rsqrt11.h	??
x86/avx/rsqrt11.h	??
x86/avx2/rsqrt11.h	??
x86/avx512_knl/rsqrt11.h	??
x86/avx512_skylake/rsqrt11.h	??
x86/sse2/rsqrt11.h	??
x86/sse42/rsqrt11.h	??
arm/aarch64/rsqrt8.h	??
arm/neon128/rsqrt8.h	??
arm/sve/rsqrt8.h	??
arm/sve1024/rsqrt8.h	??
arm/sve128/rsqrt8.h	??
arm/sve2048/rsqrt8.h	??
arm/sve256/rsqrt8.h	??
arm/sve512/rsqrt8.h	??
cpu/cpu/rsqrt8.h	??
ppc/vmx/rsqrt8.h	??
ppc/vsx/rsqrt8.h	??
x86/avx/rsqrt8.h	??
x86/avx2/rsqrt8.h	??
x86/avx512_knl/rsqrt8.h	??
x86/avx512_skylake/rsqrt8.h	??
x86/sse2/rsqrt8.h	??
x86/sse42/rsqrt8.h	??
scalar_utilities.h	??
arm/aarch64/scatter.h	??
arm/neon128/scatter.h	??
arm/sve/scatter.h	??
arm/sve1024/scatter.h	??
arm/sve128/scatter.h	??
arm/sve2048/scatter.h	??
arm/sve256/scatter.h	??
arm/sve512/scatter.h	??
cpu/cpu/scatter.h	??
ppc/vmx/scatter.h	??
ppc/vsx/scatter.h	??
x86/avx/scatter.h	??
x86/avx2/scatter.h	??

x86/avx512_knl/scatter.h	??
x86/avx512_skylake/scatter.h	??
x86/sse2/scatter.h	??
x86/sse42/scatter.h	??
arm/aarch64/scatter_linear.h	??
arm/neon128/scatter_linear.h	??
arm/sve/scatter_linear.h	??
arm/sve1024/scatter_linear.h	??
arm/sve128/scatter_linear.h	??
arm/sve2048/scatter_linear.h	??
arm/sve256/scatter_linear.h	??
arm/sve512/scatter_linear.h	??
cpu/cpu/scatter_linear.h	??
ppc/vmx/scatter_linear.h	??
ppc/vsx/scatter_linear.h	??
x86/avx/scatter_linear.h	??
x86/avx2/scatter_linear.h	??
x86/avx512_knl/scatter_linear.h	??
x86/avx512_skylake/scatter_linear.h	??
x86/sse2/scatter_linear.h	??
x86/sse42/scatter_linear.h	??
arm/aarch64/set1.h	??
arm/neon128/set1.h	??
arm/sve/set1.h	??
arm/sve1024/set1.h	??
arm/sve128/set1.h	??
arm/sve2048/set1.h	??
arm/sve256/set1.h	??
arm/sve512/set1.h	??
cpu/cpu/set1.h	??
ppc/vmx/set1.h	??
ppc/vsx/set1.h	??
x86/avx/set1.h	??
x86/avx2/set1.h	??
x86/avx512_knl/set1.h	??
x86/avx512_skylake/set1.h	??
x86/sse2/set1.h	??
x86/sse42/set1.h	??
arm/aarch64/set1l.h	??
arm/neon128/set1l.h	??
arm/sve/set1l.h	??
arm/sve1024/set1l.h	??
arm/sve128/set1l.h	??
arm/sve2048/set1l.h	??
arm/sve256/set1l.h	??
arm/sve512/set1l.h	??
cpu/cpu/set1l.h	??
ppc/vmx/set1l.h	??
ppc/vsx/set1l.h	??
x86/avx/set1l.h	??
x86/avx2/set1l.h	??
x86/avx512_knl/set1l.h	??
x86/avx512_skylake/set1l.h	??
x86/sse2/set1l.h	??
x86/sse42/set1l.h	??
arm/aarch64/shl.h	??
arm/neon128/shl.h	??
arm/sve/shl.h	??

arm/sve1024/shl.h	??
arm/sve128/shl.h	??
arm/sve2048/shl.h	??
arm/sve256/shl.h	??
arm/sve512/shl.h	??
cpu/cpu/shl.h	??
ppc/vmx/shl.h	??
ppc/vsx/shl.h	??
x86/avx/shl.h	??
x86/avx2/shl.h	??
x86/avx512_knl/shl.h	??
x86/avx512_skylake/shl.h	??
x86/sse2/shl.h	??
x86/sse42/shl.h	??
arm/aarch64/shr.h	??
arm/neon128/shr.h	??
arm/sve/shr.h	??
arm/sve1024/shr.h	??
arm/sve128/shr.h	??
arm/sve2048/shr.h	??
arm/sve256/shr.h	??
arm/sve512/shr.h	??
cpu/cpu/shr.h	??
ppc/vmx/shr.h	??
ppc/vsx/shr.h	??
x86/avx/shr.h	??
x86/avx2/shr.h	??
x86/avx512_knl/shr.h	??
x86/avx512_skylake/shr.h	??
x86/sse2/shr.h	??
x86/sse42/shr.h	??
arm/aarch64/shra.h	??
arm/neon128/shra.h	??
arm/sve/shra.h	??
arm/sve1024/shra.h	??
arm/sve128/shra.h	??
arm/sve2048/shra.h	??
arm/sve256/shra.h	??
arm/sve512/shra.h	??
cpu/cpu/shra.h	??
ppc/vmx/shra.h	??
ppc/vsx/shra.h	??
x86/avx/shra.h	??
x86/avx2/shra.h	??
x86/avx512_knl/shra.h	??
x86/avx512_skylake/shra.h	??
x86/sse2/shra.h	??
x86/sse42/shra.h	??
arm/aarch64/sin_u10.h	??
arm/neon128/sin_u10.h	??
arm/sve/sin_u10.h	??
arm/sve1024/sin_u10.h	??
arm/sve128/sin_u10.h	??
arm/sve2048/sin_u10.h	??
arm/sve256/sin_u10.h	??
arm/sve512/sin_u10.h	??
cpu/cpu/sin_u10.h	??
ppc/vmx/sin_u10.h	??

ppc/vsx/sin_u10.h	??
x86/avx/sin_u10.h	??
x86/avx2/sin_u10.h	??
x86/avx512_knl/sin_u10.h	??
x86/avx512_skylake/sin_u10.h	??
x86/sse2/sin_u10.h	??
x86/sse42/sin_u10.h	??
arm/aarch64/sin_u35.h	??
arm/neon128/sin_u35.h	??
arm/sve/sin_u35.h	??
arm/sve1024/sin_u35.h	??
arm/sve128/sin_u35.h	??
arm/sve2048/sin_u35.h	??
arm/sve256/sin_u35.h	??
arm/sve512/sin_u35.h	??
cpu/cpu/sin_u35.h	??
ppc/vmx/sin_u35.h	??
ppc/vsx/sin_u35.h	??
x86/avx/sin_u35.h	??
x86/avx2/sin_u35.h	??
x86/avx512_knl/sin_u35.h	??
x86/avx512_skylake/sin_u35.h	??
x86/sse2/sin_u35.h	??
x86/sse42/sin_u35.h	??
arm/aarch64/sinh_u10.h	??
arm/neon128/sinh_u10.h	??
arm/sve/sinh_u10.h	??
arm/sve1024/sinh_u10.h	??
arm/sve128/sinh_u10.h	??
arm/sve2048/sinh_u10.h	??
arm/sve256/sinh_u10.h	??
arm/sve512/sinh_u10.h	??
cpu/cpu/sinh_u10.h	??
ppc/vmx/sinh_u10.h	??
ppc/vsx/sinh_u10.h	??
x86/avx/sinh_u10.h	??
x86/avx2/sinh_u10.h	??
x86/avx512_knl/sinh_u10.h	??
x86/avx512_skylake/sinh_u10.h	??
x86/sse2/sinh_u10.h	??
x86/sse42/sinh_u10.h	??
arm/aarch64/sinh_u35.h	??
arm/neon128/sinh_u35.h	??
arm/sve/sinh_u35.h	??
arm/sve1024/sinh_u35.h	??
arm/sve128/sinh_u35.h	??
arm/sve2048/sinh_u35.h	??
arm/sve256/sinh_u35.h	??
arm/sve512/sinh_u35.h	??
cpu/cpu/sinh_u35.h	??
ppc/vmx/sinh_u35.h	??
ppc/vsx/sinh_u35.h	??
x86/avx/sinh_u35.h	??
x86/avx2/sinh_u35.h	??
x86/avx512_knl/sinh_u35.h	??
x86/avx512_skylake/sinh_u35.h	??
x86/sse2/sinh_u35.h	??
x86/sse42/sinh_u35.h	??

arm/aarch64/sinpi_u05.h	??
arm/neon128/sinpi_u05.h	??
arm/sve/sinpi_u05.h	??
arm/sve1024/sinpi_u05.h	??
arm/sve128/sinpi_u05.h	??
arm/sve2048/sinpi_u05.h	??
arm/sve256/sinpi_u05.h	??
arm/sve512/sinpi_u05.h	??
cpu/cpu/sinpi_u05.h	??
ppc/vmx/sinpi_u05.h	??
ppc/vsx/sinpi_u05.h	??
x86/avx/sinpi_u05.h	??
x86/avx2/sinpi_u05.h	??
x86/avx512_knl/sinpi_u05.h	??
x86/avx512_skylake/sinpi_u05.h	??
x86/sse2/sinpi_u05.h	??
x86/sse42/sinpi_u05.h	??
arm/aarch64/sqrt.h	??
arm/neon128/sqrt.h	??
arm/sve/sqrt.h	??
arm/sve1024/sqrt.h	??
arm/sve128/sqrt.h	??
arm/sve2048/sqrt.h	??
arm/sve256/sqrt.h	??
arm/sve512/sqrt.h	??
cpu/cpu/sqrt.h	??
ppc/vmx/sqrt.h	??
ppc/vsx/sqrt.h	??
x86/avx/sqrt.h	??
x86/avx2/sqrt.h	??
x86/avx512_knl/sqrt.h	??
x86/avx512_skylake/sqrt.h	??
x86/sse2/sqrt.h	??
x86/sse42/sqrt.h	??
arm/aarch64/store2a.h	??
arm/neon128/store2a.h	??
arm/sve/store2a.h	??
arm/sve1024/store2a.h	??
arm/sve128/store2a.h	??
arm/sve2048/store2a.h	??
arm/sve256/store2a.h	??
arm/sve512/store2a.h	??
cpu/cpu/store2a.h	??
ppc/vmx/store2a.h	??
ppc/vsx/store2a.h	??
x86/avx/store2a.h	??
x86/avx2/store2a.h	??
x86/avx512_knl/store2a.h	??
x86/avx512_skylake/store2a.h	??
x86/sse2/store2a.h	??
x86/sse42/store2a.h	??
arm/aarch64/store2u.h	??
arm/neon128/store2u.h	??
arm/sve/store2u.h	??
arm/sve1024/store2u.h	??
arm/sve128/store2u.h	??
arm/sve2048/store2u.h	??
arm/sve256/store2u.h	??

arm/sve512/store2u.h	??
cpu/cpu/store2u.h	??
ppc/vmx/store2u.h	??
ppc/vsx/store2u.h	??
x86/avx/store2u.h	??
x86/avx2/store2u.h	??
x86/avx512_knl/store2u.h	??
x86/avx512_skylake/store2u.h	??
x86/sse2/store2u.h	??
x86/sse42/store2u.h	??
arm/aarch64/store3a.h	??
arm/neon128/store3a.h	??
arm/sve/store3a.h	??
arm/sve1024/store3a.h	??
arm/sve128/store3a.h	??
arm/sve2048/store3a.h	??
arm/sve256/store3a.h	??
arm/sve512/store3a.h	??
cpu/cpu/store3a.h	??
ppc/vmx/store3a.h	??
ppc/vsx/store3a.h	??
x86/avx/store3a.h	??
x86/avx2/store3a.h	??
x86/avx512_knl/store3a.h	??
x86/avx512_skylake/store3a.h	??
x86/sse2/store3a.h	??
x86/sse42/store3a.h	??
arm/aarch64/store3u.h	??
arm/neon128/store3u.h	??
arm/sve/store3u.h	??
arm/sve1024/store3u.h	??
arm/sve128/store3u.h	??
arm/sve2048/store3u.h	??
arm/sve256/store3u.h	??
arm/sve512/store3u.h	??
cpu/cpu/store3u.h	??
ppc/vmx/store3u.h	??
ppc/vsx/store3u.h	??
x86/avx/store3u.h	??
x86/avx2/store3u.h	??
x86/avx512_knl/store3u.h	??
x86/avx512_skylake/store3u.h	??
x86/sse2/store3u.h	??
x86/sse42/store3u.h	??
arm/aarch64/store4a.h	??
arm/neon128/store4a.h	??
arm/sve/store4a.h	??
arm/sve1024/store4a.h	??
arm/sve128/store4a.h	??
arm/sve2048/store4a.h	??
arm/sve256/store4a.h	??
arm/sve512/store4a.h	??
cpu/cpu/store4a.h	??
ppc/vmx/store4a.h	??
ppc/vsx/store4a.h	??
x86/avx/store4a.h	??
x86/avx2/store4a.h	??
x86/avx512_knl/store4a.h	??

x86/avx512_skylake/store4a.h	??
x86/sse2/store4a.h	??
x86/sse42/store4a.h	??
arm/aarch64/store4u.h	??
arm/neon128/store4u.h	??
arm/sve/store4u.h	??
arm/sve1024/store4u.h	??
arm/sve128/store4u.h	??
arm/sve2048/store4u.h	??
arm/sve256/store4u.h	??
arm/sve512/store4u.h	??
cpu/cpu/store4u.h	??
ppc/vmx/store4u.h	??
ppc/vsx/store4u.h	??
x86/avx/store4u.h	??
x86/avx2/store4u.h	??
x86/avx512_knl/store4u.h	??
x86/avx512_skylake/store4u.h	??
x86/sse2/store4u.h	??
x86/sse42/store4u.h	??
arm/aarch64/storea.h	??
arm/neon128/storea.h	??
arm/sve/storea.h	??
arm/sve1024/storea.h	??
arm/sve128/storea.h	??
arm/sve2048/storea.h	??
arm/sve256/storea.h	??
arm/sve512/storea.h	??
cpu/cpu/storea.h	??
ppc/vmx/storea.h	??
ppc/vsx/storea.h	??
x86/avx/storea.h	??
x86/avx2/storea.h	??
x86/avx512_knl/storea.h	??
x86/avx512_skylake/storea.h	??
x86/sse2/storea.h	??
x86/sse42/storea.h	??
arm/aarch64/storela.h	??
arm/neon128/storela.h	??
arm/sve/storela.h	??
arm/sve1024/storela.h	??
arm/sve128/storela.h	??
arm/sve2048/storela.h	??
arm/sve256/storela.h	??
arm/sve512/storela.h	??
cpu/cpu/storela.h	??
ppc/vmx/storela.h	??
ppc/vsx/storela.h	??
x86/avx/storela.h	??
x86/avx2/storela.h	??
x86/avx512_knl/storela.h	??
x86/avx512_skylake/storela.h	??
x86/sse2/storela.h	??
x86/sse42/storela.h	??
arm/aarch64/storelu.h	??
arm/neon128/storelu.h	??
arm/sve/storelu.h	??
arm/sve1024/storelu.h	??

arm/sve128/storelu.h	??
arm/sve2048/storelu.h	??
arm/sve256/storelu.h	??
arm/sve512/storelu.h	??
cpu/cpu/storelu.h	??
ppc/vmx/storelu.h	??
ppc/vsx/storelu.h	??
x86/avx/storelu.h	??
x86/avx2/storelu.h	??
x86/avx512_knl/storelu.h	??
x86/avx512_skylake/storelu.h	??
x86/sse2/storelu.h	??
x86/sse42/storelu.h	??
arm/aarch64/storeu.h	??
arm/neon128/storeu.h	??
arm/sve/storeu.h	??
arm/sve1024/storeu.h	??
arm/sve128/storeu.h	??
arm/sve2048/storeu.h	??
arm/sve256/storeu.h	??
arm/sve512/storeu.h	??
cpu/cpu/storeu.h	??
ppc/vmx/storeu.h	??
ppc/vsx/storeu.h	??
x86/avx/storeu.h	??
x86/avx2/storeu.h	??
x86/avx512_knl/storeu.h	??
x86/avx512_skylake/storeu.h	??
x86/sse2/storeu.h	??
x86/sse42/storeu.h	??
arm/aarch64/sub.h	??
arm/neon128/sub.h	??
arm/sve/sub.h	??
arm/sve1024/sub.h	??
arm/sve128/sub.h	??
arm/sve2048/sub.h	??
arm/sve256/sub.h	??
arm/sve512/sub.h	??
cpu/cpu/sub.h	??
ppc/vmx/sub.h	??
ppc/vsx/sub.h	??
x86/avx/sub.h	??
x86/avx2/sub.h	??
x86/avx512_knl/sub.h	??
x86/avx512_skylake/sub.h	??
x86/sse2/sub.h	??
x86/sse42/sub.h	??
arm/aarch64/subs.h	??
arm/neon128/subs.h	??
arm/sve/subs.h	??
arm/sve1024/subs.h	??
arm/sve128/subs.h	??
arm/sve2048/subs.h	??
arm/sve256/subs.h	??
arm/sve512/subs.h	??
cpu/cpu/subs.h	??
ppc/vmx/subs.h	??
ppc/vsx/subs.h	??

x86/avx/subs.h	??
x86/avx2/subs.h	??
x86/avx512_knl/subs.h	??
x86/avx512_skylake/subs.h	??
x86/sse2/subs.h	??
x86/sse42/subs.h	??
arm/aarch64/tan_u10.h	??
arm/neon128/tan_u10.h	??
arm/sve/tan_u10.h	??
arm/sve1024/tan_u10.h	??
arm/sve128/tan_u10.h	??
arm/sve2048/tan_u10.h	??
arm/sve256/tan_u10.h	??
arm/sve512/tan_u10.h	??
cpu/cpu/tan_u10.h	??
ppc/vmx/tan_u10.h	??
ppc/vsx/tan_u10.h	??
x86/avx/tan_u10.h	??
x86/avx2/tan_u10.h	??
x86/avx512_knl/tan_u10.h	??
x86/avx512_skylake/tan_u10.h	??
x86/sse2/tan_u10.h	??
x86/sse42/tan_u10.h	??
arm/aarch64/tan_u35.h	??
arm/neon128/tan_u35.h	??
arm/sve/tan_u35.h	??
arm/sve1024/tan_u35.h	??
arm/sve128/tan_u35.h	??
arm/sve2048/tan_u35.h	??
arm/sve256/tan_u35.h	??
arm/sve512/tan_u35.h	??
cpu/cpu/tan_u35.h	??
ppc/vmx/tan_u35.h	??
ppc/vsx/tan_u35.h	??
x86/avx/tan_u35.h	??
x86/avx2/tan_u35.h	??
x86/avx512_knl/tan_u35.h	??
x86/avx512_skylake/tan_u35.h	??
x86/sse2/tan_u35.h	??
x86/sse42/tan_u35.h	??
arm/aarch64/tanh_u10.h	??
arm/neon128/tanh_u10.h	??
arm/sve/tanh_u10.h	??
arm/sve1024/tanh_u10.h	??
arm/sve128/tanh_u10.h	??
arm/sve2048/tanh_u10.h	??
arm/sve256/tanh_u10.h	??
arm/sve512/tanh_u10.h	??
cpu/cpu/tanh_u10.h	??
ppc/vmx/tanh_u10.h	??
ppc/vsx/tanh_u10.h	??
x86/avx/tanh_u10.h	??
x86/avx2/tanh_u10.h	??
x86/avx512_knl/tanh_u10.h	??
x86/avx512_skylake/tanh_u10.h	??
x86/sse2/tanh_u10.h	??
x86/sse42/tanh_u10.h	??
arm/aarch64/tanh_u35.h	??

arm/neon128/tanh_u35.h	??
arm/sve/tanh_u35.h	??
arm/sve1024/tanh_u35.h	??
arm/sve128/tanh_u35.h	??
arm/sve2048/tanh_u35.h	??
arm/sve256/tanh_u35.h	??
arm/sve512/tanh_u35.h	??
cpu/cpu/tanh_u35.h	??
ppc/vmx/tanh_u35.h	??
ppc/vsx/tanh_u35.h	??
x86/avx/tanh_u35.h	??
x86/avx2/tanh_u35.h	??
x86/avx512_knl/tanh_u35.h	??
x86/avx512_skylake/tanh_u35.h	??
x86/sse2/tanh_u35.h	??
x86/sse42/tanh_u35.h	??
arm/aarch64/tgamma_u10.h	??
arm/neon128/tgamma_u10.h	??
arm/sve/tgamma_u10.h	??
arm/sve1024/tgamma_u10.h	??
arm/sve128/tgamma_u10.h	??
arm/sve2048/tgamma_u10.h	??
arm/sve256/tgamma_u10.h	??
arm/sve512/tgamma_u10.h	??
cpu/cpu/tgamma_u10.h	??
ppc/vmx/tgamma_u10.h	??
ppc/vsx/tgamma_u10.h	??
x86/avx/tgamma_u10.h	??
x86/avx2/tgamma_u10.h	??
x86/avx512_knl/tgamma_u10.h	??
x86/avx512_skylake/tgamma_u10.h	??
x86/sse2/tgamma_u10.h	??
x86/sse42/tgamma_u10.h	??
arm/aarch64/to_logical.h	??
arm/neon128/to_logical.h	??
arm/sve/to_logical.h	??
arm/sve1024/to_logical.h	??
arm/sve128/to_logical.h	??
arm/sve2048/to_logical.h	??
arm/sve256/to_logical.h	??
arm/sve512/to_logical.h	??
cpu/cpu/to_logical.h	??
ppc/vmx/to_logical.h	??
ppc/vsx/to_logical.h	??
x86/avx/to_logical.h	??
x86/avx2/to_logical.h	??
x86/avx512_knl/to_logical.h	??
x86/avx512_skylake/to_logical.h	??
x86/sse2/to_logical.h	??
x86/sse42/to_logical.h	??
arm/aarch64/to_mask.h	??
arm/neon128/to_mask.h	??
arm/sve/to_mask.h	??
arm/sve1024/to_mask.h	??
arm/sve128/to_mask.h	??
arm/sve2048/to_mask.h	??
arm/sve256/to_mask.h	??
arm/sve512/to_mask.h	??

cpu/cpu/to_mask.h	??
ppc/vmx/to_mask.h	??
ppc/vsx/to_mask.h	??
x86/avx/to_mask.h	??
x86/avx2/to_mask.h	??
x86/avx512_knl/to_mask.h	??
x86/avx512_skylake/to_mask.h	??
x86/sse2/to_mask.h	??
x86/sse42/to_mask.h	??
arm/aarch64/trunc.h	??
arm/neon128/trunc.h	??
arm/sve/trunc.h	??
arm/sve1024/trunc.h	??
arm/sve128/trunc.h	??
arm/sve2048/trunc.h	??
arm/sve256/trunc.h	??
arm/sve512/trunc.h	??
cpu/cpu/trunc.h	??
ppc/vmx/trunc.h	??
ppc/vsx/trunc.h	??
x86/avx/trunc.h	??
x86/avx2/trunc.h	??
x86/avx512_knl/trunc.h	??
x86/avx512_skylake/trunc.h	??
x86/sse2/trunc.h	??
x86/sse42/trunc.h	??
turbojpeg.h	??
TYApi.h	
TYApi.h includes camera control and data receiving interface, which supports configuration for image resolution, frame rate, exposure time, gain, working mode,etc	151
TYCAPL.h	??
TYCoordinateMapper.h	
Coordinate Conversion API	220
TYDefs.h	
TYDefs.h includes camera control and data receiving data definitions which supports configura- tion for image resolution, frame rate, exposure time, gain, working mode,etc	231
TYFeatureList.h	
Camera feature enumeration list	254
TYImageProc.h	
Image post-process API	271
TYParameter.h	??
arm/aarch64/types.h	??
arm/neon128/types.h	??
arm/sve/types.h	??
arm/sve1024/types.h	??
arm/sve128/types.h	??
arm/sve2048/types.h	??
arm/sve256/types.h	??
arm/sve512/types.h	??
cpu/cpu/types.h	??
ppc/vmx/types.h	??
ppc/vsx/types.h	??
x86/avx/types.h	??
x86/avx2/types.h	??
x86/avx512_knl/types.h	??
x86/avx512_skylake/types.h	??

x86/sse2/types.h	??
x86/sse42/types.h	??
TYVer.h	??
arm/aarch64/unzip.h	??
arm/neon128/unzip.h	??
arm/sve/unzip.h	??
arm/sve1024/unzip.h	??
arm/sve128/unzip.h	??
arm/sve2048/unzip.h	??
arm/sve256/unzip.h	??
arm/sve512/unzip.h	??
cpu/cpu/unzip.h	??
ppc/vmx/unzip.h	??
ppc/vsx/unzip.h	??
x86/avx/unzip.h	??
x86/avx2/unzip.h	??
x86/avx512_knl/unzip.h	??
x86/avx512_skylake/unzip.h	??
x86/sse2/unzip.h	??
x86/sse42/unzip.h	??
arm/aarch64/unziphi.h	??
arm/neon128/unziphi.h	??
arm/sve/unziphi.h	??
arm/sve1024/unziphi.h	??
arm/sve128/unziphi.h	??
arm/sve2048/unziphi.h	??
arm/sve256/unziphi.h	??
arm/sve512/unziphi.h	??
cpu/cpu/unziphi.h	??
ppc/vmx/unziphi.h	??
ppc/vsx/unziphi.h	??
x86/avx/unziphi.h	??
x86/avx2/unziphi.h	??
x86/avx512_knl/unziphi.h	??
x86/avx512_skylake/unziphi.h	??
x86/sse2/unziphi.h	??
x86/sse42/unziphi.h	??
arm/aarch64/unziplo.h	??
arm/neon128/unziplo.h	??
arm/sve/unziplo.h	??
arm/sve1024/unziplo.h	??
arm/sve128/unziplo.h	??
arm/sve2048/unziplo.h	??
arm/sve256/unziplo.h	??
arm/sve512/unziplo.h	??
cpu/cpu/unziplo.h	??
ppc/vmx/unziplo.h	??
ppc/vsx/unziplo.h	??
x86/avx/unziplo.h	??
x86/avx2/unziplo.h	??
x86/avx512_knl/unziplo.h	??
x86/avx512_skylake/unziplo.h	??
x86/sse2/unziplo.h	??
x86/sse42/unziplo.h	??
arm/aarch64/upcvt.h	??
arm/neon128/upcvt.h	??
arm/sve/upcvt.h	??
arm/sve1024/upcvt.h	??

arm/sve128/upcvt.h	??
arm/sve2048/upcvt.h	??
arm/sve256/upcvt.h	??
arm/sve512/upcvt.h	??
cpu/cpu/upcvt.h	??
ppc/vmx/upcvt.h	??
ppc/vsx/upcvt.h	??
x86/avx/upcvt.h	??
x86/avx2/upcvt.h	??
x86/avx512_knl/upcvt.h	??
x86/avx512_skylake/upcvt.h	??
x86/sse2/upcvt.h	??
x86/sse42/upcvt.h	??
arm/aarch64/xorb.h	??
arm/neon128/xorb.h	??
arm/sve/xorb.h	??
arm/sve1024/xorb.h	??
arm/sve128/xorb.h	??
arm/sve2048/xorb.h	??
arm/sve256/xorb.h	??
arm/sve512/xorb.h	??
cpu/cpu/xorb.h	??
ppc/vmx/xorb.h	??
ppc/vsx/xorb.h	??
x86/avx/xorb.h	??
x86/avx2/xorb.h	??
x86/avx512_knl/xorb.h	??
x86/avx512_skylake/xorb.h	??
x86/sse2/xorb.h	??
x86/sse42/xorb.h	??
arm/aarch64/xorl.h	??
arm/neon128/xorl.h	??
arm/sve/xorl.h	??
arm/sve1024/xorl.h	??
arm/sve128/xorl.h	??
arm/sve2048/xorl.h	??
arm/sve256/xorl.h	??
arm/sve512/xorl.h	??
cpu/cpu/xorl.h	??
ppc/vmx/xorl.h	??
ppc/vsx/xorl.h	??
x86/avx/xorl.h	??
x86/avx2/xorl.h	??
x86/avx512_knl/xorl.h	??
x86/avx512_skylake/xorl.h	??
x86/sse2/xorl.h	??
x86/sse42/xorl.h	??
zconf.h	??
arm/aarch64/zip.h	??
arm/neon128/zip.h	??
arm/sve/zip.h	??
arm/sve1024/zip.h	??
arm/sve128/zip.h	??
arm/sve2048/zip.h	??
arm/sve256/zip.h	??
arm/sve512/zip.h	??
cpu/cpu/zip.h	??
ppc/vmx/zip.h	??

ppc/vsx/zip.h	??
x86/avx/zip.h	??
x86/avx2/zip.h	??
x86/avx512_knl/zip.h	??
x86/avx512_skylake/zip.h	??
x86/sse2/zip.h	??
x86/sse42/zip.h	??
arm/aarch64/ziphi.h	??
arm/neon128/ziphi.h	??
arm/sve/ziphi.h	??
arm/sve1024/ziphi.h	??
arm/sve128/ziphi.h	??
arm/sve2048/ziphi.h	??
arm/sve256/ziphi.h	??
arm/sve512/ziphi.h	??
cpu/cpu/ziphi.h	??
ppc/vmx/ziphi.h	??
ppc/vsx/ziphi.h	??
x86/avx/ziphi.h	??
x86/avx2/ziphi.h	??
x86/avx512_knl/ziphi.h	??
x86/avx512_skylake/ziphi.h	??
x86/sse2/ziphi.h	??
x86/sse42/ziphi.h	??
arm/aarch64/ziplo.h	??
arm/neon128/ziplo.h	??
arm/sve/ziplo.h	??
arm/sve1024/ziplo.h	??
arm/sve128/ziplo.h	??
arm/sve2048/ziplo.h	??
arm/sve256/ziplo.h	??
arm/sve512/ziplo.h	??
cpu/cpu/ziplo.h	??
ppc/vmx/ziplo.h	??
ppc/vsx/ziplo.h	??
x86/avx/ziplo.h	??
x86/avx2/ziplo.h	??
x86/avx512_knl/ziplo.h	??
x86/avx512_skylake/ziplo.h	??
x86/sse2/ziplo.h	??
x86/sse42/ziplo.h	??
zlib.h	??

Chapter 5

Module Documentation

5.1 TurboJPEG

Classes

- struct [tjscalingfactor](#)
- struct [tjregion](#)
- struct [tjtransform](#)

Macros

- #define [TJ_NUMSAMP](#) 6
- #define [TJ_NUMPF](#) 12
- #define [TJ_NUMCS](#) 5
- #define [TJFLAG_BOTTOMUP](#) 2
- #define [TJFLAG_FASTUPSAMPLE](#) 256
- #define [TJFLAG_NOREALLOC](#) 1024
- #define [TJFLAG_FASTDCT](#) 2048
- #define [TJFLAG_ACCURATEDCT](#) 4096
- #define [TJFLAG_STOPONWARNING](#) 8192
- #define [TJFLAG_PROGRESSIVE](#) 16384
- #define [TJ_NUMERR](#) 2
- #define [TJ_NUMXOP](#) 8
- #define [TJXOPT_PERFECT](#) 1
- #define [TJXOPT_TRIM](#) 2
- #define [TJXOPT_CROP](#) 4
- #define [TJXOPT_GRAY](#) 8
- #define [TJXOPT_NOOUTPUT](#) 16
- #define [TJXOPT_PROGRESSIVE](#) 32
- #define [TJXOPT_COPYNONE](#) 64
- #define [TJPAD](#)(width) (((width) + 3) & (~3))
- #define [TJSCALED](#)(dimension, scalingFactor)
- #define [TJFLAG_FORCEMMX](#) 8
- #define [TJFLAG_FORCESSE](#) 16
- #define [TJFLAG_FORCESSE2](#) 32
- #define [TJFLAG_FORCESSE3](#) 128
- #define [NUMSUBOPT](#) [TJ_NUMSAMP](#)

- `#define TJ_444 TJSAMP_444`
- `#define TJ_422 TJSAMP_422`
- `#define TJ_420 TJSAMP_420`
- `#define TJ_411 TJSAMP_420`
- `#define TJ_GRAYSCALE TJSAMP_GRAY`
- `#define TJ_BGR 1`
- `#define TJ_BOTTOMUP TJFLAG_BOTTOMUP`
- `#define TJ_FORCEMMX TJFLAG_FORCEMMX`
- `#define TJ_FORCESSE TJFLAG_FORCESSE`
- `#define TJ_FORCESSE2 TJFLAG_FORCESSE2`
- `#define TJ_ALPHAFIRST 64`
- `#define TJ_FORCESSE3 TJFLAG_FORCESSE3`
- `#define TJ_FASTUPSAMPLE TJFLAG_FASTUPSAMPLE`
- `#define TJ_YUV 512`

Typedefs

- typedef struct [tjtransform](#) [tjtransform](#)
- typedef void * [tjhandle](#)

Enumerations

- enum [TJSAMP](#) {
[TJSAMP_444](#) = 0, [TJSAMP_422](#), [TJSAMP_420](#), [TJSAMP_GRAY](#),
[TJSAMP_440](#), [TJSAMP_411](#) }
- enum [TJPF](#) {
[TJPF_RGB](#) = 0, [TJPF_BGR](#), [TJPF_RGBX](#), [TJPF_BGRX](#),
[TJPF_XBGR](#), [TJPF_XRGB](#), [TJPF_GRAY](#), [TJPF_RGBA](#),
[TJPF_BGRA](#), [TJPF_ABGR](#), [TJPF_ARGB](#), [TJPF_CMYK](#),
[TJPF_UNKNOWN](#) = -1 }
- enum [TJCS](#) {
[TJCS_RGB](#) = 0, [TJCS_YCbCr](#), [TJCS_GRAY](#), [TJCS_CMYK](#),
[TJCS_YCCK](#) }
- enum [TJERR](#) { [TJERR_WARNING](#) = 0, [TJERR_FATAL](#) }
- enum [TJXOP](#) {
[TJXOP_NONE](#) = 0, [TJXOP_HFLIP](#), [TJXOP_VFLIP](#), [TJXOP_TRANSPOSE](#),
[TJXOP_TRANSVERSE](#), [TJXOP_ROT90](#), [TJXOP_ROT180](#), [TJXOP_ROT270](#) }

Functions

- DLL_EXPORT [tjhandle](#) [tjInitCompress](#) (void)
- DLL_EXPORT int [tjCompress2](#) ([tjhandle](#) handle, const unsigned char *srcBuf, int width, int pitch, int height, int pixelFormat, unsigned char **jpegBuf, unsigned long *jpegSize, int jpegSubsamp, int jpegQual, int flags)
- DLL_EXPORT int [tjCompressFromYUV](#) ([tjhandle](#) handle, const unsigned char *srcBuf, int width, int pad, int height, int subsamp, unsigned char **jpegBuf, unsigned long *jpegSize, int jpegQual, int flags)
- DLL_EXPORT int [tjCompressFromYUVPlanes](#) ([tjhandle](#) handle, const unsigned char **srcPlanes, int width, const int *strides, int height, int subsamp, unsigned char **jpegBuf, unsigned long *jpegSize, int jpegQual, int flags)
- DLL_EXPORT unsigned long [tjBufSize](#) (int width, int height, int jpegSubsamp)
- DLL_EXPORT unsigned long [tjBufSizeYUV2](#) (int width, int pad, int height, int subsamp)
- DLL_EXPORT unsigned long [tjPlaneSizeYUV](#) (int componentID, int width, int stride, int height, int subsamp)
- DLL_EXPORT int [tjPlaneWidth](#) (int componentID, int width, int subsamp)

- DLLEXPORT int [tjPlaneHeight](#) (int componentID, int height, int subsamp)
- DLLEXPORT int [tjEncodeYUV3](#) ([tjhandle](#) handle, const unsigned char *srcBuf, int width, int pitch, int height, int pixelFormat, unsigned char *dstBuf, int pad, int subsamp, int flags)
- DLLEXPORT int [tjEncodeYUVPlanes](#) ([tjhandle](#) handle, const unsigned char *srcBuf, int width, int pitch, int height, int pixelFormat, unsigned char **dstPlanes, int *strides, int subsamp, int flags)
- DLLEXPORT [tjhandle](#) [tjInitDecompress](#) (void)
- DLLEXPORT int [tjDecompressHeader3](#) ([tjhandle](#) handle, const unsigned char *jpegBuf, unsigned long jpegSize, int *width, int *height, int *jpegSubsamp, int *jpegColorspace)
- DLLEXPORT [tjscalingfactor](#) * [tjGetScalingFactors](#) (int *numscalingfactors)
- DLLEXPORT int [tjDecompress2](#) ([tjhandle](#) handle, const unsigned char *jpegBuf, unsigned long jpegSize, unsigned char *dstBuf, int width, int pitch, int height, int pixelFormat, int flags)
- DLLEXPORT int [tjDecompressToYUV2](#) ([tjhandle](#) handle, const unsigned char *jpegBuf, unsigned long jpegSize, unsigned char *dstBuf, int width, int pad, int height, int flags)
- DLLEXPORT int [tjDecompressToYUVPlanes](#) ([tjhandle](#) handle, const unsigned char *jpegBuf, unsigned long jpegSize, unsigned char **dstPlanes, int width, int *strides, int height, int flags)
- DLLEXPORT int [tjDecodeYUV](#) ([tjhandle](#) handle, const unsigned char *srcBuf, int pad, int subsamp, unsigned char *dstBuf, int width, int pitch, int height, int pixelFormat, int flags)
- DLLEXPORT int [tjDecodeYUVPlanes](#) ([tjhandle](#) handle, const unsigned char **srcPlanes, const int *strides, int subsamp, unsigned char *dstBuf, int width, int pitch, int height, int pixelFormat, int flags)
- DLLEXPORT [tjhandle](#) [tjInitTransform](#) (void)
- DLLEXPORT int [tjTransform](#) ([tjhandle](#) handle, const unsigned char *jpegBuf, unsigned long jpegSize, int n, unsigned char **dstBufs, unsigned long *dstSizes, [tjtransform](#) *transforms, int flags)
- DLLEXPORT int [tjDestroy](#) ([tjhandle](#) handle)
- DLLEXPORT unsigned char * [tjAlloc](#) (int bytes)
- DLLEXPORT unsigned char * [tjLoadImage](#) (const char *filename, int *width, int align, int *height, int *pixelFormat, int flags)
- DLLEXPORT int [tjSaveImage](#) (const char *filename, unsigned char *buffer, int width, int pitch, int height, int pixelFormat, int flags)
- DLLEXPORT void [tjFree](#) (unsigned char *buffer)
- DLLEXPORT char * [tjGetErrorStr2](#) ([tjhandle](#) handle)
- DLLEXPORT int [tjGetErrorCode](#) ([tjhandle](#) handle)
- DLLEXPORT unsigned long **TJBUFSIZE** (int width, int height)
- DLLEXPORT unsigned long **TJBUFSIZEYUV** (int width, int height, int jpegSubsamp)
- DLLEXPORT unsigned long **tjBufSizeYUV** (int width, int height, int subsamp)
- DLLEXPORT int **tjCompress** ([tjhandle](#) handle, unsigned char *srcBuf, int width, int pitch, int height, int pixelSize, unsigned char *dstBuf, unsigned long *compressedSize, int jpegSubsamp, int jpegQual, int flags)
- DLLEXPORT int **tjEncodeYUV** ([tjhandle](#) handle, unsigned char *srcBuf, int width, int pitch, int height, int pixelSize, unsigned char *dstBuf, int subsamp, int flags)
- DLLEXPORT int **tjEncodeYUV2** ([tjhandle](#) handle, unsigned char *srcBuf, int width, int pitch, int height, int pixelFormat, unsigned char *dstBuf, int subsamp, int flags)
- DLLEXPORT int **tjDecompressHeader** ([tjhandle](#) handle, unsigned char *jpegBuf, unsigned long jpegSize, int *width, int *height)
- DLLEXPORT int **tjDecompressHeader2** ([tjhandle](#) handle, unsigned char *jpegBuf, unsigned long jpegSize, int *width, int *height, int *jpegSubsamp)
- DLLEXPORT int **tjDecompress** ([tjhandle](#) handle, unsigned char *jpegBuf, unsigned long jpegSize, unsigned char *dstBuf, int width, int pitch, int height, int pixelSize, int flags)
- DLLEXPORT int **tjDecompressToYUV** ([tjhandle](#) handle, unsigned char *jpegBuf, unsigned long jpegSize, unsigned char *dstBuf, int flags)
- DLLEXPORT char * **tjGetErrorStr** (void)

5.1.1 Detailed Description

TurboJPEG API. This API provides an interface for generating, decoding, and transforming planar YUV and JPEG images in memory.

YUV Image Format Notes

Technically, the JPEG format uses the YCbCr colorspace (which is technically not a colorspace but a color transform), but per the convention of the digital video community, the TurboJPEG API uses "YUV" to refer to an image format consisting of Y, Cb, and Cr image planes.

Each plane is simply a 2D array of bytes, each byte representing the value of one of the components (Y, Cb, or Cr) at a particular location in the image. The width and height of each plane are determined by the image width, height, and level of chrominance subsampling. The luminance plane width is the image width padded to the nearest multiple of the horizontal subsampling factor (2 in the case of 4:2:0 and 4:2:2, 4 in the case of 4:1:1, 1 in the case of 4:4:4 or grayscale.) Similarly, the luminance plane height is the image height padded to the nearest multiple of the vertical subsampling factor (2 in the case of 4:2:0 or 4:4:0, 1 in the case of 4:4:4 or grayscale.) This is irrespective of any additional padding that may be specified as an argument to the various YUV functions. The chrominance plane width is equal to the luminance plane width divided by the horizontal subsampling factor, and the chrominance plane height is equal to the luminance plane height divided by the vertical subsampling factor.

For example, if the source image is 35 x 35 pixels and 4:2:2 subsampling is used, then the luminance plane would be 36 x 35 bytes, and each of the chrominance planes would be 18 x 35 bytes. If you specify a line padding of 4 bytes on top of this, then the luminance plane would be 36 x 35 bytes, and each of the chrominance planes would be 20 x 35 bytes.

5.1.2 Macro Definition Documentation

5.1.2.1 TJ_NUMCS

```
#define TJ_NUMCS 5
```

The number of JPEG colorspaces

Definition at line 310 of file turbojpeg.h.

5.1.2.2 TJ_NUMERR

```
#define TJ_NUMERR 2
```

The number of error codes

Definition at line 426 of file turbojpeg.h.

5.1.2.3 TJ_NUMPF

```
#define TJ_NUMPF 12
```

The number of pixel formats

Definition at line 160 of file turbojpeg.h.

5.1.2.4 TJ_NUMSAMP

```
#define TJ_NUMSAMP 6
```

The number of chrominance subsampling options

Definition at line 81 of file turbojpeg.h.

5.1.2.5 TJ_NUMXOP

```
#define TJ_NUMXOP 8
```

The number of transform operations

Definition at line 447 of file turbojpeg.h.

5.1.2.6 TJFLAG_ACCURATEDCT

```
#define TJFLAG_ACCURATEDCT 4096
```

Use the most accurate DCT/IDCT algorithm available in the underlying codec. The default if this flag is not specified is implementation-specific. For example, the implementation of TurboJPEG for libjpeg[-turbo] uses the fast algorithm by default when compressing, because this has been shown to have only a very slight effect on accuracy, but it uses the accurate algorithm when decompressing, because this has been shown to have a larger effect.

Definition at line 406 of file turbojpeg.h.

5.1.2.7 TJFLAG_BOTTOMUP

```
#define TJFLAG_BOTTOMUP 2
```

The uncompressed source/destination image is stored in bottom-up (Windows, OpenGL) order, not top-down (X11) order.

Definition at line 372 of file turbojpeg.h.

5.1.2.8 TJFLAG_FASTDCT

```
#define TJFLAG_FASTDCT 2048
```

Use the fastest DCT/IDCT algorithm available in the underlying codec. The default if this flag is not specified is implementation-specific. For example, the implementation of TurboJPEG for libjpeg[-turbo] uses the fast algorithm by default when compressing, because this has been shown to have only a very slight effect on accuracy, but it uses the accurate algorithm when decompressing, because this has been shown to have a larger effect.

Definition at line 397 of file turbojpeg.h.

5.1.2.9 TJFLAG_FASTUPSAMPLE

```
#define TJFLAG_FASTUPSAMPLE 256
```

When decompressing an image that was compressed using chrominance subsampling, use the fastest chrominance upsampling algorithm available in the underlying codec. The default is to use smooth upsampling, which creates a smooth transition between neighboring chrominance components in order to reduce upsampling artifacts in the decompressed image.

Definition at line 380 of file turbojpeg.h.

5.1.2.10 TJFLAG_NOREALLOC

```
#define TJFLAG_NOREALLOC 1024
```

Disable buffer (re)allocation. If passed to one of the JPEG compression or transform functions, this flag will cause those functions to generate an error if the JPEG image buffer is invalid or too small rather than attempting to allocate or reallocate that buffer. This reproduces the behavior of earlier versions of TurboJPEG.

Definition at line 388 of file turbojpeg.h.

5.1.2.11 TJFLAG_PROGRESSIVE

```
#define TJFLAG_PROGRESSIVE 16384
```

Use progressive entropy coding in JPEG images generated by the compression and transform functions. Progressive entropy coding will generally improve compression relative to baseline entropy coding (the default), but it will reduce compression and decompression performance considerably.

Definition at line 420 of file turbojpeg.h.

5.1.2.12 TJFLAG_STOPONWARNING

```
#define TJFLAG_STOPONWARNING 8192
```

Immediately discontinue the current compression/decompression/transform operation if the underlying codec throws a warning (non-fatal error). The default behavior is to allow the operation to complete unless a fatal error is encountered.

Definition at line 413 of file turbojpeg.h.

5.1.2.13 TJPAD

```
#define TJPAD(  
    width ) (((width) + 3) & (~3))
```

Pad the given width to the nearest 32-bit boundary

Definition at line 657 of file turbojpeg.h.

5.1.2.14 TJSCALED

```
#define TJSCALED(  
    dimension,  
    scalingFactor )
```

Value:

```
((dimension * scalingFactor.num + scalingFactor.denom - 1) / \
```

```
scalingFactor.denom)
```

Compute the scaled value of `dimension` using the given scaling factor. This macro performs the integer equivalent of `ceil(dimension * scalingFactor)`.

Definition at line 664 of file turbojpeg.h.

5.1.2.15 TJXOPT_COPYNONE

```
#define TJXOPT_COPYNONE 64
```

This option will prevent [tjTransform\(\)](#) from copying any extra markers (including EXIF and ICC profile data) from the source image to the output image.

Definition at line 546 of file turbojpeg.h.

5.1.2.16 TJXOPT_CROP

```
#define TJXOPT_CROP 4
```

This option will enable lossless cropping. See [tjTransform\(\)](#) for more information.

Definition at line 520 of file turbojpeg.h.

5.1.2.17 TJXOPT_GRAY

```
#define TJXOPT_GRAY 8
```

This option will discard the color data in the input image and produce a grayscale output image.

Definition at line 525 of file turbojpeg.h.

5.1.2.18 TJXOPT_NOOUTPUT

```
#define TJXOPT_NOOUTPUT 16
```

This option will prevent [tjTransform\(\)](#) from outputting a JPEG image for this particular transform (this can be used in conjunction with a custom filter to capture the transformed DCT coefficients without transcoding them.)

Definition at line 532 of file turbojpeg.h.

5.1.2.19 TJXOPT_PERFECT

```
#define TJXOPT_PERFECT 1
```

This option will cause [tjTransform\(\)](#) to return an error if the transform is not perfect. Lossless transforms operate on MCU blocks, whose size depends on the level of chrominance subsampling used (see [#tjMCUWidth](#) and [#tjMCUHeight](#).) If the image's width or height is not evenly divisible by the MCU block size, then there will be partial MCU blocks on the right and/or bottom edges. It is not possible to move these partial MCU blocks to the top or left of the image, so any transform that would require that is "imperfect." If this option is not specified, then any partial MCU blocks that cannot be transformed will be left in place, which will create odd-looking strips on the right or bottom edge of the image.

Definition at line 510 of file turbojpeg.h.

5.1.2.20 TJXOPT_PROGRESSIVE

```
#define TJXOPT_PROGRESSIVE 32
```

This option will enable progressive entropy coding in the output image generated by this particular transform. Progressive entropy coding will generally improve compression relative to baseline entropy coding (the default), but it will reduce compression and decompression performance considerably.

Definition at line 540 of file turbojpeg.h.

5.1.2.21 TJXOPT_TRIM

```
#define TJXOPT_TRIM 2
```

This option will cause [tjTransform\(\)](#) to discard any partial MCU blocks that cannot be transformed.

Definition at line 515 of file turbojpeg.h.

5.1.3 Typedef Documentation

5.1.3.1 tjhandle

```
typedef void* tjhandle
```

TurboJPEG instance handle

Definition at line 651 of file turbojpeg.h.

5.1.3.2 tjtransform

```
typedef struct tjtransform tjtransform
```

Lossless transform

5.1.4 Enumeration Type Documentation

5.1.4.1 TJCS

```
enum TJCS
```

JPEG colorspaces

Enumerator

TJCS_RGB	RGB colorspace. When compressing the JPEG image, the R, G, and B components in the source image are reordered into image planes, but no colorspace conversion or subsampling is performed. RGB JPEG images can be decompressed to any of the extended RGB pixel formats or grayscale, but they cannot be decompressed to YUV images.
----------	--

Enumerator

TJCS_YCbCr	YCbCr colorspace. YCbCr is not an absolute colorspace but rather a mathematical transformation of RGB designed solely for storage and transmission. YCbCr images must be converted to RGB before they can actually be displayed. In the YCbCr colorspace, the Y (luminance) component represents the black & white portion of the original image, and the Cb and Cr (chrominance) components represent the color portion of the original image. Originally, the analog equivalent of this transformation allowed the same signal to drive both black & white and color televisions, but JPEG images use YCbCr primarily because it allows the color data to be optionally subsampled for the purposes of reducing bandwidth or disk space. YCbCr is the most common JPEG colorspace, and YCbCr JPEG images can be compressed from and decompressed to any of the extended RGB pixel formats or grayscale, or they can be decompressed to YUV planar images.
TJCS_GRAY	Grayscale colorspace. The JPEG image retains only the luminance data (Y component), and any color data from the source image is discarded. Grayscale JPEG images can be compressed from and decompressed to any of the extended RGB pixel formats or grayscale, or they can be decompressed to YUV planar images.
TJCS_CMYK	CMYK colorspace. When compressing the JPEG image, the C, M, Y, and K components in the source image are reordered into image planes, but no colorspace conversion or subsampling is performed. CMYK JPEG images can only be decompressed to CMYK pixels.
TJCS_YCCK	YCCK colorspace. YCCK (AKA "YCbCrK") is not an absolute colorspace but rather a mathematical transformation of CMYK designed solely for storage and transmission. It is to CMYK as YCbCr is to RGB. CMYK pixels can be reversibly transformed into YCCK, and as with YCbCr, the chrominance components in the YCCK pixels can be subsampled without incurring major perceptual loss. YCCK JPEG images can only be compressed from and decompressed to CMYK pixels.

Definition at line 315 of file turbojpeg.h.

5.1.4.2 TJERR

enum [TJERR](#)

Error codes

Enumerator

TJERR_WARNING	The error was non-fatal and recoverable, but the image may still be corrupt.
TJERR_FATAL	The error was fatal and non-recoverable.

Definition at line 431 of file turbojpeg.h.

5.1.4.3 TJPF

enum [TJPF](#)

Pixel formats

Enumerator

TJPF_RGB	RGB pixel format. The red, green, and blue components in the image are stored in 3-byte pixels in the order R, G, B from lowest to highest byte address within each pixel.
TJPF_BGR	BGR pixel format. The red, green, and blue components in the image are stored in 3-byte pixels in the order B, G, R from lowest to highest byte address within each pixel.
TJPF_RGBX	RGBX pixel format. The red, green, and blue components in the image are stored in 4-byte pixels in the order R, G, B from lowest to highest byte address within each pixel. The X component is ignored when compressing and undefined when decompressing.
TJPF_BGRX	BGRX pixel format. The red, green, and blue components in the image are stored in 4-byte pixels in the order B, G, R from lowest to highest byte address within each pixel. The X component is ignored when compressing and undefined when decompressing.
TJPF_XBGR	XBGR pixel format. The red, green, and blue components in the image are stored in 4-byte pixels in the order R, G, B from highest to lowest byte address within each pixel. The X component is ignored when compressing and undefined when decompressing.
TJPF_XRGB	XRGB pixel format. The red, green, and blue components in the image are stored in 4-byte pixels in the order B, G, R from highest to lowest byte address within each pixel. The X component is ignored when compressing and undefined when decompressing.
TJPF_GRAY	Grayscale pixel format. Each 1-byte pixel represents a luminance (brightness) level from 0 to 255.
TJPF_RGBA	RGBA pixel format. This is the same as TJPF_RGBX , except that when decompressing, the X component is guaranteed to be 0xFF, which can be interpreted as an opaque alpha channel.
TJPF_BGRA	BGRA pixel format. This is the same as TJPF_BGRX , except that when decompressing, the X component is guaranteed to be 0xFF, which can be interpreted as an opaque alpha channel.
TJPF_ABGR	ABGR pixel format. This is the same as TJPF_XBGR , except that when decompressing, the X component is guaranteed to be 0xFF, which can be interpreted as an opaque alpha channel.
TJPF_ARGB	ARGB pixel format. This is the same as TJPF_XRGB , except that when decompressing, the X component is guaranteed to be 0xFF, which can be interpreted as an opaque alpha channel.
TJPF_CMYK	CMYK pixel format. Unlike RGB, which is an additive color model used primarily for display, CMYK (Cyan/Magenta/Yellow/Key) is a subtractive color model used primarily for printing. In the CMYK color model, the value of each color component typically corresponds to an amount of cyan, magenta, yellow, or black ink that is applied to a white background. In order to convert between CMYK and RGB, it is necessary to use a color management system (CMS.) A CMS will attempt to map colors within the printer's gamut to perceptually similar colors in the display's gamut and vice versa, but the mapping is typically not 1:1 or reversible, nor can it be defined with a simple formula. Thus, such a conversion is out of scope for a codec library. However, the TurboJPEG API allows for compressing CMYK pixels into a YCCK JPEG image (see TJCS_YCCK) and decompressing YCCK JPEG images into CMYK pixels.
TJPF_UNKNOWN	Unknown pixel format. Currently this is only used by tjLoadImage() .

Definition at line 165 of file turbojpeg.h.

5.1.4.4 TJSAMP

enum [TJSAMP](#)

Chrominance subsampling options. When pixels are converted from RGB to YCbCr (see [TJCS_YCbCr](#)) or from CMYK to YCCK (see [TJCS_YCCK](#)) as part of the JPEG compression process, some of the Cb and Cr (chrominance) components can be discarded or averaged together to produce a smaller image with little perceptible loss of image clarity (the human eye is more sensitive to small changes in brightness than to small changes in color.) This is called "chrominance subsampling".

Enumerator

TJSAMP_444	4:4:4 chrominance subsampling (no chrominance subsampling). The JPEG or YUV image will contain one chrominance component for every pixel in the source image.
TJSAMP_422	4:2:2 chrominance subsampling. The JPEG or YUV image will contain one chrominance component for every 2x1 block of pixels in the source image.
TJSAMP_420	4:2:0 chrominance subsampling. The JPEG or YUV image will contain one chrominance component for every 2x2 block of pixels in the source image.
TJSAMP_GRAY	Grayscale. The JPEG or YUV image will contain no chrominance components.
TJSAMP_440	4:4:0 chrominance subsampling. The JPEG or YUV image will contain one chrominance component for every 1x2 block of pixels in the source image. Note 4:4:0 subsampling is not fully accelerated in libjpeg-turbo.
TJSAMP_411	4:1:1 chrominance subsampling. The JPEG or YUV image will contain one chrominance component for every 4x1 block of pixels in the source image. JPEG images compressed with 4:1:1 subsampling will be almost exactly the same size as those compressed with 4:2:0 subsampling, and in the aggregate, both subsampling methods produce approximately the same perceptual quality. However, 4:1:1 is better able to reproduce sharp horizontal features. Note 4:1:1 subsampling is not fully accelerated in libjpeg-turbo.

Definition at line 92 of file turbojpeg.h.

5.1.4.5 TJXOP

enum [TJXOP](#)

Transform operations for [tjTransform\(\)](#)

Enumerator

TJXOP_NONE	Do not transform the position of the image pixels
TJXOP_HFLIP	Flip (mirror) image horizontally. This transform is imperfect if there are any partial MCU blocks on the right edge (see TJXOPT_PERFECT .)
TJXOP_VFLIP	Flip (mirror) image vertically. This transform is imperfect if there are any partial MCU blocks on the bottom edge (see TJXOPT_PERFECT .)
TJXOP_TRANSPOSE	Transpose image (flip/mirror along upper left to lower right axis.) This transform is always perfect.
TJXOP_TRANSVERSE	Transverse transpose image (flip/mirror along upper right to lower left axis.) This transform is imperfect if there are any partial MCU blocks in the image (see TJXOPT_PERFECT .)

Enumerator

TJXOP_ROT90	Rotate image clockwise by 90 degrees. This transform is imperfect if there are any partial MCU blocks on the bottom edge (see TJXOPT_PERFECT .)
TJXOP_ROT180	Rotate image 180 degrees. This transform is imperfect if there are any partial MCU blocks in the image (see TJXOPT_PERFECT .)
TJXOP_ROT270	Rotate image counter-clockwise by 90 degrees. This transform is imperfect if there are any partial MCU blocks on the right edge (see TJXOPT_PERFECT .)

Definition at line 452 of file turbojpeg.h.

5.1.5 Function Documentation

5.1.5.1 tjAlloc()

```
DLLEXPORT unsigned char* tjAlloc (
    int bytes )
```

Allocate an image buffer for use with TurboJPEG. You should always use this function to allocate the JPEG destination buffer(s) for the compression and transform functions unless you are disabling automatic buffer (re)allocation (by setting [TJFLAG_NOREALLOC](#).)

Parameters

<i>bytes</i>	the number of bytes to allocate
--------------	---------------------------------

Returns

a pointer to a newly-allocated buffer with the specified number of bytes.

See also

[tjFree\(\)](#)

5.1.5.2 tjBufSize()

```
DLLEXPORT unsigned long tjBufSize (
    int width,
    int height,
    int jpegSubsamp )
```

The maximum size of the buffer (in bytes) required to hold a JPEG image with the given parameters. The number of bytes returned by this function is larger than the size of the uncompressed source image. The reason for this is that the JPEG format uses 16-bit coefficients, and it is thus possible for a very high-quality JPEG image with very high-frequency content to expand rather than compress when converted to the JPEG format. Such images represent a very rare corner case, but since there is no way to predict the size of a JPEG image prior to compression, the corner case has to be handled.

Parameters

<i>width</i>	width (in pixels) of the image
<i>height</i>	height (in pixels) of the image
<i>jpegSubsamp</i>	the level of chrominance subsampling to be used when generating the JPEG image (see Chrominance subsampling options.)

Returns

the maximum size of the buffer (in bytes) required to hold the image, or -1 if the arguments are out of bounds.

5.1.5.3 `tjBufSizeYUV2()`

```
DLLEXPORT unsigned long tjBufSizeYUV2 (
    int width,
    int pad,
    int height,
    int subsamp )
```

The size of the buffer (in bytes) required to hold a YUV planar image with the given parameters.

Parameters

<i>width</i>	width (in pixels) of the image
<i>pad</i>	the width of each line in each plane of the image is padded to the nearest multiple of this number of bytes (must be a power of 2.)
<i>height</i>	height (in pixels) of the image
<i>subsamp</i>	level of chrominance subsampling in the image (see Chrominance subsampling options.)

Returns

the size of the buffer (in bytes) required to hold the image, or -1 if the arguments are out of bounds.

5.1.5.4 `tjCompress2()`

```
DLLEXPORT int tjCompress2 (
    tjhandle handle,
    const unsigned char * srcBuf,
    int width,
    int pitch,
    int height,
    int pixelFormat,
    unsigned char ** jpegBuf,
    unsigned long * jpegSize,
    int jpegSubsamp,
```

```
int jpegQual,  
int flags )
```

Compress an RGB, grayscale, or CMYK image into a JPEG image.

Parameters

<i>handle</i>	a handle to a TurboJPEG compressor or transformer instance
<i>srcBuf</i>	pointer to an image buffer containing RGB, grayscale, or CMYK pixels to be compressed
<i>width</i>	width (in pixels) of the source image
<i>pitch</i>	bytes per line in the source image. Normally, this should be <code>width * #tjPixelFormat[pixelFormat]</code> if the image is unpadded, or <code>TJPAD (width * #tjPixelFormat[pixelFormat])</code> if each line of the image is padded to the nearest 32-bit boundary, as is the case for Windows bitmaps. You can also be clever and use this parameter to skip lines, etc. Setting this parameter to 0 is the equivalent of setting it to <code>width * #tjPixelFormat[pixelFormat]</code> .
<i>height</i>	height (in pixels) of the source image
<i>pixelFormat</i>	pixel format of the source image (see Pixel formats .)
<i>jpegBuf</i>	address of a pointer to an image buffer that will receive the JPEG image. TurboJPEG has the ability to reallocate the JPEG buffer to accommodate the size of the JPEG image. Thus, you can choose to: 1. pre-allocate the JPEG buffer with an arbitrary size using tjAlloc() and let TurboJPEG grow the buffer as needed, 2. set <code>*jpegBuf</code> to NULL to tell TurboJPEG to allocate the buffer for you, or 3. pre-allocate the buffer to a "worst case" size determined by calling tjBufSize(). This should ensure that the buffer never has to be re-allocated (setting TJFLAG_NOREALLOC guarantees that it won't be.) If you choose option 1, <code>*jpegSize</code> should be set to the size of your pre-allocated buffer. In any case, unless you have set TJFLAG_NOREALLOC, you should always check <code>*jpegBuf</code> upon return from this function, as it may have changed.
<i>jpegSize</i>	pointer to an unsigned long variable that holds the size of the JPEG image buffer. If <code>*jpegBuf</code> points to a pre-allocated buffer, then <code>*jpegSize</code> should be set to the size of the buffer. Upon return, <code>*jpegSize</code> will contain the size of the JPEG image (in bytes.) If <code>*jpegBuf</code> points to a JPEG image buffer that is being reused from a previous call to one of the JPEG compression functions, then <code>*jpegSize</code> is ignored.
<i>jpegSubsamp</i>	the level of chrominance subsampling to be used when generating the JPEG image (see Chrominance subsampling options .)
<i>jpegQual</i>	the image quality of the generated JPEG image (1 = worst, 100 = best)
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

5.1.5.5 tjCompressFromYUV()

```
DLLEXPORT int tjCompressFromYUV (
    tjhandle handle,
    const unsigned char * srcBuf,
    int width,
    int pad,
    int height,
    int subsamp,
```

```

unsigned char ** jpegBuf,
unsigned long * jpegSize,
int jpegQual,
int flags )

```

Compress a YUV planar image into a JPEG image.

Parameters

<i>handle</i>	a handle to a TurboJPEG compressor or transformer instance
<i>srcBuf</i>	pointer to an image buffer containing a YUV planar image to be compressed. The size of this buffer should match the value returned by tjBufSizeYUV2() for the given image width, height, padding, and level of chrominance subsampling. The Y, U (Cb), and V (Cr) image planes should be stored sequentially in the source buffer (refer to YUV Image Format Notes .)
<i>width</i>	width (in pixels) of the source image. If the width is not an even multiple of the MCU block width (see #tjMCUWidth), then an intermediate buffer copy will be performed within TurboJPEG.
<i>pad</i>	the line padding used in the source image. For instance, if each line in each plane of the YUV image is padded to the nearest multiple of 4 bytes, then <i>pad</i> should be set to 4.
<i>height</i>	height (in pixels) of the source image. If the height is not an even multiple of the MCU block height (see #tjMCUHeight), then an intermediate buffer copy will be performed within TurboJPEG.
<i>subsamp</i>	the level of chrominance subsampling used in the source image (see Chrominance subsampling options .)
<i>jpegBuf</i>	address of a pointer to an image buffer that will receive the JPEG image. TurboJPEG has the ability to reallocate the JPEG buffer to accommodate the size of the JPEG image. Thus, you can choose to: 1. pre-allocate the JPEG buffer with an arbitrary size using tjAlloc() and let TurboJPEG grow the buffer as needed, 2. set <i>*jpegBuf</i> to NULL to tell TurboJPEG to allocate the buffer for you, or 3. pre-allocate the buffer to a "worst case" size determined by calling tjBufSize(). This should ensure that the buffer never has to be re-allocated (setting TJFLAG_NOREALLOC guarantees that it won't be.) If you choose option 1, <i>*jpegSize</i> should be set to the size of your pre-allocated buffer. In any case, unless you have set TJFLAG_NOREALLOC, you should always check <i>*jpegBuf</i> upon return from this function, as it may have changed.
<i>jpegSize</i>	pointer to an unsigned long variable that holds the size of the JPEG image buffer. If <i>*jpegBuf</i> points to a pre-allocated buffer, then <i>*jpegSize</i> should be set to the size of the buffer. Upon return, <i>*jpegSize</i> will contain the size of the JPEG image (in bytes.) If <i>*jpegBuf</i> points to a JPEG image buffer that is being reused from a previous call to one of the JPEG compression functions, then <i>*jpegSize</i> is ignored.
<i>jpegQual</i>	the image quality of the generated JPEG image (1 = worst, 100 = best)
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

5.1.5.6 tjCompressFromYUVPlanes()

```

DLLEXPORT int tjCompressFromYUVPlanes (
    tjhandle handle,

```

```

const unsigned char ** srcPlanes,
int width,
const int * strides,
int height,
int subsamp,
unsigned char ** jpegBuf,
unsigned long * jpegSize,
int jpegQual,
int flags )

```

Compress a set of Y, U (Cb), and V (Cr) image planes into a JPEG image.

Parameters

<i>handle</i>	a handle to a TurboJPEG compressor or transformer instance
<i>srcPlanes</i>	an array of pointers to Y, U (Cb), and V (Cr) image planes (or just a Y plane, if compressing a grayscale image) that contain a YUV image to be compressed. These planes can be contiguous or non-contiguous in memory. The size of each plane should match the value returned by tjPlaneSizeYUV() for the given image width, height, strides, and level of chrominance subsampling. Refer to YUV Image Format Notes for more details.
<i>width</i>	width (in pixels) of the source image. If the width is not an even multiple of the MCU block width (see #tjMCUWidth), then an intermediate buffer copy will be performed within TurboJPEG.
<i>strides</i>	an array of integers, each specifying the number of bytes per line in the corresponding plane of the YUV source image. Setting the stride for any plane to 0 is the same as setting it to the plane width (see YUV Image Format Notes .) If <i>strides</i> is NULL, then the strides for all planes will be set to their respective plane widths. You can adjust the strides in order to specify an arbitrary amount of line padding in each plane or to create a JPEG image from a subregion of a larger YUV planar image.
<i>height</i>	height (in pixels) of the source image. If the height is not an even multiple of the MCU block height (see #tjMCUHeight), then an intermediate buffer copy will be performed within TurboJPEG.
<i>subsamp</i>	the level of chrominance subsampling used in the source image (see Chrominance subsampling options .)
<i>jpegBuf</i>	address of a pointer to an image buffer that will receive the JPEG image. TurboJPEG has the ability to reallocate the JPEG buffer to accommodate the size of the JPEG image. Thus, you can choose to: 1. pre-allocate the JPEG buffer with an arbitrary size using tjAlloc() and let TurboJPEG grow the buffer as needed, 2. set <i>*jpegBuf</i> to NULL to tell TurboJPEG to allocate the buffer for you, or 3. pre-allocate the buffer to a "worst case" size determined by calling tjBufSize(). This should ensure that the buffer never has to be re-allocated (setting TJFLAG_NOREALLOC guarantees that it won't be.) If you choose option 1, <i>*jpegSize</i> should be set to the size of your pre-allocated buffer. In any case, unless you have set TJFLAG_NOREALLOC, you should always check <i>*jpegBuf</i> upon return from this function, as it may have changed.
<i>jpegSize</i>	pointer to an unsigned long variable that holds the size of the JPEG image buffer. If <i>*jpegBuf</i> points to a pre-allocated buffer, then <i>*jpegSize</i> should be set to the size of the buffer. Upon return, <i>*jpegSize</i> will contain the size of the JPEG image (in bytes.) If <i>*jpegBuf</i> points to a JPEG image buffer that is being reused from a previous call to one of the JPEG compression functions, then <i>*jpegSize</i> is ignored.
<i>jpegQual</i>	the image quality of the generated JPEG image (1 = worst, 100 = best)
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

5.1.5.7 tjDecodeYUV()

```
DLLEXPORT int tjDecodeYUV (
    tjhandle handle,
    const unsigned char * srcBuf,
    int pad,
    int subsamp,
    unsigned char * dstBuf,
    int width,
    int pitch,
    int height,
    int pixelFormat,
    int flags )
```

Decode a YUV planar image into an RGB or grayscale image. This function uses the accelerated color conversion routines in the underlying codec but does not execute any of the other steps in the JPEG decompression process.

Parameters

<i>handle</i>	a handle to a TurboJPEG decompressor or transformer instance
<i>srcBuf</i>	pointer to an image buffer containing a YUV planar image to be decoded. The size of this buffer should match the value returned by tjBufSizeYUV2() for the given image width, height, padding, and level of chrominance subsampling. The Y, U (Cb), and V (Cr) image planes should be stored sequentially in the source buffer (refer to YUV Image Format Notes .)
<i>pad</i>	Use this parameter to specify that the width of each line in each plane of the YUV source image is padded to the nearest multiple of this number of bytes (must be a power of 2.)
<i>subsamp</i>	the level of chrominance subsampling used in the YUV source image (see Chrominance subsampling options .)
<i>dstBuf</i>	pointer to an image buffer that will receive the decoded image. This buffer should normally be $\text{pitch} * \text{height}$ bytes in size, but the <i>dstBuf</i> pointer can also be used to decode into a specific region of a larger buffer.
<i>width</i>	width (in pixels) of the source and destination images
<i>pitch</i>	bytes per line in the destination image. Normally, this should be $\text{width} * \text{\#tjPixelSize}[\text{pixelFormat}]$ if the destination image is unpadded, or TJPAD ($\text{width} * \text{\#tjPixelSize}[\text{pixelFormat}]$) if each line of the destination image should be padded to the nearest 32-bit boundary, as is the case for Windows bitmaps. You can also be clever and use the pitch parameter to skip lines, etc. Setting this parameter to 0 is the equivalent of setting it to $\text{width} * \text{\#tjPixelSize}[\text{pixelFormat}]$.
<i>height</i>	height (in pixels) of the source and destination images
<i>pixelFormat</i>	pixel format of the destination image (see Pixel formats .)
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

5.1.5.8 `tjDecodeYUVPlanes()`

```
DLLEXPORT int tjDecodeYUVPlanes (
    tjhandle handle,
    const unsigned char ** srcPlanes,
    const int * strides,
    int subsamp,
    unsigned char * dstBuf,
    int width,
    int pitch,
    int height,
    int pixelFormat,
    int flags )
```

Decode a set of Y, U (Cb), and V (Cr) image planes into an RGB or grayscale image. This function uses the accelerated color conversion routines in the underlying codec but does not execute any of the other steps in the JPEG decompression process.

Parameters

<i>handle</i>	a handle to a TurboJPEG decompressor or transformer instance
<i>srcPlanes</i>	an array of pointers to Y, U (Cb), and V (Cr) image planes (or just a Y plane, if decoding a grayscale image) that contain a YUV image to be decoded. These planes can be contiguous or non-contiguous in memory. The size of each plane should match the value returned by tjPlaneSizeYUV() for the given image width, height, strides, and level of chrominance subsampling. Refer to YUV Image Format Notes for more details.
<i>strides</i>	an array of integers, each specifying the number of bytes per line in the corresponding plane of the YUV source image. Setting the stride for any plane to 0 is the same as setting it to the plane width (see YUV Image Format Notes .) If <i>strides</i> is NULL, then the strides for all planes will be set to their respective plane widths. You can adjust the strides in order to specify an arbitrary amount of line padding in each plane or to decode a subregion of a larger YUV planar image.
<i>subsamp</i>	the level of chrominance subsampling used in the YUV source image (see Chrominance subsampling options .)
<i>dstBuf</i>	pointer to an image buffer that will receive the decoded image. This buffer should normally be <i>pitch</i> * <i>height</i> bytes in size, but the <i>dstBuf</i> pointer can also be used to decode into a specific region of a larger buffer.
<i>width</i>	width (in pixels) of the source and destination images
<i>pitch</i>	bytes per line in the destination image. Normally, this should be <i>width</i> * <code>#tjPixelSize[pixelFormat]</code> if the destination image is unpadded, or <code>TJPAD (width * #tjPixelSize[pixelFormat])</code> if each line of the destination image should be padded to the nearest 32-bit boundary, as is the case for Windows bitmaps. You can also be clever and use the pitch parameter to skip lines, etc. Setting this parameter to 0 is the equivalent of setting it to <i>width</i> * <code>#tjPixelSize[pixelFormat]</code> .
<i>height</i>	height (in pixels) of the source and destination images
<i>pixelFormat</i>	pixel format of the destination image (see Pixel formats .)
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

5.1.5.9 tjDecompress2()

```
DLLEXPORT int tjDecompress2 (
    tjhandle handle,
    const unsigned char * jpegBuf,
    unsigned long jpegSize,
    unsigned char * dstBuf,
    int width,
    int pitch,
    int height,
    int pixelFormat,
    int flags )
```

Decompress a JPEG image to an RGB, grayscale, or CMYK image.

Parameters

<i>handle</i>	a handle to a TurboJPEG decompressor or transformer instance
<i>jpegBuf</i>	pointer to a buffer containing the JPEG image to decompress
<i>jpegSize</i>	size of the JPEG image (in bytes)
<i>dstBuf</i>	pointer to an image buffer that will receive the decompressed image. This buffer should normally be <code>pitch * scaledHeight</code> bytes in size, where <code>scaledHeight</code> can be determined by calling TJSCALED() with the JPEG image height and one of the scaling factors returned by tjGetScalingFactors() . The <code>dstBuf</code> pointer may also be used to decompress into a specific region of a larger buffer.
<i>width</i>	desired width (in pixels) of the destination image. If this is different than the width of the JPEG image being decompressed, then TurboJPEG will use scaling in the JPEG decompressor to generate the largest possible image that will fit within the desired width. If <code>width</code> is set to 0, then only the height will be considered when determining the scaled image size.
<i>pitch</i>	bytes per line in the destination image. Normally, this is <code>scaledWidth * #tjPixelFormat[pixelFormat]</code> if the decompressed image is unpadding, else TJPAD(scaledWidth * #tjPixelFormat[pixelFormat]) if each line of the decompressed image is padded to the nearest 32-bit boundary, as is the case for Windows bitmaps. (NOTE: <code>scaledWidth</code> can be determined by calling TJSCALED() with the JPEG image width and one of the scaling factors returned by tjGetScalingFactors() .) You can also be clever and use the pitch parameter to skip lines, etc. Setting this parameter to 0 is the equivalent of setting it to <code>scaledWidth * #tjPixelFormat[pixelFormat]</code> .
<i>height</i>	desired height (in pixels) of the destination image. If this is different than the height of the JPEG image being decompressed, then TurboJPEG will use scaling in the JPEG decompressor to generate the largest possible image that will fit within the desired height. If <code>height</code> is set to 0, then only the width will be considered when determining the scaled image size.
<i>pixelFormat</i>	pixel format of the destination image (see Pixel formats .)
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

5.1.5.10 tjDecompressHeader3()

```
DLLEXPORT int tjDecompressHeader3 (
    tjhandle handle,
```

```

const unsigned char * jpegBuf,
unsigned long jpegSize,
int * width,
int * height,
int * jpegSubsamp,
int * jpegColorspace )

```

Retrieve information about a JPEG image without decompressing it.

Parameters

<i>handle</i>	a handle to a TurboJPEG decompressor or transformer instance
<i>jpegBuf</i>	pointer to a buffer containing a JPEG image
<i>jpegSize</i>	size of the JPEG image (in bytes)
<i>width</i>	pointer to an integer variable that will receive the width (in pixels) of the JPEG image
<i>height</i>	pointer to an integer variable that will receive the height (in pixels) of the JPEG image
<i>jpegSubsamp</i>	pointer to an integer variable that will receive the level of chrominance subsampling used when the JPEG image was compressed (see Chrominance subsampling options .)
<i>jpegColorspace</i>	pointer to an integer variable that will receive one of the JPEG colorspace constants, indicating the colorspace of the JPEG image (see JPEG colorspace .)

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

5.1.5.11 tjDecompressToYUV2()

```

DLLEXPORT int tjDecompressToYUV2 (
    tjhandle handle,
    const unsigned char * jpegBuf,
    unsigned long jpegSize,
    unsigned char * dstBuf,
    int width,
    int pad,
    int height,
    int flags )

```

Decompress a JPEG image to a YUV planar image. This function performs JPEG decompression but leaves out the color conversion step, so a planar YUV image is generated instead of an RGB image.

Parameters

<i>handle</i>	a handle to a TurboJPEG decompressor or transformer instance
<i>jpegBuf</i>	pointer to a buffer containing the JPEG image to decompress
<i>jpegSize</i>	size of the JPEG image (in bytes)
<i>dstBuf</i>	pointer to an image buffer that will receive the YUV image. Use tjBufSizeYUV2() to determine the appropriate size for this buffer based on the image width, height, padding, and level of subsampling. The Y, U (Cb), and V (Cr) image planes will be stored sequentially in the buffer (refer to YUV Image Format Notes .)

Parameters

<i>width</i>	desired width (in pixels) of the YUV image. If this is different than the width of the JPEG image being decompressed, then TurboJPEG will use scaling in the JPEG decompressor to generate the largest possible image that will fit within the desired width. If <i>width</i> is set to 0, then only the height will be considered when determining the scaled image size. If the scaled width is not an even multiple of the MCU block width (see <code>#tjMCUWidth</code>), then an intermediate buffer copy will be performed within TurboJPEG.
<i>pad</i>	the width of each line in each plane of the YUV image will be padded to the nearest multiple of this number of bytes (must be a power of 2.) To generate images suitable for X Video, <i>pad</i> should be set to 4.
<i>height</i>	desired height (in pixels) of the YUV image. If this is different than the height of the JPEG image being decompressed, then TurboJPEG will use scaling in the JPEG decompressor to generate the largest possible image that will fit within the desired height. If <i>height</i> is set to 0, then only the width will be considered when determining the scaled image size. If the scaled height is not an even multiple of the MCU block height (see <code>#tjMCUHeight</code>), then an intermediate buffer copy will be performed within TurboJPEG.
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

5.1.5.12 `tjDecompressToYUVPlanes()`

```

DLLEXPORT int tjDecompressToYUVPlanes (
    tjhandle handle,
    const unsigned char * jpegBuf,
    unsigned long jpegSize,
    unsigned char ** dstPlanes,
    int width,
    int * strides,
    int height,
    int flags )

```

Decompress a JPEG image into separate Y, U (Cb), and V (Cr) image planes. This function performs JPEG decompression but leaves out the color conversion step, so a planar YUV image is generated instead of an RGB image.

Parameters

<i>handle</i>	a handle to a TurboJPEG decompressor or transformer instance
<i>jpegBuf</i>	pointer to a buffer containing the JPEG image to decompress
<i>jpegSize</i>	size of the JPEG image (in bytes)
<i>dstPlanes</i>	an array of pointers to Y, U (Cb), and V (Cr) image planes (or just a Y plane, if decompressing a grayscale image) that will receive the YUV image. These planes can be contiguous or non-contiguous in memory. Use tjPlaneSizeYUV() to determine the appropriate size for each plane based on the scaled image width, scaled image height, strides, and level of chrominance subsampling. Refer to YUV Image Format Notes for more details.

Parameters

<i>width</i>	desired width (in pixels) of the YUV image. If this is different than the width of the JPEG image being decompressed, then TurboJPEG will use scaling in the JPEG decompressor to generate the largest possible image that will fit within the desired width. If <i>width</i> is set to 0, then only the height will be considered when determining the scaled image size. If the scaled width is not an even multiple of the MCU block width (see <code>#tjMCUWidth</code>), then an intermediate buffer copy will be performed within TurboJPEG.
<i>strides</i>	an array of integers, each specifying the number of bytes per line in the corresponding plane of the output image. Setting the stride for any plane to 0 is the same as setting it to the scaled plane width (see YUV Image Format Notes .) If <i>strides</i> is NULL, then the strides for all planes will be set to their respective scaled plane widths. You can adjust the strides in order to add an arbitrary amount of line padding to each plane or to decompress the JPEG image into a subregion of a larger YUV planar image.
<i>height</i>	desired height (in pixels) of the YUV image. If this is different than the height of the JPEG image being decompressed, then TurboJPEG will use scaling in the JPEG decompressor to generate the largest possible image that will fit within the desired height. If <i>height</i> is set to 0, then only the width will be considered when determining the scaled image size. If the scaled height is not an even multiple of the MCU block height (see <code>#tjMCUHeight</code>), then an intermediate buffer copy will be performed within TurboJPEG.
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

5.1.5.13 `tjDestroy()`

```
DLLEXPORT int tjDestroy (
    tjhandle handle )
```

Destroy a TurboJPEG compressor, decompressor, or transformer instance.

Parameters

<i>handle</i>	a handle to a TurboJPEG compressor, decompressor or transformer instance
---------------	--

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#).)

5.1.5.14 `tjEncodeYUV3()`

```
DLLEXPORT int tjEncodeYUV3 (
    tjhandle handle,
    const unsigned char * srcBuf,
```

```

    int width,
    int pitch,
    int height,
    int pixelFormat,
    unsigned char * dstBuf,
    int pad,
    int subsamp,
    int flags )

```

Encode an RGB or grayscale image into a YUV planar image. This function uses the accelerated color conversion routines in the underlying codec but does not execute any of the other steps in the JPEG compression process.

Parameters

<i>handle</i>	a handle to a TurboJPEG compressor or transformer instance
<i>srcBuf</i>	pointer to an image buffer containing RGB or grayscale pixels to be encoded
<i>width</i>	width (in pixels) of the source image
<i>pitch</i>	bytes per line in the source image. Normally, this should be <code>width * #tjPixelSize[pixelFormat]</code> if the image is unpadded, or <code>TJPAD (width * #tjPixelSize[pixelFormat])</code> if each line of the image is padded to the nearest 32-bit boundary, as is the case for Windows bitmaps. You can also be clever and use this parameter to skip lines, etc. Setting this parameter to 0 is the equivalent of setting it to <code>width * #tjPixelSize[pixelFormat]</code> .
<i>height</i>	height (in pixels) of the source image
<i>pixelFormat</i>	pixel format of the source image (see Pixel formats .)
<i>dstBuf</i>	pointer to an image buffer that will receive the YUV image. Use <code>tjBufSizeYUV2()</code> to determine the appropriate size for this buffer based on the image width, height, padding, and level of chrominance subsampling. The Y, U (Cb), and V (Cr) image planes will be stored sequentially in the buffer (refer to YUV Image Format Notes .)
<i>pad</i>	the width of each line in each plane of the YUV image will be padded to the nearest multiple of this number of bytes (must be a power of 2.) To generate images suitable for X Video, <code>pad</code> should be set to 4.
<i>subsamp</i>	the level of chrominance subsampling to be used when generating the YUV image (see Chrominance subsampling options .) To generate images suitable for X Video, <code>subsamp</code> should be set to <code>TJSAMP_420</code> . This produces an image compatible with the I420 (AKA "YUV420P") format.
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see `tjGetErrorStr2()` and `tjGetErrorCode()`.)

5.1.5.15 tjEncodeYUVPlanes()

```

DLLEXPORT int tjEncodeYUVPlanes (
    tjhandle handle,
    const unsigned char * srcBuf,
    int width,
    int pitch,
    int height,
    int pixelFormat,

```

```

unsigned char ** dstPlanes,
int * strides,
int subsamp,
int flags )

```

Encode an RGB or grayscale image into separate Y, U (Cb), and V (Cr) image planes. This function uses the accelerated color conversion routines in the underlying codec but does not execute any of the other steps in the JPEG compression process.

Parameters

<i>handle</i>	a handle to a TurboJPEG compressor or transformer instance
<i>srcBuf</i>	pointer to an image buffer containing RGB or grayscale pixels to be encoded
<i>width</i>	width (in pixels) of the source image
<i>pitch</i>	bytes per line in the source image. Normally, this should be <code>width * #tjPixelSize[pixelFormat]</code> if the image is unpadded, or <code>TJPAD(width * #tjPixelSize[pixelFormat])</code> if each line of the image is padded to the nearest 32-bit boundary, as is the case for Windows bitmaps. You can also be clever and use this parameter to skip lines, etc. Setting this parameter to 0 is the equivalent of setting it to <code>width * #tjPixelSize[pixelFormat]</code> .
<i>height</i>	height (in pixels) of the source image
<i>pixelFormat</i>	pixel format of the source image (see Pixel formats .)
<i>dstPlanes</i>	an array of pointers to Y, U (Cb), and V (Cr) image planes (or just a Y plane, if generating a grayscale image) that will receive the encoded image. These planes can be contiguous or non-contiguous in memory. Use tjPlaneSizeYUV() to determine the appropriate size for each plane based on the image width, height, strides, and level of chrominance subsampling. Refer to YUV Image Format Notes for more details.
<i>strides</i>	an array of integers, each specifying the number of bytes per line in the corresponding plane of the output image. Setting the stride for any plane to 0 is the same as setting it to the plane width (see YUV Image Format Notes .) If <i>strides</i> is NULL, then the strides for all planes will be set to their respective plane widths. You can adjust the strides in order to add an arbitrary amount of line padding to each plane or to encode an RGB or grayscale image into a subregion of a larger YUV planar image.
<i>subsamp</i>	the level of chrominance subsampling to be used when generating the YUV image (see Chrominance subsampling options .) To generate images suitable for X Video, <i>subsamp</i> should be set to <code>TJSAMP_420</code> . This produces an image compatible with the I420 (AKA "YUV420P") format.
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

5.1.5.16 tjFree()

```

DLLEXPORT void tjFree (
    unsigned char * buffer )

```

Free an image buffer previously allocated by TurboJPEG. You should always use this function to free JPEG destination buffer(s) that were automatically (re)allocated by the compression and transform functions or that were manually allocated using [tjAlloc\(\)](#).

Parameters

<i>buffer</i>	address of the buffer to free
---------------	-------------------------------

See also[tjAlloc\(\)](#)**5.1.5.17 tjGetErrorCode()**

```
DLLEXPORT int tjGetErrorCode (
    tjhandle handle )
```

Returns a code indicating the severity of the last error. See [Error codes](#).

Parameters

<i>handle</i>	a handle to a TurboJPEG compressor, decompressor or transformer instance
---------------	--

Returns

a code indicating the severity of the last error. See [Error codes](#).

5.1.5.18 tjGetErrorStr2()

```
DLLEXPORT char* tjGetErrorStr2 (
    tjhandle handle )
```

Returns a descriptive error message explaining why the last command failed.

Parameters

<i>handle</i>	a handle to a TurboJPEG compressor, decompressor, or transformer instance, or NULL if the error was generated by a global function (but note that retrieving the error message for a global function is not thread-safe.)
---------------	---

Returns

a descriptive error message explaining why the last command failed.

5.1.5.19 `tjGetScalingFactors()`

```
DLLEXPORT tjscalingfactor\* tjGetScalingFactors (
    int * numscalingfactors )
```

Returns a list of fractional scaling factors that the JPEG decompressor in this implementation of TurboJPEG supports.

Parameters

<i>numscalingfactors</i>	pointer to an integer variable that will receive the number of elements in the list
--------------------------	---

Returns

a pointer to a list of fractional scaling factors, or NULL if an error is encountered (see [tjGetErrorStr2\(\)](#).)

5.1.5.20 `tjInitCompress()`

```
DLLEXPORT tjhandle tjInitCompress (
    void )
```

Create a TurboJPEG compressor instance.

Returns

a handle to the newly-created instance, or NULL if an error occurred (see [tjGetErrorStr2\(\)](#).)

5.1.5.21 `tjInitDecompress()`

```
DLLEXPORT tjhandle tjInitDecompress (
    void )
```

Create a TurboJPEG decompressor instance.

Returns

a handle to the newly-created instance, or NULL if an error occurred (see [tjGetErrorStr2\(\)](#).)

5.1.5.22 tjInitTransform()

```
DLLEXPORT tjhandle tjInitTransform (
    void )
```

Create a new TurboJPEG transformer instance.

Returns

a handle to the newly-created instance, or NULL if an error occurred (see [tjGetErrorStr2\(\)](#).)

5.1.5.23 tjLoadImage()

```
DLLEXPORT unsigned char* tjLoadImage (
    const char * filename,
    int * width,
    int align,
    int * height,
    int * pixelFormat,
    int flags )
```

Load an uncompressed image from disk into memory.

Parameters

<i>filename</i>	name of a file containing an uncompressed image in Windows BMP or PBMPLUS (PPM/PGM) format
<i>width</i>	pointer to an integer variable that will receive the width (in pixels) of the uncompressed image
<i>align</i>	row alignment of the image buffer to be returned (must be a power of 2.) For instance, setting this parameter to 4 will cause all rows in the image buffer to be padded to the nearest 32-bit boundary, and setting this parameter to 1 will cause all rows in the image buffer to be unpadded.
<i>height</i>	pointer to an integer variable that will receive the height (in pixels) of the uncompressed image
<i>pixelFormat</i>	pointer to an integer variable that specifies or will receive the pixel format of the uncompressed image buffer. The behavior of tjLoadImage() will vary depending on the value of <code>*pixelFormat</code> passed to the function: • TJPF_UNKNOWN : The uncompressed image buffer returned by the function will use the most optimal pixel format for the file type, and <code>*pixelFormat</code> will contain the ID of this pixel format upon successful return from the function. • TJPF_GRAY : Only PGM files and 8-bit BMP files with a grayscale colormap can be loaded. • TJPF_CMYK : The RGB or grayscale pixels stored in the file will be converted using a quick & dirty algorithm that is suitable only for testing purposes (proper conversion between CMYK and other formats requires a color management system.) • Other pixel formats : The uncompressed image buffer will use the specified pixel format, and pixel format conversion will be performed if necessary.
<i>flags</i>	the bitwise OR of one or more of the flags .

Returns

a pointer to a newly-allocated buffer containing the uncompressed image, converted to the chosen pixel format and with the chosen row alignment, or NULL if an error occurred (see [tjGetErrorStr2\(\)](#).) This buffer should be freed using [tjFree\(\)](#).

5.1.5.24 tjPlaneHeight()

```
DLLEXPORT int tjPlaneHeight (
    int componentID,
    int height,
    int subsamp )
```

The plane height of a YUV image plane with the given parameters. Refer to [YUV Image Format Notes](#) for a description of plane height.

Parameters

<i>componentID</i>	ID number of the image plane (0 = Y, 1 = U/Cb, 2 = V/Cr)
<i>height</i>	height (in pixels) of the YUV image
<i>subsamp</i>	level of chrominance subsampling in the image (see Chrominance subsampling options.)

Returns

the plane height of a YUV image plane with the given parameters, or -1 if the arguments are out of bounds.

5.1.5.25 tjPlaneSizeYUV()

```
DLLEXPORT unsigned long tjPlaneSizeYUV (
    int componentID,
    int width,
    int stride,
    int height,
    int subsamp )
```

The size of the buffer (in bytes) required to hold a YUV image plane with the given parameters.

Parameters

<i>componentID</i>	ID number of the image plane (0 = Y, 1 = U/Cb, 2 = V/Cr)
<i>width</i>	width (in pixels) of the YUV image. NOTE: this is the width of the whole image, not the plane width.
<i>stride</i>	bytes per line in the image plane. Setting this to 0 is the equivalent of setting it to the plane width.
<i>height</i>	height (in pixels) of the YUV image. NOTE: this is the height of the whole image, not the plane height.
<i>subsamp</i>	level of chrominance subsampling in the image (see Chrominance subsampling options.)

Returns

the size of the buffer (in bytes) required to hold the YUV image plane, or -1 if the arguments are out of bounds.

5.1.5.26 tjPlaneWidth()

```
DLLEXPORT int tjPlaneWidth (
    int componentID,
    int width,
    int subsamp )
```

The plane width of a YUV image plane with the given parameters. Refer to [YUV Image Format Notes](#) for a description of plane width.

Parameters

<i>componentID</i>	ID number of the image plane (0 = Y, 1 = U/Cb, 2 = V/Cr)
<i>width</i>	width (in pixels) of the YUV image
<i>subsamp</i>	level of chrominance subsampling in the image (see Chrominance subsampling options .)

Returns

the plane width of a YUV image plane with the given parameters, or -1 if the arguments are out of bounds.

5.1.5.27 tjSaveImage()

```
DLLEXPORT int tjSaveImage (
    const char * filename,
    unsigned char * buffer,
    int width,
    int pitch,
    int height,
    int pixelFormat,
    int flags )
```

Save an uncompressed image from memory to disk.

Parameters

<i>filename</i>	name of a file to which to save the uncompressed image. The image will be stored in Windows BMP or PBMPLUS (PPM/PGM) format, depending on the file extension.
<i>buffer</i>	pointer to an image buffer containing RGB, grayscale, or CMYK pixels to be saved
<i>width</i>	width (in pixels) of the uncompressed image
<i>pitch</i>	bytes per line in the image buffer. Setting this parameter to 0 is the equivalent of setting it to <code>width * #tjPixelSize[pixelFormat]</code> .
<i>height</i>	height (in pixels) of the uncompressed image

Parameters

<i>pixelFormat</i>	pixel format of the image buffer (see Pixel formats .) If this parameter is set to TJPF_GRAY , then the image will be stored in PGM or 8-bit (indexed color) BMP format. Otherwise, the image will be stored in PPM or 24-bit BMP format. If this parameter is set to TJPF_CMYK , then the CMYK pixels will be converted to RGB using a quick & dirty algorithm that is suitable only for testing (proper conversion between CMYK and other formats requires a color management system.)
<i>flags</i>	the bitwise OR of one or more of the flags .

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#).)

5.1.5.28 `tjTransform()`

```
DLLEXPORT int tjTransform (
    tjhandle handle,
    const unsigned char * jpegBuf,
    unsigned long jpegSize,
    int n,
    unsigned char ** dstBufs,
    unsigned long * dstSizes,
    tjtransform * transforms,
    int flags )
```

Losslessly transform a JPEG image into another JPEG image. Lossless transforms work by moving the raw DCT coefficients from one JPEG image structure to another without altering the values of the coefficients. While this is typically faster than decompressing the image, transforming it, and re-compressing it, lossless transforms are not free. Each lossless transform requires reading and performing Huffman decoding on all of the coefficients in the source image, regardless of the size of the destination image. Thus, this function provides a means of generating multiple transformed images from the same source or applying multiple transformations simultaneously, in order to eliminate the need to read the source coefficients multiple times.

Parameters

<i>handle</i>	a handle to a TurboJPEG transformer instance
<i>jpegBuf</i>	pointer to a buffer containing the JPEG source image to transform
<i>jpegSize</i>	size of the JPEG source image (in bytes)
<i>n</i>	the number of transformed JPEG images to generate

Parameters

<i>dstBufs</i>	pointer to an array of <i>n</i> image buffers. <code>dstBufs[i]</code> will receive a JPEG image that has been transformed using the parameters in <code>transforms[i]</code>. TurboJPEG has the ability to reallocate the JPEG buffer to accommodate the size of the JPEG image. Thus, you can choose to: 1. pre-allocate the JPEG buffer with an arbitrary size using tjAlloc() and let TurboJPEG grow the buffer as needed, 2. set <code>dstBufs[i]</code> to NULL to tell TurboJPEG to allocate the buffer for you, or 3. pre-allocate the buffer to a "worst case" size determined by calling tjBufSize() with the transformed or cropped width and height. Under normal circumstances, this should ensure that the buffer never has to be re-allocated (setting TJFLAG_NOREALLOC guarantees that it won't be.) Note, however, that there are some rare cases (such as transforming images with a large amount of embedded EXIF or ICC profile data) in which the output image will be larger than the worst-case size, and TJFLAG_NOREALLOC cannot be used in those cases. If you choose option 1, <code>dstSizes[i]</code> should be set to the size of your pre-allocated buffer. In any case, unless you have set TJFLAG_NOREALLOC, you should always check <code>dstBufs[i]</code> upon return from this function, as it may have changed.
<i>dstSizes</i>	pointer to an array of <i>n</i> unsigned long variables that will receive the actual sizes (in bytes) of each transformed JPEG image. If <code>dstBufs[i]</code> points to a pre-allocated buffer, then <code>dstSizes[i]</code> should be set to the size of the buffer. Upon return, <code>dstSizes[i]</code> will contain the size of the JPEG image (in bytes.)
<i>transforms</i>	pointer to an array of <i>n</i> tjtransform structures, each of which specifies the transform parameters and/or cropping region for the corresponding transformed output image.
<i>flags</i>	the bitwise OR of one or more of the flags

Returns

0 if successful, or -1 if an error occurred (see [tjGetErrorStr2\(\)](#) and [tjGetErrorCode\(\)](#).)

Chapter 6

Class Documentation

6.1 AlgVersion Struct Reference

Public Attributes

- int32_t **major**
- int32_t **minor**
- int32_t **patch**
- int32_t **reserved**

6.1.1 Detailed Description

Definition at line 6 of file TYCAPL.h.

The documentation for this struct was generated from the following file:

- TYCAPL.h

6.2 DepthEnhenceParameters Struct Reference

Public Attributes

- float [sigma_s](#)
filter param on space
- float [sigma_r](#)
filter param on range
- int [outlier_win_sz](#)
outlier filter windows ize
- float **outlier_rate**

6.2.1 Detailed Description

Definition at line 194 of file TYImageProc.h.

The documentation for this struct was generated from the following file:

- [TYImageProc.h](#)

6.3 DepthInpainterParameters Struct Reference

Public Attributes

- int **kernel_size**
- int **max_internal_hole**

6.3.1 Detailed Description

Definition at line 175 of file TYImageProc.h.

The documentation for this struct was generated from the following file:

- [TYImageProc.h](#)

6.4 DepthSpeckleFilterParameters Struct Reference

Public Attributes

- int **max_speckle_size**
- int **max_speckle_diff**
- float **max_physical_size**

6.4.1 Detailed Description

Definition at line 152 of file TYImageProc.h.

The documentation for this struct was generated from the following file:

- [TYImageProc.h](#)

6.5 f16 Struct Reference

Public Attributes

- u16 **u**

6.5.1 Detailed Description

Definition at line 986 of file nsimd.h.

The documentation for this struct was generated from the following file:

- nsimd.h

6.6 gz_header_s Struct Reference

Public Attributes

- int **text**
- uLong **time**
- int **xflags**
- int **os**
- Bytef * **extra**
- uInt **extra_len**
- uInt **extra_max**
- Bytef * **name**
- uInt **name_max**
- Bytef * **comment**
- uInt **comm_max**
- int **hcrc**
- int **done**

6.6.1 Detailed Description

Definition at line 114 of file zlib.h.

The documentation for this struct was generated from the following file:

- zlib.h

6.7 gzFile_s Struct Reference

Public Attributes

- unsigned **have**
- unsigned char * **next**
- z_off64_t **pos**

6.7.1 Detailed Description

Definition at line 1837 of file zlib.h.

The documentation for this struct was generated from the following file:

- zlib.h

6.8 JHUFF_TBL Struct Reference

Public Attributes

- `UINT8 bits` [17]
- `UINT8 huffval` [256]
- `boolean sent_table`

6.8.1 Detailed Description

Definition at line 103 of file `jpeglib.h`.

The documentation for this struct was generated from the following file:

- `jpeglib.h`

6.9 jpeg_common_struct Struct Reference

Public Attributes

- `jpeg_common_fields`

6.9.1 Detailed Description

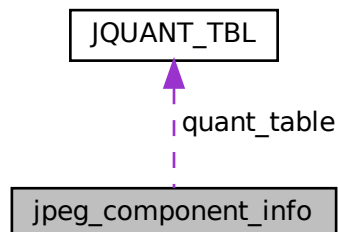
Definition at line 282 of file `jpeglib.h`.

The documentation for this struct was generated from the following file:

- `jpeglib.h`

6.10 jpeg_component_info Struct Reference

Collaboration diagram for `jpeg_component_info`:



Public Attributes

- int **component_id**
- int **component_index**
- int **h_samp_factor**
- int **v_samp_factor**
- int **quant_tbl_no**
- int **dc_tbl_no**
- int **ac_tbl_no**
- JDIMENSION **width_in_blocks**
- JDIMENSION **height_in_blocks**
- int **DCT_scaled_size**
- JDIMENSION **downsampled_width**
- JDIMENSION **downsampled_height**
- boolean **component_needed**
- int **MCU_width**
- int **MCU_height**
- int **MCU_blocks**
- int **MCU_sample_width**
- int **last_col_width**
- int **last_row_height**
- [JQUANT_TBL](#) * **quant_table**
- void * **dct_table**

6.10.1 Detailed Description

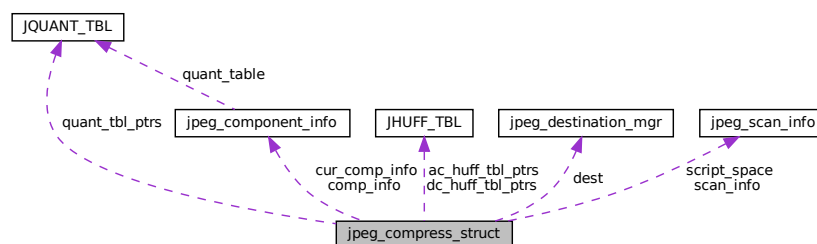
Definition at line 119 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.11 jpeg_compress_struct Struct Reference

Collaboration diagram for jpeg_compress_struct:



Public Attributes

- **jpeg_common_fields**
- struct [jpeg_destination_mgr](#) * **dest**
- JDIMENSION **image_width**
- JDIMENSION **image_height**
- int **input_components**
- J_COLOR_SPACE **in_color_space**
- double **input_gamma**
- int **data_precision**
- int **num_components**
- J_COLOR_SPACE **jpeg_color_space**
- [jpeg_component_info](#) * **comp_info**
- [JQUANT_TBL](#) * **quant_tbl_ptrs** [NUM_QUANT_TBLS]
- [JHUFF_TBL](#) * **dc_huff_tbl_ptrs** [NUM_HUFF_TBLS]
- [JHUFF_TBL](#) * **ac_huff_tbl_ptrs** [NUM_HUFF_TBLS]
- UINT8 **arith_dc_L** [NUM_ARITH_TBLS]
- UINT8 **arith_dc_U** [NUM_ARITH_TBLS]
- UINT8 **arith_ac_K** [NUM_ARITH_TBLS]
- int **num_scans**
- const [jpeg_scan_info](#) * **scan_info**
- boolean **raw_data_in**
- boolean **arith_code**
- boolean **optimize_coding**
- boolean **CCIR601_sampling**
- int **smoothing_factor**
- J_DCT_METHOD **dct_method**
- unsigned int **restart_interval**
- int **restart_in_rows**
- boolean **write_JFIF_header**
- UINT8 **JFIF_major_version**
- UINT8 **JFIF_minor_version**
- UINT8 **density_unit**
- UINT16 **X_density**
- UINT16 **Y_density**
- boolean **write_Adobe_marker**
- JDIMENSION **next_scanline**
- boolean **progressive_mode**
- int **max_h_samp_factor**
- int **max_v_samp_factor**
- JDIMENSION **total_iMCU_rows**
- int **comps_in_scan**
- [jpeg_component_info](#) * **cur_comp_info** [MAX_COMPS_IN_SCAN]
- JDIMENSION **MCUs_per_row**
- JDIMENSION **MCU_rows_in_scan**
- int **blocks_in_MCU**
- int **MCU_membership** [C_MAX_BLOCKS_IN_MCU]
- int **Ss**
- int **Se**
- int **Ah**
- int **Al**
- struct [jpeg_comp_master](#) * **master**
- struct [jpeg_c_main_controller](#) * **main**
- struct [jpeg_c_prep_controller](#) * **prep**
- struct [jpeg_c_coef_controller](#) * **coef**

- struct jpeg_marker_writer * **marker**
- struct jpeg_color_converter * **cconvert**
- struct jpeg_downsampler * **downsample**
- struct jpeg_forward_dct * **fdct**
- struct jpeg_entropy_encoder * **entropy**
- [jpeg_scan_info](#) * **script_space**
- int **script_space_size**

6.11.1 Detailed Description

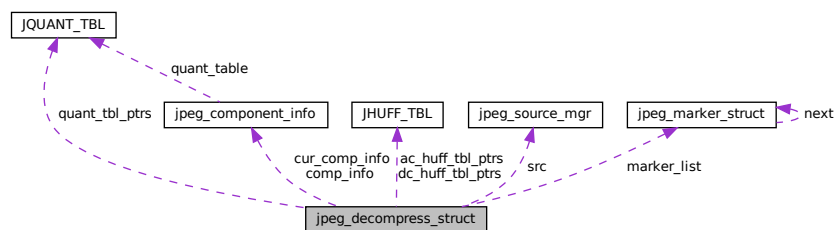
Definition at line 297 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.12 jpeg_decompress_struct Struct Reference

Collaboration diagram for jpeg_decompress_struct:



Public Attributes

- **jpeg_common_fields**
- struct [jpeg_source_mgr](#) * **src**
- JDIMENSION **image_width**
- JDIMENSION **image_height**
- int **num_components**
- J_COLOR_SPACE **jpeg_color_space**
- J_COLOR_SPACE **out_color_space**
- unsigned int **scale_num**
- unsigned int **scale_denom**
- double **output_gamma**
- boolean **buffered_image**
- boolean **raw_data_out**
- J_DCT_METHOD **dct_method**
- boolean **do_fancy_upsampling**
- boolean **do_block_smoothing**
- boolean **quantize_colors**

- J_DITHER_MODE **dither_mode**
- boolean **two_pass_quantize**
- int **desired_number_of_colors**
- boolean **enable_1pass_quant**
- boolean **enable_external_quant**
- boolean **enable_2pass_quant**
- JDIMENSION **output_width**
- JDIMENSION **output_height**
- int **out_color_components**
- int **output_components**
- int **rec_outbuf_height**
- int **actual_number_of_colors**
- JSAMPARRAY **colormap**
- JDIMENSION **output_scanline**
- int **input_scan_number**
- JDIMENSION **input_iMCU_row**
- int **output_scan_number**
- JDIMENSION **output_iMCU_row**
- int(* **coef_bits**)[DCTSIZE2]
- [JQUANT_TBL](#) * **quant_tbl_ptrs** [NUM_QUANT_TBLS]
- [JHUFF_TBL](#) * **dc_huff_tbl_ptrs** [NUM_HUFF_TBLS]
- [JHUFF_TBL](#) * **ac_huff_tbl_ptrs** [NUM_HUFF_TBLS]
- int **data_precision**
- [jpeg_component_info](#) * **comp_info**
- boolean **progressive_mode**
- boolean **arith_code**
- UINT8 **arith_dc_L** [NUM_ARITH_TBLS]
- UINT8 **arith_dc_U** [NUM_ARITH_TBLS]
- UINT8 **arith_ac_K** [NUM_ARITH_TBLS]
- unsigned int **restart_interval**
- boolean **saw_JFIF_marker**
- UINT8 **JFIF_major_version**
- UINT8 **JFIF_minor_version**
- UINT8 **density_unit**
- UINT16 **X_density**
- UINT16 **Y_density**
- boolean **saw_Adobe_marker**
- UINT8 **Adobe_transform**
- boolean **CCIR601_sampling**
- [jpeg_saved_marker_ptr](#) **marker_list**
- int **max_h_samp_factor**
- int **max_v_samp_factor**
- int **min_DCT_scaled_size**
- JDIMENSION **total_iMCU_rows**
- JSAMPLE * **sample_range_limit**
- int **comps_in_scan**
- [jpeg_component_info](#) * **cur_comp_info** [MAX_COMPS_IN_SCAN]
- JDIMENSION **MCUs_per_row**
- JDIMENSION **MCU_rows_in_scan**
- int **blocks_in_MCU**
- int **MCU_membership** [D_MAX_BLOCKS_IN_MCU]
- int **Ss**
- int **Se**
- int **Ah**
- int **Al**

- int **unread_marker**
- struct jpeg_decomp_master * **master**
- struct jpeg_d_main_controller * **main**
- struct jpeg_d_coef_controller * **coef**
- struct jpeg_d_post_controller * **post**
- struct jpeg_input_controller * **inputctl**
- struct jpeg_marker_reader * **marker**
- struct jpeg_entropy_decoder * **entropy**
- struct jpeg_inverse_dct * **idct**
- struct jpeg_upsampler * **upsample**
- struct jpeg_color_deconverter * **cconvert**
- struct jpeg_color_quantizer * **cquantize**

6.12.1 Detailed Description

Definition at line 472 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.13 jpeg_destination_mgr Struct Reference

Public Attributes

- JOCTET * **next_output_byte**
- size_t **free_in_buffer**
- void(* **init_destination**)(j_compress_ptr cinfo)
- boolean(* **empty_output_buffer**)(j_compress_ptr cinfo)
- void(* **term_destination**)(j_compress_ptr cinfo)

6.13.1 Detailed Description

Definition at line 790 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.14 jpeg_error_mgr Struct Reference

Public Attributes

- void(* **error_exit**)(j_common_ptr cinfo)
- void(* **emit_message**)(j_common_ptr cinfo, int msg_level)
- void(* **output_message**)(j_common_ptr cinfo)
- void(* **format_message**)(j_common_ptr cinfo, char *buffer)
- void(* **reset_error_mgr**)(j_common_ptr cinfo)
- int **msg_code**
-
- union {
 int i [8]
 char s [JMSG_STR_PARM_MAX]
 } **msg_parm**
- int **trace_level**
- long **num_warnings**
- const char *const * **jpeg_message_table**
- int **last_jpeg_message**
- const char *const * **addon_message_table**
- int **first_addon_message**
- int **last_addon_message**

6.14.1 Detailed Description

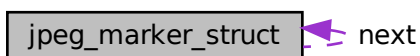
Definition at line 720 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.15 jpeg_marker_struct Struct Reference

Collaboration diagram for jpeg_marker_struct:



Public Attributes

- [jpeg_saved_marker_ptr](#) next
- UINT8 marker
- unsigned int original_length
- unsigned int data_length
- JOCTET * data

6.15.1 Detailed Description

Definition at line 203 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.16 jpeg_memory_mgr Struct Reference

Public Attributes

- void *(* **alloc_small**)(j_common_ptr cinfo, int pool_id, size_t sizeofobject)
- void *(* **alloc_large**)(j_common_ptr cinfo, int pool_id, size_t sizeofobject)
- JSAMPARRAY(* **alloc_sarray**)(j_common_ptr cinfo, int pool_id, JDIMENSION samplesperrow, JDIMENSION numrows)
- JBLOCKARRAY(* **alloc_barray**)(j_common_ptr cinfo, int pool_id, JDIMENSION blocksperrow, JDIMENSION numrows)
- jvirt_sarray_ptr(* **request_virt_sarray**)(j_common_ptr cinfo, int pool_id, boolean pre_zero, JDIMENSION samplesperrow, JDIMENSION numrows, JDIMENSION maxaccess)
- jvirt_barray_ptr(* **request_virt_barray**)(j_common_ptr cinfo, int pool_id, boolean pre_zero, JDIMENSION blocksperrow, JDIMENSION numrows, JDIMENSION maxaccess)
- void(* **realize_virt_arrays**)(j_common_ptr cinfo)
- JSAMPARRAY(* **access_virt_sarray**)(j_common_ptr cinfo, jvirt_sarray_ptr ptr, JDIMENSION start_row, JDIMENSION num_rows, boolean writable)
- JBLOCKARRAY(* **access_virt_barray**)(j_common_ptr cinfo, jvirt_barray_ptr ptr, JDIMENSION start_row, JDIMENSION num_rows, boolean writable)
- void(* **free_pool**)(j_common_ptr cinfo, int pool_id)
- void(* **self_destruct**)(j_common_ptr cinfo)
- long **max_memory_to_use**
- long **max_alloc_chunk**

6.16.1 Detailed Description

Definition at line 833 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.17 jpeg_progress_mgr Struct Reference

Public Attributes

- void(* **progress_monitor**)(j_common_ptr cinfo)
- long **pass_counter**
- long **pass_limit**
- int **completed_passes**
- int **total_passes**

6.17.1 Detailed Description

Definition at line 778 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.18 jpeg_scan_info Struct Reference

Public Attributes

- int **comps_in_scan**
- int **component_index** [MAX_COMPS_IN_SCAN]
- int **Ss**
- int **Se**
- int **Ah**
- int **Al**

6.18.1 Detailed Description

Definition at line 192 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.19 jpeg_source_mgr Struct Reference

Public Attributes

- const JOCTET * **next_input_byte**
- size_t **bytes_in_buffer**
- void(* **init_source**)(j_decompress_ptr cinfo)
- boolean(* **fill_input_buffer**)(j_decompress_ptr cinfo)
- void(* **skip_input_data**)(j_decompress_ptr cinfo, long num_bytes)
- boolean(* **resync_to_restart**)(j_decompress_ptr cinfo, int desired)
- void(* **term_source**)(j_decompress_ptr cinfo)

6.19.1 Detailed Description

Definition at line 802 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.20 JQUANT_TBL Struct Reference

Public Attributes

- UINT16 **quantval** [DCTSIZE2]
- boolean **sent_table**

6.20.1 Detailed Description

Definition at line 86 of file jpeglib.h.

The documentation for this struct was generated from the following file:

- jpeglib.h

6.21 nsimd_aarch64_vf16 Struct Reference

Public Attributes

- float32x4_t **v0**
- float32x4_t **v1**

6.21.1 Detailed Description

Definition at line 36 of file arm/aarch64/types.h.

The documentation for this struct was generated from the following file:

- arm/aarch64/types.h

6.22 nsimd_aarch64_vlf16 Struct Reference

Public Attributes

- uint32x4_t **v0**
- uint32x4_t **v1**

6.22.1 Detailed Description

Definition at line 55 of file arm/aarch64/types.h.

The documentation for this struct was generated from the following file:

- arm/aarch64/types.h

6.23 nsimd_avx2_vf16 Struct Reference

Public Attributes

- __m256 v0
- __m256 v1

6.23.1 Detailed Description

Definition at line 32 of file x86/avx2/types.h.

The documentation for this struct was generated from the following file:

- x86/avx2/types.h

6.24 nsimd_avx2_vlf16 Struct Reference

Public Attributes

- __m256 v0
- __m256 v1

6.24.1 Detailed Description

Definition at line 44 of file x86/avx2/types.h.

The documentation for this struct was generated from the following file:

- x86/avx2/types.h

6.25 nsimd_avx512_knl_vf16 Struct Reference

Public Attributes

- __m512 v0
- __m512 v1

6.25.1 Detailed Description

Definition at line 32 of file x86/avx512_knl/types.h.

The documentation for this struct was generated from the following file:

- x86/avx512_knl/types.h

6.26 nsimd_avx512_knl_vlf16 Struct Reference

Public Attributes

- __mmask16 v0
- __mmask16 v1

6.26.1 Detailed Description

Definition at line 44 of file x86/avx512_knl/types.h.

The documentation for this struct was generated from the following file:

- x86/avx512_knl/types.h

6.27 nsimd_avx512_skylake_vf16 Struct Reference

Public Attributes

- __m512 v0
- __m512 v1

6.27.1 Detailed Description

Definition at line 32 of file x86/avx512_skylake/types.h.

The documentation for this struct was generated from the following file:

- x86/avx512_skylake/types.h

6.28 nsimd_avx512_skylake_vlf16 Struct Reference

Public Attributes

- __mmask16 v0
- __mmask16 v1

6.28.1 Detailed Description

Definition at line 44 of file x86/avx512_skylake/types.h.

The documentation for this struct was generated from the following file:

- x86/avx512_skylake/types.h

6.29 nsimd_avx_vf16 Struct Reference

Public Attributes

- __m256 v0
- __m256 v1

6.29.1 Detailed Description

Definition at line 32 of file x86/avx/types.h.

The documentation for this struct was generated from the following file:

- x86/avx/types.h

6.30 nsimd_avx_vlf16 Struct Reference

Public Attributes

- __m256 v0
- __m256 v1

6.30.1 Detailed Description

Definition at line 44 of file x86/avx/types.h.

The documentation for this struct was generated from the following file:

- x86/avx/types.h

6.31 nsimd_cpu_vf16 Struct Reference

Public Attributes

- f32 v0
- f32 v1
- f32 v2
- f32 v3
- f32 v4
- f32 v5
- f32 v6
- f32 v7

6.31.1 Detailed Description

Definition at line 36 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.32 nsimd_cpu_vf32 Struct Reference

Public Attributes

- f32 v0
- f32 v1
- f32 v2
- f32 v3

6.32.1 Detailed Description

Definition at line 32 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.33 nsimd_cpu_vf64 Struct Reference

Public Attributes

- f64 v0
- f64 v1

6.33.1 Detailed Description

Definition at line 30 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.34 nsimd_cpu_vi16 Struct Reference

Public Attributes

- `i16 v0`
- `i16 v1`
- `i16 v2`
- `i16 v3`
- `i16 v4`
- `i16 v5`
- `i16 v6`
- `i16 v7`

6.34.1 Detailed Description

Definition at line 50 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.35 nsimd_cpu_vi32 Struct Reference

Public Attributes

- `i32 v0`
- `i32 v1`
- `i32 v2`
- `i32 v3`

6.35.1 Detailed Description

Definition at line 46 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.36 nsimd_cpu_vi64 Struct Reference

Public Attributes

- i64 v0
- i64 v1

6.36.1 Detailed Description

Definition at line 44 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.37 nsimd_cpu_vi8 Struct Reference

Public Attributes

- i8 v0
- i8 v1
- i8 v2
- i8 v3
- i8 v4
- i8 v5
- i8 v6
- i8 v7
- i8 v8
- i8 v9
- i8 v10
- i8 v11
- i8 v12
- i8 v13
- i8 v14
- i8 v15

6.37.1 Detailed Description

Definition at line 58 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.38 nsimd_cpu_vlf16 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**
- unsigned int **v4**
- unsigned int **v5**
- unsigned int **v6**
- unsigned int **v7**

6.38.1 Detailed Description

Definition at line 111 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.39 nsimd_cpu_vlf32 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**

6.39.1 Detailed Description

Definition at line 107 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.40 nsimd_cpu_vlf64 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**

6.40.1 Detailed Description

Definition at line 105 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.41 nsimd_cpu_vli16 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**
- unsigned int **v4**
- unsigned int **v5**
- unsigned int **v6**
- unsigned int **v7**

6.41.1 Detailed Description

Definition at line 125 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.42 nsimd_cpu_vli32 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**

6.42.1 Detailed Description

Definition at line 121 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.43 nsimd_cpu_vli64 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**

6.43.1 Detailed Description

Definition at line 119 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.44 nsimd_cpu_vli8 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**
- unsigned int **v4**
- unsigned int **v5**
- unsigned int **v6**
- unsigned int **v7**
- unsigned int **v8**
- unsigned int **v9**
- unsigned int **v10**
- unsigned int **v11**
- unsigned int **v12**
- unsigned int **v13**
- unsigned int **v14**
- unsigned int **v15**

6.44.1 Detailed Description

Definition at line 133 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.45 nsimd_cpu_vlu16 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**
- unsigned int **v4**
- unsigned int **v5**
- unsigned int **v6**
- unsigned int **v7**

6.45.1 Detailed Description

Definition at line 155 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.46 nsimd_cpu_vlu32 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**

6.46.1 Detailed Description

Definition at line 151 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.47 nsimd_cpu_vlu64 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**

6.47.1 Detailed Description

Definition at line 149 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.48 nsimd_cpu_vlu8 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**
- unsigned int **v4**
- unsigned int **v5**
- unsigned int **v6**
- unsigned int **v7**
- unsigned int **v8**
- unsigned int **v9**
- unsigned int **v10**
- unsigned int **v11**
- unsigned int **v12**
- unsigned int **v13**
- unsigned int **v14**
- unsigned int **v15**

6.48.1 Detailed Description

Definition at line 163 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.49 nsimd_cpu_vu16 Struct Reference

Public Attributes

- u16 **v0**
- u16 **v1**
- u16 **v2**
- u16 **v3**
- u16 **v4**
- u16 **v5**
- u16 **v6**
- u16 **v7**

6.49.1 Detailed Description

Definition at line 80 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.50 nsimd_cpu_vu32 Struct Reference

Public Attributes

- `u32 v0`
- `u32 v1`
- `u32 v2`
- `u32 v3`

6.50.1 Detailed Description

Definition at line 76 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.51 nsimd_cpu_vu64 Struct Reference

Public Attributes

- `u64 v0`
- `u64 v1`

6.51.1 Detailed Description

Definition at line 74 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.52 nsimd_cpu_vu8 Struct Reference

Public Attributes

- `u8 v0`
- `u8 v1`
- `u8 v2`
- `u8 v3`
- `u8 v4`
- `u8 v5`
- `u8 v6`
- `u8 v7`
- `u8 v8`
- `u8 v9`
- `u8 v10`
- `u8 v11`
- `u8 v12`
- `u8 v13`
- `u8 v14`
- `u8 v15`

6.52.1 Detailed Description

Definition at line 88 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

6.53 nsimd_neon128_vf16 Struct Reference

Public Attributes

- `float32x4_t v0`
- `float32x4_t v1`

6.53.1 Detailed Description

Definition at line 36 of file `arm/neon128/types.h`.

The documentation for this struct was generated from the following file:

- `arm/neon128/types.h`

6.54 nsimd_neon128_vf64 Struct Reference

Public Attributes

- double **v0**
- double **v1**

6.54.1 Detailed Description

Definition at line 30 of file arm/neon128/types.h.

The documentation for this struct was generated from the following file:

- arm/neon128/types.h

6.55 nsimd_neon128_vlf16 Struct Reference

Public Attributes

- uint32x4_t **v0**
- uint32x4_t **v1**

6.55.1 Detailed Description

Definition at line 55 of file arm/neon128/types.h.

The documentation for this struct was generated from the following file:

- arm/neon128/types.h

6.56 nsimd_neon128_vlf64 Struct Reference

Public Attributes

- u64 **v0**
- u64 **v1**

6.56.1 Detailed Description

Definition at line 49 of file arm/neon128/types.h.

The documentation for this struct was generated from the following file:

- arm/neon128/types.h

6.57 nsimd_sse2_vf16 Struct Reference

Public Attributes

- `__m128 v0`
- `__m128 v1`

6.57.1 Detailed Description

Definition at line 32 of file `x86/sse2/types.h`.

The documentation for this struct was generated from the following file:

- `x86/sse2/types.h`

6.58 nsimd_sse2_vlf16 Struct Reference

Public Attributes

- `__m128 v0`
- `__m128 v1`

6.58.1 Detailed Description

Definition at line 44 of file `x86/sse2/types.h`.

The documentation for this struct was generated from the following file:

- `x86/sse2/types.h`

6.59 nsimd_sse42_vf16 Struct Reference

Public Attributes

- `__m128 v0`
- `__m128 v1`

6.59.1 Detailed Description

Definition at line 32 of file `x86/sse42/types.h`.

The documentation for this struct was generated from the following file:

- `x86/sse42/types.h`

6.60 nsimd_sse42_vlf16 Struct Reference

Public Attributes

- `__m128 v0`
- `__m128 v1`

6.60.1 Detailed Description

Definition at line 44 of file `x86/sse42/types.h`.

The documentation for this struct was generated from the following file:

- `x86/sse42/types.h`

6.61 nsimd_vmx_vf16 Struct Reference

Public Attributes

- `__vector float v0`
- `__vector float v1`

6.61.1 Detailed Description

Definition at line 32 of file `ppc/vmx/types.h`.

The documentation for this struct was generated from the following file:

- `ppc/vmx/types.h`

6.62 nsimd_vmx_vf64 Struct Reference

Public Attributes

- `f64 v0`
- `f64 v1`

6.62.1 Detailed Description

Definition at line 30 of file `ppc/vmx/types.h`.

The documentation for this struct was generated from the following file:

- `ppc/vmx/types.h`

6.63 nsimd_vmx_vf64 Struct Reference

Public Attributes

- i64 v0
- i64 v1

6.63.1 Detailed Description

Definition at line 33 of file ppc/vmx/types.h.

The documentation for this struct was generated from the following file:

- ppc/vmx/types.h

6.64 nsimd_vmx_vlf16 Struct Reference

Public Attributes

- __vector __bool int v0
- __vector __bool int v1

6.64.1 Detailed Description

Definition at line 44 of file ppc/vmx/types.h.

The documentation for this struct was generated from the following file:

- ppc/vmx/types.h

6.65 nsimd_vmx_vlf64 Struct Reference

Public Attributes

- u64 v0
- u64 v1

6.65.1 Detailed Description

Definition at line 42 of file ppc/vmx/types.h.

The documentation for this struct was generated from the following file:

- ppc/vmx/types.h

6.66 nsimd_vmx_vli64 Struct Reference

Public Attributes

- u64 **v0**
- u64 **v1**

6.66.1 Detailed Description

Definition at line 45 of file `ppc/vmx/types.h`.

The documentation for this struct was generated from the following file:

- `ppc/vmx/types.h`

6.67 nsimd_vmx_vlu64 Struct Reference

Public Attributes

- u64 **v0**
- u64 **v1**

6.67.1 Detailed Description

Definition at line 49 of file `ppc/vmx/types.h`.

The documentation for this struct was generated from the following file:

- `ppc/vmx/types.h`

6.68 nsimd_vmx_vu64 Struct Reference

Public Attributes

- u64 **v0**
- u64 **v1**

6.68.1 Detailed Description

Definition at line 37 of file `ppc/vmx/types.h`.

The documentation for this struct was generated from the following file:

- `ppc/vmx/types.h`

6.69 nsimd_vsx_vf16 Struct Reference

Public Attributes

- `__vector float v0`
- `__vector float v1`

6.69.1 Detailed Description

Definition at line 32 of file `ppc/vsx/types.h`.

The documentation for this struct was generated from the following file:

- `ppc/vsx/types.h`

6.70 nsimd_vsx_vlf16 Struct Reference

Public Attributes

- `__vector __bool int v0`
- `__vector __bool int v1`

6.70.1 Detailed Description

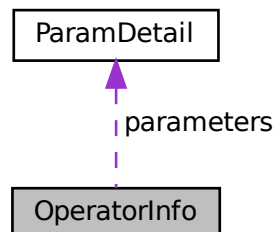
Definition at line 44 of file `ppc/vsx/types.h`.

The documentation for this struct was generated from the following file:

- `ppc/vsx/types.h`

6.71 OperatorInfo Struct Reference

Collaboration diagram for OperatorInfo:



Public Attributes

- char **name** [64]
- char **description** [128]
- bool **enabled**
- [ParamDetail](#) * **parameters**
- uint32_t **param_count**

6.71.1 Detailed Description

Definition at line 38 of file TYCAPL.h.

The documentation for this struct was generated from the following file:

- TYCAPL.h

6.72 ParamDetail Struct Reference

Public Attributes

- char **name** [64]
- AlgParamType **type**
- char **description** [128]
- ```
union {
 bool bool_value
 int32_t int_value
 float float_value
};
```
- int32\_t **min\_int**
- int32\_t **max\_int**
- int32\_t **step\_int**
- float **min\_float**
- float **max\_float**
- float **step\_float**

### 6.72.1 Detailed Description

Definition at line 19 of file TYCAPL.h.

The documentation for this struct was generated from the following file:

- TYCAPL.h

## 6.73 pattern\_bin\_param Struct Reference

### Public Attributes

- uint32\_t **offset**
- uint8\_t **data** [512]

#### 6.73.1 Detailed Description

Definition at line 1088 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.74 pattern\_gray\_param Struct Reference

### Public Attributes

- uint32\_t **phase\_num**
- uint32\_t **param1**
- uint32\_t **param2**
- uint32\_t **param3**

#### 6.74.1 Detailed Description

Definition at line 1080 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.75 pattern\_sine\_param Struct Reference

### Public Attributes

- uint32\_t **phase\_num**
- float **period**

#### 6.75.1 Detailed Description

Definition at line 1074 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)



## 6.76 TY\_PHC\_GROUP\_ATTR::phc\_group\_attr Struct Reference

### Public Attributes

- `uint8_t type`
- `uint8_t amp_thresh`
- `uint16_t ch`
- `uint8_t chn_type`
- `uint8_t rsvd [27]`

### 6.76.1 Detailed Description

Definition at line 1058 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.77 tjregion Struct Reference

```
#include <turbojpeg.h>
```

### Public Attributes

- `int x`
- `int y`
- `int w`
- `int h`

### 6.77.1 Detailed Description

Cropping region

Definition at line 566 of file turbojpeg.h.

### 6.77.2 Member Data Documentation

#### 6.77.2.1 h

```
int tjregion::h
```

The height of the cropping region. Setting this to 0 is the equivalent of setting it to the height of the source JPEG image - y.

Definition at line 586 of file turbojpeg.h.

### 6.77.2.2 w

```
int tjregion::w
```

The width of the cropping region. Setting this to 0 is the equivalent of setting it to the width of the source JPEG image - x.

Definition at line 581 of file turbojpeg.h.

### 6.77.2.3 x

```
int tjregion::x
```

The left boundary of the cropping region. This must be evenly divisible by the MCU block width (see #tjMCUWidth.)

Definition at line 571 of file turbojpeg.h.

### 6.77.2.4 y

```
int tjregion::y
```

The upper boundary of the cropping region. This must be evenly divisible by the MCU block height (see #tjMCUHeight.)

Definition at line 576 of file turbojpeg.h.

The documentation for this struct was generated from the following file:

- turbojpeg.h

## 6.78 tjscalingfactor Struct Reference

```
#include <turbojpeg.h>
```

### Public Attributes

- int [num](#)
- int [denom](#)

### 6.78.1 Detailed Description

Scaling factor

Definition at line 552 of file turbojpeg.h.

## 6.78.2 Member Data Documentation

### 6.78.2.1 denom

```
int tjscalingfactor::denom
```

Denominator

Definition at line 560 of file turbojpeg.h.

### 6.78.2.2 num

```
int tjscalingfactor::num
```

Numerator

Definition at line 556 of file turbojpeg.h.

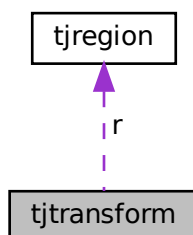
The documentation for this struct was generated from the following file:

- turbojpeg.h

## 6.79 tjtransform Struct Reference

```
#include <turbojpeg.h>
```

Collaboration diagram for tjtransform:



## Public Attributes

- [tjregion](#) *r*
- int *op*
- int *options*
- void \* *data*
- int(\* [customFilter](#) )(short \*coeffs, [tjregion](#) arrayRegion, [tjregion](#) planeRegion, int componentIndex, int transformIndex, struct [tjtransform](#) \*transform)

### 6.79.1 Detailed Description

Lossless transform

Definition at line 592 of file `turbojpeg.h`.

### 6.79.2 Member Data Documentation

#### 6.79.2.1 customFilter

```
int(* tjtransform::customFilter) (short *coeffs, tjregion arrayRegion, tjregion planeRegion,
int componentIndex, int transformIndex, struct tjtransform *transform)
```

A callback function that can be used to modify the DCT coefficients after they are losslessly transformed but before they are transcoded to a new JPEG image. This allows for custom filters or other transformations to be applied in the frequency domain.

#### Parameters

|                    |                                                                                                                                                                                                                                                                                                                                |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>coeffs</i>      | pointer to an array of transformed DCT coefficients. (NOTE: this pointer is not guaranteed to be valid once the callback returns, so applications wishing to hand off the DCT coefficients to another function or library should make a copy of them within the body of the callback.)                                         |
| <i>arrayRegion</i> | <a href="#">tjregion</a> structure containing the width and height of the array pointed to by <i>coeffs</i> as well as its offset relative to the component plane. TurboJPEG implementations may choose to split each component plane into multiple DCT coefficient arrays and call the callback function once for each array. |
| <i>planeRegion</i> | <a href="#">tjregion</a> structure containing the width and height of the component plane to which <i>coeffs</i> belongs                                                                                                                                                                                                       |
| <i>componentID</i> | ID number of the component plane to which <i>coeffs</i> belongs (Y, Cb, and Cr have, respectively, ID's of 0, 1, and 2 in typical JPEG images.)                                                                                                                                                                                |
| <i>transformID</i> | ID number of the transformed image to which <i>coeffs</i> belongs. This is the same as the index of the transform in the <code>transforms</code> array that was passed to <a href="#">tjTransform()</a> .                                                                                                                      |
| <i>transform</i>   | a pointer to a <a href="#">tjtransform</a> structure that specifies the parameters and/or cropping region for this transform                                                                                                                                                                                                   |

#### Returns

0 if the callback was successful, or -1 if an error occurred.

Definition at line 643 of file turbojpeg.h.

#### 6.79.2.2 data

```
void* tjtransform::data
```

Arbitrary data that can be accessed within the body of the callback function

Definition at line 609 of file turbojpeg.h.

#### 6.79.2.3 op

```
int tjtransform::op
```

One of the [transform operations](#)

Definition at line 600 of file turbojpeg.h.

#### 6.79.2.4 options

```
int tjtransform::options
```

The bitwise OR of one of more of the [transform options](#)

Definition at line 604 of file turbojpeg.h.

#### 6.79.2.5 r

```
tjregion tjtransform::r
```

Cropping region

Definition at line 596 of file turbojpeg.h.

The documentation for this struct was generated from the following file:

- turbojpeg.h

## 6.80 TY\_ACC\_BIAS Struct Reference

```
#include <TYDefs.h>
```

### Public Attributes

- float **data** [3]

#### 6.80.1 Detailed Description

a 3x3 matrix

| .     | .     | .     |
|-------|-------|-------|
| BIASx | BIASy | BIASz |

Definition at line 1138 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.81 TY\_ACC\_MISALIGNMENT Struct Reference

```
#include <TYDefs.h>
```

### Public Attributes

- float **data** [3 \*3]

### 6.81.1 Detailed Description

a 3x3 matrix

|.|.|

| .                   | .                   | .                   |
|---------------------|---------------------|---------------------|
| 1                   | -GAMAy <sub>z</sub> | GAMAz <sub>y</sub>  |
| GAMAx <sub>z</sub>  | 1                   | -GAMAz <sub>x</sub> |
| -GAMAx <sub>y</sub> | GAMAy <sub>x</sub>  | 1                   |

Definition at line 1150 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.82 TY\_ACC\_SCALE Struct Reference

```
#include <TYDefs.h>
```

### Public Attributes

- float **data** [3 \*3]

### 6.82.1 Detailed Description

a 3x3 matrix

| .                  | .                  | .                  |
|--------------------|--------------------|--------------------|
| SCALE <sub>x</sub> | 0                  | 0                  |
| 0                  | SCALE <sub>y</sub> | 0                  |
| 0                  | 0                  | SCALE <sub>z</sub> |

Definition at line 1161 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.83 TY\_AEC\_ROI\_PARAM Struct Reference

### Public Attributes

- uint32\_t **x**
- uint32\_t **y**
- uint32\_t **w**
- uint32\_t **h**

### 6.83.1 Detailed Description

Definition at line 1038 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.84 TY\_BYTEARRAY\_ATTR Struct Reference

byte array data structure

```
#include <TYDefs.h>
```

### Public Attributes

- int32\_t [size](#)  
*Bytes array size in bytes.*
- int32\_t [unit\\_size](#)
- int32\_t [valid\\_size](#)

### 6.84.1 Detailed Description

byte array data structure

See also

[TYGetByteArray](#)

Definition at line 906 of file TYDefs.h.

### 6.84.2 Member Data Documentation

#### 6.84.2.1 unit\_size

```
int32_t TY_BYTEARRAY_ATTR::unit_size
```

unit size in bytes for special parse

Definition at line 909 of file TYDefs.h.

#### 6.84.2.2 valid\_size

```
int32_t TY_BYTEARRAY_ATTR::valid_size
```

valid size in bytes in case has reserved member, Must be multiple of unit\_size, mem\_length = valid\_size/unit\_size

Definition at line 912 of file TYDefs.h.

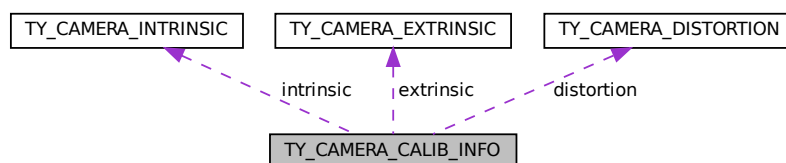
The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.85 TY\_CAMERA\_CALIB\_INFO Struct Reference

```
#include <TYDefs.h>
```

Collaboration diagram for TY\_CAMERA\_CALIB\_INFO:





## Public Attributes

- `int32_t intrinsicWidth`
- `int32_t intrinsicHeight`
- `TY_CAMERA_INTRINSIC` `intrinsic`
- `TY_CAMERA_EXTRINSIC` `extrinsic`
- `TY_CAMERA_DISTORTION` `distortion`

### 6.85.1 Detailed Description

camera 's cailbration data

See also

[TYGetStruct](#)

Definition at line 981 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.86 TY\_CAMERA\_DISTORTION Struct Reference

```
#include <TYDefs.h>
```

## Public Attributes

- `float data` [12]  
*Definition is compatible with opencv3.0+ :k1,k2,p1,p2,k3,k4,k5,k6,s1,s2,s3,s4.*

### 6.86.1 Detailed Description

camera distortion parameters

See also

[TYGetStruct](#) Usage:  
`TY_CAMERA_DISTORTION distortion;`  
`TYGetStruct(hDevice, some_compoent, TY_STRUCT_CAM_DISTORTION, &distortion, sizeof(distortion));`

Definition at line 973 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.87 TY\_CAMERA\_EXTRINSIC Struct Reference

```
#include <TYDefs.h>
```

## Public Attributes

- `float data` [4 \*4]

### 6.87.1 Detailed Description

a 4x4 matrix

| .   | .   | .   | .  |
|-----|-----|-----|----|
| r11 | r12 | r13 | t1 |
| r21 | r22 | r23 | t2 |
| r31 | r32 | r33 | t3 |
| 0   | 0   | 0   | 1  |

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_EXTRINSIC extrinsic;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_EXTRINSIC, &extrinsic, sizeof(extrinsic));
```

Definition at line 961 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.88 TY\_CAMERA\_INTRINSIC Struct Reference

```
#include <TYDefs.h>
```

### Public Attributes

- float **data** [3 \*3]

### 6.88.1 Detailed Description

a 3x3 matrix

| .  | .  | .  |
|----|----|----|
| fx | 0  | cx |
| 0  | fy | cy |
| 0  | 0  | 1  |

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_INTRINSIC intrinsic;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_CAM_INTRINSIC, &intrinsic, sizeof(intrinsic));
```

Definition at line 943 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.89 TY\_CAMERA\_ROTATION Struct Reference

```
#include <TYDefs.h>
```

### Public Attributes

- float **data** [3 \*3]

### 6.89.1 Detailed Description

a 3x3 matrix

| .   | .   | .   |
|-----|-----|-----|
| r00 | r01 | r02 |
| r10 | r11 | r12 |
| r20 | r21 | r22 |

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_ROTATION rotation;
TYGetStruct(hDevice, some_compoent, TY_STRUCTURE_CAM_ROTATION, &rotation, sizeof(rotation));
```

Definition at line 1319 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.90 TY\_CAMERA\_STATISTICS Struct Reference

### Public Attributes

- uint64\_t **packetReceived**
- uint64\_t **packetLost**
- uint64\_t **imageOutputed**
- uint64\_t **imageDropped**
- uint8\_t **rsvd** [1024]

### 6.90.1 Detailed Description

Definition at line 1112 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.91 TY\_CAMERA\_TO\_IMU Struct Reference

```
#include <TYDefs.h>
```

### Public Attributes

- float **data** [4 \*4]

### 6.91.1 Detailed Description

a 4x4 matrix

| .   | .   | .   | .  |
|-----|-----|-----|----|
| r11 | r12 | r13 | t1 |
| r21 | r22 | r23 | t2 |
| r31 | r32 | r33 | t3 |
| 0   | 0   | 0   | 1  |

Definition at line 1204 of file TYDefs.h.

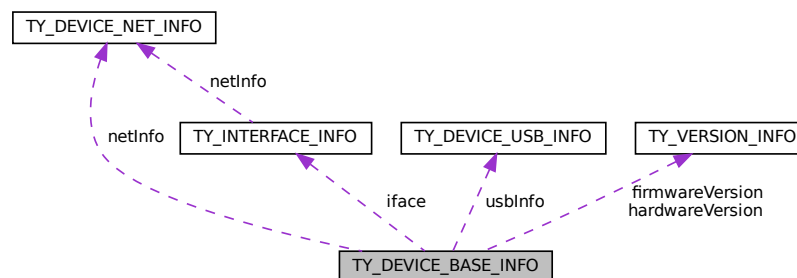
The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.92 TY\_DEVICE\_BASE\_INFO Struct Reference

```
#include <TYDefs.h>
```

Collaboration diagram for TY\_DEVICE\_BASE\_INFO:



## Public Attributes

- [TY\\_INTERFACE\\_INFO](#) **iface**
- char **id** [32]  
*device serial number*
- char **vendorName** [32]
- char **userDefinedName** [32]
- char **modelName** [32]  
*device model name*
- [TY\\_VERSION\\_INFO](#) **hardwareVersion**  
*deprecated*
- [TY\\_VERSION\\_INFO](#) **firmwareVersion**  
*deprecated*
- ```

union {
    TY\_DEVICE\_NET\_INFO netInfo
    TY\_DEVICE\_USB\_INFO usbInfo
};

```
- char **buildHash** [256]
- char **configVersion** [256]
- char **reserved** [256]

6.92.1 Detailed Description

See also

[TYGetDeviceList](#)

Definition at line 844 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.93 TY_DEVICE_NET_INFO Struct Reference

device network information

```
#include <TYDefs.h>
```

Public Attributes

- char **mac** [32]
- char **ip** [32]
- char **netmask** [32]
- char **gateway** [32]
- char **broadcast** [32]
- char **tlversion** [32]
- char **reserved** [64]

6.93.1 Detailed Description

device network information

Definition at line 814 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.94 TY_DEVICE_USB_INFO Struct Reference

Public Attributes

- int **bus**
- int **addr**
- char **tlversion** [32]
- char **reserved** [216]

6.94.1 Detailed Description

Definition at line 825 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.95 TY_DI_WORKMODE Struct Reference

Public Attributes

- TY_E_DI_MODE **mode**
- TY_E_DI_INT_ACTION **int_act**
- uint32_t **mode_supported**
- uint32_t **int_act_supported**
- uint32_t **status**
- uint32_t **reserved** [3]

6.95.1 Detailed Description

Definition at line 1286 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.96 TY_DO_WORKMODE Struct Reference

Public Attributes

- TY_E_DO_MODE **mode**
- TY_E_VOLT_T **volt**
- uint32_t **freq**
- uint32_t **duty**
- uint32_t **mode_supported**
- uint32_t **volt_supported**
- uint32_t **reserved** [3]

6.96.1 Detailed Description

Definition at line 1263 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.97 TY_ENUM_ENTRY Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- char **description** [64]
- uint32_t **value**
- uint32_t **reserved** [3]

6.97.1 Detailed Description

enum feature entry information

See also

[TYGetEnumEntryInfo](#)

Definition at line 917 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.98 TY_EVENT_INFO Struct Reference

Public Attributes

- TY_EVENT **eventId**
- char **message** [124]

6.98.1 Detailed Description

Definition at line 1257 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.99 TY_FEATURE_INFO Struct Reference

Public Attributes

- bool **isValid**
true if feature exists, false otherwise
- TY_ACCESS_MODE **accessMode**
feature access privilege
- bool **writableAtRun**
feature can be written while capturing
- char **reserved0** [1]
- TY_COMPONENT_ID **componentID**
owner of this feature
- TY_FEATURE_ID **featureID**
feature unique id
- char **name** [32]
describe string
- TY_COMPONENT_ID **bindComponentID**
component ID current feature bind to
- TY_FEATURE_ID **bindFeatureID**
feature ID current feature bind to
- TY_VISIBILITY_TYPE **visibility**
- char **reserved** [248]

6.99.1 Detailed Description

Definition at line 870 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.100 TY_FLOAT_RANGE Struct Reference

float range data structure

```
#include <TYDefs.h>
```

Public Attributes

- float **min**
- float **max**
- float **inc**
increasing step
- float **reserved** [1]

6.100.1 Detailed Description

float range data structure

See also

[TYGetFloatRange](#)

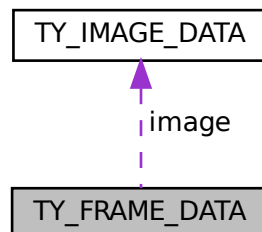
Definition at line 896 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.101 TY_FRAME_DATA Struct Reference

Collaboration diagram for TY_FRAME_DATA:



Public Attributes

- void * [userBuffer](#)
Pointer to user enqueued buffer, user should enqueue this buffer in the end of callback.
- int32_t [bufferSize](#)
Size of userBuffer.
- int32_t [validCount](#)
Number of valid data.
- int32_t [reserved](#) [6]
Reserved: reserved[0],laser_val;.
- [TY_IMAGE_DATA image](#) [10]
Buffer data, max to 10 images per frame, each buffer data could be an image or something else.

6.101.1 Detailed Description

Definition at line 1247 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.102 TY_GYRO_BIAS Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [3]

6.102.1 Detailed Description

a 3x3 matrix

.	.	.
BIASx	BIASy	BIASz

Definition at line 1170 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.103 TY_GYRO_MISALIGNMENT Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [3 *3]

6.103.1 Detailed Description

a 3x3 matrix

.	.	.
1	-ALPHAyz	ALPHAzy
0	1	-ALPHAzx
0	0	1

Definition at line 1181 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.104 TY_GYRO_SCALE Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [3 *3]

6.104.1 Detailed Description

a 3x3 matrix

.	.	.
SCALEx	0	0
0	SCALEy	0
0	0	SCALEz

Definition at line 1192 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.105 TY_IMAGE_DATA Struct Reference

Public Attributes

- uint64_t [timestamp](#)
Timestamp in microseconds.
- int32_t [imageIndex](#)
image index used in trigger mode
- int32_t [status](#)
Status of this buffer.
- [TY_COMPONENT_ID](#) [componentID](#)
Where current data come from.
- int32_t [size](#)
Buffer size.
- void * [buffer](#)
Pointer to data buffer.
- int32_t [width](#)
Image width in pixels.
- int32_t [height](#)
Image height in pixels.
- [TYPixFmt](#) [pixelFormat](#)
Pixel format, see [TYPixFmtList](#).
- int32_t [reserved](#) [9]
Reserved.

6.105.1 Detailed Description

Definition at line 1232 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.106 TY_IMU_DATA Struct Reference

Public Attributes

- uint64_t [timestamp](#)
- float [acc_x](#)
- float [acc_y](#)
- float [acc_z](#)
- float [gyro_x](#)
- float [gyro_y](#)
- float [gyro_z](#)
- float [temperature](#)
- float [reserved](#) [1]

6.106.1 Detailed Description

Definition at line 1121 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.107 TY_INT_RANGE Struct Reference

Public Attributes

- int32_t **min**
- int32_t **max**
- int32_t **inc**
increasing step
- int32_t **reserved** [1]

6.107.1 Detailed Description

Definition at line 886 of file TYDefs.h.

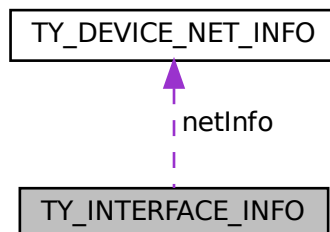
The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.108 TY_INTERFACE_INFO Struct Reference

```
#include <TYDefs.h>
```

Collaboration diagram for TY_INTERFACE_INFO:



Public Attributes

- char **name** [32]
- char **id** [32]
- TY_INTERFACE_TYPE **type**
- char **reserved** [4]
- [TY_DEVICE_NET_INFO](#) **netInfo**

6.108.1 Detailed Description

See also

[TYGetInterfaceList](#)

Definition at line 834 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.109 TY_LASER_PARAM Struct Reference

Public Attributes

- uint32_t **idx**
- uint32_t **en**
- uint32_t **power**

6.109.1 Detailed Description

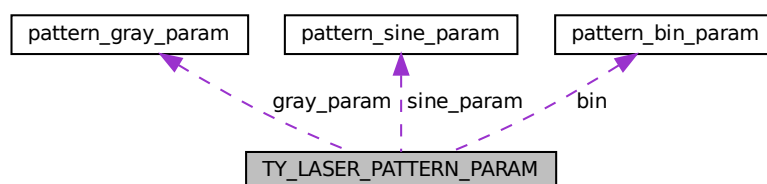
Definition at line 1222 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

6.110 TY_LASER_PATTERN_PARAM Struct Reference

Collaboration diagram for TY_LASER_PATTERN_PARAM:



Public Attributes

- uint32_t **img_index**
 - uint32_t **type**
 -
- ```

union {
 uint8_t payload [512+16]
 pattern_sine_param sine_param
 pattern_gray_param gray_param
 pattern_bin_param bin
};

```

### 6.110.1 Detailed Description

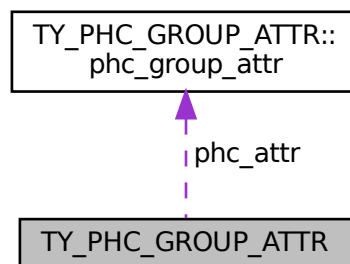
Definition at line 1094 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.111 TY\_PHC\_GROUP\_ATTR Struct Reference

Collaboration diagram for TY\_PHC\_GROUP\_ATTR:



## Classes

- struct [phc\\_group\\_attr](#)

## Public Attributes

- uint32\_t **offset**
- uint32\_t **size**
- struct [TY\\_PHC\\_GROUP\\_ATTR::phc\\_group\\_attr](#) **phc\_attr** [16]

### 6.111.1 Detailed Description

Definition at line 1054 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.112 TY\_PIXEL\_COLOR\_DESC Struct Reference

### Public Attributes

- `int16_t x`
- `int16_t y`
- `uint8_t bgr_ch1`
- `uint8_t bgr_ch2`
- `uint8_t bgr_ch3`
- `uint8_t rsvd`

### 6.112.1 Detailed Description

Definition at line 20 of file TYCoordinateMapper.h.

The documentation for this struct was generated from the following file:

- [TYCoordinateMapper.h](#)

## 6.113 TY\_PIXEL\_DESC Struct Reference

### Public Attributes

- `int16_t x`
- `int16_t y`
- `uint16_t depth`
- `uint16_t rsvd`

### 6.113.1 Detailed Description

Definition at line 12 of file TYCoordinateMapper.h.

The documentation for this struct was generated from the following file:

- [TYCoordinateMapper.h](#)



## 6.114 TY\_TEMP\_DATA Struct Reference

### Public Attributes

- uint32\_t **id**
- char **name** [16]
- char **temp** [16]
- char **desc** [16]

#### 6.114.1 Detailed Description

Definition at line 1300 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.115 TY\_TOF\_FREQ Struct Reference

### Public Attributes

- uint32\_t **freq1**
- uint32\_t **freq2**

#### 6.115.1 Detailed Description

Definition at line 1209 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.116 TY\_TRIGGER\_PARAM Struct Reference

### Public Attributes

- TY\_TRIGGER\_MODE **mode**
- int8\_t **fps**
- int8\_t **rsvd**

#### 6.116.1 Detailed Description

Definition at line 992 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.117 TY\_TRIGGER\_PARAM\_EX Struct Reference

### Public Attributes

- TY\_TRIGGER\_MODE **mode**
- 

```
union {
 struct {
 int8_t fps
 int8_t duty
 int32_t laser_stream
 int32_t led_stream
 int32_t led_expo
 int32_t led_gain
 }
 struct {
 int32_t ir_gain [2]
 }
 int32_t rsvd [32]
};
```

### 6.117.1 Detailed Description

Definition at line 1000 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.118 TY\_TRIGGER\_TIMER\_LIST Struct Reference

### Public Attributes

- uint64\_t **start\_time\_us**
- uint32\_t **offset\_us\_count**
- uint32\_t **offset\_us\_list** [50]

### 6.118.1 Detailed Description

Definition at line 1023 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.119 TY\_TRIGGER\_TIMER\_PERIOD Struct Reference

### Public Attributes

- uint64\_t **start\_time\_us**
- uint32\_t **trigger\_count**
- uint32\_t **period\_us**

### 6.119.1 Detailed Description

Definition at line 1031 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.120 TY\_VECT\_3F Struct Reference

### Public Attributes

- float **x**
- float **y**
- float **z**

### 6.120.1 Detailed Description

Definition at line 924 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.121 TY\_VERSION\_INFO Struct Reference

### Public Attributes

- int32\_t **major**
- int32\_t **minor**
- int32\_t **patch**
- int32\_t **reserved**

### 6.121.1 Detailed Description

Definition at line 805 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 6.122 TYDecodeResult Struct Reference

### Public Attributes

- uint32\_t **width**
- uint32\_t **height**
- uint32\_t **dataSize**
- TYPixFmt **format**

### 6.122.1 Detailed Description

Definition at line 44 of file TYImageProc.h.

The documentation for this struct was generated from the following file:

- [TYImageProc.h](#)

## 6.123 TYEnumEntry Struct Reference

### Public Attributes

- int64\_t **value**
- char **name** [64]
- char **tooltip** [512]
- char **description** [512]
- char **displayName** [512]

### 6.123.1 Detailed Description

Definition at line 16 of file TYParameter.h.

The documentation for this struct was generated from the following file:

- [TYParameter.h](#)

## 6.124 TYImageInfo Struct Reference

### Public Attributes

- uint32\_t **width**
- uint32\_t **height**
- TYPixFmt **format**
- uint32\_t **dataSize**
- const void \* **data**

### 6.124.1 Detailed Description

Definition at line 27 of file TYImageProc.h.

The documentation for this struct was generated from the following file:

- [TYImageProc.h](#)

## 6.125 z\_stream\_s Struct Reference

### Public Attributes

- z\_const Bytef \* **next\_in**
- uInt **avail\_in**
- uLong **total\_in**
- Bytef \* **next\_out**
- uInt **avail\_out**
- uLong **total\_out**
- z\_const char \* **msg**
- struct internal\_state FAR \* **state**
- alloc\_func **zalloc**
- free\_func **zfree**
- voidpf **opaque**
- int **data\_type**
- uLong **adler**
- uLong **reserved**

### 6.125.1 Detailed Description

Definition at line 86 of file zlib.h.

The documentation for this struct was generated from the following file:

- [zlib.h](#)



## Chapter 7

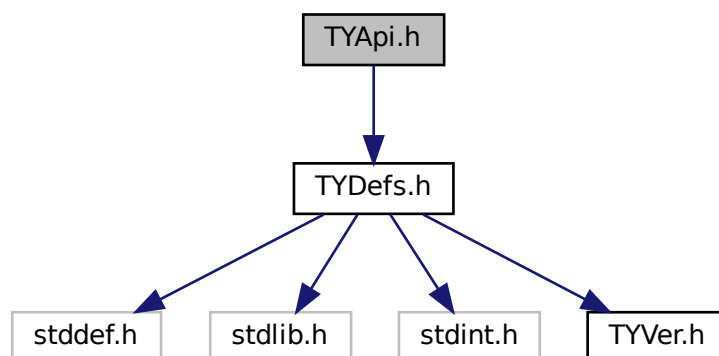
# File Documentation

### 7.1 TYApi.h File Reference

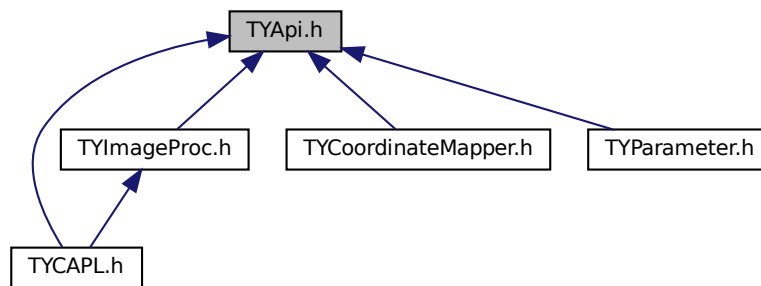
[TYApi.h](#) includes camera control and data receiving interface, which supports configuration for image resolution, frame rate, exposure time, gain, working mode, etc.

```
#include "TYDefs.h"
```

Include dependency graph for TYApi.h:



This graph shows which files directly or indirectly include this file:



## Typedefs

- typedef void(\* **TY\_EVENT\_CALLBACK**) (**TY\_EVENT\_INFO** \*, void \*userdata)
- typedef void(\* **TY\_IMU\_CALLBACK**) (**TY\_IMU\_DATA** \*, void \*userdata)

## Functions

- TY\_CAPI **TYInitLib** (void)
- TY\_CAPI **TYLibVersion** (**TY\_VERSION\_INFO** \*version)  
*Get current library version.*
- TY\_EXTC const TY\_EXPORT char \*TY\_STDC **TYErrorString** (TY\_STATUS errorID)  
*Get error information.*
- TY\_CAPI **TYDeinitLib** (void)  
*Deinit this library.*
- TY\_CAPI **TYSetLogLevel** (TY\_LOG\_TYPE type, TY\_LOG\_LEVEL lvl)  
*Set log level.*
- TY\_CAPI **TYSetLogPrefix** (const char \*prefix)  
*set log prefix*
- TY\_CAPI **TYEnableLog** (TY\_LOG\_TYPE type)  
*Enable logger by type.*
- TY\_CAPI **TYDisableLog** (TY\_LOG\_TYPE type)  
*Disable logger by type.*
- TY\_CAPI **TYCfgLogFile** (const char \*filePath, uint32\_t max\_size=10 \*1024 \*1024, uint32\_t max\_files=10)  
*Append log to specified file(s), max\_size must not be 0. If max\_files Not 0, it will use rotation files. Rotate files as: log.txt -> log.1.txt log.1.txt -> log.2.txt log.2.txt -> log.3.txt log.3.txt -> delete.*
- TY\_CAPI **TYCfgLogServer** (TY\_SERVER\_TYPE prot\_type, const char \*ip, uint16\_t port)  
*Append log to Tcp/Udp server.*
- TY\_CAPI **TYUpdateInterfaceList** (void)  
*Update current interfaces. call before TYGetInterfaceList.*
- TY\_CAPI **TYGetInterfaceNumber** (uint32\_t \*pNumIfaces)  
*Get number of current interfaces.*
- TY\_CAPI **TYGetInterfaceList** (**TY\_INTERFACE\_INFO** \*pIfaceInfos, uint32\_t bufferCount, uint32\_t \*filledCount)  
*Get interface info list.*



- TY\_CAPI [TYHasInterface](#) (const char \*ifaceID, bool \*value)  
*Check if has interface.*
- TY\_CAPI [TYOpenInterface](#) (const char \*ifaceID, [TY\\_INTERFACE\\_HANDLE](#) \*outHandle)  
*Open specified interface.*
- TY\_CAPI [TYCloseInterface](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle)  
*Close interface.*
- TY\_CAPI [TYUpdateDeviceList](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle)  
*Update current connected devices.*
- TY\_CAPI [TYUpdateAllDeviceList](#) (void)  
*Update current connected devices.*
- TY\_CAPI [TYGetDeviceNumber](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, uint32\_t \*deviceNumber)  
*Get number of current connected devices.*
- TY\_CAPI [TYGetDeviceList](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, [TY\\_DEVICE\\_BASE\\_INFO](#) \*deviceInfos, uint32\_t bufferCount, uint32\_t \*filledDeviceCount)  
*Get device info list.*
- TY\_CAPI [TYHasDevice](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, const char \*deviceID, bool \*value)  
*Check whether the interface has the specified device.*
- TY\_CAPI [TYOpenDevice](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, const char \*deviceID, [TY\\_DEV\\_HANDLE](#) \*outDeviceHandle, [TY\\_FW\\_ERRORCODE](#) \*outFwErrorcode=NULL)  
*Open device by device ID.*
- TY\_CAPI [TYOpenDeviceWithIP](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, const char \*IP, [TY\\_DEV\\_HANDLE](#) \*deviceHandle)  
*Open device by device IP, useful when a device is not listed.*
- TY\_CAPI [TYGetDeviceInterface](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_INTERFACE\\_HANDLE](#) \*piface)  
*Get interface handle by device handle.*
- TY\_CAPI [TYForceDeviceIP](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, const char \*MAC, const char \*newIP, const char \*newNetMask, const char \*newGateway)  
*Force a ethernet device to use new IP address, useful when device use persistent IP and cannot be found.*
- TY\_CAPI [TYCloseDevice](#) ([TY\\_DEV\\_HANDLE](#) hDevice, bool reboot=false)  
*Close device by device handle.*
- TY\_CAPI [TYGetDeviceInfo](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_DEVICE\\_BASE\\_INFO](#) \*info)  
*Get base info of the open device.*
- TY\_CAPI [TYGetComponentIDs](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) \*componentIDs)  
*Get all components IDs.*
- TY\_CAPI [TYGetEnabledComponents](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) \*componentIDs)  
*Get all enabled components IDs.*
- TY\_CAPI [TYEnableComponents](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentIDs)  
*Enable components.*
- TY\_CAPI [TYDisableComponents](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentIDs)  
*Disable components.*
- TY\_CAPI [TYGetFrameBufferSize](#) ([TY\\_DEV\\_HANDLE](#) hDevice, uint32\_t \*bufferSize)  
*Get total buffer size of one frame in current configuration.*
- TY\_CAPI [TYEnqueueBuffer](#) ([TY\\_DEV\\_HANDLE](#) hDevice, void \*buffer, uint32\_t bufferSize)  
*Enqueue a user allocated buffer.*
- TY\_CAPI [TYClearBufferQueue](#) ([TY\\_DEV\\_HANDLE](#) hDevice)  
*Clear the internal buffer queue, so that user can release all the buffer.*
- TY\_CAPI [TYStartCapture](#) ([TY\\_DEV\\_HANDLE](#) hDevice)  
*Start capture.*
- TY\_CAPI [TYStopCapture](#) ([TY\\_DEV\\_HANDLE](#) hDevice)  
*Stop capture.*
- TY\_CAPI [TYSendSoftTrigger](#) ([TY\\_DEV\\_HANDLE](#) hDevice)

*Send a software trigger to capture a frame when device works in trigger mode.*

- TY\_CAPI [TYRegisterEventCallback](#) (TY\_DEV\_HANDLE hDevice, TY\_EVENT\_CALLBACK callback, void \*userdata)

*Register device status callback. Register NULL to clean callback.*

- TY\_CAPI [TYRegisterImuCallback](#) (TY\_DEV\_HANDLE hDevice, TY\_IMU\_CALLBACK callback, void \*userdata)

*Register imu callback. Register NULL to clean callback.*

- TY\_CAPI [TYFetchFrame](#) (TY\_DEV\_HANDLE hDevice, TY\_FRAME\_DATA \*frame, int32\_t timeout)

*Fetch one frame.*

- TY\_CAPI [TYHasFeature](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, bool \*value)

*Check whether a component has a specific feature.*

- TY\_CAPI [TYGetFeatureInfo](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, TY\_FEATURE\_INFO \*featureInfo)

*Get feature info.*

- TY\_CAPI [TYGetIntRange](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, TY\_INT\_RANGE \*intRange)

*Get value range of integer feature.*

- TY\_CAPI [TYGetInt](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, int32\_t \*value)

*Get value of integer feature.*

- TY\_CAPI [TYSetInt](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, int32\_t value)

*Set value of integer feature.*

- TY\_CAPI [TYGetFloatRange](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, TY\_FLOAT\_RANGE \*floatRange)

*Get value range of float feature.*

- TY\_CAPI [TYGetFloat](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, float \*value)

*Get value of float feature.*

- TY\_CAPI [TYSetFloat](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, float value)

*Set value of float feature.*

- TY\_CAPI [TYGetEnumEntryCount](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, uint32\_t \*entryCount)

*Get number of enum entries.*

- TY\_CAPI [TYGetEnumEntryInfo](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, TY\_ENUM\_ENTRY \*entries, uint32\_t entryCount, uint32\_t \*filledEntryCount)

*Get list of enum entries.*

- TY\_CAPI [TYGetEnum](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, uint32\_t \*value)

*Get current value of enum feature.*

- TY\_CAPI [TYSetEnum](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, uint32\_t value)

*Set value of enum feature.*

- TY\_CAPI [TYGetBool](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, bool \*value)

*Get value of bool feature.*

- TY\_CAPI [TYSetBool](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, bool value)

*Set value of bool feature.*

- TY\_CAPI [TYGetStringLength](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, uint32\_t \*size)

*Get internal buffer size of string feature.*

- TY\_CAPI [TYGetString](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, char \*buffer, uint32\_t bufferSize)

*Get value of string feature.*

- TY\_CAPI [TYSetString](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, const char \*buffer)

*Set value of string feature.*

- TY\_CAPI [TYGetStruct](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, void \*pStruct, uint32\_t structSize)

*Get value of struct.*

- TY\_CAPI [TYSetStruct](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, void \*pStruct, uint32\_t structSize)

*Set value of struct.*

- TY\_CAPI [TYGetByteArraySize](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, uint32\_t \*pSize)

*Get the size of specified byte array zone.*

- TY\_CAPI [TYGetByteArray](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, uint8\_t \*pBuffer, uint32\_t bufferSize)

*Read byte array from device.*

- TY\_CAPI [TYSetByteArray](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, const uint8\_t \*pBuffer, uint32\_t bufferSize)

*Write byte array to device.*

- TY\_CAPI [TYGetByteArrayAttr](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, [TY\\_BYTEARRAY\\_ATTR](#) \*pAttr)

*Write byte array to device.*

- TY\_CAPI [TYGetDeviceFeatureNumber](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, uint32\_t \*size)

*Get the size of device features.*

- TY\_CAPI [TYGetDeviceFeatureInfo](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_INFO](#) \*featureInfo, uint32\_t entryCount, uint32\_t \*filledEntryCount)

*Get the all features by comp id.*

- TY\_CAPI [TYGetDeviceXMLSize](#) ([TY\\_DEV\\_HANDLE](#) hDevice, uint32\_t \*size)

*Get the Device xml size.*

- TY\_CAPI [TYGetDeviceXML](#) ([TY\\_DEV\\_HANDLE](#) hDevice, char \*xml, const uint32\_t in\_size, uint32\_t \*out\_size)

*Get the Device xml string.*

### 7.1.1 Detailed Description

[TYApi.h](#) includes camera control and data receiving interface, which supports configuration for image resolution, frame rate, exposure time, gain, working mode, etc.

### 7.1.2 Function Documentation

### 7.1.2.1 TYCfgLogFile()

```
TY_CAPI TYCfgLogFile (
 const char * filePath,
 uint32_t max_size = 10 * 1024 * 1024,
 uint32_t max_files = 10)
```

Append log to specified file(s), max\_size must not be 0. If max\_files Not 0, it will use rotation files. Rotate files as: log.txt -> log.1.txt log.1.txt -> log.2.txt log.2.txt -> log.3.txt log.3.txt -> delete.

#### Parameters

|    |                  |                                                    |
|----|------------------|----------------------------------------------------|
| in | <i>filePath</i>  | Path to the log file.                              |
| in | <i>max_size</i>  | max single files size, can't be 0, default is 10M. |
| in | <i>max_files</i> | Max files to keep, default is 10.                  |

#### Return values

|                        |                                                                                                                           |
|------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>    | Succeed.                                                                                                                  |
| <i>TY_STATUS_ERROR</i> | Failed to add file<br><br>Suggestions:<br>Please check if the file path is correct and if you have permission to write to |

### 7.1.2.2 TYCfgLogServer()

```
TY_CAPI TYCfgLogServer (
 TY_SERVER_TYPE prot_type,
 const char * ip,
 uint16_t port)
```

Append log to Tcp/Udp server.

#### Parameters

|    |                  |                                         |
|----|------------------|-----------------------------------------|
| in | <i>prot_type</i> | Protocol of the server, "tcp" or "udp". |
| in | <i>ip</i>        | IP address of the server.               |
| in | <i>port</i>      | Port of the server.                     |

#### Return values

|                        |                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>    | Succeed.                                                                                |
| <i>TY_STATUS_ERROR</i> | Failed to add server<br><br>Suggestions:<br>Please check if the ip and port are correct |

## Return values

|                                          |                                                                                         |
|------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>TY_STATUS_INVALID_PARAMETER</code> | Unsupported protocol<br><br>Suggestions:<br>Unsupported protocol, please use tcp or udp |
|------------------------------------------|-----------------------------------------------------------------------------------------|

## 7.1.2.3 TYClearBufferQueue()

```
TY_CAPI TYClearBufferQueue (
 TY_DEV_HANDLE hDevice)
```

Clear the internal buffer queue, so that user can release all the buffer.

## Parameters

|    |                      |                |
|----|----------------------|----------------|
| in | <code>hDevice</code> | Device handle. |
|----|----------------------|----------------|

## Return values

|                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TY_STATUS_OK</code>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <code>TY_STATUS_INVALID_HANDLE</code> | TYClearBufferQueue called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYClearBufferQueue(hDevice);</pre> ^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdated |
| <code>TY_STATUS_BUSY</code>           | Device is capturing.                                                                                                                                                                                                                                                                                                                                                                                                                               |

## 7.1.2.4 TYCloseDevice()

```
TY_CAPI TYCloseDevice (
 TY_DEV_HANDLE hDevice,
 bool reboot = false)
```

Close device by device handle.

## Parameters

|    |                      |                            |
|----|----------------------|----------------------------|
| in | <code>hDevice</code> | Device handle.             |
| in | <code>reboot</code>  | Reboot device after close. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYCloseDevice called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYCloseDevice(hDevice, reboot);</code><br/>                           ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/>           1.TYOpenDevice failed to open device and get correct handle<br/>           2.Memory in stack to store handle data is corrupted<br/>           3.After getting handle, you updated device list by calling TYUpdatedDeviceList</p> |
| <i>TY_STATUS_TIMEOUT</i>        | <p>Failed to close device</p> <p>Suggestions:<br/>Possible reasons:<br/>           1.Network communication is abnormal, please check whether the network is normal</p>                                                                                                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_DEVICE_ERROR</i>   | <p>Failed to close device</p> <p>Suggestions:<br/>Possible reasons:<br/>           1.Camera device is abnormal and cannot be closed normally.</p>                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>TY_STATUS_IDLE</i>           | Device has been closed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

## 7.1.2.5 TYCloseInterface()

```
TY_CAPI TYCloseInterface (
 TY_INTERFACE_HANDLE ifaceHandle)
```

Close interface.

## Parameters

|    |                    |                         |
|----|--------------------|-------------------------|
| in | <i>ifaceHandle</i> | Interface to be closed. |
|----|--------------------|-------------------------|

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>TY_STATUS_NOT_INITED</i>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <i>TY_STATUS_INVALID_INTERFACE</i> | <p>TYCloseInterface called with invalid interface handle</p> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <code>TYCloseInterface(ifaceHandle);</code><br/>                           ^ is invalid</p> <p>The ifaceHandle parameter you input is not recorded</p> <p>Possible reasons:<br/>           1.TYOpenInterface failed to open interface and get correct handle<br/>           2.Memory in stack to store handle data is corrupted<br/>           3.After getting handle, you updated interface list by calling TYUpdateInterfaceList</p> |

### 7.1.2.6 TYDeinitLib()

```
TY_CAPI TYDeinitLib (
 void)
```

Deinit this library.

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 7.1.2.7 TYDisableComponents()

```
TY_CAPI TYDisableComponents (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentIDs)
```

Disable components.

#### Parameters

|    |                     |                            |
|----|---------------------|----------------------------|
| in | <i>hDevice</i>      | Device handle.             |
| in | <i>componentIDs</i> | Components to be disabled. |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYDisableComponents called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/>    TYDisableComponents(hDevice, componentIDs);<br/>                                ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"><li>1.TYOpenDevice failed to open device and get correct handle</li><li>2.Memory in stack to store handle data is corrupted</li><li>3.After getting handle, you updated device list by calling TYUpo</li></ol> |
| <i>TY_STATUS_INVALID_PARAMETER</i> | <p>Invalid component IDs</p> <p>Suggestions:<br/>Please check componentIDs parameter<br/>Like this:<br/>    TYDisableComponents(hDevice, componentIDs);<br/>                                ^ is invalid</p> <p>componentIDs should be the value returned by TYGetComponentIDs<br/>You can also view the components of the camera by obtaining the x</p>                                                                                                                                                                                                             |
| <i>TY_STATUS_INVALID_COMPONENT</i> | Some components specified by componentIDs are invalid.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

## Return values

|                       |                                                                                                                                                                                                                                                   |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_BUSY</i> | Camera device is capturing<br><br>Suggestions:<br>Please call <code>TYEnableComponents</code> when the camera device is stopped<br>Like this:<br><code>TYStopCapture(hDevice);</code><br><code>TYDisableComponents(hDevice, componentIDs);</code> |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## See also

[TY\\_DEVICE\\_COMPONENT\\_LIST](#)

### 7.1.2.8 TYDisableLog()

```
TY_CAPI TYDisableLog (
 TY_LOG_TYPE type)
```

Disable logger by type.

## Parameters

|    |             |              |
|----|-------------|--------------|
| in | <i>type</i> | Logger type. |
|----|-------------|--------------|

## Return values

|                        |                                                                                                              |
|------------------------|--------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>    | Succeed.                                                                                                     |
| <i>TY_STATUS_ERROR</i> | Failed to disable the selected logger<br><br>Suggestions:<br>Please check if the selected logger is disabled |

### 7.1.2.9 TYEnableComponents()

```
TY_CAPI TYEnableComponents (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentIDs)
```

Enable components.

## Parameters

|    |                     |                           |
|----|---------------------|---------------------------|
| in | <i>hDevice</i>      | Device handle.            |
| in | <i>componentIDs</i> | Components to be enabled. |



## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYEnableComponents called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYEnableComponents(hDevice, componentIDs);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUp</p> |
| <i>TY_STATUS_INVALID_PARAMETER</i> | <p>Invalid component IDs</p> <p>Suggestions:<br/>Please check componentIDs parameter<br/>Like this:<br/> <pre>TYEnableComponents(hDevice, componentIDs);</pre> ^ is invalid<br/> componentIDs should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                       |
| <i>TY_STATUS_INVALID_COMPONENT</i> | Some components specified by componentIDs are invalid.                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_BUSY</i>              | <p>Camera device is capturing</p> <p>Suggestions:<br/>Please call TYEnableComponents when the camera device is stopped<br/>Like this:<br/> <pre>TYStopCapture(hDevice);</pre> <pre>TYEnableComponents(hDevice, componentIDs);</pre></p>                                                                                                                                                                                                                                          |

## See also

[TY\\_DEVICE\\_COMPONENT\\_LIST](#)

## 7.1.2.10 TYEnableLog()

```
TY_CAPI TYEnableLog (
 TY_LOG_TYPE type)
```

Enable logger by type.

## Parameters

|    |             |              |
|----|-------------|--------------|
| in | <i>type</i> | Logger type. |
|----|-------------|--------------|

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                        |                                                                                                                                            |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_ERROR</i> | Failed to enable the selected logger<br><br>Suggestions:<br>Please call TYCfgLogFile or TYCfgLogServer before enable log to file or log to |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|

## 7.1.2.11 TYEnqueueBuffer()

```

TY_CAPI TYEnqueueBuffer (
 TY_DEV_HANDLE hDevice,
 void * buffer,
 uint32_t bufferSize)

```

Enqueue a user allocated buffer.

## Parameters

|    |                   |                           |
|----|-------------------|---------------------------|
| in | <i>hDevice</i>    | Device handle.            |
| in | <i>buffer</i>     | Buffer to be enqueued.    |
| in | <i>bufferSize</i> | Size of the input buffer. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>TY_STATUS_INVALID_HANDLE</i> | TYEnqueueBuffer called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br>TYEnqueueBuffer(hDevice, buffer, bufferSize);<br>^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdated |
| <i>TY_STATUS_NULL_POINTER</i>   | TYEnqueueBuffer called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br>TYEnqueueBuffer(hDevice, buffer, bufferSize);<br>^ is NULL                                                                                                                                                                                                                                                                                       |
| <i>TY_STATUS_WRONG_SIZE</i>     | TYEnqueueBuffer called with wrong size<br><br>Suggestions:<br>Please check your code<br>Like this:<br>TYEnqueueBuffer(hDevice, buffer, bufferSize);<br>^ is 0 or negative value                                                                                                                                                                                                                                                                          |

## Return values

|                               |                                                                                                                                                  |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_TIMEOUT</i>      | Failed to enqueue frame buffer<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the network |
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to enqueue frame buffer<br><br>Suggestions:<br>Possible reasons:<br>1.Camera device is abnormal and cannot get the frame buffer size.     |

## 7.1.2.12 TYErrorString()

```
TY_EXTC const TY_EXPORT char* TY_STDC TYErrorString (
 TY_STATUS errorID)
```

Get error information.

## Parameters

|    |                |           |
|----|----------------|-----------|
| in | <i>errorID</i> | Error id. |
|----|----------------|-----------|

## Return values

|              |         |
|--------------|---------|
| <i>Error</i> | string. |
|--------------|---------|

## 7.1.2.13 TYFetchFrame()

```
TY_CAPI TYFetchFrame (
 TY_DEV_HANDLE hDevice,
 TY_FRAME_DATA * frame,
 int32_t timeout)
```

Fetch one frame.

## Parameters

|     |                |                                           |
|-----|----------------|-------------------------------------------|
| in  | <i>hDevice</i> | Device handle.                            |
| out | <i>frame</i>   | Frame data to be filled.                  |
| in  | <i>timeout</i> | Timeout in milliseconds. <0 for infinite. |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYFetchFrame called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYFetchFrame(hDevice, pFrame, timeout); ^ is invalid</pre> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>   | <p>TYFetchFrame called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYFetchFrame(hDevice, pFrame, timeout); ^ is NULL</pre>                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_IDLE</i>           | <p>Camera device is not started</p> <p>Suggestions:</p> <p>Please start the camera device first</p> <p>Like this:</p> <pre>TYStartCapture(hDevice); TYFetchFrame(hDevice, pFrame, timeout);</pre>                                                                                                                                                                                                                                                                                                                                      |
| <i>TY_STATUS_WRONG_MODE</i>     | <p>Callback has been registered, this function is disabled.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <i>TY_STATUS_TIMEOUT</i>        | <p>Failed to get frame</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Camera device is abnormal and cannot get frame.</li> <li>2.Network communication is abnormal, please check whether the network</li> <li>3.Timeout, frame acquisition timeout</li> </ol>                                                                                                                                                                                                                               |

## 7.1.2.14 TYForceDeviceIP()

```
TY_CAPI TYForceDeviceIP (
 TY_INTERFACE_HANDLE ifaceHandle,
 const char * MAC,
 const char * newIP,
 const char * newNetMask,
 const char * newGateway)
```

Force a ethernet device to use new IP address, useful when device use persistent IP and cannot be found.

## Parameters

|    |                    |                                            |
|----|--------------------|--------------------------------------------|
| in | <i>ifaceHandle</i> | Interface handle.                          |
| in | <i>MAC</i>         | Device MAC, should be "xx:xx:xx:xx:xx:xx". |
| in | <i>newIP</i>       | New IP.                                    |
| in | <i>newNetMask</i>  | New subnet mask.                           |
| in | <i>newGateway</i>  | New gateway.                               |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TY_STATUS_OK</b>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>TY_STATUS_NOT_INITED</b>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>TY_STATUS_INVALID_INTERFACE</b> | <p>TYForceDeviceIP called with invalid interface handle</p> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <pre>TYForceDeviceIP(ifaceHandle, MAC, newIP, newNetMask, newGateway)</pre> ^ is invalid<br/> The ifaceHandle parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenInterface failed to open interface and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated interface list by calling TYForceDeviceIP</p>                                                                                           |
| <b>TY_STATUS_WRONG_TYPE</b>        | <p>TYForceDeviceIP called with invalid interface type</p> <p>Suggestions:<br/>Please check interface type<br/>Usually you can get interface information by calling TYGetInterfaceList<br/>You can use TYIsNetworkInterface to check the interface type<br/>Only network interfaces can call TYForceDeviceIP<br/>Like this:<br/> <pre>TY_INTERFACE_INFO info; uint32_t num; TYGetInterfaceList(&amp;info, 1, &amp;num); if (TYIsNetworkInterface(info[0].type)) {     TY_INTERFACE_HANDLE hIface;     TYOpenInterface(info[0].id, &amp;hIface);     TYForceDeviceIP(hIface, MAC, newIP, newNetMask, newGateway); }</pre></p> |
| <b>TY_STATUS_NULL_POINTER</b>      | <p>TYForceDeviceIP called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <pre>TYForceDeviceIP(ifaceHandle, MAC, newIP, newNetMask, newGateway)</pre> ^ or ^ or ^ or ^ is NULL</p>                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>TY_STATUS_INVALID_PARAMETER</b> | <p>Invalid MAC address:</p> <p>Suggestions:<br/>Please check MAC parameter<br/>Like this:<br/> <pre>TYForceDeviceIP(ifaceHandle, MAC, newIP, newNetMask, newGateway)</pre> ^ is invalid<br/> MAC address should be six bytes of hexadecimal separated by colons<br/> For example: 00:11:22:aa:bb:cc</p>                                                                                                                                                                                                                                                                                                                     |
| <b>TY_STATUS_TIMEOUT</b>           | <p>Failed to force set IP</p> <p>Suggestions:<br/>Possible reasons:<br/> 1.Network communication is abnormal, please check whether the network is normal<br/> 2.There is no camera with a matching target MAC address in the network</p>                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>TY_STATUS_DEVICE_ERROR</b>      | <p>Failed to force set IP</p> <p>Suggestions:<br/>Possible reasons:<br/> 1.New IP, NetMask, Gateway are incorrect, camera device refuses to set IP</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

### 7.1.2.15 TYGetBool()

```
TY_CAPI TYGetBool (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 bool * value)
```

Get value of bool feature.

#### Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>value</i>       | Bool value.    |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetBool called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYGetBool(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYGetBool(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                  |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYGetBool(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGet<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                 |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <pre>TYGetBool(hDevice, componentID, featureID, pValue);</pre> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeatur</p>                                                                                                                                                                                                                      |

## Return values

|                               |                                                                                                                                                                                                          |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_NULL_POINTER</i> | TYGetBool called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetBool(hDevice, componentID, featureID, pValue); ^ is NULL</pre>                               |
| <i>TY_STATUS_TIMEOUT</i>      | Failed to get bool feature<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the network is connected                                                |
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to get bool feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not implemented<br>2.Camera device is abnormal and cannot get bool feature |

## 7.1.2.16 TYGetByteArray()

```

TY_CAPI TYGetByteArray (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 uint8_t * pBuffer,
 uint32_t bufferSize)

```

Read byte array from device.

## Parameters

|     |                    |                 |
|-----|--------------------|-----------------|
| in  | <i>hDevice</i>     | Device handle.  |
| in  | <i>componentID</i> | Component ID.   |
| in  | <i>featureID</i>   | Feature ID.     |
| out | <i>pBuffer</i>     | Byte buffer.    |
| in  | <i>bufferSize</i>  | Size of buffer. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <i>TY_STATUS_INVALID_HANDLE</i> | TYGetByteArray called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYGetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize); ^ is invalid</pre> The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdateDeviceList |

## Return values

|                                          |                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TY_STATUS_INVALID_COMPONENT</code> | <p>Invalid component ID</p> <p>Suggestions:</p> <p>Please check componentID parameter</p> <p>Like this:</p> <pre>TYGetByteArray(hDevice, componentID, featureID, buffer, bufferSize);</pre> <p>^ is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs</p> <p>You can also view the components of the camera by obtaining the x</p>                                                     |
| <code>TY_STATUS_INVALID_FEATURE</code>   | <p>Invalid feature ID</p> <p>Suggestions:</p> <p>Please check featureID parameter</p> <p>Like this:</p> <pre>TYGetByteArray(hDevice, componentID, featureID, buffer, bufferSize);</pre> <p>^ is invalid</p> <p>You entered an invalid featureID parameter</p> <p>You can get a list of features of the camera device through TYGetT</p> <p>You can also view the features of the camera device by obtaining t</p> |
| <code>TY_STATUS_WRONG_TYPE</code>        | <p>Feature type mismatch</p> <p>Suggestions:</p> <p>Please check the feature type</p> <p>Like this:</p> <pre>TYGetByteArray(hDevice, componentID, featureID, buffer, bufferSize);</pre> <p>^ type mismatch</p> <p>The feature type you entered does not match. You can use TYFeature</p>                                                                                                                          |
| <code>TY_STATUS_NULL_POINTER</code>      | <p>TYGetByteArray called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize);</pre> <p>^ is NULL</p>                                                                                                                                                                                              |
| <code>TY_STATUS_WRONG_SIZE</code>        | <p>Array size mismatch</p> <p>Suggestions:</p> <p>Please check the array size</p> <p>Like this:</p> <pre>TYGetByteArray(hDevice, componentID, featureID, buffer, bufferSize);</pre> <p>^ is inv</p> <p>The array size you entered does not match</p>                                                                                                                                                              |
| <code>TY_STATUS_TIMEOUT</code>           | <p>Failed to get byte array feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Network communication is abnormal, please check whether the ne</li> </ol>                                                                                                                                                                                                           |
| <code>TY_STATUS_DEVICE_ERROR</code>      | <p>Failed to get byte array feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.The feature of the camera device is not available or not imple</li> <li>2.Camera device is abnormal and cannot get byte array feature</li> </ol>                                                                                                                                    |

## 7.1.2.17 TYGetByteArrayAttr()

```
TY_CAPI TYGetByteArrayAttr (
 TY_DEV_HANDLE hDevice,
```



```

TY_COMPONENT_ID componentID,
TY_FEATURE_ID featureID,
TY_BYTEARRAY_ATTR * pAttr)

```

Write byte array to device.

#### Parameters

|     |                    |                                    |
|-----|--------------------|------------------------------------|
| in  | <i>hDevice</i>     | Device handle.                     |
| in  | <i>componentID</i> | Component ID.                      |
| in  | <i>featureID</i>   | Feature ID.                        |
| out | <i>pAttr</i>       | Byte array attribute to be filled. |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetByteArrayAttr called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYGetByteArrayAttr(hDevice, componentID, featureID, pAttr);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYGetByteArrayAttr(hDevice, componentID, featureID, pAttr);</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                           |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYGetByteArrayAttr(hDevice, componentID, featureID, pAttr);</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetL<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                         |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <pre>TYGetByteArrayAttr(hDevice, componentID, featureID, pAttr);</pre> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeature</p>                                                                                                                                                                                                                              |

### Return values

|                               |                                                                                                                                                                                                       |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_NULL_POINTER</i> | <p>TYGetByteArrayAttr called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:</p> <pre>TYGetByteArrayAttr(hDevice, componentID, featureID, pAttr);<br/>^ is NULL</pre> |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### 7.1.2.18 TYGetByteArraySize()

```

TY_CAPI TYGetByteArraySize (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 uint32_t * pSize)

```

Get the size of specified byte array zone.

### Parameters

|     |                    |                                    |
|-----|--------------------|------------------------------------|
| in  | <i>hDevice</i>     | Device handle.                     |
| in  | <i>componentID</i> | Component ID.                      |
| in  | <i>featureID</i>   | Feature ID.                        |
| out | <i>pSize</i>       | Size of specified byte array zone. |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetByteArraySize called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYGetByteArraySize(hDevice, componentID, featureID, pSize);</pre> ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUp</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYGetByteArraySize(hDevice, componentID, featureID, pSize);</pre> ^ is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs<br/>You can also view the components of the camera by obtaining the x</p>                                                                                                                                                             |



## Return values

|                               |                                                                                                                                                                             |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_NULL_POINTER</i> | TYGetComponentIDs called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetComponentIDs(hDevice, outComponentIDs); ^ is NULL</pre> |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## See also

[TY\\_DEVICE\\_COMPONENT\\_LIST](#)

## 7.1.2.20 TYGetDeviceFeatureInfo()

```
TY_CAPI TYGetDeviceFeatureInfo (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_INFO * featureInfo,
 uint32_t entryCount,
 uint32_t * filledEntryCount)
```

Get the all features by comp id.

## Parameters

|     |                         |                                              |
|-----|-------------------------|----------------------------------------------|
| in  | <i>hDevice</i>          | Device handle.                               |
| in  | <i>componentID</i>      | Component ID.                                |
| out | <i>featureInfo</i>      | Output feature info.                         |
| in  | <i>entryCount</i>       | Array size of input parameter "featureInfo". |
| out | <i>filledEntryCount</i> | Number of filled featureInfo.                |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>TY_STATUS_INVALID_HANDLE</i> | TYGetDeviceFeatureInfo called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYGetDeviceFeatureInfo(hDevice, componentID, featureInfo, entryCount, filledEntryCount); ^ is invalid</pre> The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdateDeviceList |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                         |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_COMPONENT</i> | Invalid component ID<br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br>TYGetDeviceFeatureInfo(hDevice, componentID, featureInfo, entryInfo, ^ is invalid<br>componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the x |
| <i>TY_STATUS_NULL_POINTER</i>      | TYGetDeviceFeatureInfo called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br>TYGetDeviceFeatureInfo(hDevice, componentID, featureInfo, entryInfo, ^ or                                                                                                                                |
| <i>TY_STATUS_TIMEOUT</i>           | Failed to get feature info<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the n                                                                                                                                                                                  |
| <i>TY_STATUS_DEVICE_ERROR</i>      | Failed to get feature info<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not impl<br>2.Camera device is abnormal and cannot get feature info                                                                                                                       |

## 7.1.2.21 TYGetDeviceFeatureNumber()

```

TY_CAPI TYGetDeviceFeatureNumber (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 uint32_t * size)

```

Get the size of device features.

## Parameters

|     |                    |                          |
|-----|--------------------|--------------------------|
| in  | <i>hDevice</i>     | Device handle.           |
| in  | <i>componentID</i> | Component ID.            |
| out | <i>size</i>        | Size of all feature cnt. |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_INVALID_HANDLE</i></b>    | <b>TYGetDeviceFeatureNumber called with invalid device handle</b><br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYGetDeviceFeatureNumber(hDevice, componentID, size);                         ^ is invalid</pre> The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUp |
| <b><i>TY_STATUS_INVALID_COMPONENT</i></b> | <b>Invalid component ID</b><br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br><pre>TYGetDeviceFeatureNumber(hDevice, componentID, size);                         ^ is invalid</pre> componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the xn                                                                                                                                                         |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | <b>TYGetDeviceFeatureNumber called with NULL pointer</b><br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetDeviceFeatureNumber(hDevice, componentID, size);                         ^ is NULL</pre>                                                                                                                                                                                                                                                                               |
| <b><i>TY_STATUS_TIMEOUT</i></b>           | <b>Failed to get feature number</b><br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the n                                                                                                                                                                                                                                                                                                                                                          |
| <b><i>TY_STATUS_DEVICE_ERROR</i></b>      | <b>Failed to get feature number</b><br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not impl<br>2.Camera device is abnormal and cannot get feature number                                                                                                                                                                                                                                                                                             |

## 7.1.2.22 TYGetDeviceInfo()

```
TY_CAPI TYGetDeviceInfo (
 TY_DEV_HANDLE hDevice,
 TY_DEVICE_BASE_INFO * info)
```

Get base info of the open device.

## Parameters

|     |                |                |
|-----|----------------|----------------|
| in  | <i>hDevice</i> | Device handle. |
| out | <i>info</i>    | Base info out. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetDeviceInfo called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetDeviceInfo(hDevice, info);                 ^ is invalid</pre> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>   | <p>TYGetDeviceInfo called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetDeviceInfo(hDevice, info);                 ^ is NULL</pre>                                                                                                                                                                                                                                                                                                                                                          |

## 7.1.2.23 TYGetDeviceInterface()

```
TY_CAPI TYGetDeviceInterface (
 TY_DEV_HANDLE hDevice,
 TY_INTERFACE_HANDLE * pIface)
```

Get interface handle by device handle.

## Parameters

|     |                |                   |
|-----|----------------|-------------------|
| in  | <i>hDevice</i> | Device handle.    |
| out | <i>pIface</i>  | Interface handle. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetDeviceInterface called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetDeviceInterface(hDevice, pIface);                 ^ is invalid</pre> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |

## Return values

|                                      |                                                                                                                                                                                                  |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_NULL_POINTER</i></b> | TYGetDeviceInterface called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetDeviceInterface(hDevice, pIface);                         ^ is NULL</pre> |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 7.1.2.24 TYGetDeviceList()

```
TY_CAPI TYGetDeviceList (
 TY_INTERFACE_HANDLE ifaceHandle,
 TY_DEVICE_BASE_INFO * deviceInfos,
 uint32_t bufferSize,
 uint32_t * filledDeviceCount)
```

Get device info list.

## Parameters

|     |                          |                                                        |
|-----|--------------------------|--------------------------------------------------------|
| in  | <i>ifaceHandle</i>       | Interface handle.                                      |
| out | <i>deviceInfos</i>       | Device info array to be filled.                        |
| in  | <i>bufferCount</i>       | Array size of deviceInfos.                             |
| out | <i>filledDeviceCount</i> | Number of filled <a href="#">TY_DEVICE_BASE_INFO</a> . |

## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_OK</i></b>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b><i>TY_STATUS_NOT_INITED</i></b>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b><i>TY_STATUS_INVALID_INTERFACE</i></b> | TYGetDeviceList called with invalid interface handle<br><br>Suggestions:<br>Please check interface handle<br>Like this:<br><pre>TYGetDeviceList(ifaceHandle, pDeviceInfos, bufferSize, pFilledDe                         ^ is invalid</pre> The ifaceHandle parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenInterface failed to open interface and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated interface list by calling TYU |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | TYGetDeviceList called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetDeviceList(ifaceHandle, pDeviceInfos, bufferSize, pFilledCo                         ^ is NULL or ^ is 0 or ^ is NULL</pre>                                                                                                                                                                                                                                                                          |



## 7.1.2.25 TYGetDeviceNumber()

```
TY_CAPI TYGetDeviceNumber (
 TY_INTERFACE_HANDLE ifaceHandle,
 uint32_t * deviceNumber)
```

Get number of current connected devices.

## Parameters

|     |                     |                              |
|-----|---------------------|------------------------------|
| in  | <i>ifaceHandle</i>  | Interface handle.            |
| out | <i>deviceNumber</i> | Number of connected devices. |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <i>TY_STATUS_NOT_INITED</i>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>TY_STATUS_INVALID_INTERFACE</i> | <p>TYGetDeviceNumber called with invalid interface handle</p> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <code>TYGetDeviceNumber(ifaceHandle, pDeviceNumber);</code><br/> ^ is invalid</p> <p>The ifaceHandle parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenInterface failed to open interface and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated interface list by calling TYU</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYGetDeviceNumber called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetDeviceNumber(ifaceHandle, deviceNumber);</code><br/> ^ is NULL</p>                                                                                                                                                                                                                                                                                                                                                                        |

## 7.1.2.26 TYGetDeviceXML()

```
TY_CAPI TYGetDeviceXML (
 TY_DEV_HANDLE hDevice,
 char * xml,
 const uint32_t in_size,
 uint32_t * out_size)
```

Get the Device xml string.

## Parameters

|     |                 |                                 |
|-----|-----------------|---------------------------------|
| in  | <i>hDevice</i>  | Device handle.                  |
| in  | <i>xml</i>      | The buffer to store xml         |
| in  | <i>in_size</i>  | The size buffer                 |
| out | <i>out_size</i> | The actual size write in buffer |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>TY_STATUS_NOT_INITED</i>     | Not call TYInitLib                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>TY_STATUS_WRONG_SIZE</i>     | <p>XML buffer size is not enough</p> <p>Suggestions:<br/>XML buffer size is not enough<br/>Like this:<br/> <code>TYGetDeviceXML(hDevice, xml, in_size, out_size);</code><br/> ^ is invalid<br/> XML buffer size is not enough, please use TYGetDeviceXMLSize to get the size</p>                                                                                                                                                                                                                         |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetDeviceXML called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetDeviceXML(hDevice, xml, in_size, out_size);</code><br/> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpdatedDeviceList</p> |
| <i>TY_STATUS_NULL_POINTER</i>   | <p>TYGetDeviceXML called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetDeviceXML(hDevice, xml, in_size, out_size);</code><br/> ^ or ^ is NULL</p>                                                                                                                                                                                                                                                                                                      |

## 7.1.2.27 TYGetDeviceXMLSize()

```

TY_CAPI TYGetDeviceXMLSize (
 TY_DEV_HANDLE hDevice,
 uint32_t * size)

```

Get the Device xml size.

## Parameters

|     |                |                               |
|-----|----------------|-------------------------------|
| in  | <i>hDevice</i> | Device handle.                |
| out | <i>size</i>    | The size of device xml string |

## Return values

|                             |                    |
|-----------------------------|--------------------|
| <i>TY_STATUS_OK</i>         | Succeed.           |
| <i>TY_STATUS_NOT_INITED</i> | Not call TYInitLib |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetDeviceXMLSize called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetDeviceXMLSize(hDevice, size);                     ^ is invalid</pre> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>   | <p>TYGetDeviceXMLSize called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetDeviceXMLSize(hDevice, size);                     ^ is NULL</pre>                                                                                                                                                                                                                                                                                                                                                          |

## 7.1.2.28 TYGetEnabledComponents()

```
TY_CAPI TYGetEnabledComponents (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID * componentIDs)
```

Get all enabled components IDs.

## Parameters

|     |                     |                                  |
|-----|---------------------|----------------------------------|
| in  | <i>hDevice</i>      | Device handle.                   |
| out | <i>componentIDs</i> | Enabled component IDs.(bit flag) |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetEnabledComponents called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetEnabledComponents(hDevice, componentIDs);                     ^ is invalid</pre> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>   | componentIDs is NULL.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

See also

[TY\\_DEVICE\\_COMPONENT\\_LIST](#)

### 7.1.2.29 TYGetEnum()

```
TY_CAPI TYGetEnum (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 uint32_t * value)
```

Get current value of enum feature.

#### Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>value</i>       | Enum value.    |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetEnum called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetEnum(hDevice, componentID, featureID, pValue);</pre> <p>^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpd</li> </ol> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID: d</p> <p>Suggestions:</p> <p>Please check componentID parameter</p> <p>Like this:</p> <pre>TYGetEnum(hDevice, componentID, featureID, pValue);</pre> <p>^ is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs</p> <p>You can also view the components of the camera by obtaining the x</p>                                                                                                                                                                                                    |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID: d</p> <p>Suggestions:</p> <p>Please check featureID parameter</p> <p>Like this:</p> <pre>TYGetEnum(hDevice, componentID, featureID, pValue);</pre> <p>^ is invalid</p> <p>You entered an invalid featureID parameter</p> <p>You can get a list of features of the camera device through TYGetI</p> <p>You can also view the features of the camera device by obtaining t</p>                                                                                                                                                |





## Parameters

|     |                         |                                          |
|-----|-------------------------|------------------------------------------|
| in  | <i>hDevice</i>          | Device handle.                           |
| in  | <i>componentID</i>      | Component ID.                            |
| in  | <i>featureID</i>        | Feature ID.                              |
| out | <i>entries</i>          | Output entries.                          |
| in  | <i>entryCount</i>       | Array size of input parameter "entries". |
| out | <i>filledEntryCount</i> | Number of filled entries.                |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetEnumEntryInfo called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetEnumEntryInfo(hDevice, componentID, featureID, entries, entryCount)</code><br/> <code>^</code> is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpdateDeviceList</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID: d</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYGetEnumEntryInfo(hDevice, componentID, featureID, entries, entryCount)</code><br/> <code>^</code> is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs</p> <p>You can also view the components of the camera by obtaining the xrt::device::component_id_list</p>                                                                                                                                        |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID: d</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetEnumEntryInfo(hDevice, componentID, featureID, entries, entryCount)</code><br/> <code>^</code> is invalid</p> <p>You entered an invalid featureID parameter</p> <p>You can get a list of features of the camera device through TYGetFeatureIDs</p> <p>You can also view the features of the camera device by obtaining the xrt::device::feature_id_list</p>                                                                         |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYGetEnumEntryInfo(hDevice, componentID, featureID, entries, entryCount, featureType)</code><br/> <code>^</code> type mismatch</p> <p>The feature type you entered does not match. You can use TYFeatureType to get the feature type</p>                                                                                                                                                                                                    |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYGetEnumEntryInfo called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetEnumEntryInfo(hDevice, componentID, featureID, pEnumDescriptions, entryCount, featureType)</code><br/> <code>^</code></p>                                                                                                                                                                                                                                                                                               |

### 7.1.2.32 TYGetFeatureInfo()

```

TY_CAPI TYGetFeatureInfo (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 TY_FEATURE_INFO * featureInfo)

```

Get feature info.

#### Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>featureInfo</i> | Feature info.  |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetFeatureInfo called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetFeatureInfo(hDevice, componentID, featureID, pFeatureInfo),</code><br/> <code>^</code> is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYGetFeatureInfo(hDevice, componentID, featureID, pFeatureInfo),</code><br/> <code>^</code> is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                         |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetFeatureInfo(hDevice, componentID, featureID, pFeatureInfo),</code><br/> <code>^</code> is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetF<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                       |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYGetFeatureInfo called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetFeatureInfo(hDevice, componentID, featureID, pFeatureInfo),</code><br/> <code>^</code> is NULL</p>                                                                                                                                                                                                                                                                                             |



## 7.1.2.33 TYGetFloat()

```

TY_CAPI TYGetFloat (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 float * value)

```

Get value of float feature.

## Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>value</i>       | Float value.   |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetFloat called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetFloat(hDevice, componentID, featureID, pValue);</code><br/> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYGetFloat(hDevice, componentID, featureID, pValue);</code><br/> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                   |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetFloat(hDevice, componentID, featureID, pValue);</code><br/> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetF<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                 |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYGetFloat(hDevice, componentID, featureID, pValue);</code><br/> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeatur</p>                                                                                                                                                                                                                       |

## Return values

|                               |                                                                                                                                                                                                            |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_NULL_POINTER</i> | TYGetFloat called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetFloat(hDevice, componentID, featureID, pValue);                 ^ is NULL</pre>               |
| <i>TY_STATUS_TIMEOUT</i>      | Failed to get float feature<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the network is normal                                                    |
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to get float feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not implemented<br>2.Camera device is abnormal and cannot get float feature |

## 7.1.2.34 TYGetFloatRange()

```

TY_CAPI TYGetFloatRange (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 TY_FLOAT_RANGE * floatRange)

```

Get value range of float feature.

## Parameters

|     |                    |                           |
|-----|--------------------|---------------------------|
| in  | <i>hDevice</i>     | Device handle.            |
| in  | <i>componentID</i> | Component ID.             |
| in  | <i>featureID</i>   | Feature ID.               |
| out | <i>floatRange</i>  | Float range to be filled. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <i>TY_STATUS_INVALID_HANDLE</i> | TYGetFloatRange called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYGetFloatRange(hDevice, componentID, featureID, pFloatRange);                 ^ is invalid</pre> The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdateDeviceList |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_COMPONENT</i> | Invalid component ID<br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br><pre>TYGetFloatRange(hDevice, componentID, featureID, pFloatRange);</pre> ^ is invalid<br><br>componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the x                                                 |
| <i>TY_STATUS_INVALID_FEATURE</i>   | Invalid feature ID<br><br>Suggestions:<br>Please check featureID parameter<br>Like this:<br><pre>TYGetFloatRange(hDevice, componentID, featureID, pFloatRange);</pre> ^ is invalid<br><br>You entered an invalid featureID parameter<br>You can get a list of features of the camera device through TYGetF<br>You can also view the features of the camera device by obtaining t |
| <i>TY_STATUS_WRONG_TYPE</i>        | Feature type mismatch<br><br>Suggestions:<br>Please check the feature type<br>Like this:<br><pre>TYGetFloatRange(hDevice, componentID, featureID, pFloatRange);</pre> ^ type mismatch<br><br>The feature type you entered does not match. You can use TYFeatur                                                                                                                   |
| <i>TY_STATUS_NULL_POINTER</i>      | TYGetFloatRange called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetFloatRange(hDevice, componentID, featureID, pFloatRange);</pre> ^ is NULL                                                                                                                                                                                      |

## 7.1.2.35 TYGetFrameBufferSize()

```
TY_CAPI TYGetFrameBufferSize (
 TY_DEV_HANDLE hDevice,
 uint32_t * bufferSize)
```

Get total buffer size of one frame in current configuration.

## Parameters

|     |                   |                        |
|-----|-------------------|------------------------|
| in  | <i>hDevice</i>    | Device handle.         |
| out | <i>bufferSize</i> | Buffer size per frame. |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetFrameBufferSize called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetFrameBufferSize(hDevice, outSize);</code><br/> <code>^</code> is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpdated</p> |
| <i>TY_STATUS_NULL_POINTER</i>   | <p>TYGetFrameBufferSize called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetFrameBufferSize(hDevice, outSize);</code><br/> <code>^</code> is NULL</p>                                                                                                                                                                                                                                                                                                     |
| <i>TY_STATUS_TIMEOUT</i>        | <p>Failed to get frame buffer size</p> <p>Suggestions:<br/>Possible reasons:<br/> 1.Network communication is abnormal, please check whether the network</p>                                                                                                                                                                                                                                                                                                                                                  |
| <i>TY_STATUS_DEVICE_ERROR</i>   | <p>Failed to get frame buffer size</p> <p>Suggestions:<br/>Possible reasons:<br/> 1.Camera device is abnormal and cannot get the frame buffer size.</p>                                                                                                                                                                                                                                                                                                                                                      |

## 7.1.2.36 TYGetInt()

```

TY_CAPI TYGetInt (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 int32_t * value)

```

Get value of integer feature.

## Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>value</i>       | Integer value. |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_INVALID_HANDLE</i></b>    | <b>TYGetInt called with invalid device handle</b><br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYGetInt(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpd |
| <b><i>TY_STATUS_INVALID_COMPONENT</i></b> | <b>Invalid component ID</b><br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br><pre>TYGetInt(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br>componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the xn                                                                                                                                          |
| <b><i>TY_STATUS_INVALID_FEATURE</i></b>   | <b>Invalid feature ID</b><br><br>Suggestions:<br>Please check featureID parameter<br>Like this:<br><pre>TYGetInt(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br>You entered an invalid featureID parameter<br>You can get a list of features of the camera device through TYGetI<br>You can also view the features of the camera device by obtaining t                                                                                           |
| <b><i>TY_STATUS_WRONG_TYPE</i></b>        | <b>Feature type mismatch</b><br><br>Suggestions:<br>Please check the feature type<br>Like this:<br><pre>TYGetInt(hDevice, componentID, featureID, pValue);</pre> ^ type mismatch<br>The feature type you entered does not match. You can use TYFeature                                                                                                                                                                                                            |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | <b>TYGetInt called with NULL pointer</b><br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetInt(hDevice, componentID, featureID, pValue);</pre> ^ is NULL                                                                                                                                                                                                                                                                                   |
| <b><i>TY_STATUS_TIMEOUT</i></b>           | <b>Failed to get int feature</b><br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the n                                                                                                                                                                                                                                                                                                                      |
| <b><i>TY_STATUS_DEVICE_ERROR</i></b>      | <b>Failed to get int feature</b><br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not imple<br>2.Camera device is abnormal and cannot get int feature                                                                                                                                                                                                                                                           |

### 7.1.2.37 TYGetInterfaceList()

```
TY_CAPI TYGetInterfaceList (
 TY_INTERFACE_INFO * pIfaceInfos,
 uint32_t bufferSize,
 uint32_t * filledCount)
```

Get interface info list.

#### Parameters

|     |                    |                                                      |
|-----|--------------------|------------------------------------------------------|
| out | <i>pIfaceInfos</i> | Array of interface infos to be filled.               |
| in  | <i>bufferCount</i> | Array size of interface infos.                       |
| out | <i>filledCount</i> | Number of filled <a href="#">TY_INTERFACE_INFO</a> . |

#### Return values

|                               |                                                                                                                                                                                                                               |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>           | Succeed.                                                                                                                                                                                                                      |
| <i>TY_STATUS_NOT_INITED</i>   | TYInitLib not called.                                                                                                                                                                                                         |
| <i>TY_STATUS_NULL_POINTER</i> | TYGetInterfaceList called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetInterfaceList (pIfaceInfos, bufferSize, filledCount);                     ^           or ^ is NULL</pre> |

### 7.1.2.38 TYGetInterfaceNumber()

```
TY_CAPI TYGetInterfaceNumber (
 uint32_t * pNumIfaces)
```

Get number of current interfaces.

#### Parameters

|     |                   |                       |
|-----|-------------------|-----------------------|
| out | <i>pNumIfaces</i> | Number of interfaces. |
|-----|-------------------|-----------------------|

#### Return values

|                               |                                                                                                                                                                                          |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>           | Succeed.                                                                                                                                                                                 |
| <i>TY_STATUS_NOT_INITED</i>   | TYInitLib not called.                                                                                                                                                                    |
| <i>TY_STATUS_NULL_POINTER</i> | TYGetInterfaceNumber called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetInterfaceNumber (pNumIfaces);                     ^ is NULL</pre> |

## 7.1.2.39 TYGetIntRange()

```

TY_CAPI TYGetIntRange (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 TY_INT_RANGE * intRange)

```

Get value range of integer feature.

## Parameters

|     |                    |                             |
|-----|--------------------|-----------------------------|
| in  | <i>hDevice</i>     | Device handle.              |
| in  | <i>componentID</i> | Component ID.               |
| in  | <i>featureID</i>   | Feature ID.                 |
| out | <i>intRange</i>    | Integer range to be filled. |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetIntRange called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetIntRange(hDevice, componentID, featureID, pIntRange);</code><br/> ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYGetIntRange(hDevice, componentID, featureID, pIntRange);</code><br/> ^ is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs<br/>You can also view the components of the camera by obtaining the x</p>                                                                                                                                                         |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetIntRange(hDevice, componentID, featureID, pIntRange);</code><br/> ^ is invalid</p> <p>You entered an invalid featureID parameter<br/>You can get a list of features of the camera device through TYGet<br/>You can also view the features of the camera device by obtaining t</p>                                                                                                         |

## Return values

|                               |                                                                                                                                                                                                                                                                                                           |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_WRONG_TYPE</i>   | <p>Feature type mismatch</p> <p>Suggestions:</p> <p>Please check the feature type</p> <p>Like this:</p> <pre>TYGetIntRange(hDevice, componentID, featureID, pintRange);</pre> <p style="text-align: right;">^ type mismatch</p> <p>The feature type you entered does not match. You can use TYFeature</p> |
| <i>TY_STATUS_NULL_POINTER</i> | <p>TYGetIntRange called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetIntRange(hDevice, componentID, featureID, pintRange);</pre> <p style="text-align: right;">^ is NULL</p>                                                                       |

## 7.1.2.40 TYGetString()

```

TY_CAPI TYGetString (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 char * buffer,
 uint32_t bufferSize)

```

Get value of string feature.

## Parameters

|     |                    |                 |
|-----|--------------------|-----------------|
| in  | <i>hDevice</i>     | Device handle.  |
| in  | <i>componentID</i> | Component ID.   |
| in  | <i>featureID</i>   | Feature ID.     |
| out | <i>buffer</i>      | String buffer.  |
| in  | <i>bufferSize</i>  | Size of buffer. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetString called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetString(hDevice, componentID, featureID, buffer, bufferSize);</pre> <p style="text-align: right;">^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUp</li> </ol> |



## Return values

|                                          |                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TY_STATUS_INVALID_COMPONENT</code> | <p>Invalid component ID</p> <p>Suggestions:</p> <p>Please check componentID parameter</p> <p>Like this:</p> <pre>TYGetString(hDevice, componentID, featureID, buffer, bufferSize)</pre> <p>^ is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs</p> <p>You can also view the components of the camera by obtaining the x</p>                                                    |
| <code>TY_STATUS_INVALID_FEATURE</code>   | <p>Invalid feature ID</p> <p>Suggestions:</p> <p>Please check featureID parameter</p> <p>Like this:</p> <pre>TYGetString(hDevice, componentID, featureID, buffer, bufferSize)</pre> <p>^ is invalid</p> <p>You entered an invalid featureID parameter</p> <p>You can get a list of features of the camera device through TYGet</p> <p>You can also view the features of the camera device by obtaining t</p> |
| <code>TY_STATUS_WRONG_TYPE</code>        | <p>Feature type mismatch</p> <p>Suggestions:</p> <p>Please check the feature type</p> <p>Like this:</p> <pre>TYGetString(hDevice, componentID, featureID, buffer, bufferSize)</pre> <p>^ type mismatch</p> <p>The feature type you entered does not match. You can use TYFeature</p>                                                                                                                         |
| <code>TY_STATUS_NULL_POINTER</code>      | <p>TYGetString called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetString(hDevice, componentID, featureID, pBuffer, bufferSize)</pre> <p>^ is NULL</p>                                                                                                                                                                                                |
| <code>TY_STATUS_TIMEOUT</code>           | <p>Failed to get string feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Network communication is abnormal, please check whether the n</li> </ol>                                                                                                                                                                                                           |
| <code>TY_STATUS_DEVICE_ERROR</code>      | <p>Failed to get string feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.The feature of the camera device is not available or not impl</li> <li>2.Camera device is abnormal and cannot get string feature</li> </ol>                                                                                                                                        |

## See also

[TYGetStringLength](#)

## 7.1.2.41 TYGetStringLength()

```
TY_CAPI TYGetStringLength (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
```

```
TY_FEATURE_ID featureID,
uint32_t * size)
```

Get internal buffer size of string feature.

## Parameters

|     |                    |                               |
|-----|--------------------|-------------------------------|
| in  | <i>hDevice</i>     | Device handle.                |
| in  | <i>componentID</i> | Component ID.                 |
| in  | <i>featureID</i>   | Feature ID.                   |
| out | <i>size</i>        | String length including '\0'. |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetString called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetString(hDevice, componentID, featureID, buffer, bufferSize);</code><br/> <code>^</code> is invalid</p> <p>The hDevice parameter you input is not recorded<br/>Possible reasons:<br/> 1. TYOpenDevice failed to open device and get correct handle<br/> 2. Memory in stack to store handle data is corrupted<br/> 3. After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYGetString(hDevice, componentID, featureID, buffer, bufferSize);</code><br/> <code>^</code> is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs<br/>You can also view the components of the camera by obtaining the x</p>                                                                                                                                                       |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetString(hDevice, componentID, featureID, buffer, bufferSize);</code><br/> <code>^</code> is invalid</p> <p>You entered an invalid featureID parameter<br/>You can get a list of features of the camera device through TYGetL<br/>You can also view the features of the camera device by obtaining t</p>                                                                                                      |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYGetString(hDevice, componentID, featureID, buffer, bufferSize);</code><br/> <code>^</code> type mismatch</p> <p>The feature type you entered does not match. You can use TYFeat</p>                                                                                                                                                                                                                            |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYGetStringLength called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetStringLength(hDevice, componentID, featureID, pLength);</code><br/> <code>^</code> is NULL</p>                                                                                                                                                                                                                                                                                                |

## See also

[TYGetString](#)

### 7.1.2.42 TYGetStruct()

```

TY_CAPI TYGetStruct (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 void * pStruct,
 uint32_t structSize)

```

Get value of struct.

#### Parameters

|     |                    |                               |
|-----|--------------------|-------------------------------|
| in  | <i>hDevice</i>     | Device handle.                |
| in  | <i>componentID</i> | Component ID.                 |
| in  | <i>featureID</i>   | Feature ID.                   |
| out | <i>pStruct</i>     | Pointer of struct.            |
| in  | <i>structSize</i>  | Size of input buffer pStruct. |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetStruct called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the xn</p>                                                                                                                                                   |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetI<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                  |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <pre>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeature</p>                                                                                                                                                                                                                       |

## Return values

|                               |                                                                                                                                                                                                                             |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_NULL_POINTER</i> | TYGetStruct called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize, ^ is NULL)                                            |
| <i>TY_STATUS_WRONG_SIZE</i>   | Struct size mismatch<br><br>Suggestions:<br>Please check the struct size<br>Like this:<br>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize, ^ is invalid)<br><br>The struct size you entered does not match |
| <i>TY_STATUS_TIMEOUT</i>      | Failed to get struct feature<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the network is normal                                                                    |
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to get struct feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not implemented<br>2.Camera device is abnormal and cannot get struct feature                |

## 7.1.2.43 TYHasDevice()

```

TY_CAPI TYHasDevice (
 TY_INTERFACE_HANDLE ifaceHandle,
 const char * deviceID,
 bool * value)

```

Check whether the interface has the specified device.

## Parameters

|     |                    |                                                                         |
|-----|--------------------|-------------------------------------------------------------------------|
| in  | <i>ifaceHandle</i> | Interface handle.                                                       |
| in  | <i>deviceID</i>    | Device ID string, can be get from <a href="#">TY_DEVICE_BASE_INFO</a> . |
| out | <i>value</i>       | True if the device exists.                                              |

## Return values

|                             |                       |
|-----------------------------|-----------------------|
| <i>TY_STATUS_OK</i>         | Succeed.              |
| <i>TY_STATUS_NOT_INITED</i> | TYInitLib not called. |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_INTERFACE</i> | <p>TYHasDevice called with invalid interface handle</p> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <code>TYHasDevice(ifaceHandle, deviceID, value);</code><br/>                           ^ is invalid</p> <p>The ifaceHandle parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenInterface failed to open interface and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated interface list by calling TYU</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYHasDevice called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYHasDevice(ifaceHandle, deviceID, value);</code><br/>                                                   ^      or      ^ is NULL</p>                                                                                                                                                                                                                                                                                                                                |

## 7.1.2.44 TYHasFeature()

```

TY_CAPI TYHasFeature (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 bool * value)

```

Check whether a component has a specific feature.

## Parameters

|     |                    |                      |
|-----|--------------------|----------------------|
| in  | <i>hDevice</i>     | Device handle.       |
| in  | <i>componentID</i> | Component ID.        |
| in  | <i>featureID</i>   | Feature ID.          |
| out | <i>value</i>       | Whether has feature. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYHasFeature called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYHasFeature(hDevice, componentID, featureID, value);</code><br/>                                                   ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUp</li> </ol> |



### 7.1.2.46 TYLibVersion()

```
TY_CAPI TYLibVersion (
 TY_VERSION_INFO * version)
```

Get current library version.

#### Parameters

|     |                |                                  |
|-----|----------------|----------------------------------|
| out | <i>version</i> | Version infomation to be filled. |
|-----|----------------|----------------------------------|

#### Return values

|                               |                                                                                                                                      |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>           | Succeed.                                                                                                                             |
| <i>TY_STATUS_NULL_POINTER</i> | TYLibVersion called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br>TYLibVersion(ver);<br>^ is NULL |

### 7.1.2.47 TYOpenDevice()

```
TY_CAPI TYOpenDevice (
 TY_INTERFACE_HANDLE ifaceHandle,
 const char * deviceID,
 TY_DEV_HANDLE * outDeviceHandle,
 TY_FW_ERRORCODE * outFwErrorcode = NULL)
```

Open device by device ID.

#### Parameters

|     |                        |                                                                                  |
|-----|------------------------|----------------------------------------------------------------------------------|
| in  | <i>ifaceHandle</i>     | Interface handle.                                                                |
| in  | <i>deviceID</i>        | Device ID string, can be get from <a href="#">TY_DEVICE_BASE_INFO</a> .          |
| out | <i>outDeviceHandle</i> | Handle of opened device. Valid only if TY_STATUS_OK or TY_FW_ERRORCODE returned. |
| out | <i>outFwErrorcode</i>  | Firmware errorcode. Valid only if TY_FW_ERRORCODE returned.                      |

#### Return values

|                             |                       |
|-----------------------------|-----------------------|
| <i>TY_STATUS_OK</i>         | Succeed.              |
| <i>TY_STATUS_NOT_INITED</i> | TYInitLib not called. |



## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_INVALID_INTERFACE</i></b> | <p><b>TYOpenDevice called with invalid interface handle</b></p> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <pre>TYOpenDevice(ifaceHandle, deviceID, outDeviceHandle, outFwErrorcode, ^);</pre> <sup>^</sup> is invalid<br/> The ifaceHandle parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenInterface failed to open interface and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated interface list by calling TYUpdateDeviceList</p>                                                                                                                                                                                                                             |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | <p><b>TYOpenDevice called with NULL pointer</b></p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <pre>TYOpenDevice(ifaceHandle, deviceID, outDeviceHandle, outFwErrorcode, ^);</pre> <sup>^</sup> or <sup>^</sup> is NULL</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b><i>TY_STATUS_INVALID_PARAMETER</i></b> | <p><b>TYOpenDevice called with invalid device ID: s</b></p> <p>Suggestions:<br/>Please check deviceID parameter<br/>Like this:<br/> <pre>TYOpenDevice(ifaceHandle, deviceID, outDeviceHandle, outFwErrorcode, ^);</pre> <sup>^</sup> is invalid<br/> Usually you get device information by calling TYUpdateDeviceList, and then open device by calling TYOpenDevice (When your device online status changes); you may need to update device list again</p>                                                                                                                                                                                                                                                                                                                              |
| <b><i>TY_STATUS_BUSY</i></b>              | <p><b>Failed to open device</b></p> <p>Suggestions:<br/>Possible reasons:<br/> 1.Camera is occupied, please check if other processes on this machine or other host machines are occupying the camera. If the camera is occupied, please wait for a while.<br/> 2.A third-party program is written into the camera, please contact the camera manufacturer.</p>                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b><i>TY_STATUS_FIRMWARE_ERROR</i></b>    | <p><b>Device opened successfully, but firmware error code is not 0</b></p> <p>Suggestions:<br/>Some functions of the device may have exceptions, please check the firmware error code.<br/> <pre>TY_FW_ERRORCODE outFwErrorcode; TYOpenDevice(ifaceHandle, deviceID, outDeviceHandle, &amp;outFwErrorcode); if(outFwErrorcode != 0) {     parse_firmware_errcode(outFwErrorcode); }</pre></p>                                                                                                                                                                                                                                                                                                                                                                                           |
| <b><i>TY_STATUS_DEVICE_ERROR</i></b>      | <p><b>Failed to open device</b></p> <p>Suggestions:<br/>Possible reasons:<br/> 1.A third-party program is written into the camera, please contact the camera manufacturer.<br/> 2.The camera IP address is not in the same network segment as the host.<br/> If the camera IP address is not in the same network segment as the host, you can modify the camera IP address or the host IP address. If there is a routing connection between your host and the camera, you can modify the camera IP address or the host IP address. Otherwise, you can modify the camera IP address or the host IP address. If you need to modify the camera IP address, please refer to the camera manual.<br/> 3.Network communication is abnormal, please check whether the network is connected.</p> |

### 7.1.2.48 TYOpenDeviceWithIP()

```
TY_CAPI TYOpenDeviceWithIP (
 TY_INTERFACE_HANDLE ifaceHandle,
 const char * IP,
 TY_DEV_HANDLE * deviceHandle)
```

Open device by device IP, useful when a device is not listed.

#### Parameters

|     |                     |                          |
|-----|---------------------|--------------------------|
| in  | <i>ifaceHandle</i>  | Interface handle.        |
| in  | <i>IP</i>           | Device IP.               |
| out | <i>deviceHandle</i> | Handle of opened device. |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_NOT_INITED</i>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <i>TY_STATUS_INVALID_INTERFACE</i> | <p>TYOpenDeviceWithIP called with invalid interface handle</p> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <code>TYOpenDeviceWithIP(ifaceHandle, IP, outDeviceHandle);</code><br/> <code>^</code> is invalid</p> <p>The ifaceHandle parameter you input is not recorded</p> <p>Possible reasons:<br/> 1.TYOpenInterface failed to open interface and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated interface list by calling TYUpdateDeviceList</p> |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYOpenDeviceWithIP called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYOpenDeviceWithIP(ifaceHandle, IP, outDeviceHandle);</code><br/> <code>^</code> or <code>^</code> is NULL</p>                                                                                                                                                                                                                                                                                                               |
| <i>TY_STATUS_INVALID_PARAMETER</i> | <p>TYOpenDeviceWithIP called with invalid IP address</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYOpenDeviceWithIP(ifaceHandle, IP, outDeviceHandle);</code><br/> <code>^</code> is invalid</p> <p>A valid IP address should be like:<br/> 192.168.31.1</p> <p>Usually you get device information by calling TYUpdateDeviceList,<br/>and then open device by calling TYOpenDevice</p> <p>When your device online status changes,<br/>you may need to update device list again</p>                                      |
| <i>TY_STATUS_BUSY</i>              | <p>Failed to open device</p> <p>Suggestions:<br/>Possible reasons:<br/> 1.Camera is occupied, please check if other processes on this machine or other host machines are occupying the camera. If the camera is occupied, please wait for a while and then try again.<br/> 2.A third-party program is written into the camera, please contact the camera manufacturer for more information.</p>                                                                                                                                                      |

## Return values

|                                      |                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_DEVICE_ERROR</i></b> | <p>Failed to open device</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.A third-party program is written into the camera, please contact the camera manufacturer.</li> <li>2.The camera IP address cannot communicate with the host IP address.</li> <li>3.Network communication is abnormal, please check whether the network is normal.</li> </ol> |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 7.1.2.49 TYOpenInterface()

```

TY_CAPI TYOpenInterface (
 const char * ifaceID,
 TY_INTERFACE_HANDLE * outHandle)

```

Open specified interface.

## Parameters

|     |                  |                                                                          |
|-----|------------------|--------------------------------------------------------------------------|
| in  | <i>ifaceID</i>   | Interface ID string, can be get from <a href="#">TY_INTERFACE_INFO</a> . |
| out | <i>outHandle</i> | Handle of opened interface.                                              |

## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_OK</i></b>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b><i>TY_STATUS_NOT_INITED</i></b>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | <p>TYOpenInterface called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYOpenInterface(ifaceID, outHandle);                 ^ or ^ is NULL</pre>                                                                                                                                                                                                                                                                 |
| <b><i>TY_STATUS_INVALID_INTERFACE</i></b> | <p>TYOpenInterface called with invalid interface ID</p> <p>Suggestions:</p> <p>Please check ifaceID parameter</p> <p>Like this:</p> <pre>TYOpenInterface(ifaceID, outHandle);                 ^ is invalid</pre> <p>Usually you get interface information by calling <a href="#">TYUpdateInterfaceList</a> and then open interface by calling TYOpenInterface. When your host interface (network or USB) changes; you may need to update interface list again.</p> |

## See also

[TYGetInterfaceList](#)

### 7.1.2.50 TYRegisterEventCallback()

```
TY_CAPI TYRegisterEventCallback (
 TY_DEV_HANDLE hDevice,
 TY_EVENT_CALLBACK callback,
 void * userdata)
```

Register device status callback. Register NULL to clean callback.

#### Parameters

|    |                 |                    |
|----|-----------------|--------------------|
| in | <i>hDevice</i>  | Device handle.     |
| in | <i>callback</i> | Callback function. |
| in | <i>userdata</i> | User private data. |

#### Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYRegisterEventCallback called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYRegisterEventCallback(hDevice, callback, userdata);</pre> <p>^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_BUSY</i>           | Device is capturing.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

### 7.1.2.51 TYRegisterImuCallback()

```
TY_CAPI TYRegisterImuCallback (
 TY_DEV_HANDLE hDevice,
 TY_IMU_CALLBACK callback,
 void * userdata)
```

Register imu callback. Register NULL to clean callback.

#### Parameters

|    |                 |                    |
|----|-----------------|--------------------|
| in | <i>hDevice</i>  | Device handle.     |
| in | <i>callback</i> | Callback function. |
| in | <i>userdata</i> | User private data. |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i> | TYRegisterImuCallback called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYRegisterImuCallback(hDevice, callback, userdata);</pre> ^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdated |
| <i>TY_STATUS_BUSY</i>           | Device is capturing.                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

## 7.1.2.52 TYSendSoftTrigger()

```
TY_CAPI TYSendSoftTrigger (
 TY_DEV_HANDLE hDevice)
```

Send a software trigger to capture a frame when device works in trigger mode.

## Parameters

|    |                |                |
|----|----------------|----------------|
| in | <i>hDevice</i> | Device handle. |
|----|----------------|----------------|

## Return values

|                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>              | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>TY_STATUS_INVALID_HANDLE</i>  | TYSendSoftTrigger called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYSendSoftTrigger(hDevice);</pre> ^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdated |
| <i>TY_STATUS_INVALID_FEATURE</i> | Not support soft trigger.                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <i>TY_STATUS_IDLE</i>            | Camera device is not started<br><br>Suggestions:<br>Please start the camera device first<br>Like this:<br><pre>TYStartCapture(hDevice);</pre> <pre>TYSendSoftTrigger(hDevice);</pre>                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_WRONG_MODE</i>      | Not in trigger mode.                                                                                                                                                                                                                                                                                                                                                                                                                             |

## Return values

|                               |                                                                                                                                                                                                                       |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to send soft trigger<br><br>Suggestions:<br>Possible reasons:<br>1.Camera device is abnormal and cannot send soft trigger.<br>2.Network communication is abnormal, please check whether the network is normal. |
| <i>TY_STATUS_BUSY</i>         | Failed to send soft trigger<br><br>Suggestions:<br>Possible reasons:<br>1.Camera is busy, the last soft trigger is not completed, please try again later.                                                             |

## 7.1.2.53 TYSetBool()

```

TY_CAPI TYSetBool (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 bool value)

```

Set value of bool feature.

## Parameters

|    |                    |                |
|----|--------------------|----------------|
| in | <i>hDevice</i>     | Device handle. |
| in | <i>componentID</i> | Component ID.  |
| in | <i>featureID</i>   | Feature ID.    |
| in | <i>value</i>       | Bool value.    |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_INVALID_HANDLE</i>    | TYSetBool called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br>TYSetBool(hDevice, componentID, featureID, value);<br>^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdateDeviceList. |
| <i>TY_STATUS_INVALID_COMPONENT</i> | Invalid component ID<br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br>TYSetBool(hDevice, componentID, featureID, value);<br>^ is invalid<br>componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the x                                                                                                                                                          |

## Return values

|                                  |                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_FEATURE</i> | Invalid feature ID<br><br>Suggestions:<br>Please check featureID parameter<br>Like this:<br><pre>TYSetBool(hDevice, componentID, featureID, value);</pre> ^ is invalid<br>You entered an invalid featureID parameter<br>You can get a list of features of the camera device through TYGetFeatures<br>You can also view the features of the camera device by obtaining the feature list |
| <i>TY_STATUS_NOT_PERMITTED</i>   | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_WRONG_TYPE</i>      | Feature type mismatch<br><br>Suggestions:<br>Please check the feature type<br>Like this:<br><pre>TYSetBool(hDevice, componentID, featureID, value);</pre> ^ type mismatch<br>The feature type you entered does not match. You can use TYFeatureType to get the feature type                                                                                                            |
| <i>TY_STATUS_BUSY</i>            | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                                                                                            |
| <i>TY_STATUS_TIMEOUT</i>         | Failed to set bool feature<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the network is connected                                                                                                                                                                                                                              |
| <i>TY_STATUS_DEVICE_ERROR</i>    | Failed to set bool feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not implemented<br>2.Camera device is abnormal and cannot set bool feature                                                                                                                                                                               |

## 7.1.2.54 TYSetByteArray()

```

TY_CAPI TYSetByteArray (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 const uint8_t * pBuffer,
 uint32_t bufferSize)

```

Write byte array to device.

## Parameters

|     |                    |                 |
|-----|--------------------|-----------------|
| in  | <i>hDevice</i>     | Device handle.  |
| in  | <i>componentID</i> | Component ID.   |
| in  | <i>featureID</i>   | Feature ID.     |
| out | <i>pBuffer</i>     | Byte buffer.    |
| in  | <i>bufferSize</i>  | Size of buffer. |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYSetByteArray called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpdateDeviceList</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the xrtDeviceList</p>                                                                                                                                                        |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetFeatureIDs<br/> You can also view the features of the camera device by obtaining the xrtDeviceList</p>                                                                                         |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeatureType to get the feature type</p>                                                                                                                                                                                                           |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYSetByteArray called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ is NULL</p>                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_WRONG_SIZE</i>        | <p>Array size mismatch</p> <p>Suggestions:<br/>Please check the array size<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ is incorrect<br/> The array size you entered does not match</p>                                                                                                                                                                                                                                                                     |
| <i>TY_STATUS_BUSY</i>              | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>TY_STATUS_TIMEOUT</i>           | <p>Failed to set byte array feature</p> <p>Suggestions:<br/>Possible reasons:<br/> 1.Network communication is abnormal, please check whether the network is connected</p>                                                                                                                                                                                                                                                                                                                                                   |



## Return values

|                               |                                                                                                                                                                                                                      |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to set byte array feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not implemented<br>2.Camera device is abnormal and cannot set byte array feature |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 7.1.2.55 TYSetEnum()

```

TY_CAPI TYSetEnum (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 uint32_t value)

```

Set value of enum feature.

## Parameters

|    |                    |                |
|----|--------------------|----------------|
| in | <i>hDevice</i>     | Device handle. |
| in | <i>componentID</i> | Component ID.  |
| in | <i>featureID</i>   | Feature ID.    |
| in | <i>value</i>       | Enum value.    |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>TY_STATUS_INVALID_HANDLE</i>    | TYSetEnum called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br>TYSetEnum(hDevice, componentID, featureID, value);<br>^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdateDeviceList |
| <i>TY_STATUS_INVALID_COMPONENT</i> | Invalid component ID<br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br>TYSetEnum(hDevice, componentID, featureID, value);<br>^ is invalid<br>componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the x                                                                                                                                                         |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_FEATURE</i>   | Invalid feature ID<br><br>Suggestions:<br>Please check featureID parameter<br>Like this:<br><pre>TYSetEnum(hDevice, componentID, featureID, value);</pre> ^ is invalid<br><br>You entered an invalid featureID parameter<br>You can get a list of features of the camera device through TYGetEnumEntryInfo<br>You can also view the features of the camera device by obtaining the feature list |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                    |
| <i>TY_STATUS_WRONG_TYPE</i>        | Feature type mismatch<br><br>Suggestions:<br>Please check the feature type<br>Like this:<br><pre>TYSetEnum(hDevice, componentID, featureID, value);</pre> ^ type mismatch<br><br>The feature type you entered does not match. You can use TYFeatureType to get the feature type                                                                                                                 |
| <i>TY_STATUS_INVALID_PARAMETER</i> | Out of range<br><br>Suggestions:<br>Please check the value<br>Like this:<br><pre>TYSetEnum(hDevice, componentID, featureID, value);</pre> ^ is out of range<br><br>The value is out of range, please use TYGetEnumEntryInfo to get the feature value range                                                                                                                                      |
| <i>TY_STATUS_BUSY</i>              | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                                                                                                     |
| <i>TY_STATUS_TIMEOUT</i>           | Failed to set enum feature<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the network is connected                                                                                                                                                                                                                                       |
| <i>TY_STATUS_DEVICE_ERROR</i>      | Failed to set enum feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not implemented<br>2.Camera device is abnormal and cannot set enum feature                                                                                                                                                                                        |

## 7.1.2.56 TYSetFloat()

```

TY_CAPI TYSetFloat (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 float value)

```

Set value of float feature.

## Parameters

|    |                    |                |
|----|--------------------|----------------|
| in | <i>hDevice</i>     | Device handle. |
| in | <i>componentID</i> | Component ID.  |
| in | <i>featureID</i>   | Feature ID.    |
| in | <i>value</i>       | Float value.   |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYSetFloat called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYSetFloat(hDevice, componentID, featureID, value);</code><br/> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUp</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYSetFloat(hDevice, componentID, featureID, value);</code><br/> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the xn</p>                                                                                                                                                 |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYSetFloat(hDevice, componentID, featureID, value);</code><br/> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetf<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYSetFloat(hDevice, componentID, featureID, value);</code><br/> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeatur</p>                                                                                                                                                                                                                      |
| <i>TY_STATUS_OUT_OF_RANGE</i>      | <p>Out of range</p> <p>Suggestions:<br/>Please check the value<br/>Like this:<br/> <code>TYSetFloat(hDevice, componentID, featureID, value);</code><br/> ^ is out of range<br/> The value is out of range, please use TYGetFloatRange to get the r</p>                                                                                                                                                                                                                                   |
| <i>TY_STATUS_BUSY</i>              | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <i>TY_STATUS_TIMEOUT</i>           | <p>Failed to set float feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.Network communication is abnormal, please check whether the ne</p>                                                                                                                                                                                                                                                                                                                                        |
| <i>TY_STATUS_DEVICE_ERROR</i>      | <p>Failed to set float feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.The feature of the camera device is not available or not imple<br/>2.Camera device is abnormal and cannot set float feature</p>                                                                                                                                                                                                                                                                           |

### 7.1.2.57 TYSetInt()

```

TY_CAPI TYSetInt (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 int32_t value)

```

Set value of integer feature.

#### Parameters

|    |                    |                |
|----|--------------------|----------------|
| in | <i>hDevice</i>     | Device handle. |
| in | <i>componentID</i> | Component ID.  |
| in | <i>featureID</i>   | Feature ID.    |
| in | <i>value</i>       | Integer value. |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYSetInt called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYSetInt(hDevice, componentID, featureID, value);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYSetInt(hDevice, componentID, featureID, value);</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                 |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYSetInt(hDevice, componentID, featureID, value);</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetf<br/> You can also view the features of the camera device by obtaining t</p>                                                                                               |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

### Return values

|                               |                                                                                                                                                                                                                                                                                                  |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_WRONG_TYPE</i>   | <p>Feature type mismatch</p> <p>Suggestions:</p> <p>Please check the feature type</p> <p>Like this:</p> <pre>TYSetInt(hDevice, componentID, featureID, value);</pre> <p style="text-align: right;">^ type mismatch</p> <p>The feature type you entered does not match. You can use TYFeature</p> |
| <i>TY_STATUS_OUT_OF_RANGE</i> | <p>Out of range</p> <p>Suggestions:</p> <p>Please check the value</p> <p>Like this:</p> <pre>TYSetInt(hDevice, componentID, featureID, value);</pre> <p style="text-align: right;">^ is out of range</p> <p>The value is out of range, please use TYGetIntRange to get the ran</p>               |
| <i>TY_STATUS_BUSY</i>         | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                      |
| <i>TY_STATUS_TIMEOUT</i>      | <p>Failed to set int feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Network communication is abnormal, please check whether the ne</li> </ol>                                                                                                 |
| <i>TY_STATUS_DEVICE_ERROR</i> | <p>Failed to set int feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.The feature of the camera device is not available or not impl</li> <li>2.Camera device is abnormal and cannot set int feature</li> </ol>                                  |

### 7.1.2.58 TYSetLogLevel()

```
TY_CAPI TYSetLogLevel (
 TY_LOG_TYPE type,
 TY_LOG_LEVEL lvl)
```

Set log level.

## Parameters

|    |             |              |
|----|-------------|--------------|
| in | <i>type</i> | Logger type. |
| in | <i>lvl</i>  | Log level.   |

### Return values

|              |          |
|--------------|----------|
| TY STATUS OK | Succeed. |
|--------------|----------|

### 7.1.2.59 TYSetLogPrefix()

```
TY_CAPI TYSetLogPrefix (
 const char * prefix)
```

set log prefix

#### Parameters

|    |               |                |
|----|---------------|----------------|
| in | <i>prefix</i> | Prefix string. |
|----|---------------|----------------|

#### Return values

|                                    |                                                                                                                                                                                                                                                                                 |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                        |
| <i>TY_STATUS_INVALID_PARAMETER</i> | Prefix is empty or prefix is too long<br><br><div> <div>Suggestions:</div> <div>Prefix is empty or prefix is too long, cannot be set</div> <div>Like this:</div> <div> <pre>TYSetLogPrefix(prefix);                 ^ prefix is empty or prefix is too long</pre> </div> </div> |

### 7.1.2.60 TYSetString()

```
TY_CAPI TYSetString (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 const char * buffer)
```

Set value of string feature.

#### Parameters

|    |                    |                |
|----|--------------------|----------------|
| in | <i>hDevice</i>     | Device handle. |
| in | <i>componentID</i> | Component ID.  |
| in | <i>featureID</i>   | Feature ID.    |
| in | <i>buffer</i>      | String buffer. |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TY_STATUS_INVALID_HANDLE</b>    | <p>TYSetString called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYSetString(hDevice, componentID, featureID, pBuffer);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <b>TY_STATUS_INVALID_COMPONENT</b> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYSetString(hDevice, componentID, featureID, pBuffer);</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                    |
| <b>TY_STATUS_INVALID_FEATURE</b>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYSetString(hDevice, componentID, featureID, pBuffer);</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetF<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                  |
| <b>TY_STATUS_NOT_PERMITTED</b>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>TY_STATUS_WRONG_TYPE</b>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <pre>TYSetString(hDevice, componentID, featureID, pBuffer);</pre> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeature</p>                                                                                                                                                                                                                       |
| <b>TY_STATUS_NULL_POINTER</b>      | <p>TYSetString called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <pre>TYSetString(hDevice, componentID, featureID, pBuffer);</pre> ^ is NULL</p>                                                                                                                                                                                                                                                                                             |
| <b>TY_STATUS_OUT_OF_RANGE</b>      | Input string is too long.                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>TY_STATUS_BUSY</b>              | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>TY_STATUS_TIMEOUT</b>           | <p>Failed to set string feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.Network communication is abnormal, please check whether the n</p>                                                                                                                                                                                                                                                                                                                                      |
| <b>TY_STATUS_DEVICE_ERROR</b>      | <p>Failed to set string feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.The feature of the camera device is not available or not impl<br/>2.Camera device is abnormal and cannot set string feature</p>                                                                                                                                                                                                                                                                        |

### 7.1.2.61 TYSetStruct()

```

TY_CAPI TYSetStruct (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 void * pStruct,
 uint32_t structSize)

```

Set value of struct.

#### Parameters

|    |                    |                    |
|----|--------------------|--------------------|
| in | <i>hDevice</i>     | Device handle.     |
| in | <i>componentID</i> | Component ID.      |
| in | <i>featureID</i>   | Feature ID.        |
| in | <i>pStruct</i>     | Pointer of struct. |
| in | <i>structSize</i>  | Size of struct.    |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYSetStruct called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs</p> <p>You can also view the components of the camera by obtaining the x</p>                                                                                                                                                    |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid</p> <p>You entered an invalid featureID parameter</p> <p>You can get a list of features of the camera device through TYGet</p> <p>You can also view the features of the camera device by obtaining t</p>                                                                                                 |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |



## Return values

|                               |                                                                                                                                                                                                                                                                                 |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_WRONG_TYPE</i>   | <p>Feature type mismatch</p> <p>Suggestions:</p> <p>Please check the feature type</p> <p>Like this:</p> <pre>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize, ^ type mismatch)</pre> <p>The feature type you entered does not match. You can use TYFeature</p> |
| <i>TY_STATUS_NULL_POINTER</i> | <p>TYSetStruct called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize, ^ is NULL)</pre>                                                                         |
| <i>TY_STATUS_WRONG_SIZE</i>   | <p>Struct size mismatch</p> <p>Suggestions:</p> <p>Please check the struct size</p> <p>Like this:</p> <pre>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize, ^ is inval</pre> <p>The struct size you entered does not match</p>                                 |
| <i>TY_STATUS_BUSY</i>         | Device is capturing, the feature is locked.                                                                                                                                                                                                                                     |
| <i>TY_STATUS_TIMEOUT</i>      | <p>Failed to set struct feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <p>1.Network communication is abnormal, please check whether the ne</p>                                                                                                                        |
| <i>TY_STATUS_DEVICE_ERROR</i> | <p>Failed to set struct feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <p>1.The feature of the camera device is not available or not impl</p> <p>2.Camera device is abnormal and cannot set struct feature</p>                                                        |

## 7.1.2.62 TYStartCapture()

```
TY_CAPI TYStartCapture (
 TY_DEV_HANDLE hDevice)
```

Start capture.

## Parameters

|    |                |                |
|----|----------------|----------------|
| in | <i>hDevice</i> | Device handle. |
|----|----------------|----------------|

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYStartCapture called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYStartCapture(hDevice);</pre> <p>^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUp</li> </ol> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>No components are enabled</p> <p>Suggestions:</p> <p>Please enable the components of the camera device first</p> <p>Like this:</p> <pre>TYEnableComponents(hDevice, componentIDs);</pre> <pre>TYStartCapture(hDevice);</pre>                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_BUSY</i>              | <p>Camera device has been started.</p> <p>Suggestions:</p> <p>Please stop the camera device first</p> <p>Like this:</p> <pre>TYStopCapture(hDevice);</pre> <pre>TYStartCapture(hDevice);</pre>                                                                                                                                                                                                                                                                                                                              |
| <i>TY_STATUS_DEVICE_ERROR</i>      | <p>Failed to start camera</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Camera device is abnormal and cannot start the camera.</li> <li>2.Network communication is abnormal, please check whether the n</li> <li>3.Camera is busy, please try again</li> </ol>                                                                                                                                                                                                                  |

## 7.1.2.63 TYStopCapture()

TY\_CAPI TYStopCapture (

[TY\\_DEV\\_HANDLE](#) hDevice )

Stop capture.

## Parameters

|    |                |                |
|----|----------------|----------------|
| in | <i>hDevice</i> | Device handle. |
|----|----------------|----------------|

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYStopCapture called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYStopCapture(hDevice);     ^     is invalid</pre> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_IDLE</i>           | <p>Camera device has been stopped</p> <p>Suggestions:</p> <p>The camera device has stopped, usually after starting</p> <p>Like this:</p> <pre>TYStartCapture(hDevice); TYStopCapture(hDevice);</pre>                                                                                                                                                                                                                                                                                                                            |
| <i>TY_STATUS_DEVICE_ERROR</i>   | <p>Failed to stop camera</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Camera device is abnormal and cannot stop the camera.</li> </ol>                                                                                                                                                                                                                                                                                                                                             |

## 7.1.2.64 TYUpdateAllDeviceList()

```
TY_CAPI TYUpdateAllDeviceList (
 void)
```

Update current connected devices.

## Return values

|                             |                       |
|-----------------------------|-----------------------|
| <i>TY_STATUS_OK</i>         | Succeed.              |
| <i>TY_STATUS_NOT_INITED</i> | TYInitLib not called. |

## 7.1.2.65 TYUpdateDeviceList()

```
TY_CAPI TYUpdateDeviceList (
 TY_INTERFACE_HANDLE ifaceHandle)
```

Update current connected devices.

## Parameters

|    |                    |                   |
|----|--------------------|-------------------|
| in | <i>ifaceHandle</i> | Interface handle. |
|----|--------------------|-------------------|

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <i>TY_STATUS_NOT_INITED</i>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <i>TY_STATUS_INVALID_INTERFACE</i> | TYUpdateDeviceList called with invalid interface handle<br><br>Suggestions:<br>Please check interface handle<br>Like this:<br><pre>TYUpdateDeviceList(ifaceHandle);</pre> ^ is invalid<br>The ifaceHandle parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenInterface failed to open interface and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated interface list by calling TYU |

## 7.1.2.66 TYUpdateInterfaceList()

```
TY_CAPI TYUpdateInterfaceList (
 void)
```

Update current interfaces. call before TYGetInterfaceList.

## Return values

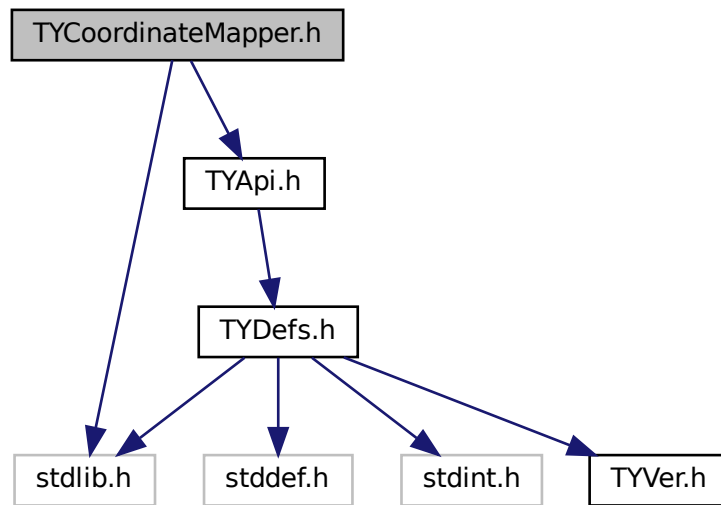
|                             |                       |
|-----------------------------|-----------------------|
| <i>TY_STATUS_OK</i>         | Succeed.              |
| <i>TY_STATUS_NOT_INITED</i> | TYInitLib not called. |

## 7.2 TYCoordinateMapper.h File Reference

Coordinate Conversion API.

```
#include <stdlib.h>
#include "TYApi.h"
```

Include dependency graph for TYCoordinateMapper.h:



## Classes

- struct [TY\\_PIXEL\\_DESC](#)
- struct [TY\\_PIXEL\\_COLOR\\_DESC](#)

## Typedefs

- typedef struct [TY\\_PIXEL\\_DESC](#) [TY\\_PIXEL\\_DESC](#)
- typedef struct [TY\\_PIXEL\\_COLOR\\_DESC](#) [TY\\_PIXEL\\_COLOR\\_DESC](#)

## Functions

- TY\_CAPI [TYInvertExtrinsic](#) (const [TY\\_CAMERA\\_EXTRINSIC](#) \*orgExtrinsic, [TY\\_CAMERA\\_EXTRINSIC](#) \*invExtrinsic)  
*Calculate 4x4 extrinsic matrix's inverse matrix.*
- TY\_CAPI [TYMapDepthToPoint3d](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*src\_calib, uint32\_t depthW, uint32\_t depthH, const [TY\\_PIXEL\\_DESC](#) \*depthPixels, uint32\_t count, [TY\\_VECT\\_3F](#) \*point3d, float f\_scale\_↵ unit=1.0f)  
*Map pixels on depth image to 3D points.*
- TY\_CAPI [TYMapPoint3dToDepth](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*dst\_calib, const [TY\\_VECT\\_3F](#) \*point3d, uint32\_t count, uint32\_t depthW, uint32\_t depthH, [TY\\_PIXEL\\_DESC](#) \*depth, float f\_scale\_↵ unit=1.0f)  
*Map 3D points to pixels on depth image. Reverse operation of TYMapDepthToPoint3d.*
- TY\_CAPI [TYMapDepthImageToPoint3d](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*src\_calib, int32\_t imageW, int32\_t imageH, const uint16\_t \*depth, [TY\\_VECT\\_3F](#) \*point3d, float f\_scale\_unit=1.0f)  
*Map depth image to 3D points. 0 depth pixels maps to (NaN, NaN, NaN).*

- TY\_CAPI [TYMapPoint3dToDepthImage](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*dst\_calib, const [TY\\_VECT\\_3F](#) \*point3d, uint32\_t count, uint32\_t depthW, uint32\_t depthH, uint16\_t \*depth, float f\_target\_scale=1.0f)  
*Map 3D points to depth image. (NAN, NAN, NAN) will be skipped.*
- TY\_CAPI [TYMapPoint3dToPoint3d](#) (const [TY\\_CAMERA\\_EXTRINSIC](#) \*extrinsic, const [TY\\_VECT\\_3F](#) \*point3dFrom, int32\_t count, [TY\\_VECT\\_3F](#) \*point3dTo)  
*Map 3D points to another coordinate.*
- TY\_CAPI [TYMapDepthToColorCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const [TY\\_PIXEL\\_DESC](#) \*depth, uint32\_t count, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t mappedW, uint32\_t mappedH, [TY\\_PIXEL\\_DESC](#) \*mappedDepth, float f\_scale\_unit=1.0f)  
*Map depth pixels to color coordinate pixels.*
- TY\_CAPI [TYMapDepthImageToColorCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t mappedW, uint32\_t mappedH, uint16\_t \*mappedDepth, float f\_scale\_unit=1.0f)  
*Map original depth image to color coordinate depth image.*
- TY\_CAPI [TYMapRGBPixelsToDepthCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t rgbW, uint32\_t rgbH, [TY\\_PIXEL\\_COLOR\\_DESC](#) \*src, uint32\_t cnt, uint32\_t min\_distance, uint32\_t max\_distance, [TY\\_PIXEL\\_COLOR\\_DESC](#) \*dst, float f\_scale\_unit=1.0f)  
*Map original RGB pixels to depth coordinate.*
- TY\_CAPI [TYMapRGBImageToDepthCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t rgbW, uint32\_t rgbH, const uint8\_t \*inRgb, uint8\_t \*mappedRgb, float f\_scale\_unit=1.0f)  
*Map original RGB image to depth coordinate RGB image.*
- TY\_CAPI [TYMapRGB48ImageToDepthCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t rgbW, uint32\_t rgbH, const uint16\_t \*inRgb, uint16\_t \*mappedRgb, float f\_scale\_unit=1.0f)  
*Map original RGB48 image to depth coordinate RGB image.*
- TY\_CAPI [TYMapMono16ImageToDepthCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t rgbW, uint32\_t rgbH, const uint16\_t \*gray, uint16\_t \*mappedGray, float f\_scale\_unit=1.0f)  
*Map original MONO16 image to depth coordinate MONO16 image.*
- TY\_CAPI [TYMapMono8ImageToDepthCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t monoW, uint32\_t monoH, const uint8\_t \*inMono, uint8\_t \*mappedMono, float f\_scale\_unit=1.0f)  
*Map original MONO8 image to depth coordinate MONO8 image.*

## 7.2.1 Detailed Description

Coordinate Conversion API.

### Note

Considering performance, we leave the responsibility of parameters check to users.

### Copyright

Copyright(C)2016-2018 Percipio All Rights Reserved

## 7.2.2 Function Documentation

### 7.2.2.1 TYInvertExtrinsic()

```
TY_CAPI TYInvertExtrinsic (
 const TY_CAMERA_EXTRINSIC * orgExtrinsic,
 TY_CAMERA_EXTRINSIC * invExtrinsic)
```

Calculate 4x4 extrinsic matrix's inverse matrix.

#### Parameters

|     |                     |                         |
|-----|---------------------|-------------------------|
| in  | <i>orgExtrinsic</i> | Input extrinsic matrix. |
| out | <i>invExtrinsic</i> | Inverse matrix.         |

#### Return values

|                        |                     |
|------------------------|---------------------|
| <i>TY_STATUS_OK</i>    | Succeed.            |
| <i>TY_STATUS_ERROR</i> | Calculation failed. |

### 7.2.2.2 TYMapDepthImageToColorCoordinate()

```
TY_CAPI TYMapDepthImageToColorCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,
 uint32_t depthH,
 const uint16_t * depth,
 const TY_CAMERA_CALIB_INFO * color_calib,
 uint32_t mappedW,
 uint32_t mappedH,
 uint16_t * mappedDepth,
 float f_scale_unit = 1.0f)
```

Map original depth image to color coordinate depth image.

#### Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>depth_calib</i> | Depth image's calibration data. |
| in  | <i>depthW</i>      | Width of current depth image.   |
| in  | <i>depthH</i>      | Height of current depth image.  |
| in  | <i>depth</i>       | Depth image.                    |
| in  | <i>color_calib</i> | Color image's calibration data. |
| in  | <i>mappedW</i>     | Width of target depth image.    |
| in  | <i>mappedH</i>     | Height of target depth image.   |
| out | <i>mappedDepth</i> | Output pixels.                  |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 7.2.2.3 TYMapDepthImageToPoint3d()

```
TY_CAPI TYMapDepthImageToPoint3d (
 const TY_CAMERA_CALIB_INFO * src_calib,
 int32_t imageW,
 int32_t imageH,
 const uint16_t * depth,
 TY_VECT_3F * point3d,
 float f_scale_unit = 1.0f)
```

Map depth image to 3D points. 0 depth pixels maps to (NaN, NaN, NaN).

#### Parameters

|     |                  |                                 |
|-----|------------------|---------------------------------|
| in  | <i>src_calib</i> | Depth image's calibration data. |
| in  | <i>depthW</i>    | Width of depth image.           |
| in  | <i>depthH</i>    | Height of depth image.          |
| in  | <i>depth</i>     | Depth image.                    |
| out | <i>point3d</i>   | Output point3D image.           |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 7.2.2.4 TYMapDepthToColorCoordinate()

```
TY_CAPI TYMapDepthToColorCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,
 uint32_t depthH,
 const TY_PIXEL_DESC * depth,
 uint32_t count,
 const TY_CAMERA_CALIB_INFO * color_calib,
 uint32_t mappedW,
 uint32_t mappedH,
 TY_PIXEL_DESC * mappedDepth,
 float f_scale_unit = 1.0f)
```

Map depth pixels to color coordinate pixels.

#### Parameters

|    |                    |                                 |
|----|--------------------|---------------------------------|
| in | <i>depth_calib</i> | Depth image's calibration data. |
| in | <i>depthW</i>      | Width of current depth image.   |
| in | <i>depthH</i>      | Height of current depth image.  |
| in | <i>depth</i>       | Depth image pixels.             |



## Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>count</i>       | Number of depth image pixels.   |
| in  | <i>color_calib</i> | Color image's calibration data. |
| in  | <i>mappedW</i>     | Width of target depth image.    |
| in  | <i>mappedH</i>     | Height of target depth image.   |
| out | <i>mappedDepth</i> | Output pixels.                  |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## 7.2.2.5 TYMapDepthToPoint3d()

```

TY_CAPI TYMapDepthToPoint3d (
 const TY_CAMERA_CALIB_INFO * src_calib,
 uint32_t depthW,
 uint32_t depthH,
 const TY_PIXEL_DESC * depthPixels,
 uint32_t count,
 TY_VECT_3F * point3d,
 float f_scale_unit = 1.0f)

```

Map pixels on depth image to 3D points.

## Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>src_calib</i>   | Depth image's calibration data. |
| in  | <i>depthW</i>      | Width of depth image.           |
| in  | <i>depthH</i>      | Height of depth image.          |
| in  | <i>depthPixels</i> | Pixels on depth image.          |
| in  | <i>count</i>       | Number of depth pixels.         |
| out | <i>point3d</i>     | Output point3D.                 |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## 7.2.2.6 TYMapMono16ImageToDepthCoordinate()

```

TY_CAPI TYMapMono16ImageToDepthCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,

```

```

uint32_t depthH,
const uint16_t * depth,
const TY_CAMERA_CALIB_INFO * color_calib,
uint32_t rgbW,
uint32_t rgbH,
const uint16_t * gray,
uint16_t * mappedGray,
float f_scale_unit = 1.0f)

```

Map original MONO16 image to depth coordinate MONO16 image.

#### Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>depth_calib</i> | Depth image's calibration data. |
| in  | <i>depthW</i>      | Width of current depth image.   |
| in  | <i>depthH</i>      | Height of current depth image.  |
| in  | <i>depth</i>       | Current depth image.            |
| in  | <i>color_calib</i> | Color image's calibration data. |
| in  | <i>rgbW</i>        | Width of MONO16 image.          |
| in  | <i>rgbH</i>        | Height of MONO16 image.         |
| in  | <i>gray</i>        | Current MONO16 image.           |
| out | <i>mappedGray</i>  | Output MONO16 image.            |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 7.2.2.7 TYMapMono8ImageToDepthCoordinate()

```

TY_CAPI TYMapMono8ImageToDepthCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,
 uint32_t depthH,
 const uint16_t * depth,
 const TY_CAMERA_CALIB_INFO * color_calib,
 uint32_t monoW,
 uint32_t monoH,
 const uint8_t * inMono,
 uint8_t * mappedMono,
 float f_scale_unit = 1.0f)

```

Map original MONO8 image to depth coordinate MONO8 image.

#### Parameters

|    |                    |                                 |
|----|--------------------|---------------------------------|
| in | <i>depth_calib</i> | Depth image's calibration data. |
| in | <i>depthW</i>      | Width of current depth image.   |
| in | <i>depthH</i>      | Height of current depth image.  |
| in | <i>depth</i>       | Current depth image.            |
| in | <i>color_calib</i> | Color image's calibration data. |

## Parameters

|     |                   |                        |
|-----|-------------------|------------------------|
| in  | <i>monoW</i>      | Width of MONO8 image.  |
| in  | <i>monoH</i>      | Height of MONO8 image. |
| in  | <i>inMono</i>     | Current MONO8 image.   |
| out | <i>mappedMono</i> | Output MONO8 image.    |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## 7.2.2.8 TYMapPoint3dToDepth()

```

TY_CAPI TYMapPoint3dToDepth (
 const TY_CAMERA_CALIB_INFO * dst_calib,
 const TY_VECT_3F * point3d,
 uint32_t count,
 uint32_t depthW,
 uint32_t depthH,
 TY_PIXEL_DESC * depth,
 float f_scale_unit = 1.0f)

```

Map 3D points to pixels on depth image. Reverse operation of TYMapDepthToPoint3d.

## Parameters

|     |                  |                                        |
|-----|------------------|----------------------------------------|
| in  | <i>dst_calib</i> | Target depth image's calibration data. |
| in  | <i>point3d</i>   | Input 3D points.                       |
| in  | <i>count</i>     | Number of points.                      |
| in  | <i>depthW</i>    | Width of target depth image.           |
| in  | <i>depthH</i>    | Height of target depth image.          |
| out | <i>depth</i>     | Output depth pixels.                   |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## 7.2.2.9 TYMapPoint3dToDepthImage()

```

TY_CAPI TYMapPoint3dToDepthImage (
 const TY_CAMERA_CALIB_INFO * dst_calib,
 const TY_VECT_3F * point3d,
 uint32_t count,
 uint32_t depthW,

```

```
uint32_t depthH,
uint16_t * depth,
float f_target_scale = 1.0f)
```

Map 3D points to depth image. (NAN, NAN, NAN) will be skipped.

#### Parameters

|         |                  |                                        |
|---------|------------------|----------------------------------------|
| in      | <i>dst_calib</i> | Target depth image's calibration data. |
| in      | <i>point3d</i>   | Input 3D points.                       |
| in      | <i>count</i>     | Number of points.                      |
| in      | <i>depthW</i>    | Width of target depth image.           |
| in      | <i>depthH</i>    | Height of target depth image.          |
| in, out | <i>depth</i>     | Depth image buffer.                    |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 7.2.2.10 TYMapPoint3dToPoint3d()

```
TY_CAPI TYMapPoint3dToPoint3d (
 const TY_CAMERA_EXTRINSIC * extrinsic,
 const TY_VECT_3F * point3dFrom,
 int32_t count,
 TY_VECT_3F * point3dTo)
```

Map 3D points to another coordinate.

#### Parameters

|     |                    |                             |
|-----|--------------------|-----------------------------|
| in  | <i>extrinsic</i>   | Extrinsic matrix.           |
| in  | <i>point3dFrom</i> | Source 3D points.           |
| in  | <i>count</i>       | Number of source 3D points. |
| out | <i>point3dTo</i>   | Target 3D points.           |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 7.2.2.11 TYMapRGB48ImageToDepthCoordinate()

```
TY_CAPI TYMapRGB48ImageToDepthCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
```

```

uint32_t depthW,
uint32_t depthH,
const uint16_t * depth,
const TY_CAMERA_CALIB_INFO * color_calib,
uint32_t rgbW,
uint32_t rgbH,
const uint16_t * inRgb,
uint16_t * mappedRgb,
float f_scale_unit = 1.0f)

```

Map original RGB48 image to depth coordinate RGB image.

#### Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>depth_calib</i> | Depth image's calibration data. |
| in  | <i>depthW</i>      | Width of current depth image.   |
| in  | <i>depthH</i>      | Height of current depth image.  |
| in  | <i>depth</i>       | Current depth image.            |
| in  | <i>color_calib</i> | Color image's calibration data. |
| in  | <i>rgbW</i>        | Width of RGB48 image.           |
| in  | <i>rgbH</i>        | Height of RGB48 image.          |
| in  | <i>inRgb</i>       | Current RGB48 image.            |
| out | <i>mappedRgb</i>   | Output RGB48 image.             |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 7.2.2.12 TYMapRGBImageToDepthCoordinate()

```

TY_CAPI TYMapRGBImageToDepthCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,
 uint32_t depthH,
 const uint16_t * depth,
 const TY_CAMERA_CALIB_INFO * color_calib,
 uint32_t rgbW,
 uint32_t rgbH,
 const uint8_t * inRgb,
 uint8_t * mappedRgb,
 float f_scale_unit = 1.0f)

```

Map original RGB image to depth coordinate RGB image.

#### Parameters

|    |                    |                                 |
|----|--------------------|---------------------------------|
| in | <i>depth_calib</i> | Depth image's calibration data. |
| in | <i>depthW</i>      | Width of current depth image.   |
| in | <i>depthH</i>      | Height of current depth image.  |
| in | <i>depth</i>       | Current depth image.            |

## Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>color_calib</i> | Color image's calibration data. |
| in  | <i>rgbW</i>        | Width of RGB image.             |
| in  | <i>rgbH</i>        | Height of RGB image.            |
| in  | <i>inRgb</i>       | Current RGB image.              |
| out | <i>mappedRgb</i>   | Output RGB image.               |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## 7.2.2.13 TYMapRGBPixelsToDepthCoordinate()

```

TY_CAPI TYMapRGBPixelsToDepthCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,
 uint32_t depthH,
 const uint16_t * depth,
 const TY_CAMERA_CALIB_INFO * color_calib,
 uint32_t rgbW,
 uint32_t rgbH,
 TY_PIXEL_COLOR_DESC * src,
 uint32_t cnt,
 uint32_t min_distance,
 uint32_t max_distance,
 TY_PIXEL_COLOR_DESC * dst,
 float f_scale_unit = 1.0f)

```

Map original RGB pixels to depth coordinate.

## Parameters

|     |                     |                                                                                                          |
|-----|---------------------|----------------------------------------------------------------------------------------------------------|
| in  | <i>depth_calib</i>  | Depth image's calibration data.                                                                          |
| in  | <i>depthW</i>       | Width of current depth image.                                                                            |
| in  | <i>depthH</i>       | Height of current depth image.                                                                           |
| in  | <i>depth</i>        | Current depth image.                                                                                     |
| in  | <i>color_calib</i>  | Color image's calibration data.                                                                          |
| in  | <i>rgbW</i>         | Width of RGB image.                                                                                      |
| in  | <i>rgbH</i>         | Height of RGB image.                                                                                     |
| in  | <i>src</i>          | Input RGB pixels info.                                                                                   |
| in  | <i>cnt</i>          | Input src RGB pixels cnt                                                                                 |
| in  | <i>min_distance</i> | The min distance(mm), which is generally set to the minimum measured distance of the current camera      |
| in  | <i>max_distance</i> | The longest distance(mm), which is generally set to the longest measuring distance of the current camera |
| out | <i>dst</i>          | Output RGB pixels info.                                                                                  |

## Return values

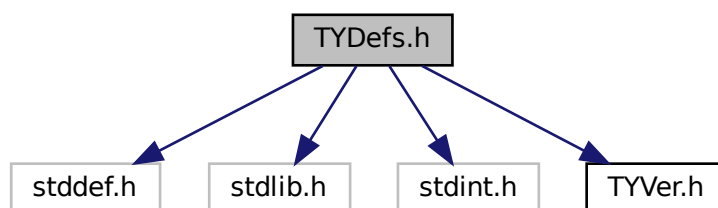
|                           |          |
|---------------------------|----------|
| <code>TY_STATUS_OK</code> | Succeed. |
|---------------------------|----------|

## 7.3 TYDefs.h File Reference

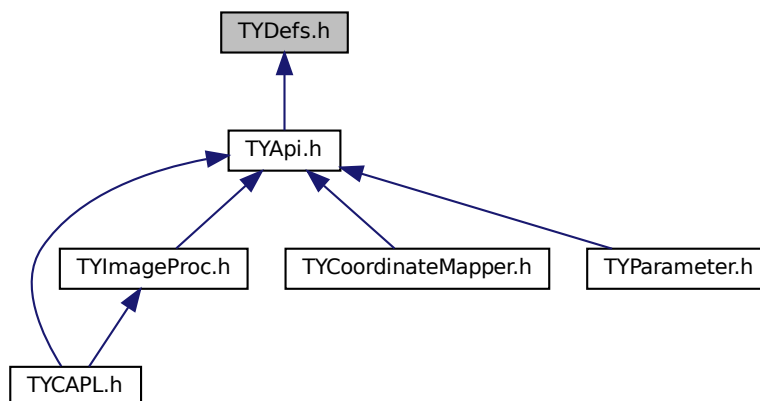
[TYDefs.h](#) includes camera control and data receiving data definitions which supports configuration for image resolution, frame rate, exposure time, gain, working mode, etc.

```
#include <stddef.h>
#include <stdlib.h>
#include <stdint.h>
#include "TYVer.h"
```

Include dependency graph for TYDefs.h:



This graph shows which files directly or indirectly include this file:



## Classes

- struct [TY\\_VERSION\\_INFO](#)
- struct [TY\\_DEVICE\\_NET\\_INFO](#)
  - device network information*
- struct [TY\\_DEVICE\\_USB\\_INFO](#)
- struct [TY\\_INTERFACE\\_INFO](#)
- struct [TY\\_DEVICE\\_BASE\\_INFO](#)
- struct [TY\\_FEATURE\\_INFO](#)
- struct [TY\\_INT\\_RANGE](#)
- struct [TY\\_FLOAT\\_RANGE](#)
  - float range data structure*
- struct [TY\\_BYTEARRAY\\_ATTR](#)
  - byte array data structure*
- struct [TY\\_ENUM\\_ENTRY](#)
- struct [TY\\_VECT\\_3F](#)
- struct [TY\\_CAMERA\\_INTRINSIC](#)
- struct [TY\\_CAMERA\\_EXTRINSIC](#)
- struct [TY\\_CAMERA\\_DISTORTION](#)
- struct [TY\\_CAMERA\\_CALIB\\_INFO](#)
- struct [TY\\_TRIGGER\\_PARAM](#)
- struct [TY\\_TRIGGER\\_PARAM\\_EX](#)
- struct [TY\\_TRIGGER\\_TIMER\\_LIST](#)
- struct [TY\\_TRIGGER\\_TIMER\\_PERIOD](#)
- struct [TY\\_AEC\\_ROI\\_PARAM](#)
- struct [TY\\_PHC\\_GROUP\\_ATTR](#)
- struct [TY\\_PHC\\_GROUP\\_ATTR::phc\\_group\\_attr](#)
- struct [pattern\\_sine\\_param](#)
- struct [pattern\\_gray\\_param](#)
- struct [pattern\\_bin\\_param](#)
- struct [TY\\_LASER\\_PATTERN\\_PARAM](#)
- struct [TY\\_CAMERA\\_STATISTICS](#)
- struct [TY\\_IMU\\_DATA](#)
- struct [TY\\_ACC\\_BIAS](#)
- struct [TY\\_ACC\\_MISALIGNMENT](#)
- struct [TY\\_ACC\\_SCALE](#)
- struct [TY\\_GYRO\\_BIAS](#)
- struct [TY\\_GYRO\\_MISALIGNMENT](#)
- struct [TY\\_GYRO\\_SCALE](#)
- struct [TY\\_CAMERA\\_TO\\_IMU](#)
- struct [TY\\_TOF\\_FREQ](#)
- struct [TY\\_LASER\\_PARAM](#)
- struct [TY\\_IMAGE\\_DATA](#)
- struct [TY\\_FRAME\\_DATA](#)
- struct [TY\\_EVENT\\_INFO](#)
- struct [TY\\_DO\\_WORKMODE](#)
- struct [TY\\_DI\\_WORKMODE](#)
- struct [TY\\_TEMP\\_DATA](#)
- struct [TY\\_CAMERA\\_ROTATION](#)



## Macros

- `#define _STDBOOL_H`
- `#define __bool_true_false_are_defined 1`
- `#define bool _Bool`
- `#define true 1`
- `#define false 0`
- `#define TY_DLLIMPORT __attribute__((visibility("default")))`
- `#define TY_DLLEXPORT __attribute__((visibility("default")))`
- `#define TY_STDC`
- `#define TY_CDEC`
- `#define TY_EXPORT TY_DLLIMPORT`
- `#define TY_EXTC`
- `#define TY_CAPI TY_EXTC TY_EXPORT TY_STATUS TY_STDC`
- `#define TY_INT_SGBM_COST_PARAM TY_INT_SGBM_UNIQUE_MAX_COST`
- `#define TY_BOOL_FLASHLIGHT TY_BOOL_IR_FLASHLIGHT`
- `#define TY_INT_FLASHLIGHT_INTENSITY TY_INT_IR_FLASHLIGHT_INTENSITY`
- `#define TY_INT_AE_TARGET_V TY_INT_AE_TARGET_Y`
- `#define TY_DECLARE_IMAGE_MODE0(pix, res) TY_IMAGE_MODE_##pix##_##res = TY_PIXEL_FOR↵  
MAT_##pix | TY_RESOLUTION_MODE_##res`
- `#define TY_DECLARE_IMAGE_MODE1(pix)`

## Typedefs

- typedef enum `TY_STATUS_LIST` `TY_STATUS_LIST`  
*API call return status.*
- typedef int32\_t `TY_STATUS`
- typedef enum `TY_FW_ERRORCODE_LIST` `TY_FW_ERRORCODE_LIST`
- typedef uint32\_t `TY_FW_ERRORCODE`
- typedef enum `TY_EVENT_LIST` `TY_ENENT_LIST`
- typedef int32\_t `TY_EVENT`
- typedef void \* `TY_INTERFACE_HANDLE`  
*Interface handle.*
- typedef void \* `TY_DEV_HANDLE`  
*Device Handle.*
- typedef enum `TY_DEVICE_COMPONENT_LIST` `TY_DEVICE_COMPONENT_LIST`
- typedef uint32\_t `TY_COMPONENT_ID`  
*component unique id*
- typedef enum `TY_FEATURE_TYPE_LIST` `TY_FEATURE_TYPE_LIST`  
*Feature Format Type definitions.*
- typedef uint32\_t `TY_FEATURE_TYPE`
- typedef enum `TY_FEATURE_ID_LIST` `TY_FEATURE_ID_LIST`  
*feature for component definitions*
- typedef uint32\_t `TY_FEATURE_ID`  
*feature unique id*
- typedef enum `TY_CONFIG_MODE_LIST` `TY_CONFIG_MODE_LIST`
- typedef uint32\_t `TY_CONFIG_MODE`
- typedef enum `TY_DEPTH_QUALITY_LIST` `TY_DEPTH_QUALITY_LIST`
- typedef uint32\_t `TY_DEPTH_QUALITY`
- typedef enum `TY_TRIGGER_POL_LIST` `TY_TRIGGER_POL_LIST`  
*set external trigger signal edge*
- typedef uint32\_t `TY_TRIGGER_POL`
- typedef enum `TY_INTERFACE_TYPE_LIST` `TY_INTERFACE_TYPE_LIST`

- typedef uint32\_t **TY\_INTERFACE\_TYPE**
- typedef enum [TY\\_ACCESS\\_MODE\\_LIST](#) **TY\_ACCESS\_MODE\_LIST**
- typedef uint8\_t **TY\_ACCESS\_MODE**
- typedef enum [TY\\_STREAM\\_ASYNC\\_MODE\\_LIST](#) **TY\_STREAM\_ASYNC\_MODE\_LIST**
  - stream async mode*
- typedef uint8\_t **TY\_STREAM\_ASYNC\_MODE**
- typedef enum TYLensOpticalType **TYLensOpticalType**
- typedef enum [TYPixFmtList](#) **TYPixFmtList**
  - pixel format definitions*
- typedef uint32\_t **TYPixFmt**
- typedef enum [TY\\_PIXEL\\_BITS\\_LIST](#) **TY\_PIXEL\_BITS\_LIST**
- typedef uint32\_t **TY\_PIXEL\_BITS**
- typedef enum [TY\\_PIXEL\\_FORMAT\\_LIST](#) **TY\_PIXEL\_FORMAT\_LIST**
  - pixel format definitions*
- typedef uint32\_t **TY\_PIXEL\_FORMAT**
- typedef enum [TY\\_RESOLUTION\\_MODE\\_LIST](#) **TY\_RESOLUTION\_MODE\_LIST**
  - predefined resolution list*
- typedef int32\_t **TY\_RESOLUTION\_MODE**
- typedef enum [TY\\_IMAGE\\_MODE\\_LIST](#) **TY\_IMAGE\_MODE\_LIST**
  - Predefined Image Mode List image mode controls image resolution & format predefined image modes named like TY\_IMAGE\_MODE\_MONO\_160x120, TY\_IMAGE\_MODE\_RGB\_1280x960.*
- typedef uint32\_t **TY\_IMAGE\_MODE**
- typedef enum [TY\\_TRIGGER\\_MODE\\_LIST](#) **TY\_TRIGGER\_MODE\_LIST**
- typedef int16\_t **TY\_TRIGGER\_MODE**
- typedef enum [TY\\_TIME\\_SYNC\\_TYPE\\_LIST](#) **TY\_TIME\_SYNC\_TYPE\_LIST**
  - type of time sync*
- typedef uint32\_t **TY\_TIME\_SYNC\_TYPE**
- typedef uint32\_t **TY\_E\_VOLT\_T**
- typedef uint32\_t **TY\_E\_DO\_MODE**
- typedef uint32\_t **TY\_E\_DI\_MODE**
- typedef uint32\_t **TY\_E\_DI\_INT\_ACTION**
- typedef uint32\_t **TY\_TEMPERATURE\_ID**
- typedef enum [TY\\_LOG\\_LEVEL\\_LIST](#) **TY\_LOG\_LEVEL\_LIST**
- typedef int32\_t **TY\_LOG\_LEVEL**
- typedef enum [TY\\_LOG\\_TYPE\\_LIST](#) **TY\_LOG\_TYPE\_LIST**
- typedef int32\_t **TY\_LOG\_TYPE**
- typedef enum [TY\\_SERVER\\_TYPE\\_LIST](#) **TY\_SERVER\_TYPE\_LIST**
- typedef int32\_t **TY\_SERVER\_TYPE**
- typedef struct [TY\\_VERSION\\_INFO](#) **TY\_VERSION\_INFO**
- typedef struct [TY\\_DEVICE\\_NET\\_INFO](#) **TY\_DEVICE\_NET\_INFO**
  - device network information*
- typedef struct [TY\\_DEVICE\\_USB\\_INFO](#) **TY\_DEVICE\_USB\_INFO**
- typedef struct [TY\\_INTERFACE\\_INFO](#) **TY\_INTERFACE\_INFO**
- typedef struct [TY\\_DEVICE\\_BASE\\_INFO](#) **TY\_DEVICE\_BASE\_INFO**
- typedef enum [TY\\_VISIBILITY\\_TYPE](#) **TY\_VISIBILITY\_TYPE**
- typedef struct [TY\\_FEATURE\\_INFO](#) **TY\_FEATURE\_INFO**
- typedef struct [TY\\_INT\\_RANGE](#) **TY\_INT\_RANGE**
- typedef struct [TY\\_FLOAT\\_RANGE](#) **TY\_FLOAT\_RANGE**
  - float range data structure*
- typedef struct [TY\\_BYTEARRAY\\_ATTR](#) **TY\_BYTEARRAY\_ATTR**
  - byte array data structure*
- typedef struct [TY\\_ENUM\\_ENTRY](#) **TY\_ENUM\_ENTRY**
- typedef struct [TY\\_VECT\\_3F](#) **TY\_VECT\_3F**

- typedef struct [TY\\_CAMERA\\_INTRINSIC](#) [TY\\_CAMERA\\_INTRINSIC](#)
- typedef struct [TY\\_CAMERA\\_EXTRINSIC](#) [TY\\_CAMERA\\_EXTRINSIC](#)
- typedef struct [TY\\_CAMERA\\_DISTORTION](#) [TY\\_CAMERA\\_DISTORTION](#)
- typedef struct [TY\\_CAMERA\\_CALIB\\_INFO](#) [TY\\_CAMERA\\_CALIB\\_INFO](#)
- typedef struct [TY\\_TRIGGER\\_PARAM](#) [TY\\_TRIGGER\\_PARAM](#)
- typedef struct [TY\\_TRIGGER\\_PARAM\\_EX](#) [TY\\_TRIGGER\\_PARAM\\_EX](#)
- typedef struct [TY\\_TRIGGER\\_TIMER\\_LIST](#) [TY\\_TRIGGER\\_TIMER\\_LIST](#)
- typedef struct [TY\\_TRIGGER\\_TIMER\\_PERIOD](#) [TY\\_TRIGGER\\_TIMER\\_PERIOD](#)
- typedef struct [TY\\_AEC\\_ROI\\_PARAM](#) [TY\\_AEC\\_ROI\\_PARAM](#)
- typedef struct [TY\\_PHC\\_GROUP\\_ATTR](#) [TY\\_PHC\\_GROUP\\_ATTR](#)
- typedef struct [TY\\_LASER\\_PATTERN\\_PARAM](#) [TY\\_LASER\\_PATTERN\\_PARAM](#)
- typedef struct [TY\\_CAMERA\\_STATISTICS](#) [TY\\_CAMERA\\_STATISTICS](#)
- typedef struct [TY\\_IMU\\_DATA](#) [TY\\_IMU\\_DATA](#)
- typedef struct [TY\\_ACC\\_BIAS](#) [TY\\_ACC\\_BIAS](#)
- typedef struct [TY\\_ACC\\_MISALIGNMENT](#) [TY\\_ACC\\_MISALIGNMENT](#)
- typedef struct [TY\\_ACC\\_SCALE](#) [TY\\_ACC\\_SCALE](#)
- typedef struct [TY\\_GYRO\\_BIAS](#) [TY\\_GYRO\\_BIAS](#)
- typedef struct [TY\\_GYRO\\_MISALIGNMENT](#) [TY\\_GYRO\\_MISALIGNMENT](#)
- typedef struct [TY\\_GYRO\\_SCALE](#) [TY\\_GYRO\\_SCALE](#)
- typedef struct [TY\\_CAMERA\\_TO\\_IMU](#) [TY\\_CAMERA\\_TO\\_IMU](#)
- typedef struct [TY\\_TOF\\_FREQ](#) [TY\\_TOF\\_FREQ](#)
- typedef enum [TY\\_IMU\\_FPS\\_LIST](#) [TY\\_IMU\\_FPS\\_LIST](#)
- typedef struct [TY\\_LASER\\_PARAM](#) [TY\\_LASER\\_PARAM](#)
- typedef struct [TY\\_IMAGE\\_DATA](#) [TY\\_IMAGE\\_DATA](#)
- typedef struct [TY\\_FRAME\\_DATA](#) [TY\\_FRAME\\_DATA](#)
- typedef struct [TY\\_EVENT\\_INFO](#) [TY\\_EVENT\\_INFO](#)
- typedef struct [TY\\_DO\\_WORKMODE](#) [TY\\_DO\\_WORKMODE](#)
- typedef struct [TY\\_DI\\_WORKMODE](#) [TY\\_DI\\_WORKMODE](#)
- typedef struct [TY\\_TEMP\\_DATA](#) [TY\\_TEMP\\_DATA](#)
- typedef struct [TY\\_CAMERA\\_ROTATION](#) [TY\\_CAMERA\\_ROTATION](#)

## Enumerations

- enum [TY\\_STATUS\\_LIST](#) : int32\_t {  
[TY\\_STATUS\\_OK](#) = 0, [TY\\_STATUS\\_ERROR](#) = -1001, [TY\\_STATUS\\_NOT\\_INITED](#) = -1002, [TY\\_STATUS\\_↵](#)  
[\\_NOT\\_IMPLEMENTED](#) = -1003,  
[TY\\_STATUS\\_NOT\\_PERMITTED](#) = -1004, [TY\\_STATUS\\_DEVICE\\_ERROR](#) = -1005, [TY\\_STATUS\\_INVA↵](#)  
[LID\\_PARAMETER](#) = -1006, [TY\\_STATUS\\_INVALID\\_HANDLE](#) = -1007,  
[TY\\_STATUS\\_INVALID\\_COMPONENT](#) = -1008, [TY\\_STATUS\\_INVALID\\_FEATURE](#) = -1009, [TY\\_STATU↵](#)  
[S\\_WRONG\\_TYPE](#) = -1010, [TY\\_STATUS\\_WRONG\\_SIZE](#) = -1011,  
[TY\\_STATUS\\_OUT\\_OF\\_MEMORY](#) = -1012, [TY\\_STATUS\\_OUT\\_OF\\_RANGE](#) = -1013, [TY\\_STATUS\\_TIM↵](#)  
[EOUT](#) = -1014, [TY\\_STATUS\\_WRONG\\_MODE](#) = -1015,  
[TY\\_STATUS\\_BUSY](#) = -1016, [TY\\_STATUS\\_IDLE](#) = -1017, [TY\\_STATUS\\_NO\\_DATA](#) = -1018, [TY\\_STATU↵](#)  
[S\\_NO\\_BUFFER](#) = -1019,  
[TY\\_STATUS\\_NULL\\_POINTER](#) = -1020, [TY\\_STATUS\\_READONLY\\_FEATURE](#) = -1021, [TY\\_STATUS\\_I↵](#)  
[NVALID\\_DESCRIPTOR](#) = -1022, [TY\\_STATUS\\_INVALID\\_INTERFACE](#) = -1023,  
[TY\\_STATUS\\_FIRMWARE\\_ERROR](#) = -1024, [TY\\_STATUS\\_ERROR\\_XML](#) = -1025, [TY\\_STATUS\\_DEV\\_E↵](#)  
[PERM](#) = -1, [TY\\_STATUS\\_DEV\\_EIO](#) = -5,  
[TY\\_STATUS\\_DEV\\_ENOMEM](#) = -12, [TY\\_STATUS\\_DEV\\_EBUSY](#) = -16, [TY\\_STATUS\\_DEV\\_EINVAL](#) = -22  
 }

*API call return status.*

- enum **TY\_FW\_ERRORCODE\_LIST** : uint32\_t {  
**TY\_FW\_ERRORCODE\_CAM0\_NOT\_DETECTED** = 0x00000001, **TY\_FW\_ERRORCODE\_CAM1\_NOT\_DETECTED** = 0x00000002, **TY\_FW\_ERRORCODE\_CAM2\_NOT\_DETECTED** = 0x00000004, **TY\_FW\_ERRORCODE\_POE\_NOT\_INIT** = 0x00000008,  
**TY\_FW\_ERRORCODE\_RECMAP\_NOT\_CORRECT** = 0x00000010, **TY\_FW\_ERRORCODE\_LOOKUPTABLE\_NOT\_CORRECT** = 0x00000020, **TY\_FW\_ERRORCODE\_DRV8899\_NOT\_INIT** = 0x00000040, **TY\_FW\_ERRORCODE\_FOC\_START\_ERR** = 0x00000080,  
**TY\_FW\_ERRORCODE\_PHASE\_CALIB\_ERR** = 0x00000100, **TY\_FW\_ERRORCODE\_CONFIG\_NOT\_FOUND** = 0x00010000, **TY\_FW\_ERRORCODE\_CONFIG\_NOT\_CORRECT** = 0x00020000, **TY\_FW\_ERRORCODE\_XML\_NOT\_FOUND** = 0x00040000,  
**TY\_FW\_ERRORCODE\_XML\_NOT\_CORRECT** = 0x00080000, **TY\_FW\_ERRORCODE\_XML\_OVERRIDE\_FAILED** = 0x00100000, **TY\_FW\_ERRORCODE\_CAM\_INIT\_FAILED** = 0x00200000, **TY\_FW\_ERRORCODE\_LASER\_INIT\_FAILED** = 0x00400000 }
- enum **TY\_EVENT\_LIST** : int32\_t { **TY\_EVENT\_DEVICE\_OFFLINE** = -2001, **TY\_EVENT\_LICENSE\_ERROR** = -2002, **TY\_EVENT\_FW\_INIT\_ERROR** = -2003 }
- enum **TY\_DEVICE\_COMPONENT\_LIST** : uint32\_t {  
**TY\_COMPONENT\_DEVICE** = 0x80000000, **TY\_COMPONENT\_DEPTH\_CAM** = 0x00010000, **TY\_COMPONENT\_IR\_CAM\_LEFT** = 0x00040000, **TY\_COMPONENT\_IR\_CAM\_RIGHT** = 0x00080000,  
**TY\_COMPONENT\_RGB\_CAM\_LEFT** = 0x00100000, **TY\_COMPONENT\_RGB\_CAM\_RIGHT** = 0x00200000, **TY\_COMPONENT\_LASER** = 0x00400000, **TY\_COMPONENT\_IMU** = 0x00800000,  
**TY\_COMPONENT\_BRIGHT\_HISTO** = 0x01000000, **TY\_COMPONENT\_STORAGE** = 0x02000000, **TY\_COMPONENT\_RGB\_CAM** = **TY\_COMPONENT\_RGB\_CAM\_LEFT** }
- enum **TY\_FEATURE\_TYPE\_LIST** : uint32\_t {  
**TY\_FEATURE\_INT** = 0x1000, **TY\_FEATURE\_FLOAT** = 0x2000, **TY\_FEATURE\_ENUM** = 0x3000, **TY\_FEATURE\_BOOL** = 0x4000,  
**TY\_FEATURE\_STRING** = 0x5000, **TY\_FEATURE\_BYTEARRAY** = 0x6000, **TY\_FEATURE\_STRUCT** = 0x7000 }

*Feature Format Type definitions.*

- enum **TY\_FEATURE\_ID\_LIST** : uint32\_t {  
**TY\_STRUCT\_CAM\_INTRINSIC** = 0x0000 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_EXTRINSIC\_TO\_DEPTH** = 0x0001 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_EXTRINSIC\_TO\_IR\_LEFT** = 0x0002 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_CAM\_RECTIFIED\_ROTATION** = 0x0003 | **TY\_FEATURE\_STRUCT**,  
**TY\_STRUCT\_CAM\_DISTORTION** = 0x0006 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_CAM\_CALIB\_DATA** = 0x0007 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_CAM\_RECTIFIED\_INTRI** = 0x0008 | **TY\_FEATURE\_STRUCT**, **TY\_BYTEARRAY\_CUSTOM\_BLOCK** = 0x000A | **TY\_FEATURE\_BYTEARRAY**,  
**TY\_BYTEARRAY\_ISP\_BLOCK** = 0x000B | **TY\_FEATURE\_BYTEARRAY**, **TY\_INT\_PERSISTENT\_IP** = 0x0010 | **TY\_FEATURE\_INT**, **TY\_INT\_PERSISTENT\_SUBMASK** = 0x0011 | **TY\_FEATURE\_INT**, **TY\_INT\_PERSISTENT\_GATEWAY** = 0x0012 | **TY\_FEATURE\_INT**,  
**TY\_BOOL\_GVSP\_RESEND** = 0x0013 | **TY\_FEATURE\_BOOL**, **TY\_INT\_PACKET\_DELAY** = 0x0014 | **TY\_FEATURE\_INT**, **TY\_INT\_ACCEPTABLE\_PERCENT** = 0x0015 | **TY\_FEATURE\_INT**, **TY\_INT\_NTP\_SERVER\_IP** = 0x0016 | **TY\_FEATURE\_INT**,  
**TY\_INT\_PACKET\_SIZE** = 0x0017 | **TY\_FEATURE\_INT**, **TY\_INT\_LINK\_CMD\_TIMEOUT** = 0x0018 | **TY\_FEATURE\_INT**, **TY\_STRUCT\_CAM\_STATISTICS** = 0x00ff | **TY\_FEATURE\_STRUCT**, **TY\_INT\_WIDTH\_MAX** = 0x0100 | **TY\_FEATURE\_INT**,  
**TY\_INT\_HEIGHT\_MAX** = 0x0101 | **TY\_FEATURE\_INT**, **TY\_INT\_OFFSET\_X** = 0x0102 | **TY\_FEATURE\_INT**, **TY\_INT\_OFFSET\_Y** = 0x0103 | **TY\_FEATURE\_INT**, **TY\_INT\_WIDTH** = 0x0104 | **TY\_FEATURE\_INT**,  
**TY\_INT\_HEIGHT** = 0x0105 | **TY\_FEATURE\_INT**, **TY\_ENUM\_IMAGE\_MODE** = 0x0109 | **TY\_FEATURE\_ENUM**, **TY\_FLOAT\_SCALE\_UNIT** = 0x010a | **TY\_FEATURE\_FLOAT**, **TY\_ENUM\_TRIGGER\_POL** = 0x0201 | **TY\_FEATURE\_ENUM**,  
**TY\_INT\_FRAME\_PER\_TRIGGER** = 0x0202 | **TY\_FEATURE\_INT**, **TY\_STRUCT\_TRIGGER\_PARAM** = 0x0523 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_TRIGGER\_PARAM\_EX** = 0x0525 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_TRIGGER\_TIMER\_LIST** = 0x0526 | **TY\_FEATURE\_STRUCT**,  
**TY\_STRUCT\_TRIGGER\_TIMER\_PERIOD** = 0x0527 | **TY\_FEATURE\_STRUCT**, **TY\_BOOL\_KEEP\_ALIVE\_ONOFF** = 0x0203 | **TY\_FEATURE\_BOOL**, **TY\_INT\_KEEP\_ALIVE\_TIMEOUT** = 0x0204 | **TY\_FEATURE\_INT**,  
**TY\_BOOL\_CMOS\_SYNC** = 0x0205 | **TY\_FEATURE\_BOOL**, **TY\_INT\_TRIGGER\_DELAY\_US** = 0x0206 | **TY\_FEATURE\_INT**, **TY\_BOOL\_TRIGGER\_OUT\_IO** = 0x0207 | **TY\_FEATURE\_BOOL**, **TY\_INT\_TRIGGER\_DURATION\_US** = 0x0208 | **TY\_FEATURE\_INT**,  
**TY\_ENUM\_STREAM\_ASYNC** = 0x0209 | **TY\_FEATURE\_ENUM**,

```

TY_INT_CAPTURE_TIME_US = 0x0210 | TY_FEATURE_INT, TY_ENUM_TIME_SYNC_TYPE =
0x0211 | TY_FEATURE_ENUM, TY_BOOL_TIME_SYNC_READY = 0x0212 | TY_FEATURE_BOOL,
TY_BOOL_IR_FLASHLIGHT = 0x0213 | TY_FEATURE_BOOL,
TY_INT_IR_FLASHLIGHT_INTENSITY = 0x0214 | TY_FEATURE_INT, TY_STRUCT_DO0_WORKMODE
= 0x0215 | TY_FEATURE_STRUCT, TY_STRUCT_DI0_WORKMODE = 0x0216 | TY_FEATURE_STRUCT,
TY_STRUCT_DO1_WORKMODE = 0x0217 | TY_FEATURE_STRUCT,
TY_STRUCT_DI1_WORKMODE = 0x0218 | TY_FEATURE_STRUCT, TY_STRUCT_DO2_WORKMODE =
0x0219 | TY_FEATURE_STRUCT, TY_STRUCT_DI2_WORKMODE = 0x0220 | TY_FEATURE_STRUCT,
TY_BOOL_RGB_FLASHLIGHT = 0x0221 | TY_FEATURE_BOOL,
TY_INT_RGB_FLASHLIGHT_INTENSITY = 0x0222 | TY_FEATURE_INT, TY_ENUM_CONFIG_MODE =
0x0221 | TY_FEATURE_ENUM, TY_ENUM_TEMPERATURE_ID = 0x0223 | TY_FEATURE_ENUM, TY_↵
STRUCT_TEMPERATURE = 0x0224 | TY_FEATURE_STRUCT,
TY_BOOL_AUTO_EXPOSURE = 0x0300 | TY_FEATURE_BOOL, TY_INT_EXPOSURE_TIME = 0x0301 |
TY_FEATURE_INT, TY_BOOL_AUTO_GAIN = 0x0302 | TY_FEATURE_BOOL, TY_INT_GAIN = 0x0303 |
TY_FEATURE_INT,
TY_BOOL_AUTO_AWB = 0x0304 | TY_FEATURE_BOOL, TY_STRUCT_AEC_ROI = 0x0305 | TY_FEAT↵
URE_STRUCT, TY_INT_TOF_HDR_RATIO = 0x0306 | TY_FEATURE_INT, TY_INT_TOF_JITTER_THRESHOLD
= 0x0307 | TY_FEATURE_INT,
TY_FLOAT_EXPOSURE_TIME_US = 0x0308 | TY_FEATURE_FLOAT, TY_INT_LASER_POWER = 0x0500
| TY_FEATURE_INT, TY_BOOL_LASER_AUTO_CTRL = 0x0501 | TY_FEATURE_BOOL, TY_STRUCT_↵
LASER_PATTERN = 0x0502 | TY_FEATURE_STRUCT,
TY_INT_LASER_CAM_TRIG_POS = 0x0503 | TY_FEATURE_INT, TY_INT_LASER_CAM_TRIG_LEN =
0x0504 | TY_FEATURE_INT, TY_INT_LASER_LUT_TRIG_POS = 0x0505 | TY_FEATURE_INT, TY_INT↵
_LASER_LUT_NUM = 0x0506 | TY_FEATURE_INT,
TY_INT_LASER_PATTERN_OFFSET = 0x0507 | TY_FEATURE_INT, TY_INT_LASER_MIRROR_NUM =
0x0508 | TY_FEATURE_INT, TY_INT_LASER_MIRROR_SEL = 0x0509 | TY_FEATURE_INT, TY_INT_L↵
ASER_LUT_IDX = 0x050a | TY_FEATURE_INT,
TY_INT_LASER_FACET_IDX = 0x050b | TY_FEATURE_INT, TY_INT_LASER_FACET_POS = 0x050c |
TY_FEATURE_INT, TY_INT_LASER_MODE = 0x050d | TY_FEATURE_INT, TY_INT_CONST_DRV_DUTY
= 0x050e | TY_FEATURE_INT,
TY_STRUCT_LASER_ENABLE_BY_IDX = 0x0530 | TY_FEATURE_STRUCT, TY_STRUCT_LASER_POWER_BY_IDX
= 0x0531 | TY_FEATURE_STRUCT, TY_STRUCT_FLOOD_ENABLE_BY_IDX = 0x0532 | TY_FEATURE↵
_STRUCT, TY_STRUCT_FLOOD_POWER_BY_IDX = 0x0533 | TY_FEATURE_STRUCT,
TY_BOOL_UNDISTORTION = 0x0510 | TY_FEATURE_BOOL, TY_BOOL_BRIGHTNESS_HISTOGRAM
= 0x0511 | TY_FEATURE_BOOL, TY_BOOL_DEPTH_POSTPROC = 0x0512 | TY_FEATURE_BOOL,
TY_INT_R_GAIN = 0x0520 | TY_FEATURE_INT,
TY_INT_G_GAIN = 0x0521 | TY_FEATURE_INT, TY_INT_B_GAIN = 0x0522 | TY_FEATURE_INT,
TY_INT_ANALOG_GAIN = 0x0524 | TY_FEATURE_INT, TY_BOOL_HDR = 0x0525 | TY_FEATURE↵
_BOOL,
TY_BYTEARRAY_HDR_PARAMETER = 0x0526 | TY_FEATURE_BYTEARRAY, TY_INT_AE_TARGET_↵
T_Y = 0x0527 | TY_FEATURE_INT, TY_BOOL_IMU_DATA_ONOFF = 0x0600 | TY_FEATURE_BOOL,
TY_STRUCT_IMU_ACC_BIAS = 0x0601 | TY_FEATURE_STRUCT,
TY_STRUCT_IMU_ACC_MISALIGNMENT = 0x0602 | TY_FEATURE_STRUCT, TY_STRUCT_IMU_ACC_SCALE
= 0x0603 | TY_FEATURE_STRUCT, TY_STRUCT_IMU_GYRO_BIAS = 0x0604 | TY_FEATURE_STRUCT,
TY_STRUCT_IMU_GYRO_MISALIGNMENT = 0x0605 | TY_FEATURE_STRUCT,
TY_STRUCT_IMU_GYRO_SCALE = 0x0606 | TY_FEATURE_STRUCT, TY_STRUCT_IMU_CAM_TO_IMU
= 0x0607 | TY_FEATURE_STRUCT, TY_ENUM_IMU_FPS = 0x0608 | TY_FEATURE_ENUM, TY_INT_SGBM_IMAGE_NUM
= 0x0610 | TY_FEATURE_INT,
TY_INT_SGBM_DISPARITY_NUM = 0x0611 | TY_FEATURE_INT, TY_INT_SGBM_DISPARITY_OFFSET
= 0x0612 | TY_FEATURE_INT, TY_INT_SGBM_MATCH_WIN_HEIGHT = 0x0613 | TY_FEATURE_INT,
TY_INT_SGBM_SEMI_PARAM_P1 = 0x0614 | TY_FEATURE_INT,
TY_INT_SGBM_SEMI_PARAM_P2 = 0x0615 | TY_FEATURE_INT, TY_INT_SGBM_UNIQUE_FACTOR
= 0x0616 | TY_FEATURE_INT, TY_INT_SGBM_UNIQUE_ABSDIFF = 0x0617 | TY_FEATURE_INT,
TY_INT_SGBM_UNIQUE_MAX_COST = 0x0618 | TY_FEATURE_INT,
TY_BOOL_SGBM_HFILTER_HALF_WIN = 0x0619 | TY_FEATURE_BOOL, TY_INT_SGBM_MATCH_WIN_WIDTH
= 0x061A | TY_FEATURE_INT, TY_BOOL_SGBM_MEDFILTER = 0x061B | TY_FEATURE_BOOL,
TY_BOOL_SGBM_LRC = 0x061C | TY_FEATURE_BOOL,
TY_INT_SGBM_LRC_DIFF = 0x061D | TY_FEATURE_INT, TY_INT_SGBM_MEDFILTER_THRESH =

```

```

0x061E | TY_FEATURE_INT, TY_INT_SGBM_SEMI_PARAM_P1_SCALE = 0x061F | TY_FEATURE_INT,
TY_INT_SGPM_PHASE_NUM = 0x0620 | TY_FEATURE_INT,
TY_INT_SGPM_NORMAL_PHASE_SCALE = 0x0621 | TY_FEATURE_INT, TY_INT_SGPM_NORMAL_PHASE_OFFSET
= 0x0622 | TY_FEATURE_INT, TY_INT_SGPM_REF_PHASE_SCALE = 0x0623 | TY_FEATURE_INT,
TY_INT_SGPM_REF_PHASE_OFFSET = 0x0624 | TY_FEATURE_INT,
TY_FLOAT_SGPM_EPI_HS = 0x0625 | TY_FEATURE_FLOAT, TY_INT_SGPM_EPI_HF = 0x0626 | TY_F↵
EATURE_INT, TY_BOOL_SGPM_EPI_EN = 0x0627 | TY_FEATURE_BOOL, TY_INT_SGPM_EPI_CH0 =
0x0628 | TY_FEATURE_INT,
TY_INT_SGPM_EPI_CH1 = 0x0629 | TY_FEATURE_INT, TY_INT_SGPM_EPI_THRESH = 0x062A
| TY_FEATURE_INT, TY_BOOL_SGPM_ORDER_FILTER_EN = 0x062B | TY_FEATURE_BOOL,
TY_INT_SGPM_ORDER_FILTER_CHN = 0x062C | TY_FEATURE_INT,
TY_INT_DEPTH_MIN_MM = 0x062D | TY_FEATURE_INT, TY_INT_DEPTH_MAX_MM = 0x062E | TY_FE↵
ATURE_INT, TY_INT_SGBM_TEXTURE_OFFSET = 0x062F | TY_FEATURE_INT, TY_INT_SGBM_TEXTURE_THRESH
= 0x0630 | TY_FEATURE_INT,
TY_STRUCT_PHC_GROUP_ATTR = 0x0710 | TY_FEATURE_STRUCT, TY_ENUM_DEPTH_QUALITY
= 0x0900 | TY_FEATURE_ENUM, TY_INT_FILTER_THRESHOLD = 0x0901 | TY_FEATURE_INT,
TY_INT_TOF_CHANNEL = 0x0902 | TY_FEATURE_INT,
TY_INT_TOF_MODULATION_THRESHOLD = 0x0903 | TY_FEATURE_INT, TY_STRUCT_TOF_FREQ =
0x0904 | TY_FEATURE_STRUCT, TY_BOOL_TOF_ANTI_INTERFERENCE = 0x0905 | TY_FEATURE_↵
BOOL, TY_INT_TOF_ANTI_SUNLIGHT_INDEX = 0x0906 | TY_FEATURE_INT,
TY_INT_MAX_SPECKLE_SIZE = 0x0907 | TY_FEATURE_INT, TY_INT_MAX_SPECKLE_DIFF =
0x0908 | TY_FEATURE_INT, TY_ENUM_LENS_OPTICAL_TYPE = 0x0909 | TY_FEATURE_ENUM,
TY_INT_TRANSMIT_ROI_OFFSET_X = 0x0910 | TY_FEATURE_INT,
TY_INT_TRANSMIT_ROI_OFFSET_Y = 0x0911 | TY_FEATURE_INT, TY_INT_TRANSMIT_ROI_WIDTH
= 0x0912 | TY_FEATURE_INT, TY_INT_TRANSMIT_ROI_HEIGHT = 0x0913 | TY_FEATURE_INT,
TY_BOOL_TRANSMIT_ROI_ENABLE = 0x0914 | TY_FEATURE_BOOL }

```

*feature for component definitions*

- enum **TY\_CONFIG\_MODE\_LIST** : uint32\_t {  
**TY\_CONFIG\_MODE\_PRESET0** = 0, **TY\_CONFIG\_MODE\_PRESET1**, **TY\_CONFIG\_MODE\_PRESET2**,  
**TY\_CONFIG\_MODE\_USERSET0** = (1 << 16),  
**TY\_CONFIG\_MODE\_USERSET1**, **TY\_CONFIG\_MODE\_USERSET2** }
- enum **TY\_DEPTH\_QUALITY\_LIST** : uint32\_t { **TY\_DEPTH\_QUALITY\_BASIC** = 1, **TY\_DEPTH\_QUALIT↵**  
**Y\_MEDIUM** = 2, **TY\_DEPTH\_QUALITY\_HIGH** = 4 }
- enum **TY\_TRIGGER\_POL\_LIST** : uint32\_t { **TY\_TRIGGER\_POL\_FALLINGEDGE** = 0, **TY\_TRIGGER\_P↵**  
**OL\_RISINGEDGE** = 1 }

*set external trigger signal edge*

- enum **TY\_INTERFACE\_TYPE\_LIST** : uint32\_t {  
**TY\_INTERFACE\_UNKNOWN** = 0, **TY\_INTERFACE\_RAW** = 1, **TY\_INTERFACE\_USB** = 2, **TY\_INTERF↵**  
**ACE\_ETHERNET** = 4,  
**TY\_INTERFACE\_IEEE80211** = 8, **TY\_INTERFACE\_ALL** = 0xffff }
- enum **TY\_ACCESS\_MODE\_LIST** : uint32\_t { **TY\_ACCESS\_READABLE** = 0x1, **TY\_ACCESS\_WRITABLE**  
= 0x2 }
- enum **TY\_STREAM\_ASYNC\_MODE\_LIST** : uint32\_t {  
**TY\_STREAM\_ASYNC\_OFF** = 0, **TY\_STREAM\_ASYNC\_DEPTH** = 1, **TY\_STREAM\_ASYNC\_RGB** = 2, **T↵**  
**Y\_STREAM\_ASYNC\_DEPTH\_RGB** = 3,  
**TY\_STREAM\_ASYNC\_ALL** = 0xff }

*stream async mode*

- enum **TYLensOpticalType** : uint32\_t { **TY\_LENS\_PINHOLE** = 0, **TY\_LENS\_FISHEYE** = 1 }
- enum **TYPixFmtList** : uint32\_t {  
**TYPixelFormatMono8** = 0x01080001, **TYPixelFormatBayerGBRG8** = 0x0108000A, **TYPixelFormat↵**  
**BayerBGGR8** = 0x0108000B, **TYPixelFormatBayerGRBG8** = 0x01080008,  
**TYPixelFormatBayerRGGB8** = 0x01080009, **TYPixelFormatPacketMono10** = 0x010A0046, **TYPixel↵**  
**FormatPacketBayerGBRG10** = 0x010A0054, **TYPixelFormatPacketBayerBGGR10** = 0x010A0052,  
**TYPixelFormatPacketBayerGRBG10** = 0x010A0056, **TYPixelFormatPacketBayerRGGB10** = 0x010↵  
A0058, **TYPixelFormatPacketMono12** = 0x010C0047, **TYPixelFormatPacketBayerGBRG12** = 0x010↵  
C0055,



```

TYPixelFormatPacketBayerBGGR12 = 0x010C0053, TYPixelFormatPacketBayerGRBG12 = 0x010C0057, TYPixelFormatPacketBayerRGGB12 = 0x010C0059, TYPixelFormatMono10 = 0x810A0046,
TYPixelFormatBayerGBRG10 = 0x810A0054, TYPixelFormatBayerBGGR10 = 0x810A0052, TYPixelFormatBayerGRBG10 = 0x810A0056, TYPixelFormatBayerRGGB10 = 0x810A0058,
TYPixelFormatMono12 = 0x810C0047, TYPixelFormatBayerGBRG12 = 0x810C0055, TYPixelFormatBayerBGGR12 = 0x810C0053, TYPixelFormatBayerGRBG12 = 0x810C0057,
TYPixelFormatBayerRGGB12 = 0x810C0059, TYPixelFormatMono14 = 0x810E0104, TYPixelFormatBayerGBRG14 = 0x810E0107, TYPixelFormatBayerBGGR14 = 0x810E0108,
TYPixelFormatBayerGRBG14 = 0x810E0105, TYPixelFormatBayerRGGB14 = 0x810E0106, TYPixelFormatMono16 = 0x01100007, TYPixelFormatBayerGBRG16 = 0x01100030,
TYPixelFormatBayerBGGR16 = 0x01100031, TYPixelFormatBayerGRBG16 = 0x0110002E, TYPixelFormatBayerRGGB16 = 0x0110002F, TYPixelFormatRGB8 = 0x02180014,
TYPixelFormatBGR8 = 0x02180015, TYPixelFormatYUV422_8 = 0x02100032, TYPixelFormatYUV422_8_UYVY = 0x0210001F, TYPixelFormatYCbCr420_8_YY_CbCr_Planar = 0x020C0113,
TYPixelFormatYCbCr420_8_YY_CrCb_Planar = 0x020C0115, TYPixelFormatYCbCr420_8_YY_CbCr_Semiplanar = 0x020C0112, TYPixelFormatYCbCr420_8_YY_CrCb_Semiplanar = 0x020C0114, TYPixelFormatCoord3D_C16 = 0x011000B8,
TYPixelFormatCoord3D_ABC16 = 0x023000B9, TYPixelFormatCoord3D_ABC32f = 0x026000C0, TYPixelFormatJPEG = 0x82180015, TYPixelFormatToIRFourGroupMono16 = 0x81400016,
TYPixelFormatInvalid = 0xFFFFFFFF }

```

*pixel format definitions*

- enum **TY\_PIXEL\_BITS\_LIST** : uint32\_t {  
**TY\_PIXEL\_8BIT** = 0x1 << 28, **TY\_PIXEL\_16BIT** = 0x2 << 28, **TY\_PIXEL\_24BIT** = 0x3 << 28, **TY\_PIXEL\_32BIT** = 0x4 << 28,  
**TY\_PIXEL\_10BIT** = 0x5 << 28, **TY\_PIXEL\_12BIT** = 0x6 << 28, **TY\_PIXEL\_14BIT** = 0x7 << 28, **TY\_PIXEL\_16BIT** = 0x8 << 28,  
**TY\_PIXEL\_48BIT** = (uint32\_t)0x8 << 28,  
**TY\_PIXEL\_64BIT** = (uint32\_t)0xa << 28 }
- enum **TY\_PIXEL\_FORMAT\_LIST** : uint32\_t {  
**TY\_PIXEL\_FORMAT\_UNDEFINED** = 0, **TY\_PIXEL\_FORMAT\_MONO** = (TY\_PIXEL\_8BIT | (0x0 << 24)),  
**TY\_PIXEL\_FORMAT\_BAYER8GB** = (TY\_PIXEL\_8BIT | (0x1 << 24)), **TY\_PIXEL\_FORMAT\_BAYER8BG** = (TY\_PIXEL\_8BIT | (0x2 << 24)),  
**TY\_PIXEL\_FORMAT\_BAYER8GR** = (TY\_PIXEL\_8BIT | (0x3 << 24)), **TY\_PIXEL\_FORMAT\_BAYER8RG** = (TY\_PIXEL\_8BIT | (0x4 << 24)), **TY\_PIXEL\_FORMAT\_BAYER8GRBG** = TY\_PIXEL\_FORMAT\_BAYER8GB,  
**TY\_PIXEL\_FORMAT\_BAYER8RGG** = TY\_PIXEL\_FORMAT\_BAYER8BG,  
**TY\_PIXEL\_FORMAT\_BAYER8GBRG** = TY\_PIXEL\_FORMAT\_BAYER8GR, **TY\_PIXEL\_FORMAT\_BAYER8BGR** = TY\_PIXEL\_FORMAT\_BAYER8RG,  
**TY\_PIXEL\_FORMAT\_CSI\_MONO10** = (TY\_PIXEL\_10BIT | (0x0 << 24)), **TY\_PIXEL\_FORMAT\_CSI\_BAYER10GRBG** = (TY\_PIXEL\_10BIT | (0x1 << 24)),  
**TY\_PIXEL\_FORMAT\_CSI\_BAYER10RGG** = (TY\_PIXEL\_10BIT | (0x2 << 24)), **TY\_PIXEL\_FORMAT\_CSI\_BAYER10BGR** = (TY\_PIXEL\_10BIT | (0x3 << 24)),  
**TY\_PIXEL\_FORMAT\_CSI\_MONO12** = (TY\_PIXEL\_12BIT | (0x0 << 24)),  
**TY\_PIXEL\_FORMAT\_CSI\_BAYER12GRBG** = (TY\_PIXEL\_12BIT | (0x1 << 24)), **TY\_PIXEL\_FORMAT\_CSI\_BAYER12RGG** = (TY\_PIXEL\_12BIT | (0x2 << 24)),  
**TY\_PIXEL\_FORMAT\_CSI\_BAYER12BGR** = (TY\_PIXEL\_12BIT | (0x3 << 24)), **TY\_PIXEL\_FORMAT\_CSI\_BAYER12BGR** = (TY\_PIXEL\_12BIT | (0x4 << 24)),  
**TY\_PIXEL\_FORMAT\_DEPTH16** = (TY\_PIXEL\_16BIT | (0x0 << 24)), **TY\_PIXEL\_FORMAT\_YVYU** = (TY\_PIXEL\_16BIT | (0x1 << 24)), **TY\_PIXEL\_FORMAT\_YUYV** = (TY\_PIXEL\_16BIT | (0x2 << 24)),  
**TY\_PIXEL\_FORMAT\_MONO16** = (TY\_PIXEL\_16BIT | (0x3 << 24)),  
**TY\_PIXEL\_FORMAT\_TOF\_IR\_MONO16** = (TY\_PIXEL\_64BIT | (0x4 << 24)), **TY\_PIXEL\_FORMAT\_RGB** = (TY\_PIXEL\_24BIT | (0x0 << 24)), **TY\_PIXEL\_FORMAT\_BGR** = (TY\_PIXEL\_24BIT | (0x1 << 24)),  
**TY\_PIXEL\_FORMAT\_JPEG** = (TY\_PIXEL\_24BIT | (0x2 << 24)),  
**TY\_PIXEL\_FORMAT\_MJPEG** = (TY\_PIXEL\_24BIT | (0x3 << 24)), **TY\_PIXEL\_FORMAT\_RGB48** = (TY\_PIXEL\_48BIT | (0x0 << 24)), **TY\_PIXEL\_FORMAT\_BGR48** = (TY\_PIXEL\_48BIT | (0x1 << 24)),  
**TY\_PIXEL\_FORMAT\_XYZ48** = (TY\_PIXEL\_48BIT | (0x2 << 24)) }

*pixel format definitions*

- enum **TY\_RESOLUTION\_MODE\_LIST** : uint32\_t {  
**TY\_RESOLUTION\_MODE\_160x100** = (160 << 12) + 100, **TY\_RESOLUTION\_MODE\_160x120** = (160 << 12) + 120,  
**TY\_RESOLUTION\_MODE\_240x320** = (240 << 12) + 320, **TY\_RESOLUTION\_MODE\_320x180** = (320 << 12) + 180,  
**TY\_RESOLUTION\_MODE\_320x200** = (320 << 12) + 200, **TY\_RESOLUTION\_MODE\_320x240** = (320 << 12) + 240,  
**TY\_RESOLUTION\_MODE\_480x640** = (480 << 12) + 640, **TY\_RESOLUTION\_MODE\_640x360** = (640 << 12) + 360,

```

TY_RESOLUTION_MODE_640x400 = (640<<12)+400, TY_RESOLUTION_MODE_640x480 = (640<<12)+480,
TY_RESOLUTION_MODE_960x1280 = (960<<12)+1280, TY_RESOLUTION_MODE_1280x720 =
(1280<<12)+720,
TY_RESOLUTION_MODE_1280x800 = (1280<<12)+800, TY_RESOLUTION_MODE_1280x960 =
(1280<<12)+960, TY_RESOLUTION_MODE_1600x1200 = (1600<<12)+1200, TY_RESOLUTION_MODE_800x600
= (800<<12)+600,
TY_RESOLUTION_MODE_1920x1080 = (1920<<12)+1080, TY_RESOLUTION_MODE_2560x1920 =
(2560<<12)+1920, TY_RESOLUTION_MODE_2592x1944 = (2592<<12)+1944, TY_RESOLUTION_MODE_1920x1440
= (1920<<12)+1440,
TY_RESOLUTION_MODE_240x96 = (240<<12)+96, TY_RESOLUTION_MODE_2048x1536 = (2048<<12)+1536
}

```

*predefined resolution list*

- enum **TY\_IMAGE\_MODE\_LIST** : uint32\_t {  
**TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_↵**  
**IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO),  
**TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_↵**  
**IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO),  
**TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_↵**  
**IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO),  
**TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_↵**  
**IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO),  
**TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_↵**  
**IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO),  
**TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_↵**  
**IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO),  
**TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_↵**  
**IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO),  
**TY\_DECLARE\_IMAGE\_MODE1**=(MONO), **TY\_DECLARE\_IMAGE\_MODE1**=(MONO) }

*Predefined Image Mode List image mode controls image resolution & format predefined image modes named like  
TY\_IMAGE\_MODE\_MONO\_160x120, TY\_IMAGE\_MODE\_RGB\_1280x960.*

- enum **TY\_TRIGGER\_MODE\_LIST** : uint32\_t {  
**TY\_TRIGGER\_MODE\_OFF** = 0, **TY\_TRIGGER\_MODE\_SLAVE** = 1, **TY\_TRIGGER\_MODE\_M\_SIG** = 2,  
**TY\_TRIGGER\_MODE\_M\_PER** = 3,  
**TY\_TRIGGER\_MODE\_SIG\_PASS** = 18, **TY\_TRIGGER\_MODE\_PER\_PASS** = 19, **TY\_TRIGGER\_MODE\_↵**  
**\_TIMER\_LIST** = 20, **TY\_TRIGGER\_MODE\_TIMER\_PERIOD** = 21,  
**TY\_TRIGGER\_MODE28** = 28, **TY\_TRIGGER\_MODE29** = 29, **TY\_TRIGGER\_MODE\_PER\_PASS2** = 30,  
**TY\_TRIGGER\_WORK\_MODE31** = 31,  
**TY\_TRIGGER\_MODE\_SIG\_LASER** = 34 }
- enum **TY\_TIME\_SYNC\_TYPE\_LIST** : uint32\_t {  
**TY\_TIME\_SYNC\_TYPE\_NONE** = 0, **TY\_TIME\_SYNC\_TYPE\_HOST** = 1, **TY\_TIME\_SYNC\_TYPE\_NTP** = 2,  
**TY\_TIME\_SYNC\_TYPE\_PTP** = 3,  
**TY\_TIME\_SYNC\_TYPE\_CAN** = 4, **TY\_TIME\_SYNC\_TYPE\_PTP\_MASTER** = 5 }

*type of time sync*

- enum **TY\_LOG\_LEVEL\_LIST** {  
**TY\_LOG\_LEVEL\_VERBOSE** = 1, **TY\_LOG\_LEVEL\_DEBUG** = 2, **TY\_LOG\_LEVEL\_INFO** = 3, **TY\_LOG\_↵**  
**\_LEVEL\_WARNING** = 4,  
**TY\_LOG\_LEVEL\_ERROR** = 5, **TY\_LOG\_LEVEL\_NEVER** = 9 }
- enum **TY\_LOG\_TYPE\_LIST** { **TY\_LOG\_TYPE\_STD**, **TY\_LOG\_TYPE\_FILE**, **TY\_LOG\_TYPE\_SERVER** }
- enum **TY\_SERVER\_TYPE\_LIST** { **TY\_SERVER\_TYPE\_UDP**, **TY\_SERVER\_TYPE\_TCP** }
- enum **TY\_VISIBILITY\_TYPE** { **BEGINNER** = 0, **EXPERT** = 1, **GURU** = 2, **INVLISIBLE** = 3 }
- enum { **TY\_PATTERN\_SINE\_TYPE** = 0, **TY\_PATTERN\_GRAY\_TYPE**, **TY\_PATTERN\_BIN\_TYPE**, **TY\_↵**  
**PATTERN\_EMPTY\_TYPE** = 0xffffffff }
- enum { **TY\_NORMAL\_PHASE\_TYPE** = 0, **TY\_REFER\_PHASE\_TYPE** }
- enum **TY\_IMU\_FPS\_LIST** { **TY\_IMU\_FPS\_100HZ** = 0, **TY\_IMU\_FPS\_200HZ**, **TY\_IMU\_FPS\_400HZ** }

## Variables

- typedef enum



- typedef **TY\_DO\_5V** = 1
- typedef **TY\_DO\_12V** = 2
- typedef **TY\_E\_VOLT\_T\_LIST**
- typedef **TY\_DO\_HIGH** = 1
- typedef **TY\_DO\_PWM** = 2
- typedef **TY\_DO\_CAM\_TRIG** = 3
- typedef **TY\_E\_DO\_MODE\_LIST**
- typedef **TY\_DI\_NE\_INT** = 1
- typedef **TY\_DI\_PE\_INT** = 2
- typedef **TY\_E\_DI\_MODE\_LIST**
- typedef **TY\_DI\_INT\_TRIG\_CAP** = 1
- typedef **TY\_DI\_INT\_EVENT** = 2
- typedef **TY\_E\_DI\_INT\_ACTION\_LIST**
- typedef **TY\_TEMPERATURE\_RIGHT** = 1
- typedef **TY\_TEMPERATURE\_COLOR** = 2
- typedef **TY\_TEMPERATURE\_CPU** = 3
- typedef **TY\_TEMPERATURE\_MAIN\_BOARD** = 4
- typedef **TY\_TEMPERATURE\_ID\_LIST**

### 7.3.1 Detailed Description

[TYDefs.h](#) includes camera control and data receiving data definitions which supports configuration for image resolution, frame rate, exposure time, gain, working mode, etc.

### 7.3.2 Macro Definition Documentation

#### 7.3.2.1 TY\_DECLARE\_IMAGE\_MODE1

```
#define TY_DECLARE_IMAGE_MODE1(
 pix)
```

**Value:**

```
TY_DECLARE_IMAGE_MODE0(pix, 160x100), \
TY_DECLARE_IMAGE_MODE0(pix, 160x120), \
TY_DECLARE_IMAGE_MODE0(pix, 320x180), \
TY_DECLARE_IMAGE_MODE0(pix, 320x200), \
TY_DECLARE_IMAGE_MODE0(pix, 320x240), \
TY_DECLARE_IMAGE_MODE0(pix, 480x640), \
TY_DECLARE_IMAGE_MODE0(pix, 640x360), \
TY_DECLARE_IMAGE_MODE0(pix, 640x400), \
TY_DECLARE_IMAGE_MODE0(pix, 640x480), \
TY_DECLARE_IMAGE_MODE0(pix, 960x1280), \
TY_DECLARE_IMAGE_MODE0(pix, 1280x720), \
TY_DECLARE_IMAGE_MODE0(pix, 1280x960), \
TY_DECLARE_IMAGE_MODE0(pix, 1280x800), \
TY_DECLARE_IMAGE_MODE0(pix, 1600x1200), \
TY_DECLARE_IMAGE_MODE0(pix, 800x600), \
TY_DECLARE_IMAGE_MODE0(pix, 1920x1080), \
TY_DECLARE_IMAGE_MODE0(pix, 2560x1920), \
TY_DECLARE_IMAGE_MODE0(pix, 2592x1944), \
TY_DECLARE_IMAGE_MODE0(pix, 1920x1440), \
TY_DECLARE_IMAGE_MODE0(pix, 2048x1536), \
TY_DECLARE_IMAGE_MODE0(pix, 240x96)
```

Definition at line 627 of file TYDefs.h.

### 7.3.3 Typedef Documentation

#### 7.3.3.1 TY\_ACC\_BIAS

```
typedef struct TY_ACC_BIAS TY_ACC_BIAS
```

a 3x3 matrix

| .     | .     | .     |
|-------|-------|-------|
| BIASx | BIASy | BIASz |

#### 7.3.3.2 TY\_ACC\_MISALIGNMENT

```
typedef struct TY_ACC_MISALIGNMENT TY_ACC_MISALIGNMENT
```

a 3x3 matrix

|.|.|

| .      | .      | .      |
|--------|--------|--------|
| 1      | -GAMAz | GAMAz  |
| GAMAxz | 1      | -GAMAx |
| -GAMAx | GAMAx  | 1      |

#### 7.3.3.3 TY\_ACC\_SCALE

```
typedef struct TY_ACC_SCALE TY_ACC_SCALE
```

a 3x3 matrix

| .      | .      | .      |
|--------|--------|--------|
| SCALEx | 0      | 0      |
| 0      | SCALEy | 0      |
| 0      | 0      | SCALEz |

#### 7.3.3.4 TY\_ACCESS\_MODE\_LIST

```
typedef enum TY_ACCESS_MODE_LIST TY_ACCESS_MODE_LIST
```

Indicate a feature is readable or writable

See also

[TYGetFeatureInfo](#)

### 7.3.3.5 TY\_BYTEARRAY\_ATTR

```
typedef struct TY_BYTEARRAY_ATTR TY_BYTEARRAY_ATTR
```

byte array data structure

See also

[TYGetByteArray](#)

### 7.3.3.6 TY\_CAMERA\_CALIB\_INFO

```
typedef struct TY_CAMERA_CALIB_INFO TY_CAMERA_CALIB_INFO
```

camera 's caillbration data

See also

[TYGetStruct](#)

### 7.3.3.7 TY\_CAMERA\_DISTORTION

```
typedef struct TY_CAMERA_DISTORTION TY_CAMERA_DISTORTION
```

camera distortion parameters

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_DISTORTION distortion;
TYGetStruct(hDevice, some_component, TY_STRUCT_CAM_DISTORTION, &distortion, sizeof(distortion));
```

### 7.3.3.8 TY\_CAMERA\_EXTRINSIC

```
typedef struct TY_CAMERA_EXTRINSIC TY_CAMERA_EXTRINSIC
```

a 4x4 matrix

| .   | .   | .   | .  |
|-----|-----|-----|----|
| r11 | r12 | r13 | t1 |
| r21 | r22 | r23 | t2 |
| r31 | r32 | r33 | t3 |
| 0   | 0   | 0   | 1  |

See also

**TYGetStruct** Usage:

```
TY_CAMERA_EXTRINSIC extrinsic;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_EXTRINSIC, &extrinsic, sizeof(extrinsic));
```

### 7.3.3.9 TY\_CAMERA\_INTRINSIC

```
typedef struct TY_CAMERA_INTRINSIC TY_CAMERA_INTRINSIC
```

a 3x3 matrix

| .  | .  | .  |
|----|----|----|
| fx | 0  | cx |
| 0  | fy | cy |
| 0  | 0  | 1  |

See also

**TYGetStruct** Usage:

```
TY_CAMERA_INTRINSIC intrinsic;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_CAM_INTRINSIC, &intrinsic, sizeof(intrinsic));
```

### 7.3.3.10 TY\_CAMERA\_ROTATION

```
typedef struct TY_CAMERA_ROTATION TY_CAMERA_ROTATION
```

a 3x3 matrix

| .   | .   | .   |
|-----|-----|-----|
| r00 | r01 | r02 |
| r10 | r11 | r12 |
| r20 | r21 | r22 |

See also

**TYGetStruct** Usage:

```
TY_CAMERA_ROTATION rotation;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_CAM_ROTATION, &rotation, sizeof(rotation));
```

### 7.3.3.11 TY\_CAMERA\_TO\_IMU

```
typedef struct TY_CAMERA_TO_IMU TY_CAMERA_TO_IMU
```

a 4x4 matrix

| .   | .   | .   | .  |
|-----|-----|-----|----|
| r11 | r12 | r13 | t1 |
| r21 | r22 | r23 | t2 |
| r31 | r32 | r33 | t3 |
| 0   | 0   | 0   | 1  |

### 7.3.3.12 TY\_COMPONENT\_ID

```
typedef uint32_t TY_COMPONENT_ID
```

component unique id

See also

[TY\\_DEVICE\\_COMPONENT\\_LIST](#)

Definition at line 192 of file TYDefs.h.

### 7.3.3.13 TY\_DEVICE\_BASE\_INFO

```
typedef struct TY_DEVICE_BASE_INFO TY_DEVICE_BASE_INFO
```

See also

[TYGetDeviceList](#)

### 7.3.3.14 TY\_DEVICE\_COMPONENT\_LIST

```
typedef enum TY_DEVICE_COMPONENT_LIST TY_DEVICE_COMPONENT_LIST
```

@brief Device Component list A device contains several component. Each component can be controlled by its own features, such as image width, exposure time, etc..

See also

To Know how to get feature information please refer to sample code DumpAllFeatures

### 7.3.3.15 TY\_ENUM\_ENTRY

```
typedef struct TY_ENUM_ENTRY TY_ENUM_ENTRY
```

enum feature entry information

See also

[TYGetEnumEntryInfo](#)

### 7.3.3.16 TY\_FEATURE\_ID

```
typedef uint32_t TY_FEATURE_ID
```

feature unique id

See also

[TY\\_FEATURE\\_ID\\_LIST](#)

Definition at line 384 of file TYDefs.h.

### 7.3.3.17 TY\_FLOAT\_RANGE

```
typedef struct TY_FLOAT_RANGE TY_FLOAT_RANGE
```

float range data structure

See also

[TYGetFloatRange](#)

### 7.3.3.18 TY\_GYRO\_BIAS

```
typedef struct TY_GYRO_BIAS TY_GYRO_BIAS
```

a 3x3 matrix

|       |       |       |
|-------|-------|-------|
| .     | .     | .     |
| BIASx | BIASy | BIASz |

### 7.3.3.19 TY\_GYRO\_MISALIGNMENT

```
typedef struct TY_GYRO_MISALIGNMENT TY_GYRO_MISALIGNMENT
```

a 3x3 matrix

| . | .        | .        |
|---|----------|----------|
| 1 | -ALPHAyz | ALPHAzy  |
| 0 | 1        | -ALPHAzx |
| 0 | 0        | 1        |

### 7.3.3.20 TY\_GYRO\_SCALE

```
typedef struct TY_GYRO_SCALE TY_GYRO_SCALE
```

a 3x3 matrix

| .      | .      | .      |
|--------|--------|--------|
| SCALEx | 0      | 0      |
| 0      | SCALEy | 0      |
| 0      | 0      | SCALEz |

### 7.3.3.21 TY\_INTERFACE\_INFO

```
typedef struct TY_INTERFACE_INFO TY_INTERFACE_INFO
```

See also

[TYGetInterfaceList](#)

### 7.3.3.22 TY\_INTERFACE\_TYPE\_LIST

```
typedef enum TY_INTERFACE_TYPE_LIST TY_INTERFACE_TYPE_LIST
```

Interface type definition

See also

[TYGetInterfaceList](#)

### 7.3.3.23 TY\_PIXEL\_BITS\_LIST

```
typedef enum TY_PIXEL_BITS_LIST TY_PIXEL_BITS_LIST
```

Pixel size type definitions to define the pixel size in bits

See also

[TY\\_PIXEL\\_FORMAT\\_LIST](#)

### 7.3.3.24 TY\_TRIGGER\_MODE\_LIST

```
typedef enum TY_TRIGGER_MODE_LIST TY_TRIGGER_MODE_LIST
```

See also

refer to sample SimpleView\_TriggerMode for detail usage

## 7.3.4 Enumeration Type Documentation

### 7.3.4.1 TY\_ACCESS\_MODE\_LIST

```
enum TY_ACCESS_MODE_LIST : uint32_t
```

Indicate a feature is readable or writable

See also

[TYGetFeatureInfo](#)

Definition at line 438 of file TYDefs.h.

### 7.3.4.2 TY\_DEVICE\_COMPONENT\_LIST

```
enum TY_DEVICE_COMPONENT_LIST : uint32_t
```

@brief Device Component list A device contains several component. Each component can be controlled by its own features, such as image width, exposure time, etc..

See also

To Know how to get feature information please refer to sample code DumpAllFeatures



## Enumerator

|                            |                                                             |
|----------------------------|-------------------------------------------------------------|
| TY_COMPONENT_DEVICE        | Abstract component stands for whole device, always enabled. |
| TY_COMPONENT_DEPTH_CAM     | Depth camera.                                               |
| TY_COMPONENT_IR_CAM_LEFT   | Left IR camera.                                             |
| TY_COMPONENT_IR_CAM_RIGHT  | Right IR camera.                                            |
| TY_COMPONENT_RGB_CAM_LEFT  | Left RGB camera.                                            |
| TY_COMPONENT_RGB_CAM_RIGHT | Right RGB camera.                                           |
| TY_COMPONENT_LASER         | Laser.                                                      |
| TY_COMPONENT_IMU           | Inertial Measurement Unit.                                  |
| TY_COMPONENT_BRIGHT_HISTO  | virtual component for brightness histogram of ir            |
| TY_COMPONENT_STORAGE       | virtual component for device storage                        |
| TY_COMPONENT_RGB_CAM       | Some device has only one RGB camera, map it to left.        |

Definition at line 177 of file TYDefs.h.

## 7.3.4.3 TY\_FEATURE\_ID\_LIST

```
enum TY_FEATURE_ID_LIST : uint32_t
```

feature for component definitions

## Enumerator

|                                  |                                                                                                                                                                                   |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TY_STRUCT_CAM_INTRINSIC          | see <a href="#">TY_CAMERA_INTRINSIC</a>                                                                                                                                           |
| TY_STRUCT_EXTRINSIC_TO_DEPTH     | extrinsic between depth cam and current component , see <a href="#">TY_CAMERA_EXTRINSIC</a>                                                                                       |
| TY_STRUCT_EXTRINSIC_TO_IR_LEFT   | extrinsic between left IR and current compoent, see <a href="#">TY_CAMERA_EXTRINSIC</a>                                                                                           |
| TY_STRUCT_CAM_RECTIFIED_ROTATION | see <a href="#">TY_CAMERA_ROTATION</a>                                                                                                                                            |
| TY_STRUCT_CAM_DISTORTION         | see <a href="#">TY_CAMERA_DISTORTION</a>                                                                                                                                          |
| TY_STRUCT_CAM_CALIB_DATA         | see <a href="#">TY_CAMERA_CALIB_INFO</a>                                                                                                                                          |
| TY_STRUCT_CAM_RECTIFIED_INTRI    | the rectified intrinsic. see <a href="#">TY_CAMERA_INTRINSIC</a>                                                                                                                  |
| TY_BYTEARRAY_CUSTOM_BLOCK        | used for reading/writing custom block                                                                                                                                             |
| TY_BYTEARRAY_ISP_BLOCK           | used for reading/writing fpn block                                                                                                                                                |
| TY_INT_PACKET_DELAY              | microseconds                                                                                                                                                                      |
| TY_INT_NTP_SERVER_IP             | Ntp server IP.                                                                                                                                                                    |
| TY_INT_LINK_CMD_TIMEOUT          | milliseconds                                                                                                                                                                      |
| TY_STRUCT_CAM_STATISTICS         | statistical information, see <a href="#">TY_CAMERA_STATISTICS</a>                                                                                                                 |
| TY_INT_WIDTH                     | Image width.                                                                                                                                                                      |
| TY_INT_HEIGHT                    | Image height.                                                                                                                                                                     |
| TY_ENUM_IMAGE_MODE               | Resolution-PixelFormat mode, see <a href="#">TY_IMAGE_MODE_LIST</a> .                                                                                                             |
| TY_FLOAT_SCALE_UNIT              | scale unit depth image is uint16 pixel format with default millimeter unit ,for some device can output Sub-millimeter accuracy data the acutal depth (mm)= PixelValue * ScaleUnit |
| TY_ENUM_TRIGGER_POL              | Trigger POL, see <a href="#">TY_TRIGGER_POL_LIST</a> .                                                                                                                            |
| TY_INT_FRAME_PER_TRIGGER         | Number of frames captured per trigger.                                                                                                                                            |

## Enumerator

|                                 |                                                                       |
|---------------------------------|-----------------------------------------------------------------------|
| TY_STRUCT_TRIGGER_PARAM         | param of trigger, see <a href="#">TY_TRIGGER_PARAM</a>                |
| TY_STRUCT_TRIGGER_PARAM_EX      | param of trigger, see <a href="#">TY_TRIGGER_PARAM_EX</a>             |
| TY_STRUCT_TRIGGER_TIMER_LIST    | param of trigger mode 20, see <a href="#">TY_TRIGGER_TIMER_LIST</a>   |
| TY_STRUCT_TRIGGER_TIMER_PERIOD  | param of trigger mode 21, see <a href="#">TY_TRIGGER_TIMER_PERIOD</a> |
| TY_BOOL_KEEP_ALIVE_ONOFF        | Keep Alive switch.                                                    |
| TY_INT_KEEP_ALIVE_TIMEOUT       | Keep Alive timeout.                                                   |
| TY_BOOL_CMOS_SYNC               | Cmos sync switch.                                                     |
| TY_INT_TRIGGER_DELAY_US         | Trigger delay time, in microseconds.                                  |
| TY_BOOL_TRIGGER_OUT_IO          | Trigger out IO.                                                       |
| TY_INT_TRIGGER_DURATION_US      | Trigger duration time, in microseconds.                               |
| TY_ENUM_STREAM_ASYNC            | stream async switch, see <a href="#">TY_STREAM_ASYNC_MODE</a>         |
| TY_INT_CAPTURE_TIME_US          | capture time in multi-ir                                              |
| TY_ENUM_TIME_SYNC_TYPE          | see <a href="#">TY_TIME_SYNC_TYPE</a>                                 |
| TY_BOOL_TIME_SYNC_READY         | time sync done status                                                 |
| TY_BOOL_IR_FLASHLIGHT           | Enable switch for floodlight used in ir component.                    |
| TY_INT_IR_FLASHLIGHT_INTENSITY  | ir component flashlight intensity level                               |
| TY_STRUCT_DO0_WORKMODE          | DO_0 workmode, see <a href="#">TY_DO_WORKMODE</a> .                   |
| TY_STRUCT_DI0_WORKMODE          | DI_0 workmode, see <a href="#">TY_DI_WORKMODE</a> .                   |
| TY_STRUCT_DO1_WORKMODE          | DO_1 workmode, see <a href="#">TY_DO_WORKMODE</a> .                   |
| TY_STRUCT_DI1_WORKMODE          | DI_1 workmode, see <a href="#">TY_DI_WORKMODE</a> .                   |
| TY_STRUCT_DO2_WORKMODE          | DO_2 workmode, see <a href="#">TY_DO_WORKMODE</a> .                   |
| TY_STRUCT_DI2_WORKMODE          | DI_2 workmode, see <a href="#">TY_DI_WORKMODE</a> .                   |
| TY_BOOL_RGB_FLASHLIGHT          | Enable switch for floodlight used in rgb component.                   |
| TY_INT_RGB_FLASHLIGHT_INTENSITY | rgb component flashlight intensity level                              |
| TY_BOOL_AUTO_EXPOSURE           | Auto exposure switch.                                                 |
| TY_INT_EXPOSURE_TIME            | Exposure time.                                                        |
| TY_BOOL_AUTO_GAIN               | Auto gain switch.                                                     |
| TY_INT_GAIN                     | Sensor Gain.                                                          |
| TY_BOOL_AUTO_AWB                | Auto white balance.                                                   |
| TY_STRUCT_AEC_ROI               | region of aec statistics, see <a href="#">TY_AEC_ROI_PARAM</a>        |
| TY_INT_TOF_HDR_RATIO            | tof sensor hdr ratio for depth                                        |
| TY_INT_TOF_JITTER_THRESHOLD     | tof jitter threshold for depth                                        |
| TY_FLOAT_EXPOSURE_TIME_US       | the exposure time, unit: us                                           |
| TY_INT_LASER_POWER              | Laser power level.                                                    |
| TY_BOOL_LASER_AUTO_CTRL         | Laser auto ctrl.                                                      |
| TY_STRUCT_LASER_ENABLE_BY_IDX   | Laser enable by device index.                                         |
| TY_STRUCT_LASER_POWER_BY_IDX    | Laser power by device index.                                          |
| TY_STRUCT_FLOOD_ENABLE_BY_IDX   | Flood enable by device index.                                         |
| TY_STRUCT_FLOOD_POWER_BY_IDX    | Flood power by device index.                                          |
| TY_BOOL_UNDISTORTION            | Output undistorted image.                                             |
| TY_BOOL_BRIGHTNESS_HISTOGRAM    | Output bright histogram.                                              |
| TY_BOOL_DEPTH_POSTPROC          | Do depth image postproc.                                              |
| TY_INT_R_GAIN                   | Gain of R channel.                                                    |
| TY_INT_G_GAIN                   | Gain of G channel.                                                    |
| TY_INT_B_GAIN                   | Gain of B channel.                                                    |
| TY_INT_ANALOG_GAIN              | Analog gain.                                                          |

## Enumerator

|                                 |                                                                          |
|---------------------------------|--------------------------------------------------------------------------|
| TY_BOOL_HDR                     | HDR func enable/disable.                                                 |
| TY_BYTEARRAY_HDR_PARAMETER      | HDR parameters.                                                          |
| TY_BOOL_IMU_DATA_ONOFF          | AE target y. IMU Data Onoff                                              |
| TY_STRUCT_IMU_ACC_BIAS          | IMU acc bias matrix, see <a href="#">TY_ACC_BIAS</a> .                   |
| TY_STRUCT_IMU_ACC_MISALIGNMENT  | IMU acc misalignment matrix, see <a href="#">TY_ACC_MISALIGNMENT</a> .   |
| TY_STRUCT_IMU_ACC_SCALE         | IMU acc scale matrix, see <a href="#">TY_ACC_SCALE</a> .                 |
| TY_STRUCT_IMU_GYRO_BIAS         | IMU gyro bias matrix, see <a href="#">TY_GYRO_BIAS</a> .                 |
| TY_STRUCT_IMU_GYRO_MISALIGNMENT | IMU gyro misalignment matrix, see <a href="#">TY_GYRO_MISALIGNMENT</a> . |
| TY_STRUCT_IMU_GYRO_SCALE        | IMU gyro scale matrix, see <a href="#">TY_GYRO_SCALE</a> .               |
| TY_STRUCT_IMU_CAM_TO_IMU        | IMU camera to imu matrix, see <a href="#">TY_CAMERA_TO_IMU</a> .         |
| TY_ENUM_IMU_FPS                 | IMU fps, see <a href="#">TY_IMU_FPS_LIST</a> .                           |
| TY_INT_SGBM_IMAGE_NUM           | SGBM image channel num.                                                  |
| TY_INT_SGBM_DISPARITY_NUM       | SGBM disparity num.                                                      |
| TY_INT_SGBM_DISPARITY_OFFSET    | SGBM disparity offset.                                                   |
| TY_INT_SGBM_MATCH_WIN_HEIGHT    | SGBM match window height.                                                |
| TY_INT_SGBM_SEMI_PARAM_P1       | SGBM semi global param p1.                                               |
| TY_INT_SGBM_SEMI_PARAM_P2       | SGBM semi global param p2.                                               |
| TY_INT_SGBM_UNIQUE_FACTOR       | SGBM uniqueness factor param.                                            |
| TY_INT_SGBM_UNIQUE_ABSDIFF      | SGBM uniqueness min absolute diff.                                       |
| TY_INT_SGBM_UNIQUE_MAX_COST     | SGBM uniqueness max cost param.                                          |
| TY_BOOL_SGBM_HFILTER_HALF_WIN   | SGBM enable half window size.                                            |
| TY_INT_SGBM_MATCH_WIN_WIDTH     | SGBM match window width.                                                 |
| TY_BOOL_SGBM_MEDFILTER          | SGBM enable median filter.                                               |
| TY_BOOL_SGBM_LRC                | SGBM enable left right consist check.                                    |
| TY_INT_SGBM_LRC_DIFF            | SGBM max diff.                                                           |
| TY_INT_SGBM_MEDFILTER_THRESH    | SGBM median filter thresh.                                               |
| TY_INT_SGBM_SEMI_PARAM_P1_SCALE | SGBM semi global param p1 scale.                                         |
| TY_INT_SGPM_PHASE_NUM           | Phase num to calc a depth.                                               |
| TY_INT_SGPM_NORMAL_PHASE_SCALE  | phase scale when calc a depth                                            |
| TY_INT_SGPM_NORMAL_PHASE_OFFSET | Phase offset when calc a depth.                                          |
| TY_INT_SGPM_REF_PHASE_SCALE     | Reference Phase scale when calc a depth.                                 |
| TY_INT_SGPM_REF_PHASE_OFFSET    | Reference Phase offset when calc a depth.                                |
| TY_FLOAT_SGPM_EPI_HS            | Epipolar Constraint pattern scale.                                       |
| TY_INT_SGPM_EPI_HF              | Epipolar Constraint pattern offset.                                      |
| TY_BOOL_SGPM_EPI_EN             | Epipolar Constraint enable.                                              |
| TY_INT_SGPM_EPI_CH0             | Epipolar Constraint channel0.                                            |
| TY_INT_SGPM_EPI_CH1             | Epipolar Constraint channel1.                                            |
| TY_INT_SGPM_EPI_THRESH          | Epipolar Constraint thresh.                                              |
| TY_BOOL_SGPM_ORDER_FILTER_EN    | Phase order filter enable.                                               |
| TY_INT_SGPM_ORDER_FILTER_CHN    | Phase order filter channel.                                              |
| TY_INT_DEPTH_MIN_MM             | min depth in mm output                                                   |
| TY_INT_DEPTH_MAX_MM             | max depth in mm ouput                                                    |
| TY_INT_SGBM_TEXTURE_OFFSET      | texture filter value offset                                              |
| TY_INT_SGBM_TEXTURE_THRESH      | texture filter threshold                                                 |
| TY_STRUCT_PHC_GROUP_ATTR        | Phase compute group attribute.                                           |

## Enumerator

|                                 |                                                               |
|---------------------------------|---------------------------------------------------------------|
| TY_ENUM_DEPTH_QUALITY           | the quality of generated depth, see TY_DEPTH_QUALITY          |
| TY_INT_FILTER_THRESHOLD         | the threshold of the noise filter, 0 for disabled             |
| TY_INT_TOF_CHANNEL              | the frequency channel of tof                                  |
| TY_INT_TOF_MODULATION_THRESHOLD | the threshold of the tof modulation                           |
| TY_STRUCT_TOF_FREQ              | the frequency of tof, see <a href="#">TY_TOF_FREQ</a>         |
| TY_BOOL_TOF_ANTI_INTERFERENCE   | cooperation if multi-device used                              |
| TY_INT_TOF_ANTI_SUNLIGHT_INDEX  | the index of anti-sunlight                                    |
| TY_INT_MAX_SPECKLE_SIZE         | the max size of speckle                                       |
| TY_INT_MAX_SPECKLE_DIFF         | the max diff of speckle                                       |
| TY_INT_TRANSMIT_ROI_OFFSET_X    | Types of optical distortion of lenses, see TYLensOpticalType. |
| TY_INT_TRANSMIT_ROI_OFFSET_Y    | Offset in pixels from image origin.                           |
| TY_INT_TRANSMIT_ROI_WIDTH       | Offset in lines from image origin.                            |
| TY_INT_TRANSMIT_ROI_HEIGHT      | ROI Image Width.                                              |
| TY_BOOL_TRANSMIT_ROI_ENABLE     | ROI Image Height. TRANSMIT_ROI enable/disable                 |

Definition at line 211 of file TYDefs.h.

#### 7.3.4.4 TY\_INTERFACE\_TYPE\_LIST

```
enum TY_INTERFACE_TYPE_LIST : uint32_t
```

Interface type definition

See also

[TYGetInterfaceList](#)

Definition at line 425 of file TYDefs.h.

#### 7.3.4.5 TY\_PIXEL\_BITS\_LIST

```
enum TY_PIXEL_BITS_LIST : uint32_t
```

Pixel size type definitions to define the pixel size in bits

See also

[TY\\_PIXEL\\_FORMAT\\_LIST](#)

Definition at line 535 of file TYDefs.h.

#### 7.3.4.6 TY\_PIXEL\_FORMAT\_LIST

```
enum TY_PIXEL_FORMAT_LIST : uint32_t
```

pixel format definitions

## Enumerator

|                                 |                     |
|---------------------------------|---------------------|
| TY_PIXEL_FORMAT_MONO            | 0x10000000          |
| TY_PIXEL_FORMAT_BAYER8GB        | 0x11000000          |
| TY_PIXEL_FORMAT_BAYER8BG        | 0x12000000          |
| TY_PIXEL_FORMAT_BAYER8GR        | 0x13000000          |
| TY_PIXEL_FORMAT_BAYER8RG        | 0x14000000          |
| TY_PIXEL_FORMAT_CSI_MONO10      | 0x50000000          |
| TY_PIXEL_FORMAT_CSI_BAYER10GRBG | 0x51000000          |
| TY_PIXEL_FORMAT_CSI_BAYER10RGGB | 0x52000000          |
| TY_PIXEL_FORMAT_CSI_BAYER10GBRG | 0x53000000          |
| TY_PIXEL_FORMAT_CSI_BAYER10BGGR | 0x54000000          |
| TY_PIXEL_FORMAT_CSI_MONO12      | 0x60000000          |
| TY_PIXEL_FORMAT_CSI_BAYER12GRBG | 0x61000000          |
| TY_PIXEL_FORMAT_CSI_BAYER12RGGB | 0x62000000          |
| TY_PIXEL_FORMAT_CSI_BAYER12GBRG | 0x63000000          |
| TY_PIXEL_FORMAT_CSI_BAYER12BGGR | 0x64000000          |
| TY_PIXEL_FORMAT_DEPTH16         | 0x20000000          |
| TY_PIXEL_FORMAT_YVYU            | 0x21000000, yvyu422 |
| TY_PIXEL_FORMAT_YUYV            | 0x22000000, yuyv422 |
| TY_PIXEL_FORMAT_MONO16          | 0x23000000,         |
| TY_PIXEL_FORMAT_TOF_IR_MONO16   | 0xa4000000,         |
| TY_PIXEL_FORMAT_RGB             | 0x30000000          |
| TY_PIXEL_FORMAT_BGR             | 0x31000000          |
| TY_PIXEL_FORMAT_JPEG            | 0x32000000          |
| TY_PIXEL_FORMAT_MJPEG           | 0x33000000          |
| TY_PIXEL_FORMAT_RGB48           | 0x80000000          |
| TY_PIXEL_FORMAT_BGR48           | 0x81000000          |
| TY_PIXEL_FORMAT_XYZ48           | 0x82000000          |

Definition at line 553 of file TYDefs.h.

### 7.3.4.7 TY\_RESOLUTION\_MODE\_LIST

```
enum TY_RESOLUTION_MODE_LIST : uint32_t
```

predefined resolution list

## Enumerator

|                            |            |
|----------------------------|------------|
| TY_RESOLUTION_MODE_160x100 | 0x000a0078 |
| TY_RESOLUTION_MODE_160x120 | 0x000a0078 |
| TY_RESOLUTION_MODE_240x320 | 0x000f0140 |
| TY_RESOLUTION_MODE_320x180 | 0x001400b4 |
| TY_RESOLUTION_MODE_320x200 | 0x001400c8 |
| TY_RESOLUTION_MODE_320x240 | 0x001400f0 |
| TY_RESOLUTION_MODE_480x640 | 0x001e0280 |
| TY_RESOLUTION_MODE_640x360 | 0x00280168 |

## Enumerator

|                              |            |
|------------------------------|------------|
| TY_RESOLUTION_MODE_640x400   | 0x00280190 |
| TY_RESOLUTION_MODE_640x480   | 0x002801e0 |
| TY_RESOLUTION_MODE_960x1280  | 0x003c0500 |
| TY_RESOLUTION_MODE_1280x720  | 0x005002d0 |
| TY_RESOLUTION_MODE_1280x800  | 0x00500320 |
| TY_RESOLUTION_MODE_1280x960  | 0x005003c0 |
| TY_RESOLUTION_MODE_1600x1200 | 0x006404b0 |
| TY_RESOLUTION_MODE_800x600   | 0x00320258 |
| TY_RESOLUTION_MODE_1920x1080 | 0x00780438 |
| TY_RESOLUTION_MODE_2560x1920 | 0x00a00780 |
| TY_RESOLUTION_MODE_2592x1944 | 0x00a20798 |
| TY_RESOLUTION_MODE_1920x1440 | 0x007805a0 |
| TY_RESOLUTION_MODE_240x96    | 0x000f0060 |
| TY_RESOLUTION_MODE_2048x1536 | 0x00800600 |

Definition at line 597 of file TYDefs.h.

### 7.3.4.8 TY\_TRIGGER\_MODE\_LIST

```
enum TY_TRIGGER_MODE_LIST : uint32_t
```

#### See also

refer to sample SimpleView\_TriggerMode for detail usage

## Enumerator

|                           |                                                                                |
|---------------------------|--------------------------------------------------------------------------------|
| TY_TRIGGER_MODE_OFF       | not trigger mode, continuous mode                                              |
| TY_TRIGGER_MODE_SLAVE     | slave mode, receive soft/hardware triggers                                     |
| TY_TRIGGER_MODE_M_SIG     | master mode 1, sending one trigger signal once received a soft trigger         |
| TY_TRIGGER_MODE_M_PER     | master mode 2, periodic sending one trigger signals, 'fps' param should be set |
| TY_TRIGGER_MODE_SIG_PASS  | discard, using TY_TRIGGER_MODE28                                               |
| TY_TRIGGER_MODE_PER_PASS  | discard, using TY_TRIGGER_MODE29                                               |
| TY_TRIGGER_MODE_PER_PASS2 | trigger mode 30, Alternate output depth image/ir image                         |

Definition at line 698 of file TYDefs.h.

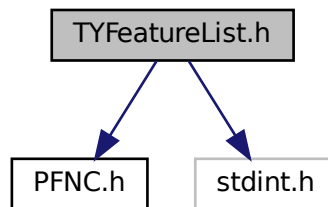
## 7.4 TYFeatureList.h File Reference

Camera feature enumeration list.

```
#include "PFNC.h"
```

```
#include <stdint.h>
```

Include dependency graph for TYFeatureList.h:



## Macros

- `#define TY_DEV_TL_VER_MAJ` "DeviceTLVersionMajor"  
*Integer. Major version of the Transport Layer of the device.*
- `#define TY_DEV_TL_VER_MIN` "DeviceTLVersionMinor"  
*Integer. Minor version of the Transport Layer of the device.*
- `#define TY_DEV_TYPE` "DeviceType"  
*Enumeration. Returns the device type.*
- `#define TY_DEV_VENDOR` "DeviceVendorName"  
*String. Name of the device vendor.*
- `#define TY_DEV_MODEL` "DeviceModelName"  
*String. Model of the device.*
- `#define TY_DEV_MANU_INFO` "DeviceManufacturerInfo"  
*String. Manufacturer information about the device.*
- `#define TY_DEV_VER` "DeviceVersion"  
*String. Version of the device.*
- `#define TY_DEV_HW_MODEL` "DeviceHardWareModel"  
*String. Hardware model of the device.*
- `#define TY_DEV_HW_VER` "DeviceHardwareVersion"  
*String. Version of the Board Hardware.*
- `#define TY_DEV_FW_VER` "DeviceFirmwareVersion"  
*String. Version of the firmware in the device.*
- `#define TY_DEV_SN` "DeviceSerialNumber"  
*String. Serial number of the device.*
- `#define TY_DEV_BUILD_HASH` "DeviceBuildHash"  
*String. Build hash of the device.*
- `#define TY_DEV_CFG_VER` "DeviceConfigVersion"  
*String. Config version of the device.*
- `#define TY_DEV_TECH_MODEL` "DeviceTechModel"  
*String. Technical model of the device.*
- `#define TY_DEV_GEN_TIME` "DeviceGeneratedTime"  
*String. Generated time of the device.*
- `#define TY_DEV_CAL_TIME` "DeviceCalibrationTime"

- String. Calibration time of the device.*

  - `#define TY_DEV_CONN_SEL "DeviceConnectionSelector"`

*Integer. Selects which Connection of the device to control.*
- `#define TY_DEV_LINK_SEL "DeviceLinkSelector"`

*Integer. Selects which Link of the device to control.*
- `#define TY_DEV_LINK_SPEED "DeviceLinkSpeed"`

*Integer. Speed of the selected link.*
- `#define TY_DEV_LINK_HB_MODE "DeviceLinkHeartbeatMode"`

*Enumeration. Activate or deactivate the Link's heartbeat.*
- `#define TY_DEV_LINK_HB_TIMEOUT "DeviceLinkHeartbeatTimeout"`

*Integer. Speed of the selected link.*
- `#define TY_DEV_STREAM_CH_COUNT "DeviceStreamChannelCount"`

*Integer. Indicates the number of streaming channels supported by the device.*
- `#define TY_DEV_STREAM_CH_SEL "DeviceStreamChannelSelector"`

*Integer. Selects the stream channel to configure.*
- `#define TY_DEV_STREAM_CH_TYPE "DeviceStreamChannelType"`

*Enumeration. Type of the selected stream channel.*
- `#define TY_DEV_STREAM_CH_LINK "DeviceStreamChannelLink"`

*Integer. Link associated with the selected stream channel.*
- `#define TY_DEV_STREAM_CH_PKT_SIZE "DeviceStreamChannelPacketSize"`

*Integer. Packet size used on the selected stream channel.*
- `#define TY_DEV_CHARSET "DeviceCharacterSet"`

*Enumeration. Character set used by the strings of the device's bootstrap registers.*
- `#define TY_DEV_RESET "DeviceReset"`

*Command. Reset the device to the state it had after power-up.*
- `#define TY_DEV_ERR_CODE "DeviceErrCode"`

*Integer. Error code of the device.*
- `#define TY_DEV_FRAME_TIMEOUT "DeviceFrameRecvTimeOut"`

*Integer. The waiting timeout for the device to send a single frame of data (unit: ms)*
- `#define TY_DEV_TEMP_SEL "DeviceTemperatureSelector"`

*Enumeration. Selects the temperature sensor to read.*
- `#define TY_DEV_TEMP "DeviceTemperature"`

*Float. Temperature of the selected by DeviceTemperatureSelector.*
- `#define TY_DEV_TEMP_STR "DeviceTemperatureString"`

*String. Temperature of the selected by DeviceTemperatureSelector.*
- `#define TY_DEV_STREAM_ASYNC "DeviceStreamAsyncMode"`

*Enumeration. Controls stream asynchronous mode.*
- `#define TY_DEV_TIME_SYNC "DeviceTimeSyncMode"`

*Enumeration. Selects the time synchronization mode.*
- `#define TY_NTP_SRV_IP "NTPServerIP"`

*Integer. NTP server IP address.*
- `#define TY_DEV_COMPRESS "DeviceDataCompressType"`

*Enumeration. Selects the data compression type.*
- `#define TY_ACQ_MODE "AcquisitionMode"`

*Enumeration. Sets the acquisition mode of the device.*
- `#define TY_ACQ_START "AcquisitionStart"`

*Command. Starts the acquisition of images.*
- `#define TY_ACQ_STOP "AcquisitionStop"`

*Command. Stops the acquisition of images.*
- `#define TY_ACQ_FPS "AcquisitionFrameRate"`

*Float. Controls the acquisition frame rate.*



- `#define TY_ACQ_FPS_EN` "AcquisitionFrameRateEnable"  
*Boolean. Enables the acquisition frame rate control.*
- `#define TY_EXP_AUTO` "ExposureAuto"  
*Boolean. Sets the automatic exposure mode.*
- `#define TY_EXP_AUTO_MIN` "ExposureAutoLowerLimit"  
*Float. Lower limit of the auto exposure time.*
- `#define TY_EXP_AUTO_MAX` "ExposureAutoUpperLimit"  
*Float. Upper limit of the auto exposure time.*
- `#define TY_EXP_TARGET` "ExposureTargetBrightness"  
*Float. Target brightness for auto exposure.*
- `#define TY_EXP_TIME` "ExposureTime"  
*Float. Sets the exposure time.*
- `#define TY_HDR_EN` "HDREnable"  
*Boolean. Enables HDR mode.*
- `#define TY_HDR_CTRL` "HDRControl"  
*Bytearray. Controls HDR parameters.*
- `#define TY_TRIG_SEL` "TriggerSelector"  
*Enumeration. Selects the trigger type to configure.*
- `#define TY_TRIG_MODE` "TriggerMode"  
*Enumeration. Controls whether the selected trigger is active.*
- `#define TY_TRIG_SW` "TriggerSoftware"  
*Command. Generates a software trigger signal.*
- `#define TY_TRIG_SRC` "TriggerSource"  
*Enumeration. Specifies the internal signal or physical input line to use as the trigger source.*
- `#define TY_TRIG_ACT` "TriggerActivation"  
*Enumeration. Specifies the activation mode of the trigger.*
- `#define TY_TRIG_DELAY` "TriggerDelay"  
*Float. Specifies the delay in microseconds to apply after the trigger reception before activating it.*
- `#define TY_TRIG_OUT` "TriggerOutIO"  
*Boolean. Controls trigger output IO.*
- `#define TY_TRIG_DUR` "TriggerDurationUs"  
*Integer. Specifies the duration in microseconds of the trigger signal.*
- `#define TY_CMOS_SYNC` "CmosSync"  
*Boolean. Controls CMOS synchronization.*
- `#define TY_TIME_SYNC_ACK` "TimeSyncAck"  
*Boolean. Time synchronization acknowledge.*
- `#define TY_GAIN_SEL` "GainSelector"  
*Enumeration. Selects which gain to control.*
- `#define TY_GAIN` "Gain"  
*Float. Controls the selected gain as a factor.*
- `#define TY_WB_AUTO` "BalanceWhiteAuto"  
*Boolean. Balance White Auto is the 'automatic' counterpart of the manual white balance feature.*
- `#define TY_AF_AOI_X` "AutoFunctionAOIOffsetX"  
*Integer. Horizontal offset of the auto function AOI.*
- `#define TY_AF_AOI_Y` "AutoFunctionAOIOffsetY"  
*Integer. Vertical offset of the auto function AOI.*
- `#define TY_AF_AOI_W` "AutoFunctionAOIWidth"  
*Integer. Width of the auto function AOI.*
- `#define TY_AF_AOI_H` "AutoFunctionAOIHeight"  
*Integer. Height of the auto function AOI.*
- `#define TY_SENSOR_W` "SensorWidth"

- Integer. Width of the camera sensor in pixels.*

  - `#define TY_SENSOR_H "SensorHeight"`
- Integer. Height of the camera sensor in pixels.*

  - `#define TY_PIX_FMT "PixelFormat"`
- Enumeration. Format of the pixels provided by the device.*

  - `#define TY_BIN_H "BinningHorizontal"`
- Enumeration. Number of horizontal pixels to combine together.*

  - `#define TY_BIN_V "BinningVertical"`
- Enumeration. Number of vertical pixels to combine together.*

  - `#define TY_COMP_EN "ComponentEnable"`
- Boolean. Enables the component.*

  - `#define TY_DEPTH_SCALE "DepthScaleUnit"`
- Float. Depth scale unit.*

  - `#define TY_DEPTH_SGBM_IMG_NUM "DepthSgbmImageNumber"`
- Integer. SGBM image number.*

  - `#define TY_DEPTH_SGBM_DISP_NUM "DepthSgbmDisparityNumber"`
- Integer. SGBM disparity number.*

  - `#define TY_DEPTH_SGBM_DISP_OFF "DepthSgbmDisparityOffset"`
- Integer. SGBM disparity offset.*

  - `#define TY_DEPTH_SGBM_WIN_H "DepthSgbmMatchWinHeight"`
- Integer. SGBM match window height.*

  - `#define TY_DEPTH_SGBM_P1 "DepthSgbmSemiParamP1"`
- Integer. SGBM semi global param p1.*

  - `#define TY_DEPTH_SGBM_P2 "DepthSgbmSemiParamP2"`
- Integer. SGBM semi global param p2.*

  - `#define TY_DEPTH_SGBM_UNIQUE "DepthSgbmUniqueFactor"`
- Integer. SGBM unique factor.*

  - `#define TY_DEPTH_SGBM_UNIQUE_DIFF "DepthSgbmUniqueAbsDiff"`
- Integer. SGBM unique absolute difference.*

  - `#define TY_DEPTH_SGBM_UNIQUE_COST "DepthSgbmUniqueMaxCost"`
- Integer. SGBM unique max cost.*

  - `#define TY_DEPTH_SGBM_H_WIN "DepthSgbmHFilterHalfWin"`
- Boolean. SGBM horizontal filter half window.*

  - `#define TY_DEPTH_SGBM_WIN_W "DepthSgbmMatchWinWidth"`
- Integer. SGBM match window width.*

  - `#define TY_DEPTH_SGBM_MED "DepthSgbmMedFilter"`
- Boolean. SGBM median filter.*

  - `#define TY_DEPTH_SGBM_LRC "DepthSgbmLRC"`
- Boolean. SGBM LRC.*

  - `#define TY_DEPTH_SGBM_LRC_DIFF "DepthSgbmLRCDiff"`
- Integer. SGBM LRC difference.*

  - `#define TY_DEPTH_SGBM_MED_THRES "DepthSgbmMedFilterThresh"`
- Integer. SGBM median filter threshold.*

  - `#define TY_DEPTH_SGBM_P1_SCALE "DepthSgbmSemiParamP1Scale"`
- Integer. SGBM semi param P1 scale.*

  - `#define TY_DEPTH_MIN "DepthRangeMin"`
- Integer. Minimum depth range.*

  - `#define TY_DEPTH_MAX "DepthRangeMax"`
- Integer. Maximum depth range.*

  - `#define TY_DEPTH_SGBM_TEX_OFF "DepthSgbmTextureFilterValueOffset"`
- Integer. Texture filter value offset.*

- `#define TY_DEPTH_SGBM_TEX_THRES` "DepthSgbmTextureFilterThreshold"  
*Integer. Texture filter threshold.*
- `#define TY_DEPTH_SGBM_TEX_EN` "DepthSgbmTextureFilterEnable"  
*Boolean. Texture filter enable.*
- `#define TY_DEPTH_SGBM_TEX_WIN` "DepthSgbmTextureFilterWindowSize"  
*Integer. Texture filter window size.*
- `#define TY_DEPTH_SGBM_TEX_MAX` "DepthSgbmTextureFilterMaxDistance"  
*Boolean. Texture filter maximum distance.*
- `#define TY_DEPTH_SGBM_SAT_EN` "DepthSgbmSaturateFilterEnable"  
*Boolean. Saturate filter enable.*
- `#define TY_DEPTH_SGBM_SAT_VAL` "DepthSgbmSaturateFilterVal"  
*Integer. Texture filter threshold.*
- `#define TY_DEPTH_SGBM_SAT_BLUR` "DepthSgbmSaturateFilterBlurSize"  
*Integer. Saturate filter blur size.*
- `#define TY_DEPTH_SGBM_SAT_DILATE` "DepthSgbmSaturateFilterDilateSize"  
*Integer. Saturate filter dilate size.*
- `#define TY_DEPTH_TOF_THRES` "DepthStreamTofFilterThreshold"  
*Integer. ToF filter threshold.*
- `#define TY_DEPTH_TOF_CH` "DepthStreamTofChannel"  
*Integer. ToF channel.*
- `#define TY_DEPTH_TOF_MOD_THRES` "DepthStreamTofModulationThreshold"  
*Integer. ToF modulation threshold.*
- `#define TY_DEPTH_TOF_QUALITY` "DepthStreamTofDepthQuality"  
*Enumeration. ToF depth quality.*
- `#define TY_DEPTH_TOF_HDR` "DepthStreamTofHdrRatio"  
*Integer. ToF HDR ratio.*
- `#define TY_DEPTH_TOF_JITTER` "DepthStreamTofJitterThreshold"  
*Integer. ToF jitter threshold.*
- `#define TY_TOF_MOD` "ToFModParam"  
*Integer. ToF modulation parameter.*
- `#define TY_TOF_FMOD0` "ToFModParam0"  
*Integer. ToF FMOD parameter 0.*
- `#define TY_TOF_FMOD1` "ToFModParam1"  
*Integer. ToF FMOD parameter 1.*
- `#define TY_TOF_DELAY0` "ToFLightDelayParam0"  
*Integer. ToF light delay parameter 0.*
- `#define TY_TOF_DELAY1` "ToFLightDelayParam1"  
*Integer. ToF light delay parameter 1.*
- `#define TY_TOF_DELAY2` "ToFLightDelayParam2"  
*Integer. ToF light delay parameter 2.*
- `#define TY_INTRIN_W` "IntrinsicWidth"  
*Integer. Intrinsic width.*
- `#define TY_INTRIN_H` "IntrinsicHeight"  
*Integer. Intrinsic height.*
- `#define TY_INTRIN` "Intrinsic"  
*Bytearray. Intrinsic parameters.*
- `#define TY_REC_INTRIN` "Intrinsic2"  
*Bytearray. Rectified Intrinsic parameters.*
- `#define TY_REC_ROT` "Rotation"  
*Bytearray. Rectified Rotation matrix.*
- `#define TY_DISTORT` "Distortion"

- *Bytearray. Distortion parameters.*
- `#define TY_EXTRIN "Extrinsic"`  
*Bytearray. Extrinsic parameters to Depth image.*
- `#define TY_PAYLOAD "PayloadSize"`  
*Integer. Unit size of data payload in bytes.*
- `#define TY_GEV_IF_SEL "GevInterfaceSelector"`  
*Integer. Selects the network interface to control.*
- `#define TY_GEV_MAC "GevMACAddress"`  
*Integer. MAC address of the network interface.*
- `#define TY_GEV_IP_CFG_LLA "GevCurrentIPConfigurationLLA"`  
*Boolean. Controls whether the Link Local Address IP configuration scheme is activated.*
- `#define TY_GEV_IP_CFG_DHCP "GevCurrentIPConfigurationDHCP"`  
*Boolean. Controls whether the DHCP IP configuration scheme is activated.*
- `#define TY_GEV_IP_CFG_PERSIST "GevCurrentIPConfigurationPersistentIP"`  
*Boolean. Controls whether the Persistent IP configuration scheme is activated.*
- `#define TY_GEV_IP "GevCurrentIPAddress"`  
*Integer. Indicates the current IP address.*
- `#define TY_GEV_SUBNET "GevCurrentSubnetMask"`  
*Integer. Indicates the current subnet mask.*
- `#define TY_GEV_GATEWAY "GevCurrentDefaultGateway"`  
*Integer. Indicates the current default gateway.*
- `#define TY_GEV_URL1 "GevFirstURL"`  
*String. The first choice of URL for the XML device description file.*
- `#define TY_GEV_URL2 "GevSecondURL"`  
*String. The second choice of URL for the XML device description file.*
- `#define TY_GEV_PERSIST_IP "GevPersistentIPAddress"`  
*Integer. Indicates the persistent IP address.*
- `#define TY_GEV_PERSIST_SUBNET "GevPersistentSubnetMask"`  
*Integer. Indicates the persistent subnet mask.*
- `#define TY_GEV_PERSIST_GATEWAY "GevPersistentDefaultGateway"`  
*Integer. Indicates the persistent default gateway.*
- `#define TY_GEV_GVCP_ACK "GevGVCPPendingAck"`  
*Boolean. If enabled the device will issue a PENDING ACK in response to all GVCP commands.*
- `#define TY_GEV_CCP "GevCCP"`  
*Enumeration. Controls the device access privilege of an application.*
- `#define TY_GEV_STREAM_CH_SEL "GevStreamChannelSelector"`  
*Integer. Selects the stream channel to configure.*
- `#define TY_GEV_SCP_IF_IDX "GevSCPInterfaceIndex"`  
*Integer. Index of network interface to use.*
- `#define TY_GEV_SCP_PORT "GevSCPHostPort"`  
*Integer. Port to which the device must send data stream images.*
- `#define TY_GEV_SCP_DIR "GevSCPDirection"`  
*Integer. Indicates the direction of the data transfer.*
- `#define TY_GEV_SCP_PKT_SIZE "GevSCPSPacketSize"`  
*Integer. Indicates the packet size in bytes.*
- `#define TY_GEV_SCPD "GevSCPD"`  
*Integer. Indicates the delay in ticks inserted between each packet.*
- `#define TY_GEV_SCD "GevSCD"`  
*Integer. Indicates the destination IP address.*
- `#define TY_GEV_SCSP "GevSCSP"`  
*Integer. Indicates the destination port.*

- `#define TY_USER_SET_CUR "UserSetCurrent"`  
*Integer. Indicates the user set that is currently in use.*
- `#define TY_USER_SET_SEL "UserSetSelector"`  
*Enumeration. Selects the feature User Set to load, save or configure.*
- `#define TY_USER_SET_DESC "UserSetDescription"`  
*String. User set description.*
- `#define TY_USER_SET_LOAD "UserSetLoad"`  
*Command. Loads the User Set specified by UserSetSelector to the device.*
- `#define TY_USER_SET_SAVE "UserSetSave"`  
*Command. Save the User Set specified by UserSetSelector to the non-volatile memory of the device.*
- `#define TY_USER_SET_DEF "UserSetDefault"`  
*Enumeration. Indicates which User Set is used as default startup set.*
- `#define TY_SRC_COUNT "SourceCount"`  
*Integer. Controls or returns the number of sources supported by the device.*
- `#define TY_SRC_SEL "SourceSelector"`  
*Enumeration. Selects the source to control.*
- `#define TY_SRC_ID "SourceIDValue"`  
*Integer. Source ID value.*
- `#define TY_CAP_TIME_STAT "CaptureTimeStatistic"`  
*Integer. Capture time statistic.*
- `#define TY_IR_UNDIST "IRUndistortion"`  
*Boolean. IR undistortion.*
- `#define TY_POST_PROC0 "PostProcessParams0"`  
*Bytearray. Post process parameters 0.*
- `#define TY_POST_PROC1 "PostProcessParams1"`  
*Bytearray. Post process parameters 1.*
- `#define TY_LIGHT_SEL "LightControllerSelector"`  
*Enumeration. Selects the light to control.*
- `#define TY_LIGHT_EN "LightEnable"`  
*Boolean. Enable/Disable the Light.*
- `#define TY_LIGHT_BRIGHTNESS "LightBrightness"`  
*Integer. Config the Light Brightness.*

## Typedefs

- `typedef enum TY_DEV_TYPE_LIST TY_DEV_TYPE_LIST`
- `typedef enum TY_DEV_LINK_HB_MODE_LIST TY_DEV_LINK_HB_MODE_LIST`
- `typedef enum TY_DEV_STREAM_CH_TYPE_LIST TY_DEV_STREAM_CH_TYPE_LIST`
- `typedef enum TY_DEV_CHARSET_LIST TY_DEV_CHARSET_LIST`
- `typedef enum TY_DEV_TEMP_SEL_LIST TY_DEV_TEMP_SEL_LIST`
- `typedef enum TY_DEV_STREAM_ASYNC_LIST TY_DEV_STREAM_ASYNC_LIST`
- `typedef enum TY_DEV_TIME_SYNC_LIST TY_DEV_TIME_SYNC_LIST`
- `typedef enum TY_DEV_COMPRESS_LIST TY_DEV_COMPRESS_LIST`
- `typedef enum TY_ACQ_MODE_LIST TY_ACQ_MODE_LIST`
- `typedef enum TY_TRIG_SEL_LIST TY_TRIG_SEL_LIST`
- `typedef enum TY_TRIG_MODE_LIST TY_TRIG_MODE_LIST`
- `typedef enum TY_TRIG_SRC_LIST TY_TRIG_SRC_LIST`
- `typedef enum TY_TRIG_ACT_LIST TY_TRIG_ACT_LIST`
- `typedef enum TY_GAIN_SEL_LIST TY_GAIN_SEL_LIST`
- `typedef enum TY_PIX_FMT_LIST TY_PIX_FMT_LIST`
- `typedef enum TY_BIN_H_LIST TY_BIN_H_LIST`

- typedef enum [TY\\_BIN\\_V\\_LIST](#) [TY\\_BIN\\_V\\_LIST](#)
- typedef enum [TY\\_DEPTH\\_TOF\\_QUALITY\\_LIST](#) [TY\\_DEPTH\\_TOF\\_QUALITY\\_LIST](#)
- typedef enum [TY\\_GEV\\_CCP\\_LIST](#) [TY\\_GEV\\_CCP\\_LIST](#)
- typedef enum [TY\\_USER\\_SET\\_SEL\\_LIST](#) [TY\\_USER\\_SET\\_SEL\\_LIST](#)
- typedef enum [TY\\_SRC\\_SEL\\_LIST](#) [TY\\_SRC\\_SEL\\_LIST](#)
- typedef enum [TY\\_LIGHT\\_SEL\\_LIST](#) [TY\\_LIGTH\\_SEL\\_LIST](#)

## Enumerations

- enum [TY\\_DEV\\_TYPE\\_LIST](#) : uint32\_t { [DEV\\_TYPE\\_TRANSMITTER](#), [DEV\\_TYPE\\_RECEIVER](#), [DEV\\_TYPE\\_TRANSCEIVER](#), [DEV\\_TYPE\\_PERIPHERAL](#) }
- enum [TY\\_DEV\\_LINK\\_HB\\_MODE\\_LIST](#) : uint32\_t { [DEV\\_LINK\\_HB\\_ON](#), [DEV\\_LINK\\_HB\\_OFF](#) }
- enum [TY\\_DEV\\_STREAM\\_CH\\_TYPE\\_LIST](#) : uint32\_t { [DEV\\_STREAM\\_CH\\_TRANSMITTER](#), [DEV\\_STREAM\\_CH\\_RECEIVER](#) }
- enum [TY\\_DEV\\_CHARSET\\_LIST](#) : uint32\_t { [DEV\\_CHARSET\\_UTF8](#) = 1, [DEV\\_CHARSET\\_ASCII](#) }
- enum [TY\\_DEV\\_TEMP\\_SEL\\_LIST](#) : uint32\_t { [DEV\\_TEMP\\_SEL\\_MAINBOARD](#), [DEV\\_TEMP\\_SEL\\_LEFT](#), [DEV\\_TEMP\\_SEL\\_RIGHT](#), [DEV\\_TEMP\\_SEL\\_TEXTURE](#), [DEV\\_TEMP\\_SEL\\_LASER](#), [DEV\\_TEMP\\_SEL\\_IRFLASH](#), [DEV\\_TEMP\\_SEL\\_TEXTURE\\_FLASH](#), [DEV\\_TEMP\\_SEL\\_CPUCORE](#) = 8 }
- enum [TY\\_DEV\\_STREAM\\_ASYNC\\_LIST](#) : uint32\_t { [DEV\\_STREAM\\_ASYNC\\_OFF](#), [DEV\\_STREAM\\_ASYNC\\_DEPTH](#), [DEV\\_STREAM\\_ASYNC\\_RGB](#), [DEV\\_STREAM\\_ASYNC\\_DEPTH](#), [DEV\\_STREAM\\_ASYNC\\_ALL](#) = 255 }
- enum [TY\\_DEV\\_TIME\\_SYNC\\_LIST](#) : uint32\_t { [DEV\\_TIME\\_SYNC\\_NONE](#), [DEV\\_TIME\\_SYNC\\_HOST](#), [DEV\\_TIME\\_SYNC\\_NTP](#), [DEV\\_TIME\\_SYNC\\_PTP\\_SLAVE](#), [DEV\\_TIME\\_SYNC\\_CAN](#), [DEV\\_TIME\\_SYNC\\_PTP\\_MASTER](#) }
- enum [TY\\_DEV\\_COMPRESS\\_LIST](#) : uint32\_t { [DEV\\_COMPRESS\\_NONE](#) = 1, [DEV\\_COMPRESS\\_HW](#) }
- enum [TY\\_ACQ\\_MODE\\_LIST](#) : uint32\_t { [ACQ\\_MODE\\_SINGLE\\_FRAME](#), [ACQ\\_MODE\\_MULTI\\_FRAME](#), [ACQ\\_MODE\\_CONTINUOUS](#) }
- enum [TY\\_TRIG\\_SEL\\_LIST](#) : uint32\_t { [TRIG\\_SEL\\_FRAME\\_ACQUISITIONSTART](#), [TRIG\\_SEL\\_FRAME\\_ACQUISITIONEND](#), [TRIG\\_SEL\\_FRAME\\_ACQUISITIONAC](#), [TRIG\\_SEL\\_FRAME\\_FRAMESTART](#), [TRIG\\_SEL\\_FRAME\\_FRAMEEND](#), [TRIG\\_SEL\\_FRAME\\_FRAMEACTIVE](#), [TRIG\\_SEL\\_FRAME\\_FRAMEBURSTSTART](#), [TRIG\\_SEL\\_FRAME\\_FRAMEBURSTEND](#), [TRIG\\_SEL\\_FRAME\\_FRAMEBURSTACTIVE](#), [TRIG\\_SEL\\_FRAME\\_LINESTART](#), [TRIG\\_SEL\\_FRAME\\_EXPOSURESTART](#), [TRIG\\_SEL\\_FRAME\\_EXPOSUREEND](#), [TRIG\\_SEL\\_FRAME\\_EXPOSUREACTIVE](#) }
- enum [TY\\_TRIG\\_MODE\\_LIST](#) : uint32\_t { [TRIG\\_MODE\\_OFF](#), [TRIG\\_MODE\\_ON](#) }
- enum [TY\\_TRIG\\_SRC\\_LIST](#) : uint32\_t { [TRIG\\_SRC\\_LINE0](#), [TRIG\\_SRC\\_LINE1](#), [TRIG\\_SRC\\_LINE2](#), [TRIG\\_SRC\\_LINE3](#), [TRIG\\_SRC\\_LINE4](#), [TRIG\\_SRC\\_LINE5](#), [TRIG\\_SRC\\_LINE6](#), [TRIG\\_SRC\\_LINE7](#), [TRIG\\_SRC\\_SOFTWARE](#) }
- enum [TY\\_TRIG\\_ACT\\_LIST](#) : uint32\_t { [TRIG\\_ACT\\_RISING\\_EDGE](#), [TRIG\\_ACT\\_FALLING\\_EDGE](#), [TRIG\\_ACT\\_ANY\\_EDGE](#), [TRIG\\_ACT\\_LEVEL\\_HIGH](#), [TRIG\\_ACT\\_LEVEL\\_LOW](#) }
- enum [TY\\_GAIN\\_SEL\\_LIST](#) : uint32\_t { [GAIN\\_SEL\\_ANALOG\\_ALL](#), [GAIN\\_SEL\\_DIGITAL\\_ALL](#), [GAIN\\_SEL\\_DIGITALRED](#), [GAIN\\_SEL\\_DIGITALGREEN](#), [GAIN\\_SEL\\_DIGITALBLUE](#) }
- enum [TY\\_PIX\\_FMT\\_LIST](#) : uint32\_t { [CSIMono10P](#) = 0x810A0046, [CSIBayerGB10P](#) = 0x810A0054, [CSIBayerBG10P](#) = 0x810A0052, [CSIBayerGR10P](#) = 0x810A0056, [CSIBayerRG10P](#) = 0x810A0058, [CSIMono12P](#) = 0x810C0047, [CSIBayerGB12P](#) = 0x810C0055, [CSIBayerBG12P](#) = 0x810C0053, [CSIBayerGR12P](#) = 0x810C0057, [CSIBayerRG12P](#) = 0x810C0059, [CSIMono14P](#) = 0x810E0104, [CSIBayerGB14P](#) = 0x810E0107, [CSIBayerBG14P](#) = 0x810E0108, [CSIBayerGR14P](#) = 0x810E0105, [CSIBayerRG14P](#) = 0x810E0106, [JPEG](#) = 0x82180015, [TofIR\\_FourGroup\\_Mono16](#) = 0x81400016 }

- enum `TY_BIN_H_LIST` : `uint32_t` { `BIN_H1` = 1, `BIN_H2`, `BIN_H3`, `BIN_H4` }
- enum `TY_BIN_V_LIST` : `uint32_t` { `BIN_V1` = 1, `BIN_V2`, `BIN_V3`, `BIN_V4` }
- enum `TY_DEPTH_TOF_QUALITY_LIST` : `uint32_t` { `DEPTH_TOF_QUALITY_LOW` = 1, `DEPTH_TOF_QUALITY_MEDIUM` = 2, `DEPTH_TOF_QUALITY_HIGH` = 4 }
- enum `TY_GEV_CCP_LIST` : `uint32_t` { `GEV_CCP_OPEN`, `GEV_CCP_EXCLUSIVE`, `GEV_CCP_CONTROL` }
- enum `TY_USER_SET_SEL_LIST` : `uint32_t` { `USER_SET_SEL_DEFAULT0`, `USER_SET_SEL_DEFAULT1`, `USER_SET_SEL_DEFAULT2`, `USER_SET_SEL_DEFAULT3`, `USER_SET_SEL_DEFAULT4`, `USER_SET_SEL_DEFAULT5`, `USER_SET_SEL_DEFAULT6`, `USER_SET_SEL_DEFAULT7`, `USER_SET_SEL_USER_SET0`, `USER_SET_SEL_USER_SET1`, `USER_SET_SEL_USER_SET2`, `USER_SET_SEL_USER_SET3`, `USER_SET_SEL_USER_SET4`, `USER_SET_SEL_USER_SET5`, `USER_SET_SEL_USER_SET6`, `USER_SET_SEL_USER_SET7` }
- enum `TY_SRC_SEL_LIST` : `uint32_t` { `SRC_SEL_DEPTH`, `SRC_SEL_TEXTURE`, `SRC_SEL_LEFT`, `SRC_SEL_RIGHT` }
- enum `TY_LIGHT_SEL_LIST` : `uint32_t` { `LIGHT_SEL_LASER_0`, `LIGHT_SEL_FLOOD_0`, `LIGHT_SEL_TEXTURE_FLOOD` }

### 7.4.1 Detailed Description

Camera feature enumeration list.

This file contains all feature names and their detailed descriptions extracted from PercipioStdGenICam.xml Integrates XML file content from both PMD and SWIFT versions

### 7.4.2 Enumeration Type Documentation

#### 7.4.2.1 TY\_ACQ\_MODE\_LIST

```
enum TY_ACQ_MODE_LIST : uint32_t
```

Enumerator

|                                    |                                |
|------------------------------------|--------------------------------|
| <code>ACQ_MODE_SINGLE_FRAME</code> | Single frame acquisition mode. |
| <code>ACQ_MODE_MULTI_FRAME</code>  | Multi frame acquisition mode.  |
| <code>ACQ_MODE_CONTINUOUS</code>   | Continuous acquisition mode.   |

Definition at line 261 of file TYFeatureList.h.

#### 7.4.2.2 TY\_BIN\_H\_LIST

```
enum TY_BIN_H_LIST : uint32_t
```

**Enumerator**

|        |                              |
|--------|------------------------------|
| BIN_H1 | Horizontal binning factor 1. |
| BIN_H2 | Horizontal binning factor 2. |
| BIN_H3 | Horizontal binning factor 3. |
| BIN_H4 | Horizontal binning factor 4. |

Definition at line 544 of file TYFeatureList.h.

**7.4.2.3 TY\_BIN\_V\_LIST**

```
enum TY_BIN_V_LIST : uint32_t
```

**Enumerator**

|        |                            |
|--------|----------------------------|
| BIN_V1 | Vertical binning factor 1. |
| BIN_V2 | Vertical binning factor 2. |
| BIN_V3 | Vertical binning factor 3. |
| BIN_V4 | Vertical binning factor 4. |

Definition at line 567 of file TYFeatureList.h.

**7.4.2.4 TY\_DEPTH\_TOF\_QUALITY\_LIST**

```
enum TY_DEPTH_TOF_QUALITY_LIST : uint32_t
```

**Enumerator**

|                          |                       |
|--------------------------|-----------------------|
| DEPTH_TOF_QUALITY_LOW    | Low depth quality.    |
| DEPTH_TOF_QUALITY_MEDIUM | Medium depth quality. |
| DEPTH_TOF_QUALITY_HIGH   | High depth quality.   |

Definition at line 625 of file TYFeatureList.h.

**7.4.2.5 TY\_DEV\_CHARSET\_LIST**

```
enum TY_DEV_CHARSET_LIST : uint32_t
```

**Enumerator**

|                   |                                 |
|-------------------|---------------------------------|
| DEV_CHARSET_UTF8  | Device use UTF8 character set.  |
| DEV_CHARSET_ASCII | Device use ASCII character set. |
| CII               |                                 |



Definition at line 130 of file TYFeatureList.h.

#### 7.4.2.6 TY\_DEV\_COMPRESS\_LIST

```
enum TY_DEV_COMPRESS_LIST : uint32_t
```

##### Enumerator

|                   |                           |
|-------------------|---------------------------|
| DEV_COMPRESS_NONE | No compression.           |
| DEV_COMPRESS_HW   | LZ4 hardware compression. |

Definition at line 241 of file TYFeatureList.h.

#### 7.4.2.7 TY\_DEV\_LINK\_HB\_MODE\_LIST

```
enum TY_DEV_LINK_HB_MODE_LIST : uint32_t
```

##### Enumerator

|                 |                             |
|-----------------|-----------------------------|
| DEV_LINK_HB_ON  | Speed of the selected link. |
| DEV_LINK_HB_OFF | Speed of the selected link. |

Definition at line 85 of file TYFeatureList.h.

#### 7.4.2.8 TY\_DEV\_STREAM\_ASYNC\_LIST

```
enum TY_DEV_STREAM_ASYNC_LIST : uint32_t
```

##### Enumerator

|                              |                                                                             |
|------------------------------|-----------------------------------------------------------------------------|
| DEV_STREAM_ASYNC_OFF         | None of the data streams are output asynchronously.                         |
| DEV_STREAM_ASYNC_DEPTH       | The acquisition of range (distance) data is output asynchronously.          |
| DEV_STREAM_ASYNC_RGB         | The acquisition of intensity data is output asynchronously.                 |
| DEV_STREAM_ASYNC_DEPTHANDRGB | Intensity and range (distance) data are acquired and output asynchronously. |
| DEV_STREAM_ASYNC_ALL         | All of the data streams are output asynchronously.                          |

Definition at line 187 of file TYFeatureList.h.

#### 7.4.2.9 TY\_DEV\_STREAM\_CH\_TYPE\_LIST

```
enum TY_DEV_STREAM_CH_TYPE_LIST : uint32_t
```

##### Enumerator

|                           |                                  |
|---------------------------|----------------------------------|
| DEV_STREAM_CH_TRANSMITTER | Data stream transmitter channel. |
| DEV_STREAM_CH_RECEIVER    | Data stream receiver channel.    |

Definition at line 108 of file TYFeatureList.h.

#### 7.4.2.10 TY\_DEV\_TEMP\_SEL\_LIST

```
enum TY_DEV_TEMP_SEL_LIST : uint32_t
```

##### Enumerator

|                            |                                            |
|----------------------------|--------------------------------------------|
| DEV_TEMP_SEL_MAINBOARD     | Temperature of the mainboard.              |
| DEV_TEMP_SEL_LEFT          | Temperature of the left binocular module.  |
| DEV_TEMP_SEL_RIGHT         | Temperature of the right binocular module. |
| DEV_TEMP_SEL_TEXTURE       | Temperature of the texture module.         |
| DEV_TEMP_SEL_LASER         | Temperature of the laser.                  |
| DEV_TEMP_SEL_IRFLASH       | Temperature of the ir flash.               |
| DEV_TEMP_SEL_TEXTURE_FLASH | Temperature of the Texture flash.          |
| DEV_TEMP_SEL_CPUCORE       | Temperature of the CPU core.               |

Definition at line 153 of file TYFeatureList.h.

#### 7.4.2.11 TY\_DEV\_TIME\_SYNC\_LIST

```
enum TY_DEV_TIME_SYNC_LIST : uint32_t
```

##### Enumerator

|                          |                             |
|--------------------------|-----------------------------|
| DEV_TIME_SYNC_NONE       | No synchronization.         |
| DEV_TIME_SYNC_HOST       | Host synchronization.       |
| DEV_TIME_SYNC_NTP        | NTP synchronization.        |
| DEV_TIME_SYNC_PTP_SLAVE  | PTP slave synchronization.  |
| DEV_TIME_SYNC_CAN        | CAN synchronization.        |
| DEV_TIME_SYNC_PTP_MASTER | PTP master synchronization. |

Definition at line 212 of file TYFeatureList.h.

#### 7.4.2.12 TY\_DEV\_TYPE\_LIST

```
enum TY_DEV_TYPE_LIST : uint32_t
```

##### Enumerator

|                      |                                                   |
|----------------------|---------------------------------------------------|
| DEV_TYPE_TRANSMITTER | Data stream transmitter device.                   |
| DEV_TYPE_RECEIVER    | Data stream receiver device.                      |
| DEV_TYPE_TRANSCEIVER | Data stream receiver and transmitter device.      |
| DEV_TYPE_PERIPHERAL  | Controlable device (with no data stream handling) |

Definition at line 45 of file TYFeatureList.h.

#### 7.4.2.13 TY\_GAIN\_SEL\_LIST

```
enum TY_GAIN_SEL_LIST : uint32_t
```

##### Enumerator

|                       |                     |
|-----------------------|---------------------|
| GAIN_SEL_ANALOG_ALL   | All analog gains.   |
| GAIN_SEL_DIGITAL_ALL  | All digital gains.  |
| GAIN_SEL_DIGITALRED   | Digital red gain.   |
| GAIN_SEL_DIGITALGREEN | Digital green gain. |
| GAIN_SEL_DIGITALBLUE  | Digital blue gain.  |

Definition at line 423 of file TYFeatureList.h.

#### 7.4.2.14 TY\_GEV\_CCP\_LIST

```
enum TY_GEV_CCP_LIST : uint32_t
```

##### Enumerator

|                   |                   |
|-------------------|-------------------|
| GEV_CCP_OPEN      | Open Access.      |
| GEV_CCP_EXCLUSIVE | Exclusive Access. |
| GEV_CCP_CONTROL   | Control Access.   |

Definition at line 685 of file TYFeatureList.h.

#### 7.4.2.15 TY\_LIGHT\_SEL\_LIST

```
enum TY_LIGHT_SEL_LIST : uint32_t
```

## Enumerator

|                         |                                                             |
|-------------------------|-------------------------------------------------------------|
| LIGHT_SEL_LASER_0       | Laser controller will effects depth.                        |
| LIGHT_SEL_FLOOD_0       | Flood controller always effects Left/Right cam when needed. |
| LIGHT_SEL_TEXTURE_FLOOD | Flood controller effects Texture cam.                       |

Definition at line 813 of file TYFeatureList.h.

## 7.4.2.16 TY\_PIX\_FMT\_LIST

```
enum TY_PIX_FMT_LIST : uint32_t
```

## Enumerator

|                        |                                           |
|------------------------|-------------------------------------------|
| CSIMono10P             | Camera Serial Interface Packed Mono10.    |
| CSIBayerGB10P          | Camera Serial Interface Packed BayerGB10. |
| CSIBayerBG10P          | Camera Serial Interface Packed BayerBG10. |
| CSIBayerGR10P          | Camera Serial Interface Packed BayerGR10. |
| CSIBayerRG10P          | Camera Serial Interface Packed BayerRG10. |
| CSIMono12P             | Camera Serial Interface Packed Mono12.    |
| CSIBayerGB12P          | Camera Serial Interface Packed BayerGB12. |
| CSIBayerBG12P          | Camera Serial Interface Packed BayerBG12. |
| CSIBayerGR12P          | Camera Serial Interface Packed BayerGR12. |
| CSIBayerRG12P          | Camera Serial Interface Packed BayerRG12. |
| CSIMono14P             | Camera Serial Interface Packed Mono14.    |
| CSIBayerGB14P          | Camera Serial Interface Packed BayerGB14. |
| CSIBayerBG14P          | Camera Serial Interface Packed BayerBG14. |
| CSIBayerGR14P          | Camera Serial Interface Packed BayerGR14. |
| CSIBayerRG14P          | Camera Serial Interface Packed BayerRG14. |
| JPEG                   | JPEG encoding format.                     |
| TofIR_FourGroup_Mono16 | TofIR_FourGroup_Mono16 encoding format.   |

Definition at line 495 of file TYFeatureList.h.

## 7.4.2.17 TY\_SRC\_SEL\_LIST

```
enum TY_SRC_SEL_LIST : uint32_t
```

## Enumerator

|                 |                         |
|-----------------|-------------------------|
| SRC_SEL_DEPTH   | Depth source.           |
| SRC_SEL_TEXTURE | Texture source.         |
| SRC_SEL_LEFT    | Left binocular source.  |
| SRC_SEL_RIGHT   | Right binocular source. |

Definition at line 779 of file TYFeatureList.h.

#### 7.4.2.18 TY\_TRIG\_ACT\_LIST

```
enum TY_TRIG_ACT_LIST : uint32_t
```

##### Enumerator

|                       |                                  |
|-----------------------|----------------------------------|
| TRIG_ACT_RISING_EDGE  | Rising edge trigger activation.  |
| TRIG_ACT_FALLING_EDGE | Falling edge trigger activation. |
| TRIG_ACT_ANY_EDGE     | Any edge trigger activation.     |
| TRIG_ACT_LEVEL_HIGH   | High level trigger activation.   |
| TRIG_ACT_LEVEL_LOW    | Low level trigger activation.    |

Definition at line 391 of file TYFeatureList.h.

#### 7.4.2.19 TY\_TRIG\_MODE\_LIST

```
enum TY_TRIG_MODE_LIST : uint32_t
```

##### Enumerator

|               |                                |
|---------------|--------------------------------|
| TRIG_MODE_OFF | Disables the selected trigger. |
| TRIG_MODE_ON  | Enable the selected trigger.   |

Definition at line 337 of file TYFeatureList.h.

#### 7.4.2.20 TY\_TRIG\_SEL\_LIST

```
enum TY_TRIG_SEL_LIST : uint32_t
```

##### Enumerator

|                                  |                                 |
|----------------------------------|---------------------------------|
| TRIG_SEL_FRAME_ACQUISITIONSTART  | Trigger for acquisition start.  |
| TRIG_SEL_FRAME_ACQUISITIONEND    | Trigger for acquisition end.    |
| TRIG_SEL_FRAME_ACQUISITIONACTIVE | Trigger for acquisition active. |
| TRIG_SEL_FRAME_FRAMESTART        | Trigger for frame start.        |
| TRIG_SEL_FRAME_FRAMEEND          | Trigger for frame end.          |
| TRIG_SEL_FRAME_FRAMEACTIVE       | Trigger for frame active.       |
| TRIG_SEL_FRAME_FRAMEBURSTSTART   | Trigger for frame burst start.  |
| TRIG_SEL_FRAME_FRAMEBURSTEND     | Trigger for frame burst end.    |

## Enumerator

|                                 |                                 |
|---------------------------------|---------------------------------|
| TRIG_SEL_FRAME_FRAMEBURSTACTIVE | Trigger for frame burst active. |
| TRIG_SEL_FRAME_LINESTART        | Trigger for line start.         |
| TRIG_SEL_FRAME_EXPOSURESTART    | Trigger for exposure start.     |
| TRIG_SEL_FRAME_EXPOSUREEND      | Trigger for exposure end.       |
| TRIG_SEL_FRAME_EXPOSUREACTIVE   | Trigger for exposure active.    |

Definition at line 296 of file TYFeatureList.h.

#### 7.4.2.21 TY\_TRIG\_SRC\_LIST

```
enum TY_TRIG_SRC_LIST : uint32_t
```

## Enumerator

|                   |                          |
|-------------------|--------------------------|
| TRIG_SRC_LINE0    | Trigger source line 0.   |
| TRIG_SRC_LINE1    | Trigger source line 1.   |
| TRIG_SRC_LINE2    | Trigger source line 2.   |
| TRIG_SRC_LINE3    | Trigger source line 3.   |
| TRIG_SRC_LINE4    | Trigger source line 4.   |
| TRIG_SRC_LINE5    | Trigger source line 5.   |
| TRIG_SRC_LINE6    | Trigger source line 6.   |
| TRIG_SRC_LINE7    | Trigger source line 7.   |
| TRIG_SRC_SOFTWARE | Software trigger source. |

Definition at line 358 of file TYFeatureList.h.

#### 7.4.2.22 TY\_USER\_SET\_SEL\_LIST

```
enum TY_USER_SET_SEL_LIST : uint32_t
```

## Enumerator

|                        |                     |
|------------------------|---------------------|
| USER_SET_SEL_DEFAULT0  | Default user set 0. |
| USER_SET_SEL_DEFAULT1  | Default user set 1. |
| USER_SET_SEL_DEFAULT2  | Default user set 2. |
| USER_SET_SEL_DEFAULT3  | Default user set 3. |
| USER_SET_SEL_DEFAULT4  | Default user set 4. |
| USER_SET_SEL_DEFAULT5  | Default user set 5. |
| USER_SET_SEL_DEFAULT6  | Default user set 6. |
| USER_SET_SEL_DEFAULT7  | Default user set 7. |
| USER_SET_SEL_USER_SET0 | User set 0.         |
| USER_SET_SEL_USER_SET1 | User set 1.         |

## Enumerator

|                        |             |
|------------------------|-------------|
| USER_SET_SEL_USER_SET2 | User set 2. |
| USER_SET_SEL_USER_SET3 | User set 3. |
| USER_SET_SEL_USER_SET4 | User set 4. |
| USER_SET_SEL_USER_SET5 | User set 5. |
| USER_SET_SEL_USER_SET6 | User set 6. |
| USER_SET_SEL_USER_SET7 | User set 7. |

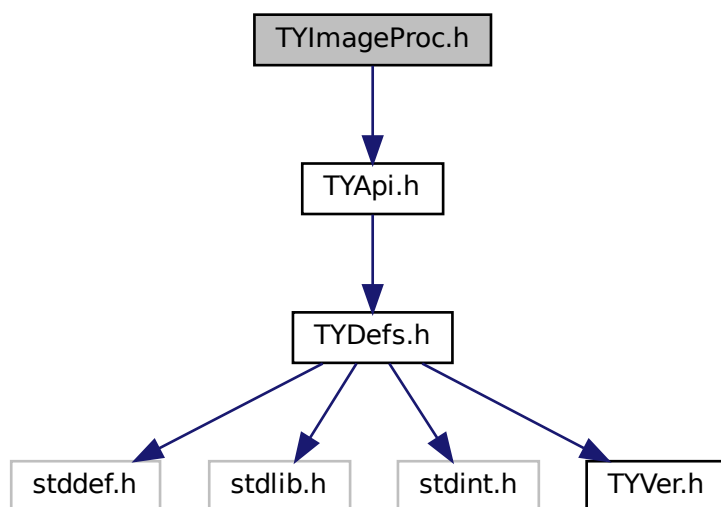
Definition at line 720 of file TYFeatureList.h.

## 7.5 TYImageProc.h File Reference

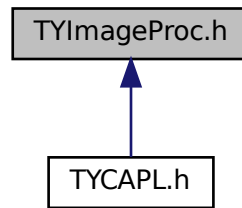
Image post-process API.

```
#include "TYApi.h"
```

Include dependency graph for TYImageProc.h:



This graph shows which files directly or indirectly include this file:



## Classes

- struct [TYImageInfo](#)
- struct [TYDecodeResult](#)
- struct [DepthSpeckleFilterParameters](#)
- struct [DepthInpainterParameters](#)
- struct [DepthEnhenceParameters](#)

## Macros

- #define **TY\_DECODE\_API** TY\_EXTC TY\_EXPORT TYDecodeError TY\_STDC
- #define **DepthSpeckleFilterParameters\_Initializer** {150, 64, 20}
- #define **DepthInpainterParameters\_Initializer** {5, 50}
- #define **DepthEnhenceParameters\_Initializer** {10, 20, 10, 0.1f}

## Typedefs

- typedef enum TYDecodeError **TYDecodeError**
- typedef struct [TYImageInfo](#) **TYImageInfo**
- typedef enum TYOutputFormat **TYOutputFormat**
- typedef struct [TYDecodeResult](#) **TYDecodeResult**

## Enumerations

- enum **TYDecodeError** {  
**TY\_DECODE\_SUCCESS** = 0, **TY\_DECODE\_NO\_DECODE\_NEEDED**, **TY\_DECODE\_ERROR\_INVALID\_↵**  
**\_PARAM**, **TY\_DECODE\_ERROR\_FORMAT\_MISMATCH**,  
**TY\_DECODE\_ERROR\_UNSUPPORTED\_FORMAT**, **TY\_DECODE\_ERROR\_BUFFER\_TOO\_SMALL**, **T↵**  
**Y\_DECODE\_ERROR\_DATA\_ERROR**, **TY\_DECODE\_ERROR\_CREATE\_WINDOW**,  
**TY\_DECODE\_ERROR\_DISPLAY\_IMAGE** }
- enum **TYOutputFormat** { **TY\_OUTPUT\_FORMAT\_AUTO** = 0, **TY\_OUTPUT\_FORMAT\_MONO8**, **TY\_O↵**  
**UTPUT\_FORMAT\_MONO16**, **TY\_OUTPUT\_FORMAT\_BGR** }



## Functions

- TY\_CAPI [TYGetImageAlgorithmVersion](#) (TY\_VERSION\_INFO \*version)  
*Get current library version.*
- TY\_CAPI [TYImageProcesAcceEnable](#) (bool en)  
*Image processing acceleration switch.*
- TY\_DECODE\_API [TYGetDecodeTargetPixFmt](#) (const TYImageInfo \*input, TYPixFmt \*outFmt, const TY↵  
OutputFormat config=TY\_OUTPUT\_FORMAT\_AUTO)  
*Get the target pixel format for image decoding.*
- TY\_DECODE\_API [TYGetDecodeBufferSize](#) (const TYImageInfo \*input, uint32\_t \*totalSize, const TY↵  
OutputFormat config=TY\_OUTPUT\_FORMAT\_AUTO)  
*Calculate the required buffer size for image decoding.*
- TY\_DECODE\_API [TYDecodeImage](#) (const TYImageInfo \*input, const TYOutputFormat config, void  
\*outputBuffer, uint32\_t outputBufferSize, TYDecodeResult \*result)  
*Decode an image from compressed format to specified output format.*
- TY\_CAPI [TYUndistortImage](#) (const TY\_CAMERA\_CALIB\_INFO \*srcCalibInfo, const TY\_IMAGE\_DATA  
\*srcImage, const TY\_CAMERA\_INTRINSIC \*cameraNewIntrinsic, TY\_IMAGE\_DATA \*dstImage, const  
TYLensOpticalType type=TY\_LENS\_PINHOLE)  
*Do image undistortion, only support TYPixelFormatMono8, TYPixelFormatMono16, TYPixelFormatRGB8, TYPixel↵  
FormatBGR8, TYPixelFormatCoord3D\_C16.*
- TY\_CAPI [TYUndistortImage2](#) (const TY\_CAMERA\_CALIB\_INFO \*calib\_info, const TY\_IMAGE\_DATA  
\*srcImage, const TY\_CAMERA\_ROTATION \*cameraRotation, const TY\_CAMERA\_INTRINSIC \*camera↵  
NewIntrinsic, TY\_IMAGE\_DATA \*dstImage, const TYLensOpticalType type=TY\_LENS\_PINHOLE)  
*Do image undistortion, only support TYPixelFormatMono8, TYPixelFormatMono16, TYPixelFormatRGB8, TYPixel↵  
FormatBGR8, TYPixelFormatCoord3D\_C16.*
- TY\_CAPI [TYDepthSpeckleFilter](#) (TY\_IMAGE\_DATA \*depthImage, const [DepthSpeckleFilterParameters](#)  
\*param, const TY\_CAMERA\_CALIB\_INFO \*calib\_data=NULLptr, const float depth\_scale\_unit=1.f)  
*Remove speckles on depth image.*
- TY\_CAPI [TYDepthImageInpainter](#) (TY\_IMAGE\_DATA \*depth, const [DepthInpainterParameters](#) \*param)  
*Repair invalid pixels in depth image, filling small holes.*
- TY\_CAPI [TYDepthEnhanceFilter](#) (const TY\_IMAGE\_DATA \*depthImages, int imageNum, TY\_IMAGE\_DATA  
\*guide, TY\_IMAGE\_DATA \*output, const [DepthEnhanceParameters](#) \*param)  
*Remove speckles on depth image.*

### 7.5.1 Detailed Description

Image post-process API.

Copyright

Copyright(C)2016-2018 Percipio All Rights Reserved

### 7.5.2 Function Documentation

#### 7.5.2.1 TYDecodeImage()

```
TY_DECODE_API TYDecodeImage (
 const TYImageInfo * input,
 const TYOutputFormat config,
 void * outputBuffer,
 uint32_t outputBufferSize,
 TYDecodeResult * result)
```

Decode an image from compressed format to specified output format.

**Parameters**

|                         |                                                          |
|-------------------------|----------------------------------------------------------|
| <i>input</i>            | Pointer to input image information                       |
| <i>config</i>           | Decode configuration parameters specifying output format |
| <i>outputBuffer</i>     | User-allocated output buffer for decoded image data      |
| <i>outputBufferSize</i> | Size of the output buffer in bytes                       |
| <i>result</i>           | Output parameter containing decode result information    |

**Returns**

int Error code, TY\_DECODE\_SUCCESS (0) indicates success

**7.5.2.2 TYDepthEnhenceFilter()**

```
TY_CAPI TYDepthEnhenceFilter (
 const TY_IMAGE_DATA * depthImages,
 int imageNum,
 TY_IMAGE_DATA * guide,
 TY_IMAGE_DATA * output,
 const DepthEnhenceParameters * param)
```

Remove speckles on depth image.

**Parameters**

|         |                   |                               |
|---------|-------------------|-------------------------------|
| in      | <i>depthImage</i> | Pointer to depth image array. |
| in      | <i>imageNum</i>   | Depth image array size.       |
| in, out | <i>guide</i>      | Guide image.                  |
| out     | <i>output</i>     | Output depth image.           |
| in      | <i>param</i>      | Algorithm parameters.         |

**Return values**

|                                    |                                                          |
|------------------------------------|----------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                 |
| <i>TY_STATUS_NULL_POINTER</i>      | Any depthImage, param, output or output->buffer is NULL. |
| <i>TY_STATUS_INVALID_PARAMETER</i> | imageNum >= 11 or imageNum <= 0, or any image invalid    |
| <i>TY_STATUS_OUT_OF_MEMORY</i>     | Output image not suitable.                               |

**7.5.2.3 TYDepthImageInpainter()**

```
TY_CAPI TYDepthImageInpainter (
 TY_IMAGE_DATA * depth,
 const DepthInpainterParameters * param)
```

Repair invalid pixels in depth image, filling small holes.

## Parameters

|         |              |                              |
|---------|--------------|------------------------------|
| in, out | <i>depth</i> | Depth image to be processed. |
| in      | <i>param</i> | Algorithm parameters.        |

## Return values

|                                    |                                                       |
|------------------------------------|-------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Success.                                              |
| <i>TY_STATUS_NULL_POINTER</i>      | depth, param or depth->buffer is NULL.                |
| <i>TY_STATUS_INVALID_PARAMETER</i> | param->kernel_size or param->kernel_size out of range |

## 7.5.2.4 TYDepthSpeckleFilter()

```

TY_CAPI TYDepthSpeckleFilter (
 TY_IMAGE_DATA * depthImage,
 const DepthSpeckleFilterParameters * param,
 const TY_CAMERA_CALIB_INFO * calib_data = nullptr,
 const float depth_scale_unit = 1.f)

```

Remove speckles on depth image.

## Parameters

|         |                   |                              |
|---------|-------------------|------------------------------|
| in, out | <i>depthImage</i> | Depth image to be processed. |
| in      | <i>param</i>      | Algorithm parameters.        |
| in      | <i>calib_data</i> | Image calibration data.      |

## Return values

|                                    |                                                              |
|------------------------------------|--------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                     |
| <i>TY_STATUS_NULL_POINTER</i>      | Any depth, param or depth->buffer is NULL.                   |
| <i>TY_STATUS_INVALID_PARAMETER</i> | param->max_speckle_size <= 0 or param->max_speckle_diff <= 0 |

## 7.5.2.5 TYGetDecodeBufferSize()

```

TY_DECODE_API TYGetDecodeBufferSize (
 const TYImageInfo * input,
 uint32_t * totalSize,
 const TYOutputFormat config = TY_OUTPUT_FORMAT_AUTO)

```

Calculate the required buffer size for image decoding.

This function calculates the required buffer size (in bytes) for storing the decoded image based on the input image format, dimensions, and the specified output configuration. The calculation takes into account the target pixel format, image dimensions, and memory alignment requirements (4-byte aligned stride).

## Parameters

|                  |                                                                    |
|------------------|--------------------------------------------------------------------|
| <i>input</i>     | Pointer to input image information                                 |
| <i>totalSize</i> | Output parameter for required buffer size in bytes                 |
| <i>config</i>    | Decode configuration parameters, defaults to TY_OUTPUT_FORMAT_AUTO |

## Returns

TYDecodeError

- TY\_DECODE\_SUCCESS: Successfully calculated buffer size
- TY\_DECODE\_ERROR\_INVALID\_PARAM: Invalid input parameters (null pointers, zero dimensions, etc.)
- TY\_DECODE\_ERROR\_UNSUPPORTED\_FORMAT: Unsupported output format or unable to determine format
- TY\_DECODE\_ERROR\_FORMAT\_MISMATCH: Incompatible input and output formats
- TY\_DECODE\_NO\_DECODE\_NEEDED: No decoding needed, output format same as input

## 7.5.2.6 TYGetDecodeTargetPixFmt()

```
TY_DECODE_API TYGetDecodeTargetPixFmt (
 const TYImageInfo * input,
 TYPixFmt * outFmt,
 const TYOutputFormat config = TY_OUTPUT_FORMAT_AUTO)
```

Get the target pixel format for image decoding.

This function determines the target pixel format for decoding based on the input image format and the specified output configuration. It can be used to query the pixel format that will be produced by the decode operation before actually performing the decode.

## Parameters

|               |                                                                    |
|---------------|--------------------------------------------------------------------|
| <i>input</i>  | Pointer to input image information                                 |
| <i>outFmt</i> | Output parameter for the target pixel format                       |
| <i>config</i> | Decode configuration parameters, defaults to TY_OUTPUT_FORMAT_AUTO |

## Returns

TYDecodeError

- TY\_DECODE\_SUCCESS: Successfully determined target format
- TY\_DECODE\_NO\_DECODE\_NEEDED: No decoding needed, output format same as input
- TY\_DECODE\_ERROR\_INVALID\_PARAM: Invalid input parameters
- TY\_DECODE\_ERROR\_UNSUPPORTED\_FORMAT: Unsupported output format
- TY\_DECODE\_ERROR\_FORMAT\_MISMATCH: Incompatible input and output formats

### 7.5.2.7 TYGetImageAlgorithmVersion()

```
TY_CAPI TYGetImageAlgorithmVersion (
 TY_VERSION_INFO * version)
```

Get current library version.

#### Parameters

|     |                |                                   |
|-----|----------------|-----------------------------------|
| out | <i>version</i> | Version information to be filled. |
|-----|----------------|-----------------------------------|

#### Return values

|                               |                                                     |
|-------------------------------|-----------------------------------------------------|
| <i>TY_STATUS_OK</i>           | Succeed.                                            |
| <i>TY_STATUS_NULL_POINTER</i> | TYGetImageAlgorithmVersion called with NULL pointer |

### 7.5.2.8 TYImageProcesAcceEnable()

```
TY_CAPI TYImageProcesAcceEnable (
 bool en)
```

Image processing acceleration switch.

#### Parameters

|    |           |                                          |
|----|-----------|------------------------------------------|
| in | <i>en</i> | Enable image process acceleration switch |
|----|-----------|------------------------------------------|

### 7.5.2.9 TYUndistortImage()

```
TY_CAPI TYUndistortImage (
 const TY_CAMERA_CALIB_INFO * srcCalibInfo,
 const TY_IMAGE_DATA * srcImage,
 const TY_CAMERA_INTRINSIC * cameraNewIntrinsic,
 TY_IMAGE_DATA * dstImage,
 const TYLensOpticalType type = TY_LENS_PINHOLE)
```

Do image undistortion, only support TYPixelFormatMono8, TYPixelFormatMono16, TYPixelFormatRGB8, TYPixelFormatBGR8, TYPixelFormatCoord3D\_C16.

#### Parameters

|     |                           |                                                                                             |
|-----|---------------------------|---------------------------------------------------------------------------------------------|
| in  | <i>srcCalibInfo</i>       | Image calibration data.                                                                     |
| in  | <i>srcImage</i>           | Source image.                                                                               |
| in  | <i>cameraNewIntrinsic</i> | Expected new image intrinsic, will use srcCalibInfo for new image intrinsic if set to NULL. |
| out | <i>dstImage</i>           | Output image.                                                                               |

## Return values

|                                    |                                                                                                           |
|------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                  |
| <i>TY_STATUS_NULL_POINTER</i>      | Any srcCalibInfo, srcImage, dstImage, srcImage->buffer, dstImage->buffer is NULL.                         |
| <i>TY_STATUS_INVALID_PARAMETER</i> | Invalid srcImage->width, srcImage->height, dstImage->width, dstImage->height or unsupported pixel format. |

## 7.5.2.10 TYUndistortImage2()

```

TY_CAPI TYUndistortImage2 (
 const TY_CAMERA_CALIB_INFO * calib_info,
 const TY_IMAGE_DATA * srcImage,
 const TY_CAMERA_ROTATION * cameraRotation,
 const TY_CAMERA_INTRINSIC * cameraNewIntrinsic,
 TY_IMAGE_DATA * dstImage,
 const TYLensOpticalType type = TY_LENS_PINHOLE)

```

Do image undistortion, only support TYPixelFormatMono8, TYPixelFormatMono16, TYPixelFormatRGB8, TY↵ PixelFormatBGR8, TYPixelFormatCoord3D\_C16.

## Parameters

|     |                           |                                                                                             |
|-----|---------------------------|---------------------------------------------------------------------------------------------|
| in  | <i>srcCalibInfo</i>       | Image calibration data.                                                                     |
| in  | <i>srcImage</i>           | Source image.                                                                               |
| in  | <i>cameraRotation</i>     | Camera rotation parameter for image orientation adjustment.                                 |
| in  | <i>cameraNewIntrinsic</i> | Expected new image intrinsic, will use srcCalibInfo for new image intrinsic if set to NULL. |
| out | <i>dstImage</i>           | Output image.                                                                               |

## Return values

|                                    |                                                                                                           |
|------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                  |
| <i>TY_STATUS_NULL_POINTER</i>      | Any srcCalibInfo, srcImage, dstImage, srcImage->buffer, dstImage->buffer is NULL.                         |
| <i>TY_STATUS_INVALID_PARAMETER</i> | Invalid srcImage->width, srcImage->height, dstImage->width, dstImage->height or unsupported pixel format. |

# Index

ACQ\_MODE\_CONTINUOUS  
    TYFeatureList.h, [263](#)  
ACQ\_MODE\_MULTI\_FRAME  
    TYFeatureList.h, [263](#)  
ACQ\_MODE\_SINGLE\_FRAME  
    TYFeatureList.h, [263](#)  
AlgVersion, [87](#)

BIN\_H1  
    TYFeatureList.h, [264](#)  
BIN\_H2  
    TYFeatureList.h, [264](#)  
BIN\_H3  
    TYFeatureList.h, [264](#)  
BIN\_H4  
    TYFeatureList.h, [264](#)  
BIN\_V1  
    TYFeatureList.h, [264](#)  
BIN\_V2  
    TYFeatureList.h, [264](#)  
BIN\_V3  
    TYFeatureList.h, [264](#)  
BIN\_V4  
    TYFeatureList.h, [264](#)

CSIBayerBG10P  
    TYFeatureList.h, [268](#)  
CSIBayerBG12P  
    TYFeatureList.h, [268](#)  
CSIBayerBG14P  
    TYFeatureList.h, [268](#)  
CSIBayerGB10P  
    TYFeatureList.h, [268](#)  
CSIBayerGB12P  
    TYFeatureList.h, [268](#)  
CSIBayerGB14P  
    TYFeatureList.h, [268](#)  
CSIBayerGR10P  
    TYFeatureList.h, [268](#)  
CSIBayerGR12P  
    TYFeatureList.h, [268](#)  
CSIBayerGR14P  
    TYFeatureList.h, [268](#)  
CSIBayerRG10P  
    TYFeatureList.h, [268](#)  
CSIBayerRG12P  
    TYFeatureList.h, [268](#)  
CSIBayerRG14P  
    TYFeatureList.h, [268](#)  
CSIMono10P

    TYFeatureList.h, [268](#)  
CSIMono12P  
    TYFeatureList.h, [268](#)  
CSIMono14P  
    TYFeatureList.h, [268](#)  
customFilter  
    tjtransform, [124](#)  
  
data  
    tjtransform, [125](#)  
denom  
    tjscalingfactor, [123](#)  
DEPTH\_TOF\_QUALITY\_HIGH  
    TYFeatureList.h, [264](#)  
DEPTH\_TOF\_QUALITY\_LOW  
    TYFeatureList.h, [264](#)  
DEPTH\_TOF\_QUALITY\_MEDIUM  
    TYFeatureList.h, [264](#)  
DepthEnhanceParameters, [87](#)  
DepthInpainterParameters, [88](#)  
DepthSpeckleFilterParameters, [88](#)  
DEV\_CHARSET\_ASCII  
    TYFeatureList.h, [264](#)  
DEV\_CHARSET\_UTF8  
    TYFeatureList.h, [264](#)  
DEV\_COMPRESS\_HW  
    TYFeatureList.h, [265](#)  
DEV\_COMPRESS\_NONE  
    TYFeatureList.h, [265](#)  
DEV\_LINK\_HB\_OFF  
    TYFeatureList.h, [265](#)  
DEV\_LINK\_HB\_ON  
    TYFeatureList.h, [265](#)  
DEV\_STREAM\_ASYNC\_ALL  
    TYFeatureList.h, [265](#)  
DEV\_STREAM\_ASYNC\_DEPTH  
    TYFeatureList.h, [265](#)  
DEV\_STREAM\_ASYNC\_DEPTHANDRGB  
    TYFeatureList.h, [265](#)  
DEV\_STREAM\_ASYNC\_OFF  
    TYFeatureList.h, [265](#)  
DEV\_STREAM\_ASYNC\_RGB  
    TYFeatureList.h, [265](#)  
DEV\_STREAM\_CH\_RECEIVER  
    TYFeatureList.h, [266](#)  
DEV\_STREAM\_CH\_TRANSMITTER  
    TYFeatureList.h, [266](#)  
DEV\_TEMP\_SEL\_CPUCORE  
    TYFeatureList.h, [266](#)  
DEV\_TEMP\_SEL\_IRFLASH

TYFeatureList.h, [266](#)  
 DEV\_TEMP\_SEL\_LASER  
     TYFeatureList.h, [266](#)  
 DEV\_TEMP\_SEL\_LEFT  
     TYFeatureList.h, [266](#)  
 DEV\_TEMP\_SEL\_MAINBOARD  
     TYFeatureList.h, [266](#)  
 DEV\_TEMP\_SEL\_RIGHT  
     TYFeatureList.h, [266](#)  
 DEV\_TEMP\_SEL\_TEXTURE  
     TYFeatureList.h, [266](#)  
 DEV\_TEMP\_SEL\_TEXTURE\_FLASH  
     TYFeatureList.h, [266](#)  
 DEV\_TIME\_SYNC\_CAN  
     TYFeatureList.h, [266](#)  
 DEV\_TIME\_SYNC\_HOST  
     TYFeatureList.h, [266](#)  
 DEV\_TIME\_SYNC\_NONE  
     TYFeatureList.h, [266](#)  
 DEV\_TIME\_SYNC\_NTP  
     TYFeatureList.h, [266](#)  
 DEV\_TIME\_SYNC\_PTP\_MASTER  
     TYFeatureList.h, [266](#)  
 DEV\_TIME\_SYNC\_PTP\_SLAVE  
     TYFeatureList.h, [266](#)  
 DEV\_TYPE\_PERIPHERAL  
     TYFeatureList.h, [267](#)  
 DEV\_TYPE\_RECEIVER  
     TYFeatureList.h, [267](#)  
 DEV\_TYPE\_TRANSCEIVER  
     TYFeatureList.h, [267](#)  
 DEV\_TYPE\_TRANSMITTER  
     TYFeatureList.h, [267](#)

f16, [88](#)

GAIN\_SEL\_ANALOG\_ALL  
     TYFeatureList.h, [267](#)  
 GAIN\_SEL\_DIGITAL\_ALL  
     TYFeatureList.h, [267](#)  
 GAIN\_SEL\_DIGITALBLUE  
     TYFeatureList.h, [267](#)  
 GAIN\_SEL\_DIGITALGREEN  
     TYFeatureList.h, [267](#)  
 GAIN\_SEL\_DIGITALRED  
     TYFeatureList.h, [267](#)  
 GEV\_CCP\_CONTROL  
     TYFeatureList.h, [267](#)  
 GEV\_CCP\_EXCLUSIVE  
     TYFeatureList.h, [267](#)  
 GEV\_CCP\_OPEN  
     TYFeatureList.h, [267](#)

gz\_header\_s, [89](#)

gzFile\_s, [89](#)

h

    tjregion, [121](#)

JHUFF\_TBL, [90](#)

JPEG

    TYFeatureList.h, [268](#)

jpeg\_common\_struct, [90](#)

jpeg\_component\_info, [90](#)

jpeg\_compress\_struct, [91](#)

jpeg\_decompress\_struct, [93](#)

jpeg\_destination\_mgr, [95](#)

jpeg\_error\_mgr, [96](#)

jpeg\_marker\_struct, [96](#)

jpeg\_memory\_mgr, [97](#)

jpeg\_progress\_mgr, [98](#)

jpeg\_scan\_info, [98](#)

jpeg\_source\_mgr, [98](#)

JQUANT\_TBL, [99](#)

LIGHT\_SEL\_FLOOD\_0

    TYFeatureList.h, [268](#)

LIGHT\_SEL\_LASER\_0

    TYFeatureList.h, [268](#)

LIGHT\_SEL\_TEXTURE\_FLOOD

    TYFeatureList.h, [268](#)

nsimd\_aarch64\_vf16, [99](#)

nsimd\_aarch64\_vlf16, [99](#)

nsimd\_avx2\_vf16, [100](#)

nsimd\_avx2\_vlf16, [100](#)

nsimd\_avx512\_knl\_vf16, [100](#)

nsimd\_avx512\_knl\_vlf16, [101](#)

nsimd\_avx512\_skylake\_vf16, [101](#)

nsimd\_avx512\_skylake\_vlf16, [101](#)

nsimd\_avx\_vf16, [102](#)

nsimd\_avx\_vlf16, [102](#)

nsimd\_cpu\_vf16, [103](#)

nsimd\_cpu\_vf32, [103](#)

nsimd\_cpu\_vf64, [103](#)

nsimd\_cpu\_vf16, [104](#)

nsimd\_cpu\_vf32, [104](#)

nsimd\_cpu\_vf64, [105](#)

nsimd\_cpu\_vf8, [105](#)

nsimd\_cpu\_vlf16, [106](#)

nsimd\_cpu\_vlf32, [106](#)

nsimd\_cpu\_vlf64, [106](#)

nsimd\_cpu\_vli16, [107](#)

nsimd\_cpu\_vli32, [107](#)

nsimd\_cpu\_vli64, [108](#)

nsimd\_cpu\_vli8, [108](#)

nsimd\_cpu\_vlu16, [109](#)

nsimd\_cpu\_vlu32, [109](#)

nsimd\_cpu\_vlu64, [109](#)

nsimd\_cpu\_vlu8, [110](#)

nsimd\_cpu\_vu16, [110](#)

nsimd\_cpu\_vu32, [111](#)

nsimd\_cpu\_vu64, [111](#)

nsimd\_cpu\_vu8, [112](#)

nsimd\_neon128\_vf16, [112](#)

nsimd\_neon128\_vf64, [113](#)

nsimd\_neon128\_vlf16, [113](#)

nsimd\_neon128\_vlf64, [113](#)

nsimd\_sse2\_vf16, [114](#)



- nsimd\_sse2\_vlf16, [114](#)
- nsimd\_sse42\_vf16, [114](#)
- nsimd\_sse42\_vlf16, [115](#)
- nsimd\_vmx\_vf16, [115](#)
- nsimd\_vmx\_vf64, [115](#)
- nsimd\_vmx\_vf64, [116](#)
- nsimd\_vmx\_vlf16, [116](#)
- nsimd\_vmx\_vlf64, [116](#)
- nsimd\_vmx\_vli64, [117](#)
- nsimd\_vmx\_vlu64, [117](#)
- nsimd\_vmx\_vu64, [117](#)
- nsimd\_vsx\_vf16, [118](#)
- nsimd\_vsx\_vlf16, [118](#)
- num
  - tjscalingfactor, [123](#)
- op
  - tjtransform, [125](#)
- OperatorInfo, [118](#)
- options
  - tjtransform, [125](#)
- ParamDetail, [119](#)
- pattern\_bin\_param, [120](#)
- pattern\_gray\_param, [120](#)
- pattern\_sine\_param, [120](#)
- r
  - tjtransform, [125](#)
- SRC\_SEL\_DEPTH
  - TYFeatureList.h, [268](#)
- SRC\_SEL\_LEFT
  - TYFeatureList.h, [268](#)
- SRC\_SEL\_RIGHT
  - TYFeatureList.h, [268](#)
- SRC\_SEL\_TEXTURE
  - TYFeatureList.h, [268](#)
- TJ\_NUMCS
  - TurboJPEG, [56](#)
- TJ\_NUMERR
  - TurboJPEG, [56](#)
- TJ\_NUMPF
  - TurboJPEG, [56](#)
- TJ\_NUMSAMP
  - TurboJPEG, [56](#)
- TJ\_NUMXOP
  - TurboJPEG, [57](#)
- tjAlloc
  - TurboJPEG, [65](#)
- tjBufSize
  - TurboJPEG, [65](#)
- tjBufSizeYUV2
  - TurboJPEG, [66](#)
- tjCompress2
  - TurboJPEG, [66](#)
- tjCompressFromYUV
  - TurboJPEG, [68](#)
- tjCompressFromYUVPlanes
  - TurboJPEG, [69](#)
- TJCS
  - TurboJPEG, [61](#)
- TJCS\_CMYK
  - TurboJPEG, [62](#)
- TJCS\_GRAY
  - TurboJPEG, [62](#)
- TJCS\_RGB
  - TurboJPEG, [61](#)
- TJCS\_YCbCr
  - TurboJPEG, [62](#)
- TJCS\_YCCK
  - TurboJPEG, [62](#)
- tjDecodeYUV
  - TurboJPEG, [71](#)
- tjDecodeYUVPlanes
  - TurboJPEG, [71](#)
- tjDecompress2
  - TurboJPEG, [72](#)
- tjDecompressHeader3
  - TurboJPEG, [73](#)
- tjDecompressToYUV2
  - TurboJPEG, [74](#)
- tjDecompressToYUVPlanes
  - TurboJPEG, [75](#)
- tjDestroy
  - TurboJPEG, [76](#)
- tjEncodeYUV3
  - TurboJPEG, [76](#)
- tjEncodeYUVPlanes
  - TurboJPEG, [77](#)
- TJERR
  - TurboJPEG, [62](#)
- TJERR\_FATAL
  - TurboJPEG, [62](#)
- TJERR\_WARNING
  - TurboJPEG, [62](#)
- TJFLAG\_ACCURATEDCT
  - TurboJPEG, [57](#)
- TJFLAG\_BOTTOMUP
  - TurboJPEG, [57](#)
- TJFLAG\_FASTDCT
  - TurboJPEG, [57](#)
- TJFLAG\_FASTUPSAMPLE
  - TurboJPEG, [57](#)
- TJFLAG\_NOREALLOC
  - TurboJPEG, [58](#)
- TJFLAG\_PROGRESSIVE
  - TurboJPEG, [58](#)
- TJFLAG\_STOPONWARNING
  - TurboJPEG, [58](#)
- tjFree
  - TurboJPEG, [78](#)
- tjGetErrorCode
  - TurboJPEG, [79](#)
- tjGetErrorStr2
  - TurboJPEG, [79](#)

- tjGetScalingFactors
  - TurboJPEG, [79](#)
- tjhandle
  - TurboJPEG, [61](#)
- tjInitCompress
  - TurboJPEG, [80](#)
- tjInitDecompress
  - TurboJPEG, [80](#)
- tjInitTransform
  - TurboJPEG, [80](#)
- tjLoadImage
  - TurboJPEG, [81](#)
- TJPAD
  - TurboJPEG, [58](#)
- TJPF
  - TurboJPEG, [62](#)
- TJPF\_ABGR
  - TurboJPEG, [63](#)
- TJPF\_ARGB
  - TurboJPEG, [63](#)
- TJPF\_BGR
  - TurboJPEG, [63](#)
- TJPF\_BGRA
  - TurboJPEG, [63](#)
- TJPF\_BGRX
  - TurboJPEG, [63](#)
- TJPF\_CMYK
  - TurboJPEG, [63](#)
- TJPF\_GRAY
  - TurboJPEG, [63](#)
- TJPF\_RGB
  - TurboJPEG, [63](#)
- TJPF\_RGBA
  - TurboJPEG, [63](#)
- TJPF\_RGBX
  - TurboJPEG, [63](#)
- TJPF\_UNKNOWN
  - TurboJPEG, [63](#)
- TJPF\_XBGR
  - TurboJPEG, [63](#)
- TJPF\_XRGB
  - TurboJPEG, [63](#)
- tjPlaneHeight
  - TurboJPEG, [82](#)
- tjPlaneSizeYUV
  - TurboJPEG, [82](#)
- tjPlaneWidth
  - TurboJPEG, [83](#)
- tjregion, [121](#)
  - h, [121](#)
  - w, [121](#)
  - x, [122](#)
  - y, [122](#)
- TJSAMP
  - TurboJPEG, [63](#)
- TJSAMP\_411
  - TurboJPEG, [64](#)
- TJSAMP\_420
  - TurboJPEG, [64](#)
- TJSAMP\_422
  - TurboJPEG, [64](#)
- TJSAMP\_440
  - TurboJPEG, [64](#)
- TJSAMP\_444
  - TurboJPEG, [64](#)
- TJSAMP\_GRAY
  - TurboJPEG, [64](#)
- tjSaveImage
  - TurboJPEG, [83](#)
- TJSCALED
  - TurboJPEG, [59](#)
- tjscalingfactor, [122](#)
  - denom, [123](#)
  - num, [123](#)
- tjTransform
  - TurboJPEG, [84](#)
- tjtransform, [123](#)
  - customFilter, [124](#)
  - data, [125](#)
  - op, [125](#)
  - options, [125](#)
  - r, [125](#)
- TJXOP
  - TurboJPEG, [64](#)
- TJXOP\_HFLIP
  - TurboJPEG, [64](#)
- TJXOP\_NONE
  - TurboJPEG, [64](#)
- TJXOP\_ROT180
  - TurboJPEG, [65](#)
- TJXOP\_ROT270
  - TurboJPEG, [65](#)
- TJXOP\_ROT90
  - TurboJPEG, [65](#)
- TJXOP\_TRANSPOSE
  - TurboJPEG, [64](#)
- TJXOP\_TRANSVERSE
  - TurboJPEG, [64](#)
- TJXOP\_VFLIP
  - TurboJPEG, [64](#)
- TJXOPT\_COPYNONE
  - TurboJPEG, [59](#)
- TJXOPT\_CROP
  - TurboJPEG, [59](#)
- TJXOPT\_GRAY
  - TurboJPEG, [59](#)
- TJXOPT\_NOOUTPUT
  - TurboJPEG, [60](#)
- TJXOPT\_PERFECT
  - TurboJPEG, [60](#)
- TJXOPT\_PROGRESSIVE
  - TurboJPEG, [60](#)
- TJXOPT\_TRIM
  - TurboJPEG, [60](#)
- TofIR\_FourGroup\_Mono16

- TYFeatureList.h, [268](#)
- TRIG\_ACT\_ANY\_EDGE
  - TYFeatureList.h, [269](#)
- TRIG\_ACT\_FALLING\_EDGE
  - TYFeatureList.h, [269](#)
- TRIG\_ACT\_LEVEL\_HIGH
  - TYFeatureList.h, [269](#)
- TRIG\_ACT\_LEVEL\_LOW
  - TYFeatureList.h, [269](#)
- TRIG\_ACT\_RISING\_EDGE
  - TYFeatureList.h, [269](#)
- TRIG\_MODE\_OFF
  - TYFeatureList.h, [269](#)
- TRIG\_MODE\_ON
  - TYFeatureList.h, [269](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONACTIVE
  - TYFeatureList.h, [269](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONEND
  - TYFeatureList.h, [269](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONSTART
  - TYFeatureList.h, [269](#)
- TRIG\_SEL\_FRAME\_EXPOSUREACTIVE
  - TYFeatureList.h, [270](#)
- TRIG\_SEL\_FRAME\_EXPOSUREEND
  - TYFeatureList.h, [270](#)
- TRIG\_SEL\_FRAME\_EXPOSURESTART
  - TYFeatureList.h, [270](#)
- TRIG\_SEL\_FRAME\_FRAMEACTIVE
  - TYFeatureList.h, [269](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTACTIVE
  - TYFeatureList.h, [270](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTEND
  - TYFeatureList.h, [269](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTSTART
  - TYFeatureList.h, [269](#)
- TRIG\_SEL\_FRAME\_FRAMEEND
  - TYFeatureList.h, [269](#)
- TRIG\_SEL\_FRAME\_FRAMESTART
  - TYFeatureList.h, [269](#)
- TRIG\_SEL\_FRAME\_LINESTART
  - TYFeatureList.h, [270](#)
- TRIG\_SRC\_LINE0
  - TYFeatureList.h, [270](#)
- TRIG\_SRC\_LINE1
  - TYFeatureList.h, [270](#)
- TRIG\_SRC\_LINE2
  - TYFeatureList.h, [270](#)
- TRIG\_SRC\_LINE3
  - TYFeatureList.h, [270](#)
- TRIG\_SRC\_LINE4
  - TYFeatureList.h, [270](#)
- TRIG\_SRC\_LINE5
  - TYFeatureList.h, [270](#)
- TRIG\_SRC\_LINE6
  - TYFeatureList.h, [270](#)
- TRIG\_SRC\_LINE7
  - TYFeatureList.h, [270](#)
- TRIG\_SRC\_SOFTWARE
  - TYFeatureList.h, [270](#)
- TurboJPEG, [53](#)
- TJ\_NUMCS, [56](#)
- TJ\_NUMERR, [56](#)
- TJ\_NUMPF, [56](#)
- TJ\_NUMSAMP, [56](#)
- TJ\_NUMXOP, [57](#)
- tjAlloc, [65](#)
- tjBufSize, [65](#)
- tjBufSizeYUV2, [66](#)
- tjCompress2, [66](#)
- tjCompressFromYUV, [68](#)
- tjCompressFromYUVPlanes, [69](#)
- TJCS, [61](#)
- TJCS\_CMYK, [62](#)
- TJCS\_GRAY, [62](#)
- TJCS\_RGB, [61](#)
- TJCS\_YCbCr, [62](#)
- TJCS\_YCCK, [62](#)
- tjDecodeYUV, [71](#)
- tjDecodeYUVPlanes, [71](#)
- tjDecompress2, [72](#)
- tjDecompressHeader3, [73](#)
- tjDecompressToYUV2, [74](#)
- tjDecompressToYUVPlanes, [75](#)
- tjDestroy, [76](#)
- tjEncodeYUV3, [76](#)
- tjEncodeYUVPlanes, [77](#)
- TJERR, [62](#)
- TJERR\_FATAL, [62](#)
- TJERR\_WARNING, [62](#)
- TJFLAG\_ACCURATEDCT, [57](#)
- TJFLAG\_BOTTOMUP, [57](#)
- TJFLAG\_FASTDCT, [57](#)
- TJFLAG\_FASTUPSAMPLE, [57](#)
- TJFLAG\_NOREALLOC, [58](#)
- TJFLAG\_PROGRESSIVE, [58](#)
- TJFLAG\_STOPONWARNING, [58](#)
- tjFree, [78](#)
- tjGetErrorCode, [79](#)
- tjGetErrorStr2, [79](#)
- tjGetScalingFactors, [79](#)
- tjhandle, [61](#)
- tjInitCompress, [80](#)
- tjInitDecompress, [80](#)
- tjInitTransform, [80](#)
- tjLoadImage, [81](#)
- TJPAD, [58](#)
- TJPF, [62](#)
- TJPF\_ABGR, [63](#)
- TJPF\_ARGB, [63](#)
- TJPF\_BGR, [63](#)
- TJPF\_BGRA, [63](#)
- TJPF\_BGRX, [63](#)
- TJPF\_CMYK, [63](#)
- TJPF\_GRAY, [63](#)
- TJPF\_RGB, [63](#)
- TJPF\_RGBA, [63](#)

- TJPF\_RGBX, [63](#)
- TJPF\_UNKNOWN, [63](#)
- TJPF\_XBGR, [63](#)
- TJPF\_XRGB, [63](#)
- tjPlaneHeight, [82](#)
- tjPlaneSizeYUV, [82](#)
- tjPlaneWidth, [83](#)
- TJSAMP, [63](#)
- TJSAMP\_411, [64](#)
- TJSAMP\_420, [64](#)
- TJSAMP\_422, [64](#)
- TJSAMP\_440, [64](#)
- TJSAMP\_444, [64](#)
- TJSAMP\_GRAY, [64](#)
- tjSaveImage, [83](#)
- TJSCALED, [59](#)
- tjTransform, [84](#)
- tjtransform, [61](#)
- TJXOP, [64](#)
- TJXOP\_HFLIP, [64](#)
- TJXOP\_NONE, [64](#)
- TJXOP\_ROT180, [65](#)
- TJXOP\_ROT270, [65](#)
- TJXOP\_ROT90, [65](#)
- TJXOP\_TRANSPOSE, [64](#)
- TJXOP\_TRANSVERSE, [64](#)
- TJXOP\_VFLIP, [64](#)
- TJXOPT\_COPYNONE, [59](#)
- TJXOPT\_CROP, [59](#)
- TJXOPT\_GRAY, [59](#)
- TJXOPT\_NOOUTPUT, [60](#)
- TJXOPT\_PERFECT, [60](#)
- TJXOPT\_PROGRESSIVE, [60](#)
- TJXOPT\_TRIM, [60](#)
- TY\_ACC\_BIAS, [125](#)
- TYDefs.h, [242](#)
- TY\_ACC\_MISALIGNMENT, [126](#)
- TYDefs.h, [242](#)
- TY\_ACC\_SCALE, [126](#)
- TYDefs.h, [242](#)
- TY\_ACCESS\_MODE\_LIST
- TYDefs.h, [242](#), [248](#)
- TY\_ACQ\_MODE\_LIST
- TYFeatureList.h, [263](#)
- TY\_AEC\_ROI\_PARAM, [127](#)
- TY\_BIN\_H\_LIST
- TYFeatureList.h, [263](#)
- TY\_BIN\_V\_LIST
- TYFeatureList.h, [264](#)
- TY\_BOOL\_AUTO\_AWB
- TYDefs.h, [250](#)
- TY\_BOOL\_AUTO\_EXPOSURE
- TYDefs.h, [250](#)
- TY\_BOOL\_AUTO\_GAIN
- TYDefs.h, [250](#)
- TY\_BOOL\_BRIGHTNESS\_HISTOGRAM
- TYDefs.h, [250](#)
- TY\_BOOL\_CMOS\_SYNC
- TYDefs.h, [250](#)
- TY\_BOOL\_DEPTH\_POSTPROC
- TYDefs.h, [250](#)
- TY\_BOOL\_HDR
- TYDefs.h, [251](#)
- TY\_BOOL\_IMU\_DATA\_ONOFF
- TYDefs.h, [251](#)
- TY\_BOOL\_IR\_FLASHLIGHT
- TYDefs.h, [250](#)
- TY\_BOOL\_KEEP\_ALIVE\_ONOFF
- TYDefs.h, [250](#)
- TY\_BOOL\_LASER\_AUTO\_CTRL
- TYDefs.h, [250](#)
- TY\_BOOL\_RGB\_FLASHLIGHT
- TYDefs.h, [250](#)
- TY\_BOOL\_SGBM\_HFILTER\_HALF\_WIN
- TYDefs.h, [251](#)
- TY\_BOOL\_SGBM\_LRC
- TYDefs.h, [251](#)
- TY\_BOOL\_SGBM\_MEDFILTER
- TYDefs.h, [251](#)
- TY\_BOOL\_SGPM\_EPI\_EN
- TYDefs.h, [251](#)
- TY\_BOOL\_SGPM\_ORDER\_FILTER\_EN
- TYDefs.h, [251](#)
- TY\_BOOL\_TIME\_SYNC\_READY
- TYDefs.h, [250](#)
- TY\_BOOL\_TOF\_ANTI\_INTERFERENCE
- TYDefs.h, [252](#)
- TY\_BOOL\_TRANSMIT\_ROI\_ENABLE
- TYDefs.h, [252](#)
- TY\_BOOL\_TRIGGER\_OUT\_IO
- TYDefs.h, [250](#)
- TY\_BOOL\_UNDISTORTION
- TYDefs.h, [250](#)
- TY\_BYTEARRAY\_ATTR, [127](#)
- TYDefs.h, [243](#)
- unit\_size, [128](#)
- valid\_size, [128](#)
- TY\_BYTEARRAY\_CUSTOM\_BLOCK
- TYDefs.h, [249](#)
- TY\_BYTEARRAY\_HDR\_PARAMETER
- TYDefs.h, [251](#)
- TY\_BYTEARRAY\_ISP\_BLOCK
- TYDefs.h, [249](#)
- TY\_CAMERA\_CALIB\_INFO, [128](#)
- TYDefs.h, [243](#)
- TY\_CAMERA\_DISTORTION, [129](#)
- TYDefs.h, [243](#)
- TY\_CAMERA\_EXTRINSIC, [129](#)
- TYDefs.h, [243](#)
- TY\_CAMERA\_INTRINSIC, [130](#)
- TYDefs.h, [244](#)
- TY\_CAMERA\_ROTATION, [131](#)
- TYDefs.h, [244](#)
- TY\_CAMERA\_STATISTICS, [131](#)
- TY\_CAMERA\_TO\_IMU, [132](#)
- TYDefs.h, [244](#)

TY\_COMPONENT\_BRIGHT\_HISTO  
    TYDefs.h, [249](#)

TY\_COMPONENT\_DEPTH\_CAM  
    TYDefs.h, [249](#)

TY\_COMPONENT\_DEVICE  
    TYDefs.h, [249](#)

TY\_COMPONENT\_ID  
    TYDefs.h, [245](#)

TY\_COMPONENT\_IMU  
    TYDefs.h, [249](#)

TY\_COMPONENT\_IR\_CAM\_LEFT  
    TYDefs.h, [249](#)

TY\_COMPONENT\_IR\_CAM\_RIGHT  
    TYDefs.h, [249](#)

TY\_COMPONENT\_LASER  
    TYDefs.h, [249](#)

TY\_COMPONENT\_RGB\_CAM  
    TYDefs.h, [249](#)

TY\_COMPONENT\_RGB\_CAM\_LEFT  
    TYDefs.h, [249](#)

TY\_COMPONENT\_RGB\_CAM\_RIGHT  
    TYDefs.h, [249](#)

TY\_COMPONENT\_STORAGE  
    TYDefs.h, [249](#)

TY\_DECLARE\_IMAGE\_MODE1  
    TYDefs.h, [241](#)

TY\_DEPTH\_TOF\_QUALITY\_LIST  
    TYFeatureList.h, [264](#)

TY\_DEV\_CHARSET\_LIST  
    TYFeatureList.h, [264](#)

TY\_DEV\_COMPRESS\_LIST  
    TYFeatureList.h, [265](#)

TY\_DEV\_LINK\_HB\_MODE\_LIST  
    TYFeatureList.h, [265](#)

TY\_DEV\_STREAM\_ASYNC\_LIST  
    TYFeatureList.h, [265](#)

TY\_DEV\_STREAM\_CH\_TYPE\_LIST  
    TYFeatureList.h, [265](#)

TY\_DEV\_TEMP\_SEL\_LIST  
    TYFeatureList.h, [266](#)

TY\_DEV\_TIME\_SYNC\_LIST  
    TYFeatureList.h, [266](#)

TY\_DEV\_TYPE\_LIST  
    TYFeatureList.h, [266](#)

TY\_DEVICE\_BASE\_INFO, [132](#)  
    TYDefs.h, [245](#)

TY\_DEVICE\_COMPONENT\_LIST  
    TYDefs.h, [245](#), [248](#)

TY\_DEVICE\_NET\_INFO, [133](#)

TY\_DEVICE\_USB\_INFO, [134](#)

TY\_DI\_WORKMODE, [134](#)

TY\_DO\_WORKMODE, [135](#)

TY\_ENUM\_DEPTH\_QUALITY  
    TYDefs.h, [252](#)

TY\_ENUM\_ENTRY, [135](#)  
    TYDefs.h, [245](#)

TY\_ENUM\_IMAGE\_MODE  
    TYDefs.h, [249](#)

TY\_ENUM\_IMU\_FPS  
    TYDefs.h, [251](#)

TY\_ENUM\_STREAM\_ASYNC  
    TYDefs.h, [250](#)

TY\_ENUM\_TIME\_SYNC\_TYPE  
    TYDefs.h, [250](#)

TY\_ENUM\_TRIGGER\_POL  
    TYDefs.h, [249](#)

TY\_EVENT\_INFO, [136](#)

TY\_FEATURE\_ID  
    TYDefs.h, [246](#)

TY\_FEATURE\_ID\_LIST  
    TYDefs.h, [249](#)

TY\_FEATURE\_INFO, [136](#)

TY\_FLOAT\_EXPOSURE\_TIME\_US  
    TYDefs.h, [250](#)

TY\_FLOAT\_RANGE, [137](#)  
    TYDefs.h, [246](#)

TY\_FLOAT\_SCALE\_UNIT  
    TYDefs.h, [249](#)

TY\_FLOAT\_SGPM\_EPI\_HS  
    TYDefs.h, [251](#)

TY\_FRAME\_DATA, [137](#)

TY\_GAIN\_SEL\_LIST  
    TYFeatureList.h, [267](#)

TY\_GEV\_CCP\_LIST  
    TYFeatureList.h, [267](#)

TY\_GYRO\_BIAS, [138](#)  
    TYDefs.h, [246](#)

TY\_GYRO\_MISALIGNMENT, [138](#)  
    TYDefs.h, [247](#)

TY\_GYRO\_SCALE, [139](#)  
    TYDefs.h, [247](#)

TY\_IMAGE\_DATA, [140](#)

TY\_IMU\_DATA, [140](#)

TY\_INT\_ANALOG\_GAIN  
    TYDefs.h, [250](#)

TY\_INT\_B\_GAIN  
    TYDefs.h, [250](#)

TY\_INT\_CAPTURE\_TIME\_US  
    TYDefs.h, [250](#)

TY\_INT\_DEPTH\_MAX\_MM  
    TYDefs.h, [251](#)

TY\_INT\_DEPTH\_MIN\_MM  
    TYDefs.h, [251](#)

TY\_INT\_EXPOSURE\_TIME  
    TYDefs.h, [250](#)

TY\_INT\_FILTER\_THRESHOLD  
    TYDefs.h, [252](#)

TY\_INT\_FRAME\_PER\_TRIGGER  
    TYDefs.h, [249](#)

TY\_INT\_G\_GAIN  
    TYDefs.h, [250](#)

TY\_INT\_GAIN  
    TYDefs.h, [250](#)

TY\_INT\_HEIGHT  
    TYDefs.h, [249](#)

TY\_INT\_IR\_FLASHLIGHT\_INTENSITY

- TYDefs.h, [250](#)
- TY\_INT\_KEEP\_ALIVE\_TIMEOUT
  - TYDefs.h, [250](#)
- TY\_INT\_LASER\_POWER
  - TYDefs.h, [250](#)
- TY\_INT\_LINK\_CMD\_TIMEOUT
  - TYDefs.h, [249](#)
- TY\_INT\_MAX\_SPECKLE\_DIFF
  - TYDefs.h, [252](#)
- TY\_INT\_MAX\_SPECKLE\_SIZE
  - TYDefs.h, [252](#)
- TY\_INT\_NTP\_SERVER\_IP
  - TYDefs.h, [249](#)
- TY\_INT\_PACKET\_DELAY
  - TYDefs.h, [249](#)
- TY\_INT\_R\_GAIN
  - TYDefs.h, [250](#)
- TY\_INT\_RANGE, [141](#)
- TY\_INT\_RGB\_FLASHLIGHT\_INTENSITY
  - TYDefs.h, [250](#)
- TY\_INT\_SGBM\_DISPARITY\_NUM
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_DISPARITY\_OFFSET
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_IMAGE\_NUM
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_LRC\_DIFF
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_MATCH\_WIN\_HEIGHT
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_MATCH\_WIN\_WIDTH
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_MEDFILTER\_THRESH
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_SEMI\_PARAM\_P1
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_SEMI\_PARAM\_P1\_SCALE
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_SEMI\_PARAM\_P2
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_TEXTURE\_OFFSET
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_TEXTURE\_THRESH
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_UNIQUE\_ABSDIFF
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_UNIQUE\_FACTOR
  - TYDefs.h, [251](#)
- TY\_INT\_SGBM\_UNIQUE\_MAX\_COST
  - TYDefs.h, [251](#)
- TY\_INT\_SGPM\_EPI\_CH0
  - TYDefs.h, [251](#)
- TY\_INT\_SGPM\_EPI\_CH1
  - TYDefs.h, [251](#)
- TY\_INT\_SGPM\_EPI\_HF
  - TYDefs.h, [251](#)
- TY\_INT\_SGPM\_EPI\_THRESH
  - TYDefs.h, [251](#)
- TY\_INT\_SGPM\_NORMAL\_PHASE\_OFFSET
  - TYDefs.h, [251](#)
- TY\_INT\_SGPM\_NORMAL\_PHASE\_SCALE
  - TYDefs.h, [251](#)
- TY\_INT\_SGPM\_ORDER\_FILTER\_CHN
  - TYDefs.h, [251](#)
- TY\_INT\_SGPM\_PHASE\_NUM
  - TYDefs.h, [251](#)
- TY\_INT\_SGPM\_REF\_PHASE\_OFFSET
  - TYDefs.h, [251](#)
- TY\_INT\_SGPM\_REF\_PHASE\_SCALE
  - TYDefs.h, [251](#)
- TY\_INT\_TOF\_ANTI\_SUNLIGHT\_INDEX
  - TYDefs.h, [252](#)
- TY\_INT\_TOF\_CHANNEL
  - TYDefs.h, [252](#)
- TY\_INT\_TOF\_HDR\_RATIO
  - TYDefs.h, [250](#)
- TY\_INT\_TOF\_JITTER\_THRESHOLD
  - TYDefs.h, [250](#)
- TY\_INT\_TOF\_MODULATION\_THRESHOLD
  - TYDefs.h, [252](#)
- TY\_INT\_TRANSMIT\_ROI\_HEIGHT
  - TYDefs.h, [252](#)
- TY\_INT\_TRANSMIT\_ROI\_OFFSET\_X
  - TYDefs.h, [252](#)
- TY\_INT\_TRANSMIT\_ROI\_OFFSET\_Y
  - TYDefs.h, [252](#)
- TY\_INT\_TRANSMIT\_ROI\_WIDTH
  - TYDefs.h, [252](#)
- TY\_INT\_TRIGGER\_DELAY\_US
  - TYDefs.h, [250](#)
- TY\_INT\_TRIGGER\_DURATION\_US
  - TYDefs.h, [250](#)
- TY\_INT\_WIDTH
  - TYDefs.h, [249](#)
- TY\_INTERFACE\_INFO, [141](#)
  - TYDefs.h, [247](#)
- TY\_INTERFACE\_TYPE\_LIST
  - TYDefs.h, [247](#), [252](#)
- TY\_LASER\_PARAM, [142](#)
- TY\_LASER\_PATTERN\_PARAM, [142](#)
- TY\_LIGHT\_SEL\_LIST
  - TYFeatureList.h, [267](#)
- TY\_PHC\_GROUP\_ATTR, [143](#)
- TY\_PHC\_GROUP\_ATTR::phc\_group\_attr, [121](#)
- TY\_PIX\_FMT\_LIST
  - TYFeatureList.h, [268](#)
- TY\_PIXEL\_BITS\_LIST
  - TYDefs.h, [247](#), [252](#)
- TY\_PIXEL\_COLOR\_DESC, [144](#)
- TY\_PIXEL\_DESC, [144](#)
- TY\_PIXEL\_FORMAT\_BAYER8BG
  - TYDefs.h, [253](#)
- TY\_PIXEL\_FORMAT\_BAYER8GB
  - TYDefs.h, [253](#)
- TY\_PIXEL\_FORMAT\_BAYER8GR
  - TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_BAYER8RG  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_BGR  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_BGR48  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_CSI\_BAYER10BGGR  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_CSI\_BAYER10GBRG  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_CSI\_BAYER10GRBG  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_CSI\_BAYER10RGGG  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_CSI\_BAYER12BGGR  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_CSI\_BAYER12GBRG  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_CSI\_BAYER12GRBG  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_CSI\_BAYER12RGGG  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_CSI\_MONO10  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_CSI\_MONO12  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_DEPTH16  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_JPEG  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_LIST  
TYDefs.h, [252](#)

TY\_PIXEL\_FORMAT\_MJPG  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_MONO  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_MONO16  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_RGB  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_RGB48  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_TOF\_IR\_MONO16  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_XYZ48  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_YUYV  
TYDefs.h, [253](#)

TY\_PIXEL\_FORMAT\_YVYU  
TYDefs.h, [253](#)

TY\_RESOLUTION\_MODE\_1280x720  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_1280x800  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_1280x960  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_1600x1200  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_160x100  
TYDefs.h, [253](#)

TY\_RESOLUTION\_MODE\_160x120  
TYDefs.h, [253](#)

TY\_RESOLUTION\_MODE\_1920x1080  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_1920x1440  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_2048x1536  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_240x320  
TYDefs.h, [253](#)

TY\_RESOLUTION\_MODE\_240x96  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_2560x1920  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_2592x1944  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_320x180  
TYDefs.h, [253](#)

TY\_RESOLUTION\_MODE\_320x200  
TYDefs.h, [253](#)

TY\_RESOLUTION\_MODE\_320x240  
TYDefs.h, [253](#)

TY\_RESOLUTION\_MODE\_480x640  
TYDefs.h, [253](#)

TY\_RESOLUTION\_MODE\_640x360  
TYDefs.h, [253](#)

TY\_RESOLUTION\_MODE\_640x400  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_640x480  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_800x600  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_960x1280  
TYDefs.h, [254](#)

TY\_RESOLUTION\_MODE\_LIST  
TYDefs.h, [253](#)

TY\_SRC\_SEL\_LIST  
TYFeatureList.h, [268](#)

TY\_STRUCT\_AEC\_ROI  
TYDefs.h, [250](#)

TY\_STRUCT\_CAM\_CALIB\_DATA  
TYDefs.h, [249](#)

TY\_STRUCT\_CAM\_DISTORTION  
TYDefs.h, [249](#)

TY\_STRUCT\_CAM\_INTRINSIC  
TYDefs.h, [249](#)

TY\_STRUCT\_CAM\_RECTIFIED\_INTRI  
TYDefs.h, [249](#)

TY\_STRUCT\_CAM\_RECTIFIED\_ROTATION  
TYDefs.h, [249](#)

TY\_STRUCT\_CAM\_STATISTICS  
TYDefs.h, [249](#)

TY\_STRUCT\_DIO\_WORKMODE  
TYDefs.h, [250](#)

TY\_STRUCT\_DI1\_WORKMODE  
TYDefs.h, [250](#)

- TY\_STRUCT\_DI2\_WORKMODE  
TYDefs.h, [250](#)
- TY\_STRUCT\_DO0\_WORKMODE  
TYDefs.h, [250](#)
- TY\_STRUCT\_DO1\_WORKMODE  
TYDefs.h, [250](#)
- TY\_STRUCT\_DO2\_WORKMODE  
TYDefs.h, [250](#)
- TY\_STRUCT\_EXTRINSIC\_TO\_DEPTH  
TYDefs.h, [249](#)
- TY\_STRUCT\_EXTRINSIC\_TO\_IR\_LEFT  
TYDefs.h, [249](#)
- TY\_STRUCT\_FLOOD\_ENABLE\_BY\_IDX  
TYDefs.h, [250](#)
- TY\_STRUCT\_FLOOD\_POWER\_BY\_IDX  
TYDefs.h, [250](#)
- TY\_STRUCT\_IMU\_ACC\_BIAS  
TYDefs.h, [251](#)
- TY\_STRUCT\_IMU\_ACC\_MISALIGNMENT  
TYDefs.h, [251](#)
- TY\_STRUCT\_IMU\_ACC\_SCALE  
TYDefs.h, [251](#)
- TY\_STRUCT\_IMU\_CAM\_TO\_IMU  
TYDefs.h, [251](#)
- TY\_STRUCT\_IMU\_GYRO\_BIAS  
TYDefs.h, [251](#)
- TY\_STRUCT\_IMU\_GYRO\_MISALIGNMENT  
TYDefs.h, [251](#)
- TY\_STRUCT\_IMU\_GYRO\_SCALE  
TYDefs.h, [251](#)
- TY\_STRUCT\_LASER\_ENABLE\_BY\_IDX  
TYDefs.h, [250](#)
- TY\_STRUCT\_LASER\_POWER\_BY\_IDX  
TYDefs.h, [250](#)
- TY\_STRUCT\_PHC\_GROUP\_ATTR  
TYDefs.h, [251](#)
- TY\_STRUCT\_TOF\_FREQ  
TYDefs.h, [252](#)
- TY\_STRUCT\_TRIGGER\_PARAM  
TYDefs.h, [250](#)
- TY\_STRUCT\_TRIGGER\_PARAM\_EX  
TYDefs.h, [250](#)
- TY\_STRUCT\_TRIGGER\_TIMER\_LIST  
TYDefs.h, [250](#)
- TY\_STRUCT\_TRIGGER\_TIMER\_PERIOD  
TYDefs.h, [250](#)
- TY\_TEMP\_DATA, [145](#)
- TY\_TOF\_FREQ, [145](#)
- TY\_TRIG\_ACT\_LIST  
TYFeatureList.h, [269](#)
- TY\_TRIG\_MODE\_LIST  
TYFeatureList.h, [269](#)
- TY\_TRIG\_SEL\_LIST  
TYFeatureList.h, [269](#)
- TY\_TRIG\_SRC\_LIST  
TYFeatureList.h, [270](#)
- TY\_TRIGGER\_MODE\_LIST  
TYDefs.h, [248](#), [254](#)
- TY\_TRIGGER\_MODE\_M\_PER  
TYDefs.h, [254](#)
- TY\_TRIGGER\_MODE\_M\_SIG  
TYDefs.h, [254](#)
- TY\_TRIGGER\_MODE\_OFF  
TYDefs.h, [254](#)
- TY\_TRIGGER\_MODE\_PER\_PASS  
TYDefs.h, [254](#)
- TY\_TRIGGER\_MODE\_PER\_PASS2  
TYDefs.h, [254](#)
- TY\_TRIGGER\_MODE\_SIG\_PASS  
TYDefs.h, [254](#)
- TY\_TRIGGER\_MODE\_SLAVE  
TYDefs.h, [254](#)
- TY\_TRIGGER\_PARAM, [145](#)
- TY\_TRIGGER\_PARAM\_EX, [146](#)
- TY\_TRIGGER\_TIMER\_LIST, [146](#)
- TY\_TRIGGER\_TIMER\_PERIOD, [147](#)
- TY\_USER\_SET\_SEL\_LIST  
TYFeatureList.h, [270](#)
- TY\_VECT\_3F, [147](#)
- TY\_VERSION\_INFO, [147](#)
- TYApi.h, [151](#)
- TYCfgLogFile, [155](#)
- TYCfgLogServer, [156](#)
- TYClearBufferQueue, [157](#)
- TYCloseDevice, [157](#)
- TYCloseInterface, [158](#)
- TYDeinitLib, [159](#)
- TYDisableComponents, [159](#)
- TYDisableLog, [160](#)
- TYEnableComponents, [160](#)
- TYEnableLog, [161](#)
- TYEnqueueBuffer, [162](#)
- TYErrorString, [163](#)
- TYFetchFrame, [163](#)
- TYForceDeviceIP, [164](#)
- TYGetBool, [165](#)
- TYGetByteArray, [167](#)
- TYGetByteArrayAttr, [168](#)
- TYGetByteArraySize, [170](#)
- TYGetComponentIDs, [171](#)
- TYGetDeviceFeatureInfo, [172](#)
- TYGetDeviceFeatureNumber, [173](#)
- TYGetDeviceInfo, [174](#)
- TYGetDeviceInterface, [175](#)
- TYGetDeviceList, [176](#)
- TYGetDeviceNumber, [176](#)
- TYGetDeviceXML, [177](#)
- TYGetDeviceXMLSize, [178](#)
- TYGetEnabledComponents, [179](#)
- TYGetEnum, [180](#)
- TYGetEnumEntryCount, [181](#)
- TYGetEnumEntryInfo, [182](#)
- TYGetFeatureInfo, [183](#)
- TYGetFloat, [184](#)
- TYGetFloatRange, [186](#)
- TYGetFrameBufferSize, [187](#)



- TYGetInt, [188](#)
- TYGetInterfaceList, [189](#)
- TYGetInterfaceNumber, [190](#)
- TYGetIntRange, [191](#)
- TYGetString, [192](#)
- TYGetStringLength, [193](#)
- TYGetStruct, [195](#)
- TYHasDevice, [197](#)
- TYHasFeature, [198](#)
- TYHasInterface, [199](#)
- TYLibVersion, [199](#)
- TYOpenDevice, [200](#)
- TYOpenDeviceWithIP, [201](#)
- TYOpenInterface, [203](#)
- TYRegisterEventCallback, [203](#)
- TYRegisterImuCallback, [204](#)
- TYSendSoftTrigger, [205](#)
- TYSetBool, [206](#)
- TYSetByteArray, [207](#)
- TYSetEnum, [209](#)
- TYSetFloat, [210](#)
- TYSetInt, [212](#)
- TYSetLogLevel, [213](#)
- TYSetLogPrefix, [213](#)
- TYSetString, [214](#)
- TYSetStruct, [216](#)
- TYStartCapture, [217](#)
- TYStopCapture, [218](#)
- TYUpdateAllDeviceList, [219](#)
- TYUpdateDeviceList, [219](#)
- TYUpdateInterfaceList, [220](#)
- TYCfgLogFile
  - TYApi.h, [155](#)
- TYCfgLogServer
  - TYApi.h, [156](#)
- TYClearBufferQueue
  - TYApi.h, [157](#)
- TYCloseDevice
  - TYApi.h, [157](#)
- TYCloseInterface
  - TYApi.h, [158](#)
- TYCoordinateMapper.h, [220](#)
  - TYInvertExtrinsic, [222](#)
  - TYMapDepthImageToColorCoordinate, [223](#)
  - TYMapDepthImageToPoint3d, [224](#)
  - TYMapDepthToColorCoordinate, [224](#)
  - TYMapDepthToPoint3d, [225](#)
  - TYMapMono16ImageToDepthCoordinate, [225](#)
  - TYMapMono8ImageToDepthCoordinate, [226](#)
  - TYMapPoint3dToDepth, [227](#)
  - TYMapPoint3dToDepthImage, [227](#)
  - TYMapPoint3dToPoint3d, [228](#)
  - TYMapRGB48ImageToDepthCoordinate, [228](#)
  - TYMapRGBImageToDepthCoordinate, [229](#)
  - TYMapRGBPixelsToDepthCoordinate, [230](#)
- TYDecodeImage
  - TYImageProc.h, [273](#)
- TYDecodeResult, [148](#)
- TYDefs.h, [231](#)
  - TY\_ACC\_BIAS, [242](#)
  - TY\_ACC\_MISALIGNMENT, [242](#)
  - TY\_ACC\_SCALE, [242](#)
  - TY\_ACCESS\_MODE\_LIST, [242](#), [248](#)
  - TY\_BOOL\_AUTO\_AWB, [250](#)
  - TY\_BOOL\_AUTO\_EXPOSURE, [250](#)
  - TY\_BOOL\_AUTO\_GAIN, [250](#)
  - TY\_BOOL\_BRIGHTNESS\_HISTOGRAM, [250](#)
  - TY\_BOOL\_CMOS\_SYNC, [250](#)
  - TY\_BOOL\_DEPTH\_POSTPROC, [250](#)
  - TY\_BOOL\_HDR, [251](#)
  - TY\_BOOL\_IMU\_DATA\_ONOFF, [251](#)
  - TY\_BOOL\_IR\_FLASHLIGHT, [250](#)
  - TY\_BOOL\_KEEP\_ALIVE\_ONOFF, [250](#)
  - TY\_BOOL\_LASER\_AUTO\_CTRL, [250](#)
  - TY\_BOOL\_RGB\_FLASHLIGHT, [250](#)
  - TY\_BOOL\_SGBM\_HFILTER\_HALF\_WIN, [251](#)
  - TY\_BOOL\_SGBM\_LRC, [251](#)
  - TY\_BOOL\_SGBM\_MEDFILTER, [251](#)
  - TY\_BOOL\_SGPM\_EPI\_EN, [251](#)
  - TY\_BOOL\_SGPM\_ORDER\_FILTER\_EN, [251](#)
  - TY\_BOOL\_TIME\_SYNC\_READY, [250](#)
  - TY\_BOOL\_TOF\_ANTI\_INTERFERENCE, [252](#)
  - TY\_BOOL\_TRANSMIT\_ROI\_ENABLE, [252](#)
  - TY\_BOOL\_TRIGGER\_OUT\_IO, [250](#)
  - TY\_BOOL\_UNDISTORTION, [250](#)
  - TY\_BYTEARRAY\_ATTR, [243](#)
  - TY\_BYTEARRAY\_CUSTOM\_BLOCK, [249](#)
  - TY\_BYTEARRAY\_HDR\_PARAMETER, [251](#)
  - TY\_BYTEARRAY\_ISP\_BLOCK, [249](#)
  - TY\_CAMERA\_CALIB\_INFO, [243](#)
  - TY\_CAMERA\_DISTORTION, [243](#)
  - TY\_CAMERA\_EXTRINSIC, [243](#)
  - TY\_CAMERA\_INTRINSIC, [244](#)
  - TY\_CAMERA\_ROTATION, [244](#)
  - TY\_CAMERA\_TO\_IMU, [244](#)
  - TY\_COMPONENT\_BRIGHT\_HISTO, [249](#)
  - TY\_COMPONENT\_DEPTH\_CAM, [249](#)
  - TY\_COMPONENT\_DEVICE, [249](#)
  - TY\_COMPONENT\_ID, [245](#)
  - TY\_COMPONENT\_IMU, [249](#)
  - TY\_COMPONENT\_IR\_CAM\_LEFT, [249](#)
  - TY\_COMPONENT\_IR\_CAM\_RIGHT, [249](#)
  - TY\_COMPONENT\_LASER, [249](#)
  - TY\_COMPONENT\_RGB\_CAM, [249](#)
  - TY\_COMPONENT\_RGB\_CAM\_LEFT, [249](#)
  - TY\_COMPONENT\_RGB\_CAM\_RIGHT, [249](#)
  - TY\_COMPONENT\_STORAGE, [249](#)
  - TY\_DECLARE\_IMAGE\_MODE1, [241](#)
  - TY\_DEVICE\_BASE\_INFO, [245](#)
  - TY\_DEVICE\_COMPONENT\_LIST, [245](#), [248](#)
  - TY\_ENUM\_DEPTH\_QUALITY, [252](#)
  - TY\_ENUM\_ENTRY, [245](#)
  - TY\_ENUM\_IMAGE\_MODE, [249](#)
  - TY\_ENUM\_IMU\_FPS, [251](#)
  - TY\_ENUM\_STREAM\_ASYNC, [250](#)
  - TY\_ENUM\_TIME\_SYNC\_TYPE, [250](#)

- TY\_ENUM\_TRIGGER\_POL, 249
- TY\_FEATURE\_ID, 246
- TY\_FEATURE\_ID\_LIST, 249
- TY\_FLOAT\_EXPOSURE\_TIME\_US, 250
- TY\_FLOAT\_RANGE, 246
- TY\_FLOAT\_SCALE\_UNIT, 249
- TY\_FLOAT\_SGPM\_EPI\_HS, 251
- TY\_GYRO\_BIAS, 246
- TY\_GYRO\_MISALIGNMENT, 247
- TY\_GYRO\_SCALE, 247
- TY\_INT\_ANALOG\_GAIN, 250
- TY\_INT\_B\_GAIN, 250
- TY\_INT\_CAPTURE\_TIME\_US, 250
- TY\_INT\_DEPTH\_MAX\_MM, 251
- TY\_INT\_DEPTH\_MIN\_MM, 251
- TY\_INT\_EXPOSURE\_TIME, 250
- TY\_INT\_FILTER\_THRESHOLD, 252
- TY\_INT\_FRAME\_PER\_TRIGGER, 249
- TY\_INT\_G\_GAIN, 250
- TY\_INT\_GAIN, 250
- TY\_INT\_HEIGHT, 249
- TY\_INT\_IR\_FLASHLIGHT\_INTENSITY, 250
- TY\_INT\_KEEP\_ALIVE\_TIMEOUT, 250
- TY\_INT\_LASER\_POWER, 250
- TY\_INT\_LINK\_CMD\_TIMEOUT, 249
- TY\_INT\_MAX\_SPECKLE\_DIFF, 252
- TY\_INT\_MAX\_SPECKLE\_SIZE, 252
- TY\_INT\_NTP\_SERVER\_IP, 249
- TY\_INT\_PACKET\_DELAY, 249
- TY\_INT\_R\_GAIN, 250
- TY\_INT\_RGB\_FLASHLIGHT\_INTENSITY, 250
- TY\_INT\_SGBM\_DISPARITY\_NUM, 251
- TY\_INT\_SGBM\_DISPARITY\_OFFSET, 251
- TY\_INT\_SGBM\_IMAGE\_NUM, 251
- TY\_INT\_SGBM\_LRC\_DIFF, 251
- TY\_INT\_SGBM\_MATCH\_WIN\_HEIGHT, 251
- TY\_INT\_SGBM\_MATCH\_WIN\_WIDTH, 251
- TY\_INT\_SGBM\_MEDFILTER\_THRESH, 251
- TY\_INT\_SGBM\_SEMI\_PARAM\_P1, 251
- TY\_INT\_SGBM\_SEMI\_PARAM\_P1\_SCALE, 251
- TY\_INT\_SGBM\_SEMI\_PARAM\_P2, 251
- TY\_INT\_SGBM\_TEXTURE\_OFFSET, 251
- TY\_INT\_SGBM\_TEXTURE\_THRESH, 251
- TY\_INT\_SGBM\_UNIQUE\_ABSDIFF, 251
- TY\_INT\_SGBM\_UNIQUE\_FACTOR, 251
- TY\_INT\_SGBM\_UNIQUE\_MAX\_COST, 251
- TY\_INT\_SGPM\_EPI\_CH0, 251
- TY\_INT\_SGPM\_EPI\_CH1, 251
- TY\_INT\_SGPM\_EPI\_HF, 251
- TY\_INT\_SGPM\_EPI\_THRESH, 251
- TY\_INT\_SGPM\_NORMAL\_PHASE\_OFFSET, 251
- TY\_INT\_SGPM\_NORMAL\_PHASE\_SCALE, 251
- TY\_INT\_SGPM\_ORDER\_FILTER\_CHN, 251
- TY\_INT\_SGPM\_PHASE\_NUM, 251
- TY\_INT\_SGPM\_REF\_PHASE\_OFFSET, 251
- TY\_INT\_SGPM\_REF\_PHASE\_SCALE, 251
- TY\_INT\_TOF\_ANTI\_SUNLIGHT\_INDEX, 252
- TY\_INT\_TOF\_CHANNEL, 252
- TY\_INT\_TOF\_HDR\_RATIO, 250
- TY\_INT\_TOF\_JITTER\_THRESHOLD, 250
- TY\_INT\_TOF\_MODULATION\_THRESHOLD, 252
- TY\_INT\_TRANSMIT\_ROI\_HEIGHT, 252
- TY\_INT\_TRANSMIT\_ROI\_OFFSET\_X, 252
- TY\_INT\_TRANSMIT\_ROI\_OFFSET\_Y, 252
- TY\_INT\_TRANSMIT\_ROI\_WIDTH, 252
- TY\_INT\_TRIGGER\_DELAY\_US, 250
- TY\_INT\_TRIGGER\_DURATION\_US, 250
- TY\_INT\_WIDTH, 249
- TY\_INTERFACE\_INFO, 247
- TY\_INTERFACE\_TYPE\_LIST, 247, 252
- TY\_PIXEL\_BITS\_LIST, 247, 252
- TY\_PIXEL\_FORMAT\_BAYER8BG, 253
- TY\_PIXEL\_FORMAT\_BAYER8GB, 253
- TY\_PIXEL\_FORMAT\_BAYER8GR, 253
- TY\_PIXEL\_FORMAT\_BAYER8RG, 253
- TY\_PIXEL\_FORMAT\_BGR, 253
- TY\_PIXEL\_FORMAT\_BGR48, 253
- TY\_PIXEL\_FORMAT\_CSI\_BAYER10BGGR, 253
- TY\_PIXEL\_FORMAT\_CSI\_BAYER10GBRG, 253
- TY\_PIXEL\_FORMAT\_CSI\_BAYER10GRBG, 253
- TY\_PIXEL\_FORMAT\_CSI\_BAYER10RGGG, 253
- TY\_PIXEL\_FORMAT\_CSI\_BAYER12BGGR, 253
- TY\_PIXEL\_FORMAT\_CSI\_BAYER12GBRG, 253
- TY\_PIXEL\_FORMAT\_CSI\_BAYER12GRBG, 253
- TY\_PIXEL\_FORMAT\_CSI\_BAYER12RGGG, 253
- TY\_PIXEL\_FORMAT\_CSI\_MONO10, 253
- TY\_PIXEL\_FORMAT\_CSI\_MONO12, 253
- TY\_PIXEL\_FORMAT\_DEPTH16, 253
- TY\_PIXEL\_FORMAT\_JPEG, 253
- TY\_PIXEL\_FORMAT\_LIST, 252
- TY\_PIXEL\_FORMAT\_MJPEG, 253
- TY\_PIXEL\_FORMAT\_MONO, 253
- TY\_PIXEL\_FORMAT\_MONO16, 253
- TY\_PIXEL\_FORMAT\_RGB, 253
- TY\_PIXEL\_FORMAT\_RGB48, 253
- TY\_PIXEL\_FORMAT\_TOF\_IR\_MONO16, 253
- TY\_PIXEL\_FORMAT\_XYZ48, 253
- TY\_PIXEL\_FORMAT\_YUYV, 253
- TY\_PIXEL\_FORMAT\_YVYU, 253
- TY\_RESOLUTION\_MODE\_1280x720, 254
- TY\_RESOLUTION\_MODE\_1280x800, 254
- TY\_RESOLUTION\_MODE\_1280x960, 254
- TY\_RESOLUTION\_MODE\_1600x1200, 254
- TY\_RESOLUTION\_MODE\_160x100, 253
- TY\_RESOLUTION\_MODE\_160x120, 253
- TY\_RESOLUTION\_MODE\_1920x1080, 254
- TY\_RESOLUTION\_MODE\_1920x1440, 254
- TY\_RESOLUTION\_MODE\_2048x1536, 254
- TY\_RESOLUTION\_MODE\_240x320, 253
- TY\_RESOLUTION\_MODE\_240x96, 254
- TY\_RESOLUTION\_MODE\_2560x1920, 254
- TY\_RESOLUTION\_MODE\_2592x1944, 254
- TY\_RESOLUTION\_MODE\_320x180, 253
- TY\_RESOLUTION\_MODE\_320x200, 253
- TY\_RESOLUTION\_MODE\_320x240, 253
- TY\_RESOLUTION\_MODE\_480x640, 253

- TY\_RESOLUTION\_MODE\_640x360, [253](#)
- TY\_RESOLUTION\_MODE\_640x400, [254](#)
- TY\_RESOLUTION\_MODE\_640x480, [254](#)
- TY\_RESOLUTION\_MODE\_800x600, [254](#)
- TY\_RESOLUTION\_MODE\_960x1280, [254](#)
- TY\_RESOLUTION\_MODE\_LIST, [253](#)
- TY\_STRUCT\_AEC\_ROI, [250](#)
- TY\_STRUCT\_CAM\_CALIB\_DATA, [249](#)
- TY\_STRUCT\_CAM\_DISTORTION, [249](#)
- TY\_STRUCT\_CAM\_INTRINSIC, [249](#)
- TY\_STRUCT\_CAM\_RECTIFIED\_INTRI, [249](#)
- TY\_STRUCT\_CAM\_RECTIFIED\_ROTATION, [249](#)
- TY\_STRUCT\_CAM\_STATISTICS, [249](#)
- TY\_STRUCT\_DI0\_WORKMODE, [250](#)
- TY\_STRUCT\_DI1\_WORKMODE, [250](#)
- TY\_STRUCT\_DI2\_WORKMODE, [250](#)
- TY\_STRUCT\_DO0\_WORKMODE, [250](#)
- TY\_STRUCT\_DO1\_WORKMODE, [250](#)
- TY\_STRUCT\_DO2\_WORKMODE, [250](#)
- TY\_STRUCT\_EXTRINSIC\_TO\_DEPTH, [249](#)
- TY\_STRUCT\_EXTRINSIC\_TO\_IR\_LEFT, [249](#)
- TY\_STRUCT\_FLOOD\_ENABLE\_BY\_IDX, [250](#)
- TY\_STRUCT\_FLOOD\_POWER\_BY\_IDX, [250](#)
- TY\_STRUCT\_IMU\_ACC\_BIAS, [251](#)
- TY\_STRUCT\_IMU\_ACC\_MISALIGNMENT, [251](#)
- TY\_STRUCT\_IMU\_ACC\_SCALE, [251](#)
- TY\_STRUCT\_IMU\_CAM\_TO\_IMU, [251](#)
- TY\_STRUCT\_IMU\_GYRO\_BIAS, [251](#)
- TY\_STRUCT\_IMU\_GYRO\_MISALIGNMENT, [251](#)
- TY\_STRUCT\_IMU\_GYRO\_SCALE, [251](#)
- TY\_STRUCT\_LASER\_ENABLE\_BY\_IDX, [250](#)
- TY\_STRUCT\_LASER\_POWER\_BY\_IDX, [250](#)
- TY\_STRUCT\_PHC\_GROUP\_ATTR, [251](#)
- TY\_STRUCT\_TOF\_FREQ, [252](#)
- TY\_STRUCT\_TRIGGER\_PARAM, [250](#)
- TY\_STRUCT\_TRIGGER\_PARAM\_EX, [250](#)
- TY\_STRUCT\_TRIGGER\_TIMER\_LIST, [250](#)
- TY\_STRUCT\_TRIGGER\_TIMER\_PERIOD, [250](#)
- TY\_TRIGGER\_MODE\_LIST, [248](#), [254](#)
- TY\_TRIGGER\_MODE\_M\_PER, [254](#)
- TY\_TRIGGER\_MODE\_M\_SIG, [254](#)
- TY\_TRIGGER\_MODE\_OFF, [254](#)
- TY\_TRIGGER\_MODE\_PER\_PASS, [254](#)
- TY\_TRIGGER\_MODE\_PER\_PASS2, [254](#)
- TY\_TRIGGER\_MODE\_SIG\_PASS, [254](#)
- TY\_TRIGGER\_MODE\_SLAVE, [254](#)
- TYDeinitLib
  - TYApi.h, [159](#)
- TYDepthEnhanceFilter
  - TYImageProc.h, [274](#)
- TYDepthImageInpainter
  - TYImageProc.h, [274](#)
- TYDepthSpeckleFilter
  - TYImageProc.h, [275](#)
- TYDisableComponents
  - TYApi.h, [159](#)
- TYDisableLog
  - TYApi.h, [160](#)
- TYEnableComponents
  - TYApi.h, [160](#)
- TYEnableLog
  - TYApi.h, [161](#)
- TYEnqueueBuffer
  - TYApi.h, [162](#)
- TYEnumEntry, [148](#)
- TYErrorString
  - TYApi.h, [163](#)
- TYFeatureList.h, [254](#)
  - ACQ\_MODE\_CONTINUOUS, [263](#)
  - ACQ\_MODE\_MULTI\_FRAME, [263](#)
  - ACQ\_MODE\_SINGLE\_FRAME, [263](#)
  - BIN\_H1, [264](#)
  - BIN\_H2, [264](#)
  - BIN\_H3, [264](#)
  - BIN\_H4, [264](#)
  - BIN\_V1, [264](#)
  - BIN\_V2, [264](#)
  - BIN\_V3, [264](#)
  - BIN\_V4, [264](#)
  - CSIBayerBG10P, [268](#)
  - CSIBayerBG12P, [268](#)
  - CSIBayerBG14P, [268](#)
  - CSIBayerGB10P, [268](#)
  - CSIBayerGB12P, [268](#)
  - CSIBayerGB14P, [268](#)
  - CSIBayerGR10P, [268](#)
  - CSIBayerGR12P, [268](#)
  - CSIBayerGR14P, [268](#)
  - CSIBayerRG10P, [268](#)
  - CSIBayerRG12P, [268](#)
  - CSIBayerRG14P, [268](#)
  - CSIMono10P, [268](#)
  - CSIMono12P, [268](#)
  - CSIMono14P, [268](#)
  - DEPTH\_TOF\_QUALITY\_HIGH, [264](#)
  - DEPTH\_TOF\_QUALITY\_LOW, [264](#)
  - DEPTH\_TOF\_QUALITY\_MEDIUM, [264](#)
  - DEV\_CHARSET\_ASCII, [264](#)
  - DEV\_CHARSET\_UTF8, [264](#)
  - DEV\_COMPRESS\_HW, [265](#)
  - DEV\_COMPRESS\_NONE, [265](#)
  - DEV\_LINK\_HB\_OFF, [265](#)
  - DEV\_LINK\_HB\_ON, [265](#)
  - DEV\_STREAM\_ASYNC\_ALL, [265](#)
  - DEV\_STREAM\_ASYNC\_DEPTH, [265](#)
  - DEV\_STREAM\_ASYNC\_DEPTHANDRGB, [265](#)
  - DEV\_STREAM\_ASYNC\_OFF, [265](#)
  - DEV\_STREAM\_ASYNC\_RGB, [265](#)
  - DEV\_STREAM\_CH\_RECEIVER, [266](#)
  - DEV\_STREAM\_CH\_TRANSMITTER, [266](#)
  - DEV\_TEMP\_SEL\_CPUCORE, [266](#)
  - DEV\_TEMP\_SEL\_IRFLASH, [266](#)
  - DEV\_TEMP\_SEL\_LASER, [266](#)
  - DEV\_TEMP\_SEL\_LEFT, [266](#)
  - DEV\_TEMP\_SEL\_MAINBOARD, [266](#)
  - DEV\_TEMP\_SEL\_RIGHT, [266](#)

- DEV\_TEMP\_SEL\_TEXTURE, [266](#)
- DEV\_TEMP\_SEL\_TEXTURE\_FLASH, [266](#)
- DEV\_TIME\_SYNC\_CAN, [266](#)
- DEV\_TIME\_SYNC\_HOST, [266](#)
- DEV\_TIME\_SYNC\_NONE, [266](#)
- DEV\_TIME\_SYNC\_NTP, [266](#)
- DEV\_TIME\_SYNC\_PTP\_MASTER, [266](#)
- DEV\_TIME\_SYNC\_PTP\_SLAVE, [266](#)
- DEV\_TYPE\_PERIPHERAL, [267](#)
- DEV\_TYPE\_RECEIVER, [267](#)
- DEV\_TYPE\_TRANSCEIVER, [267](#)
- DEV\_TYPE\_TRANSMITTER, [267](#)
- GAIN\_SEL\_ANALOG\_ALL, [267](#)
- GAIN\_SEL\_DIGITAL\_ALL, [267](#)
- GAIN\_SEL\_DIGITALBLUE, [267](#)
- GAIN\_SEL\_DIGITALGREEN, [267](#)
- GAIN\_SEL\_DIGITALRED, [267](#)
- GEV\_CCP\_CONTROL, [267](#)
- GEV\_CCP\_EXCLUSIVE, [267](#)
- GEV\_CCP\_OPEN, [267](#)
- JPEG, [268](#)
- LIGHT\_SEL\_FLOOD\_0, [268](#)
- LIGHT\_SEL\_LASER\_0, [268](#)
- LIGHT\_SEL\_TEXTURE\_FLOOD, [268](#)
- SRC\_SEL\_DEPTH, [268](#)
- SRC\_SEL\_LEFT, [268](#)
- SRC\_SEL\_RIGHT, [268](#)
- SRC\_SEL\_TEXTURE, [268](#)
- TofIR\_FourGroup\_Mono16, [268](#)
- TRIG\_ACT\_ANY\_EDGE, [269](#)
- TRIG\_ACT\_FALLING\_EDGE, [269](#)
- TRIG\_ACT\_LEVEL\_HIGH, [269](#)
- TRIG\_ACT\_LEVEL\_LOW, [269](#)
- TRIG\_ACT\_RISING\_EDGE, [269](#)
- TRIG\_MODE\_OFF, [269](#)
- TRIG\_MODE\_ON, [269](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONACTIVE, [269](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONEND, [269](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONSTART, [269](#)
- TRIG\_SEL\_FRAME\_EXPOSUREACTIVE, [270](#)
- TRIG\_SEL\_FRAME\_EXPOSUREEND, [270](#)
- TRIG\_SEL\_FRAME\_EXPOSURESTART, [270](#)
- TRIG\_SEL\_FRAME\_FRAMEACTIVE, [269](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTACTIVE, [270](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTEND, [269](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTSTART, [269](#)
- TRIG\_SEL\_FRAME\_FRAMEEND, [269](#)
- TRIG\_SEL\_FRAME\_FRAMESTART, [269](#)
- TRIG\_SEL\_FRAME\_LINESTART, [270](#)
- TRIG\_SRC\_LINE0, [270](#)
- TRIG\_SRC\_LINE1, [270](#)
- TRIG\_SRC\_LINE2, [270](#)
- TRIG\_SRC\_LINE3, [270](#)
- TRIG\_SRC\_LINE4, [270](#)
- TRIG\_SRC\_LINE5, [270](#)
- TRIG\_SRC\_LINE6, [270](#)
- TRIG\_SRC\_LINE7, [270](#)
- TRIG\_SRC\_SOFTWARE, [270](#)
- TY\_ACQ\_MODE\_LIST, [263](#)
- TY\_BIN\_H\_LIST, [263](#)
- TY\_BIN\_V\_LIST, [264](#)
- TY\_DEPTH\_TOF\_QUALITY\_LIST, [264](#)
- TY\_DEV\_CHARSET\_LIST, [264](#)
- TY\_DEV\_COMPRESS\_LIST, [265](#)
- TY\_DEV\_LINK\_HB\_MODE\_LIST, [265](#)
- TY\_DEV\_STREAM\_ASYNC\_LIST, [265](#)
- TY\_DEV\_STREAM\_CH\_TYPE\_LIST, [265](#)
- TY\_DEV\_TEMP\_SEL\_LIST, [266](#)
- TY\_DEV\_TIME\_SYNC\_LIST, [266](#)
- TY\_DEV\_TYPE\_LIST, [266](#)
- TY\_GAIN\_SEL\_LIST, [267](#)
- TY\_GEV\_CCP\_LIST, [267](#)
- TY\_LIGHT\_SEL\_LIST, [267](#)
- TY\_PIX\_FMT\_LIST, [268](#)
- TY\_SRC\_SEL\_LIST, [268](#)
- TY\_TRIG\_ACT\_LIST, [269](#)
- TY\_TRIG\_MODE\_LIST, [269](#)
- TY\_TRIG\_SEL\_LIST, [269](#)
- TY\_TRIG\_SRC\_LIST, [270](#)
- TY\_USER\_SET\_SEL\_LIST, [270](#)
- USER\_SET\_SEL\_DEFAULT0, [270](#)
- USER\_SET\_SEL\_DEFAULT1, [270](#)
- USER\_SET\_SEL\_DEFAULT2, [270](#)
- USER\_SET\_SEL\_DEFAULT3, [270](#)
- USER\_SET\_SEL\_DEFAULT4, [270](#)
- USER\_SET\_SEL\_DEFAULT5, [270](#)
- USER\_SET\_SEL\_DEFAULT6, [270](#)
- USER\_SET\_SEL\_DEFAULT7, [270](#)
- USER\_SET\_SEL\_USER\_SET0, [270](#)
- USER\_SET\_SEL\_USER\_SET1, [270](#)
- USER\_SET\_SEL\_USER\_SET2, [271](#)
- USER\_SET\_SEL\_USER\_SET3, [271](#)
- USER\_SET\_SEL\_USER\_SET4, [271](#)
- USER\_SET\_SEL\_USER\_SET5, [271](#)
- USER\_SET\_SEL\_USER\_SET6, [271](#)
- USER\_SET\_SEL\_USER\_SET7, [271](#)
- TYFetchFrame
  - TYApi.h, [163](#)
- TYForceDeviceIP
  - TYApi.h, [164](#)
- TYGetBool
  - TYApi.h, [165](#)
- TYGetByteArray
  - TYApi.h, [167](#)
- TYGetByteArrayAttr
  - TYApi.h, [168](#)
- TYGetByteArraySize
  - TYApi.h, [170](#)
- TYGetComponentIDs
  - TYApi.h, [171](#)
- TYGetDecodeBufferSize
  - TYImageProc.h, [275](#)
- TYGetDecodeTargetPixFmt
  - TYImageProc.h, [276](#)
- TYGetDeviceFeatureInfo
  - TYApi.h, [172](#)

- TYGetDeviceFeatureNumber
  - TYApi.h, [173](#)
- TYGetDeviceInfo
  - TYApi.h, [174](#)
- TYGetDeviceInterface
  - TYApi.h, [175](#)
- TYGetDeviceList
  - TYApi.h, [176](#)
- TYGetDeviceNumber
  - TYApi.h, [176](#)
- TYGetDeviceXML
  - TYApi.h, [177](#)
- TYGetDeviceXMLSize
  - TYApi.h, [178](#)
- TYGetEnabledComponents
  - TYApi.h, [179](#)
- TYGetEnum
  - TYApi.h, [180](#)
- TYGetEnumEntryCount
  - TYApi.h, [181](#)
- TYGetEnumEntryInfo
  - TYApi.h, [182](#)
- TYGetFeatureInfo
  - TYApi.h, [183](#)
- TYGetFloat
  - TYApi.h, [184](#)
- TYGetFloatRange
  - TYApi.h, [186](#)
- TYGetFrameBufferSize
  - TYApi.h, [187](#)
- TYGetImageAlgorithmVersion
  - TYImageProc.h, [276](#)
- TYGetInt
  - TYApi.h, [188](#)
- TYGetInterfaceList
  - TYApi.h, [189](#)
- TYGetInterfaceNumber
  - TYApi.h, [190](#)
- TYGetIntRange
  - TYApi.h, [191](#)
- TYGetString
  - TYApi.h, [192](#)
- TYGetStringLength
  - TYApi.h, [193](#)
- TYGetStruct
  - TYApi.h, [195](#)
- TYHasDevice
  - TYApi.h, [197](#)
- TYHasFeature
  - TYApi.h, [198](#)
- TYHasInterface
  - TYApi.h, [199](#)
- TYImageInfo, [149](#)
- TYImageProc.h, [271](#)
  - TYDecodeImage, [273](#)
  - TYDepthEnhanceFilter, [274](#)
  - TYDepthImageInpainter, [274](#)
  - TYDepthSpeckleFilter, [275](#)
  - TYGetDecodeBufferSize, [275](#)
  - TYGetDecodeTargetPixFmt, [276](#)
  - TYGetImageAlgorithmVersion, [276](#)
  - TYImageProcesAcceEnable, [277](#)
  - TYUndistortImage, [277](#)
  - TYUndistortImage2, [278](#)
- TYImageProcesAcceEnable
  - TYImageProc.h, [277](#)
- TYInvertExtrinsic
  - TYCoordinateMapper.h, [222](#)
- TYLibVersion
  - TYApi.h, [199](#)
- TYMapDepthImageToColorCoordinate
  - TYCoordinateMapper.h, [223](#)
- TYMapDepthImageToPoint3d
  - TYCoordinateMapper.h, [224](#)
- TYMapDepthToColorCoordinate
  - TYCoordinateMapper.h, [224](#)
- TYMapDepthToPoint3d
  - TYCoordinateMapper.h, [225](#)
- TYMapMono16ImageToDepthCoordinate
  - TYCoordinateMapper.h, [225](#)
- TYMapMono8ImageToDepthCoordinate
  - TYCoordinateMapper.h, [226](#)
- TYMapPoint3dToDepth
  - TYCoordinateMapper.h, [227](#)
- TYMapPoint3dToDepthImage
  - TYCoordinateMapper.h, [227](#)
- TYMapPoint3dToPoint3d
  - TYCoordinateMapper.h, [228](#)
- TYMapRGB48ImageToDepthCoordinate
  - TYCoordinateMapper.h, [228](#)
- TYMapRGBImageToDepthCoordinate
  - TYCoordinateMapper.h, [229](#)
- TYMapRGBPixelsToDepthCoordinate
  - TYCoordinateMapper.h, [230](#)
- TYOpenDevice
  - TYApi.h, [200](#)
- TYOpenDeviceWithIP
  - TYApi.h, [201](#)
- TYOpenInterface
  - TYApi.h, [203](#)
- TYRegisterEventCallback
  - TYApi.h, [203](#)
- TYRegisterImuCallback
  - TYApi.h, [204](#)
- TYSendSoftTrigger
  - TYApi.h, [205](#)
- TYSetBool
  - TYApi.h, [206](#)
- TYSetByteArray
  - TYApi.h, [207](#)
- TYSetEnum
  - TYApi.h, [209](#)
- TYSetFloat
  - TYApi.h, [210](#)
- TYSetInt
  - TYApi.h, [212](#)

- TYSetLogLevel
  - TYApi.h, [213](#)
- TYSetLogPrefix
  - TYApi.h, [213](#)
- TYSetString
  - TYApi.h, [214](#)
- TYSetStruct
  - TYApi.h, [216](#)
- TYStartCapture
  - TYApi.h, [217](#)
- TYStopCapture
  - TYApi.h, [218](#)
- TYUndistortImage
  - TYImageProc.h, [277](#)
- TYUndistortImage2
  - TYImageProc.h, [278](#)
- TYUpdateAllDeviceList
  - TYApi.h, [219](#)
- TYUpdateDeviceList
  - TYApi.h, [219](#)
- TYUpdateInterfaceList
  - TYApi.h, [220](#)
- unit\_size
  - TY\_BYTEARRAY\_ATTR, [128](#)
- USER\_SET\_SEL\_DEFAULT0
  - TYFeatureList.h, [270](#)
- USER\_SET\_SEL\_DEFAULT1
  - TYFeatureList.h, [270](#)
- USER\_SET\_SEL\_DEFAULT2
  - TYFeatureList.h, [270](#)
- USER\_SET\_SEL\_DEFAULT3
  - TYFeatureList.h, [270](#)
- USER\_SET\_SEL\_DEFAULT4
  - TYFeatureList.h, [270](#)
- USER\_SET\_SEL\_DEFAULT5
  - TYFeatureList.h, [270](#)
- USER\_SET\_SEL\_DEFAULT6
  - TYFeatureList.h, [270](#)
- USER\_SET\_SEL\_DEFAULT7
  - TYFeatureList.h, [270](#)
- USER\_SET\_SEL\_USER\_SET0
  - TYFeatureList.h, [270](#)
- USER\_SET\_SEL\_USER\_SET1
  - TYFeatureList.h, [270](#)
- USER\_SET\_SEL\_USER\_SET2
  - TYFeatureList.h, [271](#)
- USER\_SET\_SEL\_USER\_SET3
  - TYFeatureList.h, [271](#)
- USER\_SET\_SEL\_USER\_SET4
  - TYFeatureList.h, [271](#)
- USER\_SET\_SEL\_USER\_SET5
  - TYFeatureList.h, [271](#)
- USER\_SET\_SEL\_USER\_SET6
  - TYFeatureList.h, [271](#)
- USER\_SET\_SEL\_USER\_SET7
  - TYFeatureList.h, [271](#)
- valid\_size
  - TY\_BYTEARRAY\_ATTR, [128](#)
- w
  - tjregion, [121](#)
- x
  - tjregion, [122](#)
- y
  - tjregion, [122](#)
- z\_stream\_s, [149](#)